



MACHINE LEARNING
MASTERY

Deep Learning with PyTorch

Learn Basic Deep Learning
with Minimal Code in
PyTorch 2.0



Adrian Tam

Disclaimer

The information contained within this eBook is strictly for educational purposes. If you wish to apply ideas contained in this eBook, you are taking full responsibility for your actions.

The author has made every effort to ensure the accuracy of the information within this book was correct at time of publication. The author does not assume and hereby disclaims any liability to any party for any loss, damage, or disruption caused by errors or omissions, whether such errors or omissions result from accident, negligence, or any other cause.

No part of this eBook may be reproduced or transmitted in any form or by any means, electronic or mechanical, recording or by any information storage and retrieval system, without written permission from the author.

Credits

Founder: Jason Brownlee

Authors: Adrian Tam

Technical Reviewers: Darci Heikkinen, Amy Lam, and Devansh Sethi

Copyright

Deep Learning with PyTorch

© 2023 MachineLearningMastery.com. All Rights Reserved.

Edition: v1.00

Contents

Preface	ix
Introduction	x
I Background	1
1 Overview of Some Deep Learning Libraries	2
The C++ Libraries	2
Python Libraries	3
PyTorch and TensorFlow	5
Further Reading	7
Summary	8
2 Introduction to PyTorch	9
PyTorch at a High Level	9
Getting Started with PyTorch	10
Applications of PyTorch.	11
Advantages and Limitations of PyTorch	11
Comparing PyTorch to TensorFlow	12
Further Readings.	13
Summary	13
3 Manipulating Tensors in PyTorch	14
Creating Tensors	14
Checking a Tensor	17
Manipulating Tensors.	19
Tensor Functions	24
Further Reading	29
Summary	29
4 Using Autograd in PyTorch to Solve a Regression Problem	30
Autograd in PyTorch	30

Using Autograd for Polynomial Regression.	32
Using Autograd to Solve a Math Puzzle	35
Further Reading	36
Summary	36

II Multilayer Perceptron Models 37

5 A Crash Course to Deep Learning 38

From Multilayer Perceptrons to Deep Learning.	38
Neurons: Weights and Activations	39
Networks of Neurons	40
Training Networks	41
More Resources	43
Summary	43

6 Multilayer Perceptron Building Blocks in PyTorch 44

Neural Network Models in PyTorch.	44
Model Inputs	46
Layers, Activations, and Layer Properties	46
Loss Function and Model Optimizers	47
Model Training and Inference	48
Examination of a Model.	48
Further Readings.	49
Summary	50

7 Your First Neural Network in PyTorch, Step by Step 51

Load Data.	51
Define the Model.	53
Preparation for Training	56
Training a Model.	56
Evaluate the Model.	57
Make Predictions.	60
Further Reading	62
Summary	62

8 Creating a Training Loop for Your Models 63

Elements of Training a Deep Learning Model	63
Collecting Statistics During Training	67
Using <code>tqdm</code> to Report the Training Progress	70
Further Readings.	73
Summary	74

9 Evaluating PyTorch Models 75

Empirical Evaluation of Models	75
Data Splitting	76
Training a PyTorch Model with Validation	77

k -Fold Cross-Validation	82
Further Readings.	86
Summary	86
10 Project: Building a Multiclass Classification Model in PyTorch	87
Problem Description	87
Load the Dataset.	88
Encode the Categorical Variable	88
Define the Neural Network Model	89
Benchmark the Model.	92
Complete Example	96
Further Readings.	99
Summary	99
11 Project: Building a Binary Classification Model in PyTorch	100
Description of the Dataset.	100
Load the Dataset.	101
Creating a Model.	102
Comparing Models with Cross-Validation	104
Retrain the Final Model.	107
Complete Code	109
Further Readings.	113
Summary	113
12 Project: Building a Regression Model in PyTorch	114
Description of the Dataset.	114
Building a Model and Train	115
Improving the Model with Preprocessing	120
Further Readings.	124
Summary	124
III Techniques for Better Deep Learning Models	125
13 Save and Load Your PyTorch Models	126
Build an Example Model	126
What's Inside a PyTorch Model	128
Accessing <code>state_dict</code> of a Model	129
Further Readings.	133
Summary	133
14 Using Activation Functions in Deep Learning Models	135
A Toy Model of Binary Classification	135
Why Nonlinear Functions?.	140
The Effect of Activation Functions	140
Further Readings.	149
Summary	150

15	Loss Functions in PyTorch Models	151
	What Are Loss Functions?	151
	Loss Functions for Regression	152
	Loss Functions for Classification	154
	Custom Loss Function in PyTorch	157
	Further Readings.	160
	Summary	160
16	Using Dropout Regularization in PyTorch Models	161
	Dropout Regularization for Neural Networks.	161
	Dropout Regularization in PyTorch.	162
	Using Dropout on the Input Layer	165
	Using Dropout on Hidden Layers.	167
	Dropout in Evaluation Mode.	169
	Tips for Using Dropout	169
	Further Readings.	169
	Summary	170
17	Using Learning Rate Schedule in PyTorch Training	171
	Learning Rate Schedule for Training Models	171
	Applying Learning Rate Schedules in PyTorch Training	172
	Custom Learning Rate Schedules	177
	Tips for Using Learning Rate Schedules	179
	Further Readings.	180
	Summary	180
18	Training a PyTorch Model with DataLoader and Dataset	181
	What is DataLoader?	181
	Using DataLoader in a Training Loop.	183
	Create Data Iterator using Dataset Class	185
	Further Readings.	188
	Summary	188
19	Use PyTorch Deep Learning Models with scikit-learn	189
	Overview of skorch	189
	Evaluate Deep Learning Models with Cross-Validation	190
	Running k -Fold Cross-validation with scikit-learn.	192
	Grid Search Deep Learning Model Parameters	195
	Further Readings.	200
	Summary	200
20	Optimize Hyperparameters with Grid Search	202
	How to Use PyTorch Models in scikit-learn	203
	How to Use Grid Search in scikit-learn	205
	Problem Description	205
	How to Tune Batch Size and Number of Epochs	206
	How to Tune the Training Optimization Algorithm	207

How to Tune Learning Rate and Momentum	209
How to Tune Network Weight Initialization	211
How to Tune the Neuron Activation Function	214
How to Tune Dropout Regularization	216
How to Tune the Number of Neurons in the Hidden Layer	218
Tips for Hyperparameter Optimization	220
Further Readings.	221
Summary	221
21 Managing Training Process with Checkpoints and Early Stopping	223
Checkpointing Neural Network Models	223
Checkpointing with Early Stopping	228
Further Readings.	231
Summary	232
22 Visualizing a PyTorch Model	233
Why Graphical Represetnation of a PyTorch Model is Hard	233
How to Use Netron to Create a Model Graph	234
Further Readings.	236
Summary	237
23 Understand Model Behavior During Training by Visualizing Metrics	238
Collecting Metrics from a Training Loop	238
Plotting the Training History	243
Further Readings.	246
Summary	246
IV Advanced Models	247
24 From MLP to CNN and RNN	248
Introduction to Convolutional Neural Network	248
Introduction to Recurrent Neural Network.	250
Further Readings.	251
Summary	251
25 Building a Convolutional Neural Network in PyTorch	252
The Case for Convolutional Neural Networks.	252
Building Blocks of Convolutional Neural Networks	253
An Example of Convolutional Neural Network	254
What Are in Feature Maps?	256
Further Readings.	260
Summary	261
26 Handwritten Digit Recognition with LeNet5 Model in PyTorch	262
The MNIST Handwritten Digit Recognition Problem	262
Loading the MNIST Dataset in PyTorch	263

Baseline Model with Multilayer Perceptrons	264
Simple Convolutional Neural Network for MNIST	267
LeNet5 for MNIST	270
Resources on MNIST	274
Summary	274
27 LSTM for Time Series Prediction in PyTorch	276
Overview of LSTM Network	276
LSTM for Time Series Prediction.	278
Training and Verifying Your LSTM Network.	281
Further Readings.	286
Summary	286
28 Text Generation with LSTM in PyTorch	287
What is a Generative Model	288
Getting Text Data	288
A Small LSTM Network to Predict the Next Character	289
Generating Text with an LSTM Model	294
Using a Larger LSTM Network.	296
Faster Training with GPU.	300
Further Readings.	304
Summary	304
V Appendix	305
A How to Setup Python on Your Workstation	306
Overview	306
Download Anaconda	306
Install Anaconda	308
Start and Update Anaconda	309
Install Deep Learning Libraries.	311
Install Visual Studio Code for Python.	312
Further Reading	315
Summary	315
B How to Setup Amazon EC2 for Deep Learning on GPUs	316
Overview	316
Setup Your AWS Account	317
Launch Your Server Instance.	318
Login, Configure and Run	320
Build and Run Models on AWS	323
Close Your EC2 Instance	323
Tips and Tricks for Using Keras on AWS	324
Further Reading	325
Summary	325

How Far You Have Come

326

Preface

No explanation is needed for why deep learning is amazing. But where to start?

Nowadays, probably the ecosystem for deep learning is based on either of the two libraries: TensorFlow and PyTorch. To learn about deep learning, it is inevitable to study the program in either frameworks. Neither is perfect and either has some edge over the other.

This book is to assemble some quick tips to get started on PyTorch. You will need some basic knowledge on what deep learning is about and some high level idea on the steps to build and use a deep learning model. Then, you can learn how to implement all these idea in PyTorch.

This is the guide you may find useful to bring you up to speed quickly in developing a deep learning model in PyTorch as well as understanding other people's code.

Adrian Tam
New York
2023

Introduction

Welcome to Deep Learning with PyTorch. This book is your guide to deep learning. You will discover the PyTorch library for deep learning and how to use it to develop and evaluate deep learning models. In this book you will discover the techniques, recipes and skills in deep learning that you can then bring to your own machine learning projects.

Deep learning does have a lot of fascinating math under the covers, but you do not need to know it to be able to pick it up as a tool and wield it on important projects and deliver real value. From the applied perspective, deep learning is quite a shallow field and a motivated developer can quickly pick it up and start making very real and impactful contributions. This is our goal for you and this book is your ticket to that outcome.

Deep Learning the Wrong Way

If you ask a deep learning practitioner how to get started with neural networks and deep learning, what do they say? They say things like

- ▷ You must have a strong foundation in linear algebra.
- ▷ You must have a deep knowledge of traditional neural network techniques.
- ▷ You really must know about probability and statistics.
- ▷ You should really have a deep knowledge of machine learning.
- ▷ You probably need to be a PhD in computer science.
- ▷ You probably need 10 years of experience as a machine learning developer.

You can see that the *common sense* advice means that it is not until after you have completed years of study and experience that you are ready to actually start developing and evaluating machine learning model for your machine learning projects.

This advice is dead wrong.

Deep Learning with Python

The approach taken with this book and with all of Machine Learning Mastery is to flip the traditional approach. If you are interested in deep learning, start by developing and evaluating deep learning models. Then if you discover you really like it or have a knack for it, later you can step deeper and deeper into the background and theory, as you need it in order to serve you in developing better and more valuable results. This book is your ticket to jumping in and making a ruckus with deep learning.

Unlike R, Python is a fully featured programming language allowing you to use the same libraries and code for model development as you can use in production. Unlike Java, Python has the SciPy stack for scientific computing and scikit-learn which is a professional grade machine learning library.

Machine Learning Mastery has published a similar book on TensorFlow and Keras, but it is impossible to ignore the existence of the PyTorch library from Facebook/Meta. These libraries are developed for use in Python, with concise API, and allows you to focus on the deep learning model that you want to develop rather than the detail on how numbers and matrices change values. This book is solely in PyTorch.

You will develop your own and perhaps your first neural network and deep learning models while working through this book. You will have the skills to bring this amazing new technology to your own projects. It is going to be a fun journey and we can't wait to start.

Book Organization

There are three kinds of chapters in this book.

- ▷ **Lessons**, where you learn about specific features of neural network models and how to use specific aspects of PyTorch.
- ▷ **Projects**, where you will pull together multiple lessons into an end-to-end project and deliver a result, providing a template for your own projects.
- ▷ **Recipes**, where you can copy and paste the standalone code into your own project, including all of the code presented in this book.

Lessons and Projects

Lessons are discrete and are focused on one topic, designed for you to complete in one sitting. You can take as long as you need, from 20 minutes if you are racing through, to hours if you want to experiment with the code or ideas and improve upon the presented results. Your lessons are divided into five parts:

- ▷ Background
- ▷ Multilayer perceptron models
- ▷ Techniques for better deep learning models
- ▷ Advanced models

Part 1: Background

In this part you will learn about the PyTorch library that lay the foundation for your deep learning journey. This part of the book includes the following lessons:

- ▷ Overview of Some Deep Learning Libraries
- ▷ Introduction to PyTorch
- ▷ Manipulating Tensors in PyTorch
- ▷ Using Autograd in PyTorch to Solve a Regression Problem

The lessons will introduce you to the library that you need to install and use on your workstation. You will also learn about how PyTorch can help deep learning. At the end of this part you will be ready to start developing models in PyTorch on your workstation.

Part 2: Multilayer Perceptron Models

In this part you will learn about feedforward neural networks that may be deep or not and how to expertly develop your own networks and evaluate them efficiently using PyTorch. This part of the book includes the following lessons:

- ▷ A Crash Course to Deep Learning
- ▷ Multilayer Perceptron Building Blocks in PyTorch
- ▷ Your First Neural Network in PyTorch, Step by Step
- ▷ Creating a Training Loop for Your Models
- ▷ Evaluating PyTorch Models

These important lessons are tied together with three foundation projects. These projects demonstrate how you can quickly and efficiently develop neural network models for tabular data and provide project templates that you can use on your own regression and classification machine learning problems. These projects include:

- ▷ Project: Building a Multiclass Classification Model in PyTorch
- ▷ Project: Building a Binary Classification Model in PyTorch
- ▷ Project: Building a Regression Model in PyTorch

Not only the classification and regression are major application of deep learning models, these projects are also good opportunities to learn about data preprocessing techniques for a more effective model. At the end of this part you will be ready to discover the finer points of deep learning.

Part 3: Techniques for Better Deep Learning Models

In this part you will learn about some finer points of the PyTorch library and API for practical machine learning projects and some of the more important developments in applied neural networks that you need to know in order to deliver a world-class results. This part of the book includes the following lessons:

- ▷ Save and Load Your PyTorch Models

- ▷ Using Activation Functions in Deep Learning Models
- ▷ Loss Functions in PyTorch Models
- ▷ Using Dropout Regularization in PyTorch Models
- ▷ Using Learning Rate Schedule in PyTorch Models
- ▷ Training a PyTorch Model with DataLoader and Dataset
- ▷ Use PyTorch Deep Learning Models with scikit-learn
- ▷ Optimize Hyperparameters with Grid Search
- ▷ Managing Training Process with Checkpoints and Early Stopping
- ▷ Visualizing a PyTorch Model
- ▷ Understanding Model Behavior During Training by Visualizing Metrics

At the end of this part you will know how to confidently wield PyTorch on your machine learning projects with a focus on the finer points of investigating model performance, persisting models for later use and gaining lifts in performance over baseline models.

Part 4: Advanced Models

Neural networks started their history with fully-connected multilayer perceptron models. However, the recent advance in neural networks proposed various different designs. In this part you will receive a crash course in the dominant model for computer vision machine learning problems and some natural language problems and learn how you can best exploit the capabilities of the PyTorch for your own projects. This part of the book includes the following lessons and projects:

- ▷ From MLP to CNN and RNN
- ▷ Building a Convolutional Neural Network in PyTorch
- ▷ Handwritten Digit Recognition with LeNet5 Model in PyTorch
- ▷ LSTM for Time Series Prediction in PyTorch
- ▷ Text Generation with LSTM in PyTorch

The best way to learn about this impressive type of neural network model is to apply it. The last four chapters each has a project to let you practice what you learned earlier while learning new types of network models.

After completing the lessons and projects in this part, you will have the skills and the confidence to use the templates and recipes to tackle your own deep learning projects using convolutional neural networks and recurrent neural networks.

Conclusions

The book concludes with some resources that you can use to learn more information about a specific topic or find help if you need it, as you start to develop and evaluate your own deep learning models.

Recipes

Building up a catalog of code recipes is an important part of your deep learning journey. Each time you learn about a new technique or new problem type, you should write up a short code recipe that demonstrates it. This will give you a starting point to use on your next deep learning or machine learning project.

As part of this book you will receive a catalog of deep learning recipes. This includes recipes for all of the lessons presented in this book, as well as complete code for all of the projects. You are strongly encouraged to add to and build upon this catalog of recipes as you expand your use and knowledge of deep learning in Python.

Requirements for This Book

Python, PyTorch, and NumPy

You do not need to be a Python expert, but it would be helpful if you know how to install and setup Python and NumPy. The lessons and projects assume that you have a Python environment available. This may be on your workstation or laptop, it may be in a VM or a Docker instance that you run, or it may be a server instance that you can configure in the cloud. You will be guided as to how to install the PyTorch library in Part I of the book. If you have trouble, you can follow the step-by-step tutorial in Appendix A.

Machine Learning

You do not need to be a machine learning expert, but it would be helpful if you knew how to navigate a small machine learning problem using scikit-learn. Basic concepts like cross-validation and one-hot encoding used in lessons and projects are described, but only briefly. There are resources to go into these topics in more detail at the end of the book, but some knowledge of these areas might make things easier for you.

Deep Learning

You do not need to know the math and theory of deep learning algorithms, but it would be helpful to have some basic ideas of the field. You will get a crash course in neural network terminology and models, but I will not go into much detail. Again, there will be resources for more information at the end of the book, but it might be helpful if you can start with some ideas about neural networks.

Note: All tutorials can be completed on standard workstation hardware with a CPU. GPU is not required. Some tutorials later in the book can be speed up significantly by running on the GPU and a suggestion is provided to consider using GPU hardware at the beginning of those sections. You can access GPU hardware easily and cheaply in the cloud and a step-by-step procedure is taught on how to do this in Appendix B.

Your Outcomes from Reading This Book

This book will lead you from being a developer who is interested in deep learning with Python to a developer who has the resources and capabilities to work through a new dataset end-to-end using Python and develop accurate deep learning models. Specifically, you will know:

- ▷ How to develop and evaluate neural network models end-to-end.
- ▷ How to use more advanced techniques required for developing state-of-the-art deep learning models.
- ▷ How to build larger models for image.
- ▷ How to get help with deep learning in Python.

From here you can start to dive into the specifics of the functions, techniques and algorithms used with the goal of learning how to use them better in order to deliver more accurate predictive models, more reliably in less time. There are a few ways you can read this book. You can dip into the lessons and projects as your need or interests motivate you. Alternatively, you can work through the book end-to-end and take advantage of how the lessons and projects built toward complexity and range. I recommend the latter approach.

To get the very most from this book, I recommend taking each lesson and project and build upon them. Attempt to improve the results, apply the method to a similar but different problem, and so on. Write up what you tried or learned and share it on your blog, social media or send me an email at jason@MachineLearningMastery.com. This book is really what you make of it and by putting in a little extra, you can quickly become a true force in applied deep learning.

What This Book Is Not

This book solves a specific problem of getting you, a developer, up to speed applying deep learning to your own machine learning projects in Python. As such, this book was not intended to be everything to everyone and it is very important to calibrate your expectations. Specifically:

- ▷ **This is not a deep learning textbook.** You will not be getting into the basic theory of artificial neural networks or deep learning algorithms. You are also expected to have some familiarity with machine learning basics, or be able to pick them up yourself.
- ▷ **This is not an algorithm book.** You will not be working through the details of how specific deep learning algorithms work. You are expected to have some basic knowledge of deep learning algorithms or be able to pick up this knowledge yourself.
- ▷ **This is not a Python programming book.** You will not be spending a lot of time on Python syntax and programming (e.g., basic programming tasks in Python). You are expected to already be familiar with Python or be a developer who can pick up a new C-like language relatively quickly.

You can still get a lot out of this book if you are weak in one or two of these areas, but you may struggle picking up the language or require some more explanation of the techniques.

If this is the case, see the Getting More Help chapter at the end of the book and seek out a good companion reference text.

Summary

It is a special time right now. The tools for applied deep learning have never been so good. The pace of change with neural networks and deep learning feels like it has never been so fast, spurred by the amazing results that the methods are showing in such a broad range of fields. This is the start of your journey into deep learning and I am excited for you. Take your time, have fun and I am so excited to see where you can take this amazing new technology to.

Next

Let's dive in. Next up is Part I where you will take a whirlwind tour of the foundation libraries for deep learning in Python, namely the PyTorch library that you will be using throughout this book.

Background I

Overview of Some Deep Learning Libraries

Machine learning is a broad topic. Deep learning, in particular, is a way of using neural networks for machine learning. A neural network is probably a concept older than machine learning, dating back to the 1950s. Unsurprisingly, there were many libraries created for it.

The following aims to give an overview of some of the famous libraries for neural networks and deep learning. After finishing this chapter, you will learn

- ▷ Some of the deep learning or neural network libraries
- ▷ The functional difference between two common libraries, PyTorch and TensorFlow

Let's get started.

Overview

This chapter is in three parts; they are:

- ▷ The C++ Libraries
- ▷ Python Libraries
- ▷ PyTorch and TensorFlow

1.1 The C++ Libraries

Deep learning has gained attention in the last decade. Before that, there was little confidence in how to train a neural network with many layers. However, understanding how to build a multilayer perceptron was around for many years.

Before we had deep learning, probably the most famous neural network library was `libann`. It is a library for C++, and the functionality is limited due to its age. This library has stopped development since 2004. A newer library for C++ is `OpenNN`, which allows modern C++ syntax.

But that's pretty much all for C++. The rigid syntax of C++ may be why we do not have too many libraries for deep learning. The training phase of a deep learning project is

about experiments. We want some tools that allow us to iterate faster. Hence a dynamic programming language could be a better fit. Therefore, you see Python come on the scene.

1.2 Python Libraries

One of the earliest libraries for deep learning is Caffe. It was developed at U.C. Berkeley specifically for computer vision problems. While it is developed in C++, it serves as a library with a Python interface. Hence we can build our project in Python with the network defined in a JSON-like syntax.

Chainer is another library in Python. It is an influential one because the syntax makes a lot of sense. While it is less common nowadays, the API in Keras and PyTorch bears a resemblance to Chainer. The following is an example from Chainer's documentation, and you may mistake it as Keras or PyTorch:

```
import chainer
import chainer.functions as F
import chainer.links as L
from chainer import iterators, optimizer, training, Chain
from chainer.datasets import mnist

train, test = mnist.get_mnist()
batchsize = 128
max_epoch = 10

train_iter = iterators.SerialIterator(train, batchsize)

class MLP(Chain):
    def __init__(self, n_mid_units=100, n_out=10):
        super(MLP, self).__init__()
        with self.init_scope():
            self.l1 = L.Linear(None, n_mid_units)
            self.l2 = L.Linear(None, n_mid_units)
            self.l3 = L.Linear(None, n_out)

    def forward(self, x):
        h1 = F.relu(self.l1(x))
        h2 = F.relu(self.l2(h1))
        return self.l3(h2)

# create model
model = MLP()
model = L.Classifier(model) # using softmax cross entropy

# set up optimizer
optimizer = optimizers.MomentumSGD()
optimizer.setup(model)

# connect train iterator and optimizer to an updater
updater = training.updaters.StandardUpdater(train_iter, optimizer)
```



```
# set up trainer and run
trainer = training.Trainer(updater, (max_epoch, 'epoch'), out='mnist_result')
trainer.run()
```

Listing 1.1: Example Chainer code

The other obsoleted library is Theano. It has ceased development, but once upon a time, it was a major library for deep learning. The earlier version of the Keras library allows you to choose between a Theano or TensorFlow backend. Indeed, neither Theano nor TensorFlow are deep learning libraries precisely. Rather, they are tensor libraries that make matrix operations and differentiation handy, upon where deep learning operations can be built. Hence these two are considered replacements for each other from Keras' perspective.

CNTK from Microsoft and Apache MXNet are two other libraries that worth mentioning. They are large with interfaces for multiple languages. Python, of course, is one of them. CNTK has C# and C++ interfaces, while MXNet provides interfaces for Java, Scala, R, Julia, C++, Clojure, and Perl. But recently, Microsoft decided to stop developing CNTK. MXNet does have some momentum, and it is probably the most popular library after TensorFlow and PyTorch.

Below is an example of using MXNet via the R interface. Conceptually, you see the syntax is similar to Keras' functional API:

```
require(mxnet)

train <- read.csv('data/train.csv', header=TRUE)
train <- data.matrix(train)
train.x <- train[,-1]
train.y <- train[,1]
train.x <- t(train.x/255)

data <- mx.symbol.Variable("data")
fc1 <- mx.symbol.FullyConnected(data, name="fc1", num_hidden=128)
act1 <- mx.symbol.Activation(fc1, name="relu1", act_type="relu")
fc2 <- mx.symbol.FullyConnected(act1, name="fc2", num_hidden=64)
act2 <- mx.symbol.Activation(fc2, name="relu2", act_type="relu")
fc3 <- mx.symbol.FullyConnected(act2, name="fc3", num_hidden=10)
softmax <- mx.symbol.SoftmaxOutput(fc3, name="sm")

devices <- mx.cpu()
mx.set.seed(0)
model <- mx.model.FeedForward.create(softmax, X=train.x, y=train.y,
                                     ctx=devices, num.round=10, array.batch.size=100,
                                     learning.rate=0.07, momentum=0.9,
                                     eval.metric=mx.metric.accuracy,
                                     initializer=mx.init.uniform(0.07),
                                     epoch.end.callback=mx.callback.log.train.metric(100))
```

Listing 1.2: Example MXNet code in R

1.3 PyTorch and TensorFlow

PyTorch and TensorFlow are the two major libraries nowadays. In the past, when TensorFlow was in version 1.x, they were vastly different. But since TensorFlow absorbed Keras as part of its library, these two libraries mostly work similarly.

PyTorch is backed by Facebook, and its syntax has been stable over the years. There are also a lot of existing models that we can borrow. The common way of defining a deep learning model in PyTorch is to create a class:

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, kernel_size=(5,5), stride=1, padding=2)
        self.pool1 = nn.AvgPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5, stride=1, padding=0)
        self.pool2 = nn.AvgPool2d(kernel_size=2, stride=2)
        self.conv3 = nn.Conv2d(16, 120, kernel_size=5, stride=1, padding=0)
        self.flatten = nn.Flatten()
        self.linear4 = nn.Linear(120, 84)
        self.linear5 = nn.Linear(84, 10)
        self.softmax = nn.LogSoftMax(dim=1)

    def forward(self, x):
        x = F.tanh(self.conv1(x))
        x = self.pool1(x)
        x = F.tanh(self.conv2(x))
        x = self.pool2(x)
        x = F.tanh(self.conv3(x))
        x = self.flatten(x)
        x = F.tanh(self.linear4(x))
        x = self.linear5(x)
        return self.softmax(x)

model = Model()
```

Listing 1.3: Example PyTorch code using sub-classing syntax

But there is also a sequential syntax to make the code more concise:

```
import torch
import torch.nn as nn

model = nn.Sequential(
    # assume input 1x28x28
    nn.Conv2d(1, 6, kernel_size=(5,5), stride=1, padding=2),
    nn.Tanh(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Conv2d(6, 16, kernel_size=5, stride=1, padding=0),
```

```

nn.Tanh(),
nn.AvgPool2d(kernel_size=2, stride=2),
nn.Conv2d(16, 120, kernel_size=5, stride=1, padding=0),
nn.Tanh(),
nn.Flatten(),
nn.Linear(120, 84),
nn.Tanh(),
nn.Linear(84, 10),
nn.LogSoftmax(dim=1)
)

```

Listing 1.4: Example PyTorch code using sequential syntax

TensorFlow in version 2.x adopted Keras as part of its libraries. In the past, these two were separate projects. In TensorFlow 1.x, we need to build a computation graph, setup a session, and derive gradients from a session for the deep learning model. Hence it is a bit too verbose. Keras is designed as a library to hide all these low-level details.

The same network as above can be produced by TensorFlow's Keras syntax as follows:

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, Dense, AveragePooling2D, Flatten

model = Sequential([
    Conv2D(6, (5,5), input_shape=(28,28,1), padding="same", activation="tanh"),
    AveragePooling2D((2,2), strides=2),
    Conv2D(16, (5,5), activation="tanh"),
    AveragePooling2D((2,2), strides=2),
    Conv2D(120, (5,5), activation="tanh"),
    Flatten(),
    Dense(84, activation="tanh"),
    Dense(10, activation="softmax")
])

```

Listing 1.5: Example TensorFlow code using sequential syntax

One major difference between PyTorch and Keras syntax is in the training loop. In Keras, we just need to assign the loss function, the optimization algorithm, the dataset, and some other parameters to the model. Then we have a `fit()` function to do all the training work, as follows:

```

...
model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=100, batch_size=32)

```

Listing 1.6: Using `fit()` function in TensorFlow

But in PyTorch, we need to write our own training loop code:

```

# self-defined training loop function
def training_loop(model, optimizer, loss_fn, train_loader, val_loader=None, n_epochs=100):
    best_loss, best_epoch = np.inf, -1
    best_state = model.state_dict()

```

```

for epoch in range(n_epochs):
    # Training
    model.train()
    train_loss = 0
    for data, target in train_loader:
        output = model(data)
        loss = loss_fn(output, target)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        train_loss += loss.item()
    # Validation
    model.eval()
    status = (f"{str(datetime.datetime.now())} End of epoch {epoch}, "
              f"training loss={train_loss/len(train_loader)}")
    if val_loader:
        val_loss = 0
        for data, target in val_loader:
            output = model(data)
            loss = loss_fn(output, target)
            val_loss += loss.item()
        status += f", validation loss={val_loss/len(val_loader)}"
    print(status)

optimizer = optim.Adam(model.parameters())
criterion = nn.NLLLoss()
training_loop(model, optimizer, criterion, train_loader, test_loader, n_epochs=100)

```

Listing 1.7: User-defined training loop function for PyTorch

This may not be an issue if you're experimenting with a new design of network in which you want to have more control over how the loss is calculated and how the optimizer updates the model weights. But otherwise, you will appreciate the simpler syntax from Keras.

Note that both PyTorch and TensorFlow are libraries with a Python interface. Therefore, it is possible to have an interface for other languages too. For example, there are Torch for R and TensorFlow for R.

Also, note that the libraries we mentioned above are full-featured libraries that include training and prediction. If you consider a production environment where you make use of a trained model, there could be a wider choice. TensorFlow has a "TensorFlow Lite" counterpart that allows a trained model to be run on a mobile device or the web. Intel also has an OpenVINO library that aims to optimize the performance in prediction.

1.4 Further Reading

Below are the links to the libraries we mentioned above:

libann.

<https://www.nongnu.org/libann/>

OpenNN.

<https://www.opennn.net/>

Chainer.

<https://chainer.org/>

Caffe.

<https://caffe.berkeleyvision.org/>

CNTK.

<https://docs.microsoft.com/en-us/cognitive-toolkit/>

MXNet.

<https://mxnet.apache.org/versions/1.9.1/>

Theano.

<https://github.com/Theano/Theano>

PyTorch.

<https://pytorch.org/>

TensorFlow.

<https://www.tensorflow.org/>

Torch for R.

<https://torch.mlverse.org/>

TensorFlow for R.

<https://tensorflow.rstudio.com/>

1.5 Summary

In this chapter, you discovered various deep learning libraries and some of their characteristics. Specifically, you learned:

- ▷ What are the libraries available for C++ and Python
- ▷ How the Chainer library influenced the syntax in building a deep learning model
- ▷ The relationship between Keras and TensorFlow 2.x
- ▷ What are the differences between PyTorch and TensorFlow

In the next chapter, you will learn more about the library of choice in this book, PyTorch.

Introduction to PyTorch

PyTorch is an open-source machine learning library developed by Facebook's AI Research team. It is a Python-based library used for building deep neural networks and supports dynamic computation graphs. In the following, you will learn about this amazing library and what it can do at a high level. After finishing this chapter, you will know:

- ▷ What are the main features of PyTorch to make it a deep learning library
- ▷ How to install PyTorch
- ▷ Some basic examples of PyTorch code

Let's get started.

Overview

This chapter is in four parts; they are:

- ▷ PyTorch at a High Level
- ▷ Applications of PyTorch
- ▷ Advantages and Limitations of PyTorch
- ▷ Comparing PyTorch to TensorFlow

2.1 PyTorch at a High Level

PyTorch is a library that provides two important high-level features:

1. Tensor computation (like NumPy), optionally with acceleration via graphics processing units (GPU).
2. Deep neural networks built on automatic differentiation

PyTorch is a deep learning library because of these two key features. *Tensor* is a general term for arrays and matrices. Components in a deep learning model are represented as matrices and matrix formulas mathematically. The tensor computation allows numbers in matrices to be manipulated concisely. GPU is sometimes used to speed up the matrix computation. A deep learning model can run (in *inference mode* or *forward pass*) with tensor computation,

but you need automatic differentiation to train the model or run the *backward pass*. These two features together allow us to develop deep learning models in PyTorch.

PyTorch has become very popular among machine learning enthusiasts due to its simplicity and ease of use. It is also gaining popularity in the research community due to its dynamic nature, which allows researchers to change the model architecture on the fly.

Some key features of PyTorch include:

- ▷ A simple and intuitive API that allows developers to quickly prototype their models.
- ▷ Seamless integration with NumPy, making it easy to move data between the two libraries.
- ▷ Dynamic computation graphs that allow for easier debugging and faster experimentation.
- ▷ Built-in support for distributed training across multiple GPUs and machines.
- ▷ A large and active community that provides support and contributes to the development of the library.

In summary, PyTorch is a powerful and easy-to-use deep learning library that is widely used in both academia and industry. Its dynamic nature, ease of use, and strong community support make it an excellent choice for anyone looking to get started with deep learning.

2.2 Getting Started with PyTorch

To get started with PyTorch, you first need to install it. You can do this easily using `pip` by running the following command:

```
pip install torch
```

Listing 2.1: Installing PyTorch using `pip`

If you're using `conda`, the package name is different and the command would be:

```
conda install pytorch
```

Listing 2.2: Installing PyTorch using `conda`

This is the simplest, but you may also need to install CUDA library before your PyTorch installation can run on GPU. Once you have installed PyTorch, you can start experimenting with it. Here is a simple example to let you verify your PyTorch is working:

```
import torch

# create a tensor of size (2,3)
x = torch.tensor([[1, 2, 3], [4, 5, 6]])

# create a tensor of size (3,2)
y = torch.tensor([[1, 2], [3, 4], [5, 6]])
```

```
# matrix multiplication of x and y
z = torch.matmul(x, y)

print(z)
```

Listing 2.3: A simple PyTorch test program

This will output:

```
tensor([[22, 28],\
        [49, 64]])
```

Output 2.1: Output from the test program

This is just a simple example, but it demonstrates how PyTorch performs tensor computation.

2.3 Applications of PyTorch

PyTorch is widely used in various applications of machine learning and deep learning such as:

- ▷ **Computer Vision:** PyTorch is used in computer vision tasks such as image classification, object detection, and segmentation. Some popular computer vision models such as ResNet, YOLO, and Faster R-CNN have a PyTorch implementation.
- ▷ **Natural Language Processing (NLP):** PyTorch is used in NLP tasks such as sentiment analysis, machine translation, and text generation. Some popular NLP models including BERT and Transformer have a PyTorch implementation.
- ▷ **Reinforcement Learning:** PyTorch is also used in reinforcement learning tasks such as game playing and robot control. Some popular reinforcement learning models such as DQN, PPO, and A3C have a PyTorch implementation.
- ▷ **Other Applications:** Apart from the above-mentioned applications, PyTorch is also used in other areas such as speech recognition, time series forecasting, and anomaly detection.

2.4 Advantages and Limitations of PyTorch

PyTorch has several advantages as a deep learning library:

- ▷ **Dynamic Computation Graphs:** PyTorch uses dynamic computation graphs, which means that the graph is generated on the fly as the operations are executed. This allows you to code at high level without worrying how the computer should execute it. In other words, you can experiment faster.
- ▷ **Easy to Use:** PyTorch has a simple and intuitive API that allows developers to quickly prototype their models. It is also easy to install and setup.
- ▷ **GPU Acceleration** PyTorch supports strong acceleration via graphics processing units (GPU), making it faster than some other deep learning libraries.

- ▷ **Active Community:** PyTorch has a large and active community that provides support and contributes to the development of the library. This means that users can quickly get help and access to new features.

PyTorch has some limitations that users should be aware of:

- ▷ **Limited Production Use** PyTorch is mainly used for research and prototyping. It is not yet widely used in production environments.
- ▷ **Steep Learning Curve** Although PyTorch has a simple API, it can still have a steep learning curve, especially for users who are new to deep learning.

2.5 Comparing PyTorch to TensorFlow

PyTorch and TensorFlow are considered rivals. They have different design but similar in functionality.

TensorFlow has a more primitive interface that you can drill down to a lower level than what PyTorch can provide. However, when developing a deep learning model, you would prefer to work at conceptual levels such as layers and optimization algorithms rather than arithmetic of a single value. Therefore, in TensorFlow 2, it is tightly coupled with the high level library Keras. You will find Keras and PyTorch syntax are comparable as both allows you to design a deep learning model at a high level.

However, you will find Keras influenced by the design of an older library scikit-learn that sometimes it hid too much detail. For example, in TensorFlow and Keras, you normally train a model with a function call `fit()` which all the automatic differentiation and gradient update are done implicitly. In PyTorch, you need to write lines of code to clearly specify how exactly these are done. In cases that you are innovative on the details of how a deep learning model is trained, you will find PyTorch more flexible. This is also one of the reasons why PyTorch is popular in research communities.

In detail, there are some subtle design choice that make one library better than another. In Keras, for example, there is a concept of Keras tensor which is distinct from a tensor from TensorFlow that works as a placeholder. This is sometimes confusing to consider whether a tensor should contain value or not. PyTorch doesn't use such placeholder tensors to build a model. But the downside of this is each component in a deep learning model are disconnected. The logic of how a tensor *flows* between components is unclear until the model actually runs. You will see this creates trouble when you want to convert a model into a picture, which Keras has a `plot_model()` function and no PyTorch counterpart exists.

The other aspect that set these two libraries apart are the family of their sister projects. PyTorch has a torchvision library that is specific for computer vision models. You will find a lot of useful image processing routines in the library that make your computer vision project much easier. TensorFlow, on the other hand, has very rich mobile and edge device support. You can easily convert a TensorFlow model into a TensorFlow.js model to run as JavaScript on a browser, or into a TensorFlow Lite model that can run on memory-limited mobile and edge devices. The TensorBoard project that provides a dashboard to visualize various aspects of a model and its training is appreciated by many. It is so helpful that PyTorch even provided built-in support to write data to TensorBoard.

2.6 Further Readings

It is easy to find books on PyTorch, but the most current documentation is always from PyTorch's official website:

PyTorch.

<https://pytorch.org/>

PyTorch documentation.

<https://pytorch.org/docs/stable/index.html>

PyTorch tutorials.

<https://pytorch.org/tutorials/>

2.7 Summary

PyTorch is a powerful and easy-to-use deep learning library that is widely used in both academia and industry. Its dynamic nature, ease of use, and strong community support make it an excellent choice for anyone looking to get started with deep learning. PyTorch is widely used in various applications such as computer vision, NLP, and reinforcement learning. It has several advantages over other deep learning libraries, such as dynamic computation graphs, easy-to-use API, GPU acceleration, and an active community. However, it also has some limitations, such as limited production use and relatively steep learning curve, PyTorch is an excellent choice for anyone looking to get started with deep learning.

In the next chapter, you will learn how to use PyTorch as a tensor library to manipulate an array of numbers.

Manipulating Tensors in PyTorch

PyTorch is a deep-learning library. Just like some other deep learning libraries, it applies operations on numerical arrays called *tensors*. In the simplest terms, tensors are just multidimensional arrays. When we deal with the tensors, some operations are used very often. In PyTorch, there are some functions defined specifically for dealing with tensors.

In the following, we will give a brief overview of what PyTorch provides on tensors and how we can use them. After finishing this chapter, you will know:

- ▷ How to create and operate on PyTorch tensors
- ▷ PyTorch's tensor syntax is similar to NumPy
- ▷ The common functions you can use from PyTorch to manipulate a tensor

Let's get started.

Overview

This chapter is in four parts; they are:

- ▷ Creating Tensors
- ▷ Checking a Tensor
- ▷ Manipulating Tensors
- ▷ Tensor Functions

3.1 Creating Tensors

If you're familiar with NumPy, you should recall that there are multiple ways of creating an array. The same is true in PyTorch for creating tensors. The simplest way to create a specific constant matrix like the following:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

is by using:

```
import torch
a = torch.tensor([[1,2,3], [4,5,6]], dtype=torch.int32)
print(a)
```

Listing 3.1: Creating a tensor in PyTorch

It prints:

```
tensor([[1, 2, 3],
        [4, 5, 6]], dtype=torch.int32)
```

Output 3.1: A PyTorch tensor

The `dtype` argument specifies the data type of the values in the tensor. It is optional. You can also provide the values from a NumPy array and convert it to a PyTorch tensor.

Usually, you would create a tensor for some specific purpose. For example, if you want to have ten values evenly distributed between -1 and 1, you can use the `linspace()` function:

```
import torch
a = torch.linspace(-1, 1, 10)
print(a)
```

Listing 3.2: Creating a tensor in PyTorch

It prints:

```
tensor([-1.0000, -0.7778, -0.5556, -0.3333, -0.1111,  0.1111,  0.3333,  0.5556,
         0.7778,  1.0000])
```

Output 3.2: Tensor as produced by `linspace()`

However, if you want to have a tensor of random values (which is very useful in testing your functions), you can make one like the following:

```
import torch
a = torch.rand(3,4)
print(a)
```

Listing 3.3: Creating a random tensor

It prints, for example:

```
tensor([[0.4252, 0.1029, 0.9858, 0.7502],
        [0.1993, 0.6412, 0.2424, 0.6451],
        [0.7878, 0.7615, 0.9170, 0.8534]])
```

Output 3.3: A random tensor of shape 3×4

This resulting tensor is of dimension 3×4 , and each value is uniformly distributed between 0 and 1. If you want to have the values normally distributed, just change the function to `randn()`:

```
a = torch.randn(3,4)
```

Listing 3.4: Creating a tensor of normally distributed random values

If you want to have the random values be an integer, e.g., between 3 to 10, you can use the `randint()` function:

```
import torch
a = torch.randint(3, 10, size=(3,4))
print(a)
```

Listing 3.5: Creating a tensor of random integers

This will give, for example:

```
tensor([[4, 5, 7, 9],
        [3, 8, 8, 9],
        [4, 7, 7, 6]])
```

Output 3.4: A tensor of random integers

The values are in the range $3 \leq x < 10$. By default, the lower bound is zero, so if you want the values to be $0 \leq x < 10$, you can use:

```
a = torch.randint(10, size=(3,4))
```

Listing 3.6: Creating a tensor of random integers

The other commonly used tensors are the zero tensor and tensors with all values the same. To create a zero tensor (e.g., of dimension $2 \times 3 \times 4$), you can use:

```
import torch
a = torch.zeros(2, 3, 4)
print(a)
```

Listing 3.7: Creating a zero tensor

It prints:

```
tensor([[[0., 0., 0., 0.],
         [0., 0., 0., 0.],
         [0., 0., 0., 0.]],
        [[0., 0., 0., 0.],
         [0., 0., 0., 0.],
         [0., 0., 0., 0.]])
```

Output 3.5: A zero tensor

And to create a tensor of all values are 5, you can use:

```
import torch
a = torch.full((2,3,4), 5)
print(a)
```

Listing 3.8: Creating a full tensor

It prints:

```
tensor([[[[5, 5, 5, 5],
          [5, 5, 5, 5],
          [5, 5, 5, 5]],

        [[5, 5, 5, 5],
          [5, 5, 5, 5],
          [5, 5, 5, 5]]]])
```

Output 3.6: A full tensor is a tensor of identical values

But if you want all values to be one, there is a simpler function:

```
a = torch.ones(2,3,4)
```

Listing 3.9: Creating a tensor of 1's

Finally, if you want an identity matrix, you can get it with `diag()` or `eye()`:

```
import torch
a = torch.eye(4)
print(a)
```

Listing 3.10: Creating an identity tensor

It prints:

```
tensor([[1., 0., 0., 0.],
        [0., 1., 0., 0.],
        [0., 0., 1., 0.],
        [0., 0., 0., 1.]])
```

Output 3.7: An identity matrix

3.2 Checking a Tensor

Once you have a tensor and you want to know more about it, you can simply print it to the screen using `print()`. But if the tensor is too big, it is easier to show its dimension by checking its shape:

```
import torch
a = torch.zeros(2, 3, 4)
print(a.shape)
print(a.size())
```

Listing 3.11: Checking tensor shape

It prints:

```
torch.Size([2, 3, 4])
torch.Size([2, 3, 4])
```

Output 3.8: Tensor shape

The shape of a tensor can be accessed using the `shape` property or the `size()` function. If you want to see how many dimensions you have (i.e., $2 \times 3 \times 4$ is 3 and 3×4 is 2), you can read the `ndim` property:

```
print(a.ndim)
```

Listing 3.12: Find the number of dimensions of a tensor

This will give you “3”. If you use `len()` to check a tensor, it will only give you the size of the first dimension, e.g.:

```
import torch
a = torch.zeros(2, 3, 4)
print(len(a))
```

Listing 3.13: Find the size of the first dimension

It prints:

```
2
```

Output 3.9: Size of the first dimension

Another property that you want to learn about a tensor is its data type. Usually, you use floating points in deep learning, but sometimes, the tensors should be in integers (e.g., in an image as pixel values). To check the data type, you can read the `dtype` property:

```
print(a.dtype)
```

Listing 3.14: Find the data type of a tensor

It prints:

```
torch.float32
```

Output 3.10: The data type of a tensor

If you want to change the data type, you can recreate the tensor with a new type:

```
import torch
a = torch.zeros(2, 3, 4)
b = a.type(torch.int32)
print(a.dtype)
print(b.dtype)
```

Listing 3.15: Changing the data type of a tensor is to create another one

The above prints:

```
torch.float32
torch.int32
```

Output 3.11: Data type of the tensors as printed

3.3 Manipulating Tensors

One common operation on tensors in deep learning is to change the tensor shape. For example, you may want to convert a 2D tensor into 1D or add a dummy dimension to a tensor. You may also want to extract a sub-tensor from a larger tensor.

For example, you can create a tensor like the following:

```
a = torch.randn(3,4,5)
print(a)
```

Listing 3.16: Creating a random tensor

If you get:

```
tensor([[[[-1.1271e-01, -7.1124e-01,  1.1335e+00, -8.5644e-01, -1.4191e+00],
          [-1.9065e+00, -6.8386e-02,  5.8727e-01,  6.5890e-03, -2.6947e-01],
          [ 6.3194e-01, -7.7426e-01,  1.6546e+00,  1.2647e-01, -1.0944e+00],
          [ 3.7819e-01, -8.8670e-01,  5.3772e-01,  1.4985e+00,  5.8396e-01]],

        [[ 1.8704e+00,  2.0914e+00, -9.1604e-01,  1.2317e+00, -1.5722e-01],
          [ 2.4689e-01, -2.3157e-01, -3.3033e-01,  1.4021e+00, -6.9540e-01],
          [ 3.0298e-01, -1.4936e-01, -6.8863e-01,  1.6977e-01,  2.4682e+00],
          [-8.1375e-01,  4.8489e-01, -1.2024e+00, -4.9771e-01,  1.1728e-01]],

        [[-1.6011e+00,  1.5686e-03, -1.0560e-01, -1.2938e+00,  5.3077e-01],
          [-9.7636e-01, -9.1854e-01, -1.0002e+00,  1.1852e+00,  1.0328e+00],
          [ 9.6664e-01,  5.3752e-01, -3.1340e-02, -6.7852e-02, -7.2317e-01],
          [-5.5263e-01,  9.4754e-01, -5.4503e-01,  6.3850e-02,  1.2879e+00]]])
```

Output 3.12: A random tensor

It allows you to take a slice using the same syntax as in NumPy:

```
print(a[1])
```

Listing 3.17: Slicing a tensor

This will be:

```
tensor([[ 1.8704,  2.0914, -0.9160,  1.2317, -0.1572],
        [ 0.2469, -0.2316, -0.3303,  1.4021, -0.6954],
        [ 0.3030, -0.1494, -0.6886,  0.1698,  2.4682],
        [-0.8137,  0.4849, -1.2024, -0.4977,  0.1173]])
```

Output 3.13: A sliced tensor

Or if you use:

```
print(a[1:, 2:4])
```

Listing 3.18: Slicing a tensor in two dimensions

It will be:


```
tensor([[[ 0.3030, -0.1494, -0.6886,  0.1698,  2.4682],
         [-0.8137,  0.4849, -1.2024, -0.4977,  0.1173]],

        [[ 0.9666,  0.5375, -0.0313, -0.0679, -0.7232],
         [-0.5526,  0.9475, -0.5450,  0.0638,  1.2879]]])
```

Output 3.14: A sliced tensor

You can also make use of the same slicing syntax to *add* a new dimension. For example,

```
print(a[:, None, :, None].shape)
```

Listing 3.19: Adding a new dimension to a tensor with `None`

You will see:

```
torch.Size([3, 1, 4, 1, 5])
```

Output 3.15: A tensor with more dimensions added

Here you use `None` to insert a new dimension at a specific place. This is useful if, for example, you need to convert an image into a batch of only one image. If you're familiar with NumPy, you may recall there is a function `expand_dims()` for this purpose, but PyTorch doesn't provide it. A similar function is `unsqueeze()`, which is demonstrated below:

```
b = torch.unsqueeze(a, dim=2)
print(a.shape)
print(b.shape)
```

Listing 3.20: Adding a new dimension using `unsqueeze()`

This prints:

```
torch.Size([3, 4, 5])
torch.Size([3, 4, 1, 5])
```

Output 3.16: Tensor shape before and after adding a dimension

One powerful nature of NumPy slicing syntax is Boolean indexing. This is also supported with PyTorch tensors. For example:

```
import torch
a = torch.randn(3,4)
print(a)
print(a[:, (a > -1).all(axis=0)])
```

Listing 3.21: Boolean indexing on a tensor

You may see:

```
tensor([[ 1.2548,  0.4078,  0.5548, -0.7016],
        [-0.3720, -0.5682, -0.3430,  0.0886],
        [ 0.2151,  0.3626, -2.0275,  1.8121]])
tensor([[ 1.2548,  0.4078, -0.7016],
```

```
[-0.3720, -0.5682,  0.0886],
[ 0.2151,  0.3626,  1.8121]])
```

Output 3.17: Result of boolean indexing

The above selects the columns where all elements are greater than -1 . You can also manipulate the tensor by selecting specific columns:

```
print(a[:, [1,0,0,1]])
```

Listing 3.22: Creating a tensor by indexing on a dimension

This results in:

```
tensor([[ 0.4078,  1.2548,  1.2548,  0.4078],
        [-0.5682, -0.3720, -0.3720, -0.5682],
        [ 0.3626,  0.2151,  0.2151,  0.3626]])
```

Output 3.18: A tensor created by selecting columns from another tensor

To convert a 2D tensor into 1D, you can use:

```
import torch
a = torch.randn(3,4)
print(a)
print(a.ravel())
```

Listing 3.23: Flattening a 2D tensor to 1D

The result will be:

```
tensor([[ -0.2718, -0.8309,  0.6263, -0.2499],
        [-0.1780,  1.1735, -1.3530, -1.2374],
        [-0.6050, -1.5524, -0.1008, -1.2782]])
tensor([ -0.2718, -0.8309,  0.6263, -0.2499, -0.1780,  1.1735, -1.3530, -1.2374,
        -0.6050, -1.5524, -0.1008, -1.2782])
```

Output 3.19: The 2D tensor and its 1D equivalent

You may also use the `reshape()` function to achieve the same:

```
print(a.reshape(-1))
```

Listing 3.24: Reshape a tensor

The result should be the same as that of `ravel()`. But usually, the `reshape()` function is for more complicated target shapes:

```
print(a.reshape(3,2,2))
```

Listing 3.25: Reshape a tensor

This will print:

```
tensor([[[[-0.2718, -0.8309],
          [ 0.6263, -0.2499]],

        [[-0.1780,  1.1735],
          [-1.3530, -1.2374]],

        [[-0.6050, -1.5524],
          [-0.1008, -1.2782]]]])
```

Output 3.20: The tensor as reshaped

One common case of reshaping tensors is to do matrix transpose. For a 2D matrix, it is easily done in the same way as NumPy:

```
print(a.T)
```

Listing 3.26: Transpose a tensor

which prints:

```
tensor([[[-0.2718, -0.1780, -0.6050],
        [-0.8309,  1.1735, -1.5524],
        [ 0.6263, -1.3530, -0.1008],
        [-0.2499, -1.2374, -1.2782]])
```

Output 3.21: A tensor transposed

But the `transpose()` function in PyTorch, unlike NumPy, requires you to explicitly specify the two axes to swap:

```
print(a.transpose(0, 1))
```

Listing 3.27: Transpose a tensor as swapping axes

This result is same as above. If you have multiple tensors, you can combine them by stacking them (`vstack()` for vertically and `hstack()` for horizontally). For example:

```
import torch
a = torch.randn(3,3)
b = torch.randn(3,3)
print(a)
print(b)
print(torch.vstack([a,b]))
```

Listing 3.28: Stacking tensors

This may print:

```
tensor([[ 1.1739,  1.3546, -0.2886],
        [ 1.0444,  0.4437, -2.7933],
        [ 0.6805,  0.8401, -1.2527]])
tensor([[ 1.6273,  1.2622, -0.4362],
        [-1.6529,  0.6457, -0.1454],
        [-2.0960, -1.3024, -0.1033]])
```

```
tensor([[ 1.1739,  1.3546, -0.2886],
        [ 1.0444,  0.4437, -2.7933],
        [ 0.6805,  0.8401, -1.2527],
        [ 1.6273,  1.2622, -0.4362],
        [-1.6529,  0.6457, -0.1454],
        [-2.0960, -1.3024, -0.1033]])
```

Output 3.22: Stacked tensors

The concatenate function is similar:

```
c = torch.concatenate([a, b])
print(c)
```

Listing 3.29: Creating a new tensor by concatenating

You will get the same tensor:

```
tensor([[ 1.1739,  1.3546, -0.2886],
        [ 1.0444,  0.4437, -2.7933],
        [ 0.6805,  0.8401, -1.2527],
        [ 1.6273,  1.2622, -0.4362],
        [-1.6529,  0.6457, -0.1454],
        [-2.0960, -1.3024, -0.1033]])
```

Output 3.23: Tensor created by `concatenate()` is same as `vstack()`

The reverse is to split, e.g.,

```
print(torch.vsplit(c, 2))
```

Listing 3.30: Splitting a tensor into two

It prints

```
(tensor([[ 1.1739,  1.3546, -0.2886],
        [ 1.0444,  0.4437, -2.7933],
        [ 0.6805,  0.8401, -1.2527]]), tensor([[ 1.6273,  1.2622, -0.4362],
        [-1.6529,  0.6457, -0.1454],
        [-2.0960, -1.3024, -0.1033]]))
```

Output 3.24: Two tensors as the result of splitting

This function tells how many tensors to split into, rather than what size each tensor is. The latter is indeed more useful in deep learning (e.g., to split a tensor of a large dataset into many tensors of small batches). The equivalent function would be:

```
print(torch.split(c, 3, dim=0))
```

Listing 3.31: Splitting a tensor on a particular dimension

This should give you the same result as before. So `split(c, 3, dim=0)` means to split on dimension 0 such that each resulting tensor will be of size 3.

3.4 Tensor Functions

PyTorch tensors can be treated as arrays. So you can often use it in a similar way as NumPy arrays. For example, you have the functions of common mathematical functions:

```
import torch
a = torch.randn(2,3)
print(a)
print("exp =", torch.exp(a))
print("log =", torch.log(a))
print("sin =", torch.sin(a))
print("arctan =", torch.arctan(a))
print("abs =", torch.abs(a))
print("square =", torch.square(a))
print("sqrt =", torch.sqrt(a))
print("ceil =", torch.ceil(a))
print("round =", torch.round(a))
print("clip =", torch.clip(a, 0.1, 0.9))
```

Listing 3.32: Some tensor functions in PyTorch

This prints:

```
tensor([[ 1.2001,  1.0952,  1.1952],
        [ 0.1955, -2.8078,  0.3427]])
exp = tensor([[3.3206, 2.9897, 3.3042],
             [1.2159, 0.0603, 1.4087]])
log = tensor([[ 0.1824,  0.0909,  0.1783],
             [-1.6323,   nan, -1.0709]])
sin = tensor([[ 0.9321,  0.8890,  0.9303],
             [ 0.1942, -0.3277,  0.3360]])
arctan = tensor([[ 0.8761,  0.8308,  0.8741],
                [ 0.1930, -1.2286,  0.3302]])
abs = tensor([[1.2001, 1.0952, 1.1952],
             [0.1955, 2.8078, 0.3427]])
square = tensor([[1.4403, 1.1994, 1.4285],
                [0.0382, 7.8835, 0.1174]])
sqrt = tensor([[1.0955, 1.0465, 1.0932],
              [0.4421,   nan, 0.5854]])
ceil = tensor([[ 2.,  2.,  2.],
              [ 1., -2.,  1.]])
round = tensor([[ 1.,  1.,  1.],
               [ 0., -3.,  0.]])
clip = tensor([[0.9000, 0.9000, 0.9000],
              [0.1955, 0.1000, 0.3427]])
```

Output 3.25: Result of the tensor functions

Note that if a function is undefined (e.g., square root of negative numbers), `nan` will be the result, but no exception will be raised. In PyTorch, you have a function to check if the values of a tensor are `nan`:

```
import torch
b = torch.sqrt(a)
print(b)
print(torch.isnan(b))
```

Listing 3.33: Finding which value of a tensor is nan

You will get:

```
tensor([[1.0280,  nan,  nan],
        [ nan, 1.1473,  nan]])
tensor([[False,  True,  True],
        [ True, False,  True]])
```

Output 3.26: A Boolean tensor indicating if a value in a tensor is nan

Indeed, besides these defined functions, the Python operators can be applied to the tensors too:

```
import torch
a = torch.randn(2, 3)
b = torch.randn(2, 3)
print(a)
print(b)
print(a+b)
print(a/b)
print(a ** 2)
```

Listing 3.34: Tensor operations

You get:

```
tensor([[ 0.7378, -0.3469,  1.3089],
        [-1.9152,  0.3745, -0.7248]])
tensor([[ -0.3650, -0.4768,  0.9331],
        [ 0.5095,  1.7169, -0.5463]])
tensor([[ 0.3729, -0.8237,  2.2421],
        [-1.4058,  2.0914, -1.2711]])
tensor([[ -2.0216,  0.7275,  1.4027],
        [-3.7594,  0.2181,  1.3269]])
tensor([[0.5444, 0.1203, 1.7133],
        [3.6682, 0.1403, 0.5254]])
```

Output 3.27: Result of tensor operations

But among the operators, matrix multiplications are very important in deep learning. You can do this with:

```
print(torch.matmul(a, b.T))
print(a @ b.T)
```

Listing 3.35: Matrix multiplications

This prints

```
tensor([[ 1.1176, -0.9347],
        [-0.1560,  0.0632]])
tensor([[ 1.1176, -0.9347],
        [-0.1560,  0.0632]])
```

Output 3.28: Result of matrix multiplication

These two are the same. Indeed, the @ operator from Python can also be used for vector dot-product, e.g.:

```
import torch
a = torch.randn(3)
b = torch.randn(3)
print(a)
print(b)
print(torch.dot(a, b))
print(a @ b)
```

Listing 3.36: Computing dot-product in PyTorch

It prints:

```
tensor([-0.8986, -0.6994,  1.1443])
tensor([-1.0666,  0.1455,  0.1322])
tensor(1.0081)
tensor(1.0081)
```

Output 3.29: Result of dot-product

If you treat the values in a tensor as samples, you may also want to find some statistics about it. Some are provided in PyTorch too:

```
import torch
a = torch.randn(3,4)
print(a)
print(torch.mean(a, dim=0))
print(torch.std(a, dim=0))
print(torch.cumsum(a, dim=0))
print(torch.cumprod(a, dim=0))
```

Listing 3.37: Basic statistics on a tensor

It prints:

```
tensor([[ 0.3331, -0.0190,  0.4814, -1.1484],
        [-0.5712,  0.8430, -1.6147, -1.1664],
        [ 1.7298, -1.7665, -0.5918,  0.3024]])
tensor([ 0.4972, -0.3142, -0.5750, -0.6708])
tensor([1.1593, 1.3295, 1.0482, 0.8429])
tensor([[ 0.3331, -0.0190,  0.4814, -1.1484],
        [-0.2381,  0.8240, -1.1333, -2.3148],
        [ 1.4917, -0.9425, -1.7251, -2.0124]])
tensor([[ 0.3331, -0.0190,  0.4814, -1.1484],
```

```
[-0.1903, -0.0160, -0.7774,  1.3395],
[-0.3291,  0.0283,  0.4601,  0.4051]])
```

Output 3.30: Result of some basic statistics on a tensor

But for linear algebra functions, you should find it in PyTorch's `linalg` submodule. For example:

```
print(torch.linalg.svd(a))
```

Listing 3.38: Using the `linalg` submodule

You will see:

```
torch.return_types.linalg_svd(
U=tensor([[ -0.0353,  0.1313,  0.9907],
          [-0.5576,  0.8201, -0.1286],
          [ 0.8294,  0.5569, -0.0443]]),
S=tensor([2.7956, 1.9465, 1.2715]),
Vh=tensor([[ 0.6229, -0.6919,  0.1404,  0.3369],
           [ 0.2767, -0.1515, -0.8172, -0.4824],
           [ 0.2570, -0.0385,  0.5590, -0.7874],
           [ 0.6851,  0.7048, -0.0073,  0.1840]]))
```

Output 3.31: Result of singular value decomposition on a tensor

And specifically for convolution neural networks, padding a tensor is done with the following:

```
b = torch.nn.functional.pad(a, (1,1,0,2), value=0)
print(b)
```

Listing 3.39: Padding a tensor with a value

This prints:

```
tensor([[ 0.0000,  0.3331, -0.0190,  0.4814, -1.1484,  0.0000],
        [ 0.0000, -0.5712,  0.8430, -1.6147, -1.1664,  0.0000],
        [ 0.0000,  1.7298, -1.7665, -0.5918,  0.3024,  0.0000],
        [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000],
        [ 0.0000,  0.0000,  0.0000,  0.0000,  0.0000,  0.0000]])
```

Output 3.32: A padded tensor

This example of the `pad()` function is to create (1,1) padding on dimension 0 and (0,2) on dimension 1. In other words, for each dimension 0 (rows), we add one dummy value (0) at the beginning and the end. For each dimension 1 (columns), we add zero dummy values at the beginning but two dummy values at the end.

Finally, since PyTorch tensors can be considered arrays, you can use them directly with other tools such as `matplotlib`. Below is an example of plotting a surface using PyTorch tensors:


```

import matplotlib.pyplot as plt
import torch

# create tensors
x = torch.linspace(-1, 1, 100)
y = torch.linspace(-2, 2, 100)
# create the surface
xx, yy = torch.meshgrid(x, y, indexing="xy") # xy-indexing is matching numpy
z = torch.sqrt(1 - xx**2 - (yy/2)**2)
print(xx)

fig = plt.figure(figsize=(8,8))
ax = plt.axes(projection="3d")
ax.set_xlim([-2, 2])
ax.set_ylim([-2, 2])
ax.set_zlim([0, 2])
ax.plot_surface(xx, yy, z, cmap="cividis")
ax.view_init(45, 35)
plt.show()

```

Listing 3.40: Plotting a surface using matplotlib from a PyTorch tensor

The mesh grid produced the tensor `xx` as:

```

tensor([[ -1.0000, -0.9798, -0.9596, ...,  0.9596,  0.9798,  1.0000],
        [ -1.0000, -0.9798, -0.9596, ...,  0.9596,  0.9798,  1.0000],
        [ -1.0000, -0.9798, -0.9596, ...,  0.9596,  0.9798,  1.0000],
        ...,
        [ -1.0000, -0.9798, -0.9596, ...,  0.9596,  0.9798,  1.0000],
        [ -1.0000, -0.9798, -0.9596, ...,  0.9596,  0.9798,  1.0000],
        [ -1.0000, -0.9798, -0.9596, ...,  0.9596,  0.9798,  1.0000]])

```

Output 3.33: Result of `meshgrid()` function

Which values are identical along the second dimension. Similarly for the tensor `yy`, except the repetition is on the other dimension. The plot created is:

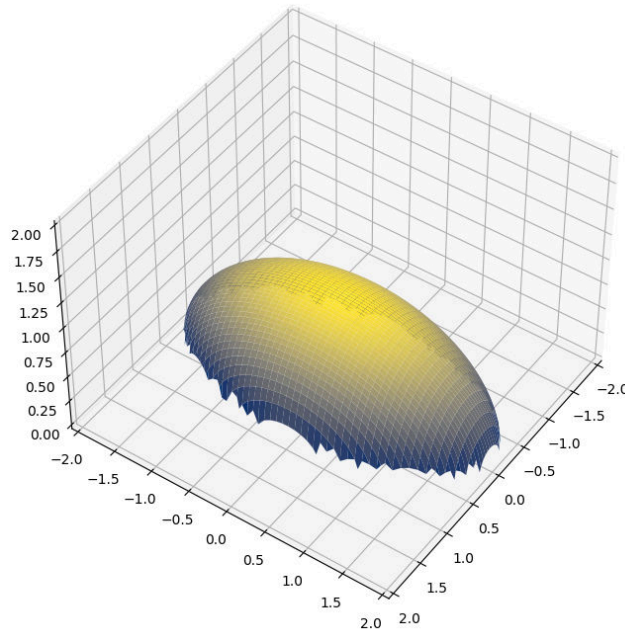


Figure 3.1: Surface plot from matplotlib

3.5 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

Articles

Tensors. PyTorch tutorials.

https://pytorch.org/tutorials/beginner/basics/tensorqs_tutorial.html

Introduction to PyTorch Tensors. PyTorch tutorials.

https://pytorch.org/tutorials/beginner/introyt/tensors_deeper_tutorial.html

3.6 Summary

In this chapter, you discovered how to manipulate PyTorch tensors. Specifically, you learned:

- ▷ What is a tensor
- ▷ How to create various kinds of tensors in PyTorch
- ▷ How to reshape, slice, and manipulate tensors in PyTorch
- ▷ The common functions that can be applied to PyTorch tensors

In the next chapter, you will learn how to use the automatic differentiation feature in PyTorch to solve some simple problems.

Using Autograd in PyTorch to Solve a Regression Problem

We usually use PyTorch to build a neural network. However, PyTorch can do more than this. Because PyTorch is also a tensor library with automatic differentiation capability, you can easily use it to solve a numerical optimization problem with gradient descent. In this chapter, you will learn how PyTorch's automatic differentiation engine, autograd, works.

After finishing this chapter, you will learn:

- ▷ What is autograd in PyTorch
- ▷ How to make use of autograd and an optimizer to solve an optimization problem

Let's get started.

Overview

This chapter is in three parts; they are:

- ▷ Autograd in PyTorch
- ▷ Using Autograd for Polynomial Regression
- ▷ Using Autograd to Solve a Math Puzzle

4.1 Autograd in PyTorch

In PyTorch, you can create tensors as variables or constants and build an expression with them. The expression is essentially a function of the variable tensors. Therefore, you may derive its derivative function, i.e., the differentiation or the gradient. This is the foundation of the training loop in a deep learning model. PyTorch comes with this feature at its core.

It is easier to explain autograd with an example. In PyTorch, you can create a constant matrix as follows:

```
import torch

x = torch.tensor([1, 2, 3])
print(x)
```

```
print(x.shape)
print(x.dtype)
```

Listing 4.1: Setting up a constant matrix in PyTorch

The above prints:

```
tensor([1, 2, 3])
torch.Size([3])
torch.int64
```

Output 4.1: The constant tensor created

This creates an integer vector (in the form of a PyTorch tensor). This vector can work like a NumPy vector in most cases. For example, you can do `x+x` or `2*x`, and the result is just what you would expect. PyTorch comes with many functions for array manipulation that match NumPy, such as `torch.transpose` or `torch.concatenate`.

But this tensor is not assumed to be a variable for a function in the sense that differentiation with it is not supported. You can create tensors that work like a variable with an extra option:

```
import torch

x = torch.tensor([1., 2., 3.], requires_grad=True)
print(x)
print(x.shape)
print(x.dtype)
```

Listing 4.2: Setting up a tensor to support differentiation

This will print:

```
tensor([1., 2., 3.], requires_grad=True)
torch.Size([3])
torch.float32
```

Output 4.2: A tensor to support differentiation

Note that, in the above, a tensor of floating point values was created. It is required because differentiation requires floating points, not integers.

The operations (such as `x+x` and `2*x`) can still be applied, but in this case, the tensor will remember how it got its values. You can demonstrate this feature in the following:

```
import torch

x = torch.tensor(3.6, requires_grad=True)
y = x * x
y.backward()
print("x =", x)
print("y =", y)
print("x.grad =", x.grad)
```

Listing 4.3: Tensor operations

This prints:

```
x = tensor(3.6000, requires_grad=True)
y = tensor(12.9600, grad_fn=<MulBackward0>)
x.grad = tensor(7.2000)
```

Output 4.3: Tensor operation

What it does is the following: This defined a variable x (with value 3.6) and then computed $y=x*x$ or $y = x^2$. Then you ask for the differentiation of y . Since y obtained its value from x , you can find the derivative $\frac{dy}{dx}$ at $x.grad$, in the form of a tensor, immediately after you run `y.backward()`. You know $y = x^2$ means $y' = 2x$. Hence the output would give you a value of $3.6 \times 2 = 7.2$.

4.2 Using Autograd for Polynomial Regression

How is this feature in PyTorch helpful? Let's consider a case where you have a polynomial in the form of $y = f(x)$, and you are given several (x, y) samples. How can you recover the polynomial $f(x)$? One way is to assume a random coefficient for the polynomial and feed in the samples (x, y) . If the polynomial is found, you should see the value of y matches $f(x)$. The closer they are, the closer your estimate is to the correct polynomial.

This is indeed a numerical optimization problem where you want to minimize the difference between y and $f(x)$. You can use gradient descent to solve it.

Let's consider an example. You can build a polynomial $f(x) = x^2 + 2x + 3$ in NumPy as follows:

```
import numpy as np

polynomial = np.poly1d([1, 2, 3])
print(polynomial)
```

Listing 4.4: Creating a polynomial in NumPy

This prints:

```
2
1 x + 2 x + 3
```

Output 4.4: A polynomial

You may use the polynomial as a function, such as:

```
print(polynomial(1.5))
```

Listing 4.5: Using a NumPy polynomial as a function

And this prints 8.25, for $(1.5)^2 + 2 \times (1.5) + 3 = 8.25$.

Now you can generate a number of samples from this function using NumPy:

```
N = 20    # number of samples

# Generate random samples roughly between -10 to +10
X = np.random.randn(N,1) * 5
Y = polynomial(X)
```

Listing 4.6: Generating samples from a polynomial

In the above, both X and Y are NumPy arrays of the shape $(20,1)$, and they are related as $y = f(x)$ for the polynomial $f(x)$.

Now, assume you do not know what the polynomial is except it is quadratic. And you want to recover the coefficients. Since a quadratic polynomial is in the form of $Ax^2 + Bx + C$, you have three unknowns to find. You can find them using the gradient descent algorithm you implement or an existing gradient descent optimizer. The following demonstrates how it works:

```
import torch

# Assume samples X and Y are prepared elsewhere

XX = np.hstack([X*X, X, np.ones_like(X)])

w = torch.randn(3, 1, requires_grad=True) # the 3 coefficients
x = torch.tensor(XX, dtype=torch.float32) # input sample
y = torch.tensor(Y, dtype=torch.float32)  # output sample
optimizer = torch.optim.NAdam([w], lr=0.01)
print(w)

for _ in range(1000):
    y_pred = x @ w
    mse = torch.mean(torch.square(y - y_pred))
    optimizer.zero_grad()
    mse.backward()
    optimizer.step()

print(w)
```

Listing 4.7: Gradient descent in PyTorch

The `print` statement before the `for` loop gives three random numbers, such as:

```
tensor([[1.3827],
        [0.8629],
        [0.2357]], requires_grad=True)
```

Output 4.5: The three random numbers from the initialized tensor

But the one after the `for` loop gives you the coefficients very close to that in the polynomial:

```
tensor([[1.0004],
        [1.9924],
        [2.9159]], requires_grad=True)
```

Output 4.6: The solution produced by gradient descent

What the above code does is the following: First, it creates a variable vector w of 3 values, namely the coefficients A, B, C . Then you create an array of shape $(N, 3)$, in which N is the number of samples in the array X . This array has 3 columns: the values of x^2 , x , and 1, respectively. Such an array is built from the vector X using the `np.hstack()` function. Similarly, you build the tensor y from the NumPy array Y .

Afterward, you use a for loop to run the gradient descent in 1,000 iterations. In each iteration, you compute $x \times w$ in matrix form to find $Ax^2 + Bx + C$ and assign it to the variable `y_pred`. Then, compare y and `y_pred` and find the mean square error. Next, derive the gradient, i.e., the rate of change of the mean square error with respect to the coefficients w using the `backward()` function. And based on this gradient, you use gradient descent to update w via the optimizer.

In essence, the above code will find the coefficients w that minimizes the mean square error.

Putting everything together, the following is the complete code:

```
import numpy as np
import torch

polynomial = np.poly1d([1, 2, 3])
N = 20 # number of samples

# Generate random samples roughly between -10 to +10
X = np.random.randn(N,1) * 5
Y = polynomial(X)

# Prepare input as an array of shape (N,3)
XX = np.hstack([X*X, X, np.ones_like(X)])

# Prepare tensors
w = torch.randn(3, 1, requires_grad=True) # the 3 coefficients
x = torch.tensor(XX, dtype=torch.float32) # input sample
y = torch.tensor(Y, dtype=torch.float32) # output sample
optimizer = torch.optim.NAdam([w], lr=0.01)
print(w)

# Run optimizer
for _ in range(1000):
    optimizer.zero_grad()
    y_pred = x @ w
    mse = torch.mean(torch.square(y - y_pred))
    mse.backward()
    optimizer.step()

print(w)
```

Listing 4.8: Using gradient descent and automatic differentiation to fit a polynomial

4.3 Using Autograd to Solve a Math Puzzle

In the above, 20 samples were used, which is more than enough to fit a quadratic equation. You may use gradient descent to solve some math puzzles as well. For example, the following problem:

$$\begin{array}{rcl} A & + & B = 9 \\ + & & - \\ C & - & D = 1 \\ = & & = \\ 8 & & 2 \end{array}$$

In other words, to find the values of A, B, C, D such that:

$$A + B = 9$$

$$C - D = 1$$

$$A + C = 8$$

$$B - D = 2$$

This can also be solved using autograd, as follows:

```
import random
import torch

A = torch.tensor(random.random(), requires_grad=True)
B = torch.tensor(random.random(), requires_grad=True)
C = torch.tensor(random.random(), requires_grad=True)
D = torch.tensor(random.random(), requires_grad=True)

# Gradient descent loop
EPOCHS = 2000
optimizer = torch.optim.NAdam([A, B, C, D], lr=0.01)
for _ in range(EPOCHS):
    y1 = A + B - 9
    y2 = C - D - 1
    y3 = A + C - 8
    y4 = B - D - 2
    sqerr = y1*y1 + y2*y2 + y3*y3 + y4*y4
    optimizer.zero_grad()
    sqerr.backward()
    optimizer.step()

print(A)
print(B)
print(C)
print(D)
```

Listing 4.9: Solving a math puzzle with autograd in PyTorch

There can be multiple solutions to this problem. One solution is the following:


```
tensor(4.7191, requires_grad=True)
tensor(4.2808, requires_grad=True)
tensor(3.2808, requires_grad=True)
tensor(2.2808, requires_grad=True)
```

Output 4.7: Solution to the math puzzle

Which means $A = 4.72$, $B = 4.28$, $C = 3.28$, and $D = 2.28$. You can verify this solution fits the problem.

The above code defines the four unknowns as variables with a random initial value. Then you compute the result of the four equations and compare it to the expected answer. You then sum up the squared error and ask PyTorch's optimizer to minimize it. The minimum possible square error is zero, attained when our solution exactly fits the problem.

Note the way PyTorch produces the gradient: You ask for the gradient of `sqerr`, which it noticed that, among other things, only `A`, `B`, `C`, and `D` are its dependencies that `requires_grad=True`. Hence four gradients are found. You then apply each gradient to the respective variables in each iteration via the optimizer.

4.4 Further Reading

This section provides more resources on the topic if you are looking to go deeper.

Articles

Autograd mechanics. PyTorch documentation.

<https://pytorch.org/docs/stable/notes/autograd.html>

Automatic differentiation package — torch.autograd. PyTorch API.

<https://pytorch.org/docs/stable/autograd.html>

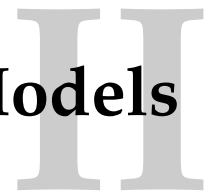
4.5 Summary

In this chapter, we demonstrated how PyTorch's automatic differentiation works. This is the building block for carrying out deep learning training. Specifically, you learned:

- ▷ What is automatic differentiation in PyTorch
- ▷ How you can use gradient tape to carry out automatic differentiation
- ▷ How you can use automatic differentiation to solve an optimization problem

Starting the next chapter, you will learn about the major application of PyTorch — the deep learning models.

Multilayer Perceptron Models



A Crash Course to Deep Learning

Artificial neural networks form a fascinating area of study, although they can be intimidating when you are just getting started. There are a lot of specialized terms in use to describe the data structures and algorithms used in the field. In this chapter, you will get a crash course in the terminology and processes used in the field of multilayer perceptron artificial neural networks. After completing this chapter, you will know:

- ▷ The building blocks of neural networks, including neurons, weights, and activation functions
- ▷ How the building blocks are used in layers to create networks
- ▷ How networks are trained from example data

Let's get started.

Overview

This chapter is in four parts; they are:

- ▷ From Multilayer Perceptrons to Deep Learning
- ▷ Neurons: Weights and Activations
- ▷ Networks of Neurons
- ▷ Training Networks

5.1 From Multilayer Perceptrons to Deep Learning

The field of artificial neural networks is often just called *neural networks* or *multilayer perceptrons*, as multilayer perceptron is perhaps the most useful type of neural network. A perceptron is a single-neuron model that was a precursor to larger neural networks. It is a field of study that investigates how simple models of biological brains can be used to solve difficult computational tasks like the predictive modeling tasks we see in machine learning. The goal is not to create realistic models of the brain but instead to develop robust algorithms and data structures that we can use to model difficult problems.

The power of neural networks comes from their ability to learn the correlations in your training data and how best to relate it to the output variable you want to predict. In this sense, neural networks learn mapping. Mathematically, they are capable of learning any mapping function and have been proven to be a *universal approximation algorithm*. The predictive capability of neural networks comes from the hierarchical or multilayered structure of the networks. The data structure can pick out (learn to represent) features at different scales or resolutions and combine them into higher-order features, for example, from lines to collections of lines to shapes.

Neural networks are usually constructed by stacking up layers. Traditional neural network models has only three layers: The input, output, and one *hidden* layer in-between. Only recently when computers are powerful enough, we learned how to effectively create a neural network with more layers. Hence the term *deep learning* is coined to emphasize the distinction from the older simple neural network models.

5.2 Neurons: Weights and Activations

The building blocks for neural networks are artificial neurons. These are simple computational units that have weighted input signals and produce an output signal using an activation function.

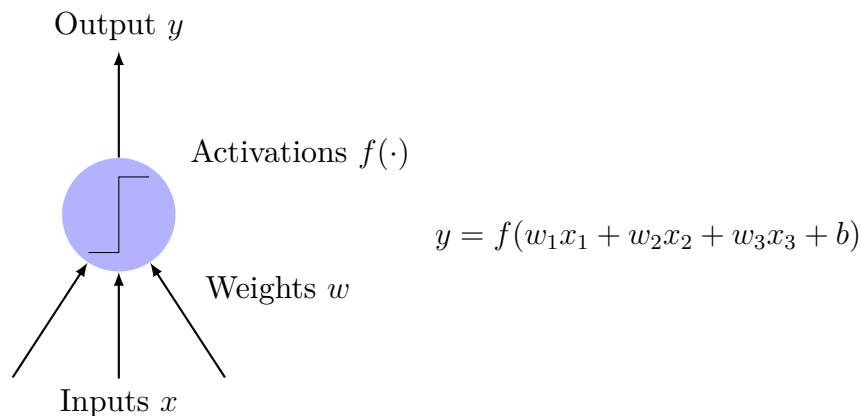


Figure 5.1: Model of a simple neuron with its mathematical representation

Neuron Weights

You may be familiar with linear regression, where the weights on the inputs are very much like the coefficients used in a regression equation. Like linear regression, each neuron also has a bias which can be thought of as an input that always has the value 1.0, and it, too, must be weighted. For example, a neuron may have two inputs, which requires three weights — one for each input and one for the bias.

Weights are often initialized to small random values, such as values from 0 to 0.3, although more complex initialization schemes can be used. Like linear regression, larger weights indicate increased complexity and fragility of the model. Keeping weights in the network small is desirable, and regularization techniques can be used.

Activation

The weighted inputs are summed and passed through an activation function, sometimes called a transfer function. An activation function is a simple mapping of summed weighted input to the output of the neuron. It is called an activation function because it governs the threshold at which the neuron is activated and the strength of the output signal. Historically, simple step activation functions were used. When the summed input was above a threshold of 0.5, for example, then the neuron would output a value of 1.0; otherwise it would output a 0.0.

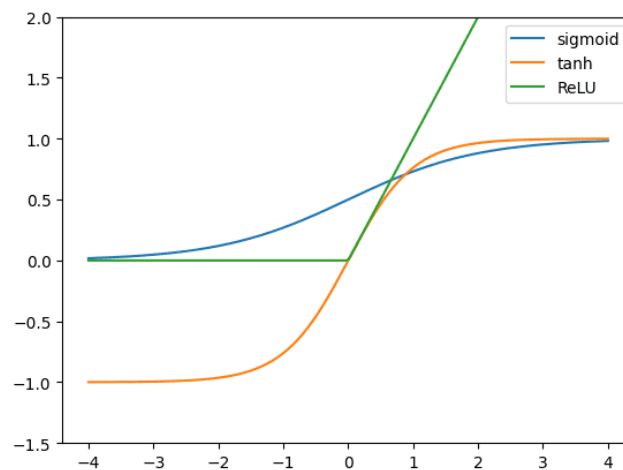


Figure 5.2: Plot of different activation functions

Traditionally, nonlinear activation functions are used. This allows the network to combine the inputs in more complex ways and, in turn, provide a richer capability in the functions they can model. Nonlinear functions like the logistic function, also called the sigmoid function, were used to output a value between 0 and 1 with an S-shaped distribution. The hyperbolic tangent function, also called tanh, were used to output the same distribution over the range -1 to $+1$. More recently, the rectifier activation function (ReLU) has been shown to provide better results.

5.3 Networks of Neurons

Neurons are arranged into networks of neurons, the figure below is an example. A row of neurons is called a layer, and one network can have multiple layers. The architecture of the neurons in the network is often called the network topology.

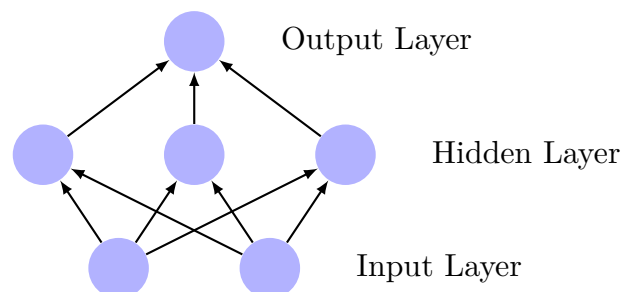


Figure 5.3: Model of a simple network

Input or Visible Layers

The bottom layer that takes input from your dataset is called the visible layer because it is the exposed part of the network. Often a neural network is drawn with a visible layer with one neuron per input value or column in your dataset. These are not actual neurons as described above, but simply drawn as nodes to represent input values that will be passed to the next layer.

Hidden Layers

Layers after the input layer are called hidden layers because they are not directly exposed to the input. The simplest network structure is to have a single neuron in the hidden layer that directly outputs the value. Given increases in computing power and efficient libraries, very deep neural networks can be constructed. Deep learning can refer to having many hidden layers in your neural network. They are deep because they would have been unimaginably slow to train historically but may take seconds or minutes to train using modern techniques and hardware.

Output Layer

The final hidden layer is called the output layer, and it is responsible for outputting a value or vector of values that correspond to the format required for the problem. The choice of activation function in the output layer is strongly constrained by the type of problem that you are modeling. For example:

- ▷ A regression problem may have a single output neuron, and the neuron may have no activation function.
- ▷ A binary classification problem may have a single output neuron and use a sigmoid activation function to output a value between 0 and 1 to represent the probability of the primary class as predicted. This can be turned into a crisp class value by using a threshold of 0.5 and snap values less than the threshold to 0, otherwise to 1.
- ▷ A multiclass classification problem may have multiple neurons in the output layer, one for each class (e.g., three neurons for the three classes in the famous iris flowers classification problem). In this case, a softmax activation function may be used to output a probability of the network predicting each of the class values. Selecting the output with the highest probability can be used to produce a crisp classification value.

5.4 Training Networks

Once configured, the neural network needs to be trained on your dataset.

Data Preparation

You must first prepare your data for training on a neural network. Data must be numerical, for example, real values. If you have categorical data, such as a *sex* attribute with the values “male” and “female”, you can convert it to a real-valued representation called one-hot encoding.

This is where one new column is added for each class value (two columns in the case of sex of male and female), and a 0 or 1 is added for each row depending on the class value for that row.

This same one-hot encoding can be used on the output variable in classification problems with more than one class. A binary vector is created from a single column that would be easy to directly compare to the output of the neuron in the network's output layer. That, as described above, would output one value for each class. Neural networks require the input to be scaled consistently. You can rescale it to the range between 0 and 1, this is called normalization. Another popular technique is to standardize it so that the distribution of each column has a mean of zero and a standard deviation of 1. Scaling also applies to image pixel data. Data such as words can be converted to integers, such as the frequency rank of the word in the dataset and other encoding techniques.

Stochastic Gradient Descent

The classical and still preferred training algorithm for neural networks is called stochastic gradient descent. This is where one row of data is exposed to the network at a time as input. The network processes the input upward, activating neurons as it goes to finally produce an output value. This is called a forward pass on the network. It is the type of pass that is also used after the network is trained in order to make predictions on new data.

The output of the network is compared to the expected output, and an error is calculated. This error is then propagated back through the network, one layer at a time, and the weights are updated according to the amount they contributed to the error. This clever bit of math is called the backpropagation algorithm. The process is repeated for all of the examples in your training data. One round of updating the network for the entire training dataset is called an epoch. A network may be trained for tens, hundreds, or many thousands of epochs.

Weight Updates

The weights in the network can be updated from the errors calculated for each training example, and this is called *online learning*. It can result in fast but also chaotic changes to the network. Alternatively, the errors can be saved across all the training examples, and the network can be updated at the end. This is called *batch learning* and is often more stable.

Because datasets are so large and because of computational efficiencies, the size of the batch, the number of examples the network is shown before an update, is often reduced to a small number, such as tens or hundreds of examples. The amount that weights are updated is controlled by a configuration parameter called the *learning rate*. It is also called the *step size* and controls the step or change made to network weights for a given error. Often small learning rates are used, such as 0.1 or 0.01 or smaller. The update equation can be complemented with additional configuration terms that you can set.

- ▷ **Momentum** is a term that carry over the weight update of the previous step to the current, so the weight update will continue in the same direction even when there is less error being calculated.

- ▷ **Learning Rate Decay** is used to decrease the learning rate over epochs to allow the network to make large changes to the weights at the beginning and smaller fine-tuning changes later in the training schedule.

Prediction

Once a neural network has been trained, it can be used to make predictions. You can make predictions on test or validation data in order to estimate the skill of the model on unseen data. You can also deploy it operationally and use it to make predictions continuously. The network topology and the final set of weights are all you need to save from the model. Predictions are made by providing the input to the network and performing a forward-pass, allowing it to generate an output you can use as a prediction.

5.5 More Resources

There are decades of papers and books on the topic of artificial neural networks. If you are new to the field, check out the following resources as further reading:

Books

- Christopher M. Bishop. *Neural Networks for Pattern Recognition*. Oxford University Press, 1996.
<https://www.amazon.com/dp/0198538642>
- Russell D. Reed and Robert J. Marks II. *Neural Smithing: Supervised Learning in Feedforward Artificial Neural Networks*. MIT Press, 1999.
<https://www.amazon.com/dp/0262527014>
- Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
<https://www.amazon.com/dp/0262035618>
(Online version at <http://www.deeplearningbook.org>).

5.6 Summary

In this chapter, you discovered artificial neural networks for machine learning. You learned:

- ▷ How neural networks are not models of the brain but are instead computational models for solving complex machine learning problems.
- ▷ That neural networks are comprised of neurons that have weights and activation functions.
- ▷ The networks are organized into layers of neurons and are trained using stochastic gradient descent.
- ▷ That it is a good idea to prepare your data before training a neural network model.

In the next chapter, you will learn what PyTorch provides to build a deep learning model.

Multilayer Perceptron Building Blocks in PyTorch

The PyTorch library is for deep learning. Deep learning, indeed, is just another name for a large-scale neural network or multilayer perceptron network. In its simplest form, multilayer perceptrons are a sequence of layers connected in tandem. In this chapter, you will discover the simple components you can use to create neural networks and simple deep learning models in PyTorch.

Let's get started.

Overview

This chapter is in six parts; they are:

- ▷ Neural Network Models in PyTorch
- ▷ Model Inputs
- ▷ Layers, Activations, and Layer Properties
- ▷ Loss Functions and Model Optimizers
- ▷ Model Training and Inference
- ▷ Examination of a Model

6.1 Neural Network Models in PyTorch

PyTorch can do a lot of things, but the most common use case is to build a deep learning model. The simplest model can be defined using `Sequential` class, which is just a linear stack of layers connected in tandem. You can create a `Sequential` model and define all the layers in one shot; for example:

```
import torch
import torch.nn as nn

model = nn.Sequential(...)
```

Listing 6.1: Building a MLP with the `Sequential` class

You should have all your layers defined inside the parentheses in the processing order from input to output. For example:

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(764, 100),
    nn.ReLU(),
    nn.Linear(100, 50),
    nn.ReLU(),
    nn.Linear(50, 10),
    nn.Sigmoid()
)
print(model)
```

Listing 6.2: An example MLP model

The other way of using `Sequential` is to pass in an ordered dictionary in which you can assign names to each layer:

```
from collections import OrderedDict
import torch.nn as nn

model = nn.Sequential(OrderedDict([
    ('dense1', nn.Linear(764, 100)),
    ('act1', nn.ReLU()),
    ('dense2', nn.Linear(100, 50)),
    ('act2', nn.ReLU()),
    ('output', nn.Linear(50, 10)),
    ('outact', nn.Sigmoid()),
]))
print(model)
```

Listing 6.3: An MLP model built using `OrderedDict`

And if you would like to build the layers one by one instead of doing everything in one shot, you can do the following:

```
import torch.nn as nn

model = nn.Sequential()
model.add_module("dense1", nn.Linear(8, 12))
model.add_module("act1", nn.ReLU())
model.add_module("dense2", nn.Linear(12, 8))
model.add_module("act2", nn.ReLU())
model.add_module("output", nn.Linear(8, 1))
model.add_module("outact", nn.Sigmoid())
print(model)
```

Listing 6.4: An MLP model created by adding one layer at a time

You will find this helpful in a more complex case where you need to build a model based on some conditions.

6.2 Model Inputs

The first layer in your model hints at the shape of the input. In the example above, you have `nn.Linear(764, 100)` as the first layer. Depending on the different layer type you use, the arguments may bear different meanings. But in this case, it is a `Linear` layer (also known as a dense layer or fully connected layer), and the two arguments tell the input and output dimensions of *this layer*.

Note that the size of a batch is implicit. In this example, you should pass in a PyTorch tensor of shape `(n, 764)` into this layer and expect a tensor of shape `(n, 100)` in return, where `n` is the size of a batch.

6.3 Layers, Activations, and Layer Properties

There are many kinds of neural network layers defined in PyTorch. In fact, it is easy to define your own layer if you want to. Below are some common layers that you may see often:

- ▷ `nn.Linear(input, output)`: The fully-connected layer
- ▷ `nn.Conv2d(in_channel, out_channel, kernel_size)`: The 2D convolution layer, popular in image processing networks
- ▷ `nn.Dropout(probability)`: Dropout layer, usually added to a network to introduce regularization
- ▷ `nn.Flatten()`: Reshape a high-dimensional input tensor into 1-dimensional (per each sample in a batch)

Besides layers, there are also activation functions. These are functions applied to each element of a tensor. Usually, you take the output of a layer and apply the activation before feeding it as input to a subsequent layer. Some common activation functions are:

- ▷ `nn.ReLU()`: Rectified linear unit, the most common activation nowadays
- ▷ `nn.Sigmoid()` and `nn.Tanh()`: Sigmoid and hyperbolic tangent functions, which are the usual choice in older literature
- ▷ `nn.Softmax()`: To convert a vector into probability-like values; popular in classification networks

You can find a list of all the different layers and activation functions in PyTorch's documentation.

The design of PyTorch is very modular. Therefore, you don't have much to adjust in each component. Take this `Linear` layer as an example. You can only specify the input and output shape but not other details, such as how to initialize the weights. However, almost all the components can take two additional arguments: the device and the data type.

A PyTorch device specifies where this layer will execute. Normally, you choose between the CPU and the GPU or omit it and let PyTorch decide. To specify a device, you do the following:

```
nn.Linear(764, 100, device="cpu")
```

Listing 6.5: Creating part of a model at CPU

or (CUDA means a supported nVidia GPU):

```
nn.Linear(764, 100, device="cuda:0")
```

Listing 6.6: Creating part of a model at GPU

The data type argument (`dtype`) specifies what kind of data type this layer should operate on. Usually, it is a 32-bit float, and usually, you don't want to change that. But if you need to specify a different type, you must do so using PyTorch types, e.g.,

```
nn.Linear(764, 100, dtype=torch.float16)
```

Listing 6.7: Setting operation data type

6.4 Loss Function and Model Optimizers

A neural network model is a sequence of matrix operations. The matrices that are independent of the input and kept inside the model are called weights. Training a neural network will *optimize* these weights so that they produce the output you want. In deep learning, the algorithm to optimize these weights is gradient descent.

There are many variations of gradient descent. You can make your choice by preparing an optimizer for your model. It is not part of the model, but you will use it alongside the model during training. The way you use it includes defining a *loss function* and minimizing the loss function using the optimizer. The loss function will give a *distance score* to tell how far away the model's output is from your desired output. It compares the output tensor of the model to the expected tensor, which is called the *label* or the *ground truth* in a different context. Because it is provided as part of the training dataset, a neural network model is a supervised learning model.

In PyTorch, you can simply take the model's output tensor and manipulate it to calculate the loss. But you can also make use of the functions provided in PyTorch for that, e.g.,

```
loss_fn = nn.CrossEntropyLoss()  
loss = loss_fn(output, label)
```

Listing 6.8: Creating and using a loss function

In this example, the `loss_fn` is a function, and `loss` is a tensor that supports automatic differentiation. You can trigger the differentiation by calling `loss.backward()`.

Below are some common loss functions in PyTorch:

- ▷ `nn.MSELoss()`: Mean square error, useful in regression problems
- ▷ `nn.CrossEntropyLoss()`: Cross-entropy loss, useful in classification problems
- ▷ `nn.BCELoss()`: Binary cross-entropy loss, useful in binary classification problems

Creating an optimizer is similar:

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

Listing 6.9: Creating an optimizer

All optimizers require a list of all parameters that it needs to optimize. It is because the optimizer is created outside the model, and you need to tell it where to look for the parameters (i.e., model weights). Then, the optimizer will take the gradient as computed by the `backward()` function call and apply it to the parameters based on the optimization algorithm.

This is a list of some common optimizers:

- ▷ `torch.optim.Adam()`: The Adam algorithm (adaptive moment estimation)
- ▷ `torch.optim.NAdam()`: The Adam algorithm with Nesterov momentum
- ▷ `torch.optim.SGD()`: Stochastic gradient descent
- ▷ `torch.optim.RMSprop()`: The RMSprop algorithm

You can find a list of all provided loss functions and optimizers in PyTorch's documentation, where you can also learn about the mathematical formula of each optimization algorithm.

6.5 Model Training and Inference

PyTorch doesn't have a dedicated function for model training and evaluation. A defined model by itself is like a function. You pass in an input tensor and get back the output tensor. Therefore, it is your responsibility to write the training loop. A minimal training loop is like the following:

```
for n in range(num_epochs):  
    y_pred = model(X)  
    loss = loss_fn(y_pred, y)  
    optimizer.zero_grad()  
    loss.backward()  
    optimizer.step()
```

Listing 6.10: Training a model with an optimizer

If you already have a model, you can simply take `y_pred = model(X)` and use the output tensor `y_pred` for other purposes. That's how you use the model for prediction or inference. A model, however, does not expect one input sample but a batch of input samples in one tensor. If the model is to take an input vector (which is one-dimensional), you should provide a two-dimensional tensor to the model. Usually, in the case of inference, you deliberately create a batch of one sample.

6.6 Examination of a Model

Once you have a model, you can check what it is by printing it:

```
print(model)
```

Listing 6.11: Printing a model

This will give you, for example, the following:

```
Sequential(
  (0): Linear(in_features=8, out_features=12, bias=True)
  (1): ReLU()
  (2): Linear(in_features=12, out_features=8, bias=True)
  (3): ReLU()
  (4): Linear(in_features=8, out_features=1, bias=True)
  (5): Sigmoid()
)
```

Output 6.1: A model printed

If you would like to save the model, you can use the `pickle` library from Python. But you can also access it using PyTorch:

```
torch.save(model, "my_model.pth")
```

Listing 6.12: Saving a PyTorch model

This way, you have the entire model object saved. You can retrieve the model with:

```
model = torch.load("my_model.pth")
```

Listing 6.13: Reloading a PyTorch model

But the recommended way of saving a model is to leave the model design in code and keep only the weights (more details on Chapter 13). You can do so with:

```
torch.save(model.state_dict(), "my_model.pth")
```

Listing 6.14: Saving a PyTorch model with the `save()` function

The `state_dict()` function extracts only the states (i.e., weights in a model). To retrieve it, you need to rebuild the model from scratch and then load the weights like this:

```
model = nn.Sequential(...)
model.load_state_dict(torch.load("my_model.pth"))
```

Listing 6.15: Reloading a model using PyTorch's `load()` function

6.7 Further Readings

You can learn more about how to create simple neural networks and deep learning models in PyTorch using the following resources:

Articles

PyTorch tutorials.

<https://pytorch.org/tutorials/>

torch.nn. PyTorch API.

<https://pytorch.org/docs/stable/nn.html>

torch.optim. PyTorch API.

<https://pytorch.org/docs/stable/optim.html>

6.8 Summary

In this chapter, you discovered the PyTorch API that you can use to create artificial neural networks and deep learning models. Specifically, you learned about the life cycle of a PyTorch model, including:

- ▷ Constructing a model
- ▷ Creating and adding layers and activations
- ▷ Preparing a model for training and inference

In the next chapter, you will build your first deep learning model using PyTorch.

Your First Neural Network in PyTorch, Step by Step

PyTorch is a powerful Python library for building deep learning models. It provides everything you need to define and train a neural network and use it for inference. You don't need to write much code to complete all this. In this chapter, you will discover how to create your first deep learning neural network model in Python using PyTorch. After completing this chapter, you will know:

- ▷ How to load a CSV dataset and prepare it for use with PyTorch
- ▷ How to define a Multilayer Perceptron model in PyTorch
- ▷ How to train and evaluate a PyTorch model on a validation dataset

Let's get started.

Overview

There is not a lot of code required. You will go over it slowly so that you will know how to create your own models in the future. The steps you will learn in this chapter are as follows:

- ▷ Load Data
- ▷ Define PyTorch Model
- ▷ Define Loss Function and Optimizers
- ▷ Run a Training Loop
- ▷ Evaluate the Model
- ▷ Make Predictions

7.1 Load Data

The first step is to define the functions and classes you intend to use in this chapter. You will use the NumPy library to load your dataset and the PyTorch library for deep learning models.

The imports required are listed below:


```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
```

Listing 7.1: Dependencies

You can now load your dataset. You can download it here:

▷ <https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv>

In this chapter, you will use the Pima Indians onset of diabetes dataset. This has been a standard machine learning dataset since the early days of the field. It describes patient medical record data for Pima Indians and whether they had an onset of diabetes within five years.

It is a binary classification problem (onset of diabetes as 1 or not as 0). All the input variables that describe each patient are transformed and numerical. This makes it easy to use directly with neural networks that expect numerical input and output values and is an ideal choice for our first neural network in PyTorch.

Download the dataset and place it in your local working directory, the same location as your Python file. Save it with the filename `pima-indians-diabetes.csv`. Take a look inside the file; you should see rows of data like the following:

```
6,148,72,35,0,33.6,0.627,50,1
1,85,66,29,0,26.6,0.351,31,0
8,183,64,0,0,23.3,0.672,32,1
1,89,66,23,94,28.1,0.167,21,0
0,137,40,35,168,43.1,2.288,33,1
...
```

Output 7.1: Dataset in CSV format

You can now load the file as a matrix of numbers using the NumPy function `loadtxt()`. There are eight input variables and one output variable (the last column). You will be learning a model to map rows of input variables (X) to an output variable (y), which is often summarized as $y = f(X)$. The variables are summarized as follows:

Input Variables (X):

1. Number of times pregnant
2. Plasma glucose concentration at 2 hours in an oral glucose tolerance test
3. Diastolic blood pressure (mm Hg)
4. Triceps skin fold thickness (mm)
5. 2-hour serum insulin ($\mu\text{IU}/\text{ml}$)
6. Body mass index (weight in kg/(height in m)²)
7. Diabetes pedigree function
8. Age (years)

Output Variables (y):

▷ Class label (0 or 1)

Once the CSV file is loaded into memory, you can split the columns of data into input and output variables.

The data will be stored in a 2D array where the first dimension is rows and the second dimension is columns, e.g., (rows, columns). You can split the array into two arrays by selecting subsets of columns using the standard NumPy slice operator “:”. You can select the first eight columns from index 0 to index 7 via the slice `0:8`. You can then select the output column (the 9th variable) via index 8.

```
...  
  
# load the dataset, split into input (X) and output (y) variables  
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')  
X = dataset[:,0:8]  
y = dataset[:,8]
```

Listing 7.2: Loading data from a CSV file

But these data should be converted to PyTorch tensors first. One reason is that PyTorch usually operates in a 32-bit floating point while NumPy, by default, uses a 64-bit floating point. Mix-and-match is not allowed in most operations. Converting to PyTorch tensors can avoid the implicit conversion that may cause problems. You can also take this chance to correct the shape to fit what PyTorch would expect, e.g., prefer $n \times 1$ matrix over n -vectors.

To convert, create a tensor out of NumPy arrays:

```
X = torch.tensor(X, dtype=torch.float32)  
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
```

Listing 7.3: Converting a NumPy array into PyTorch tensor

You are now ready to define your neural network model.

7.2 Define the Model

Indeed, there are two ways to define a model in PyTorch. The goal is to make it like a function that takes an input and returns an output.

A model can be defined as a sequence of layers. You create a `Sequential` model with the layers listed out. The first thing you need to do to get this right is to ensure the first layer has the correct number of input features. In this example, you can specify the input dimension 8 for the eight input variables as one vector.

The other parameters for a layer or how many layers you need for a model is not an easy question. You may use heuristics to help you design the model, or you can refer to other people’s designs in dealing with a similar problem. Often, the best neural network structure is found through a process of trial-and-error experimentation. Generally, you need a network

large enough to capture the structure of the problem but small enough to make it fast. In this example, let's use a fully-connected network structure with three layers.

Fully connected layers or dense layers are defined using the `Linear` class in PyTorch. It simply means an operation similar to matrix multiplication. You can specify the number of inputs as the first argument and the number of outputs as the second argument. The number of outputs is sometimes called the number of neurons or number of nodes in the layer.

You also need an activation function *after* the layer. If not provided, you just take the output of the matrix multiplication to the next step, or sometimes you call it using `linear activation`, hence the name of the layer.

In this example, you will use the *rectified linear unit* activation function, referred to as ReLU, on the first two layers and the sigmoid function in the output layer.

A sigmoid on the output layer ensures the output is between 0 and 1, which is easy to map to either a probability of class 1 or snap to a hard classification of either class by a cut-off threshold of 0.5. In the past, you might have used sigmoid and tanh activation functions for all layers, but it turns out that sigmoid activation can lead to the problem of vanishing gradient in deep neural networks, and ReLU activation is found to provide better performance in terms of both speed and accuracy.

You can make a complete model by adding each layer such that:

- ▷ The model expects rows of data with 8 variables (the first argument at the first layer set to 8)
- ▷ The first hidden layer has 12 neurons, followed by a ReLU activation function
- ▷ The second hidden layer has 8 neurons, followed by another ReLU activation function
- ▷ The output layer has one neuron, followed by a sigmoid activation function

```
...  
  
model = nn.Sequential(  
    nn.Linear(8, 12),  
    nn.ReLU(),  
    nn.Linear(12, 8),  
    nn.ReLU(),  
    nn.Linear(8, 1),  
    nn.Sigmoid()  
)
```

Listing 7.4: Creating an MLP model

You can check the model by printing it out as follows:

```
print(model)
```

Listing 7.5: Showing the MLP model created

You will see:

```

Sequential(
  (0): Linear(in_features=8, out_features=12, bias=True)
  (1): ReLU()
  (2): Linear(in_features=12, out_features=8, bias=True)
  (3): ReLU()
  (4): Linear(in_features=8, out_features=1, bias=True)
  (5): Sigmoid()
)

```

Output 7.2: The MLP model created

You are free to change the design and see if you get a better or worse result than the subsequent part of this chapter.

But note that, in PyTorch, there is a more verbose way of creating a model. The model above can be created as a Python `class` inherited from the `nn.Module`:

```

...

class PimaClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden1 = nn.Linear(8, 12)
        self.act1 = nn.ReLU()
        self.hidden2 = nn.Linear(12, 8)
        self.act2 = nn.ReLU()
        self.output = nn.Linear(8, 1)
        self.act_output = nn.Sigmoid()

    def forward(self, x):
        x = self.act1(self.hidden1(x))
        x = self.act2(self.hidden2(x))
        x = self.act_output(self.output(x))
        return x

model = PimaClassifier()
print(model)

```

Listing 7.6: The same MLP model created as a subclass of `nn.Module`

In this case, the model printed will be:

```

PimaClassifier(
  (hidden1): Linear(in_features=8, out_features=12, bias=True)
  (act1): ReLU()
  (hidden2): Linear(in_features=12, out_features=8, bias=True)
  (act2): ReLU()
  (output): Linear(in_features=8, out_features=1, bias=True)
  (act_output): Sigmoid()
)

```

Output 7.3: The MLP model created

In this approach, a class needs to have all the layers defined in the constructor because you need to prepare all its components when it is created, but the input is not yet provided.

Note that you also need to call the parent class's constructor (the line `super().__init__()`) to bootstrap your model. You also need to define a `forward()` function in the class to tell, if an input tensor `x` is provided, how you produce the output tensor in return.

You can see from the output above that the model remembers how you named each layer.

7.3 Preparation for Training

A defined model is ready for training, but you need to specify what the goal of the training is. In this example, the data has the input features X and the output label y . You want the neural network model to produce an output that is as close to y as possible. Training a network means finding the best set of weights to map inputs to outputs in your dataset. The *loss function* is the metric to measure the prediction's distance to y . In this example, you should use binary cross-entropy because it is a binary classification problem.

Once you decide on the loss function, you also need an optimizer. The optimizer is the algorithm you use to adjust the model weights progressively to produce a better output. There are many optimizers to choose from, and in this example, Adam is used. This popular version of gradient descent can automatically tune itself and gives good results in a wide range of problems.

```
loss_fn = nn.BCELoss()    # binary cross-entropy
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

Listing 7.7: Creating a loss function and an optimizer

The optimizer usually has some configuration parameters. Most notably, the learning rate `lr`. But all optimizers need to know what to optimize. Therefore we pass on `model.parameters()`, which is a generator of all parameters from the model we created.

7.4 Training a Model

You have defined your model, the loss metric, and the optimizer. It is ready for training by executing the model on some data.

Training a neural network model usually takes in epochs and batches. They are idioms for how data is passed to a model:

- ▷ *Epoch*: Passes the entire training dataset to the model once
- ▷ *Batch*: One or more samples passed to the model, from which the gradient descent algorithm will be executed for one iteration

Simply speaking, the entire dataset is split into batches, and you pass the batches one by one into a model using a training loop. Once you have exhausted all the batches, you have finished one epoch. Then you can start over again with the same dataset and start the second epoch, continuing to refine the model. This process repeats until you are satisfied with the model's output.

The size of a batch is limited by the system's memory. Also, the number of computations required is linearly proportional to the size of a batch. The total number of batches over many epochs is how many times you run the gradient descent to refine the model. It is a trade off that you want more iterations for the gradient descent so you can produce a better model, but at the same time, you do not want the training to take too long time to complete. The number of epochs and the size of a batch can be chosen experimentally by trial and error.

The goal of training a model is to ensure it learns a good enough mapping of input data to output classification. It will not be perfect, and errors are inevitable. Usually, you will see the amount of error reducing when in the later epochs, but it will eventually level out. This is called model convergence.

The simplest way to build a training loop is to use two nested for-loops, one for epochs and one for batches:

```
n_epochs = 100
batch_size = 10

for epoch in range(n_epochs):
    for i in range(0, len(X), batch_size):
        Xbatch = X[i:i+batch_size]
        y_pred = model(Xbatch)
        ybatch = y[i:i+batch_size]
        loss = loss_fn(y_pred, ybatch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    print(f'Finished epoch {epoch}, latest loss {loss}')
```

Listing 7.8: The training loop

When this runs, it will print the following:

```
Finished epoch 0, latest loss 0.6271069645881653
Finished epoch 1, latest loss 0.6056771874427795
Finished epoch 2, latest loss 0.5916517972946167
Finished epoch 3, latest loss 0.5822567939758301
Finished epoch 4, latest loss 0.5682642459869385
Finished epoch 5, latest loss 0.5640913248062134
...
```

Output 7.4: Progress printed while training the model

7.5 Evaluate the Model

You have trained our neural network on the entire dataset, and you can evaluate the performance of the network on the same dataset. This will only give you an idea of how well you have modeled the dataset (e.g., train accuracy) but no idea of how well the algorithm might perform on new data. This was done for simplicity, but ideally, you could separate your data into train and test datasets for training and evaluation of your model.

You can evaluate your model on your training dataset in the same way you invoked the model in training. This will generate predictions for each input, but then you still need to compute a score for the evaluation. This score can be the same as your loss function or something different. Because you are doing binary classification, you can use accuracy as your evaluation score by converting the output (a floating point in the range of 0 to 1) to an integer (0 or 1) and compare to the label we know.

This is done as follows:

```
# compute accuracy (no_grad is optional)
with torch.no_grad():
    y_pred = model(X)

accuracy = (y_pred.round() == y).float().mean()
print(f"Accuracy {accuracy}")
```

Listing 7.9: Evaluating the trained model

The `round()` function rounds off the floating point to the nearest integer. The `==` operator compares and returns a Boolean tensor, which can be converted to floating point numbers 1.0 and 0.0. The `mean()` function will provide you the count of the number of 1's (i.e., prediction matches the label) divided by the total number of samples. The `no_grad()` context is optional but suggested, so you relieve `y_pred` from remembering how it comes up with the number since you are not going to do differentiation on it.

Putting everything together, the following is the complete code.

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

# load the dataset, split into input (X) and output (y) variables
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
X = dataset[:,0:8]
y = dataset[:,8]

X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# define the model
model = nn.Sequential(
    nn.Linear(8, 12),
    nn.ReLU(),
    nn.Linear(12, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
    nn.Sigmoid()
)
print(model)

# train the model
loss_fn = nn.BCELoss() # binary cross-entropy
```

```

optimizer = optim.Adam(model.parameters(), lr=0.001)

n_epochs = 100
batch_size = 10

for epoch in range(n_epochs):
    for i in range(0, len(X), batch_size):
        Xbatch = X[i:i+batch_size]
        y_pred = model(Xbatch)
        ybatch = y[i:i+batch_size]
        loss = loss_fn(y_pred, ybatch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    print(f'Finished epoch {epoch}, latest loss {loss}')

# compute accuracy (no_grad is optional)
with torch.no_grad():
    y_pred = model(X)
accuracy = (y_pred.round() == y).float().mean()
print(f"Accuracy {accuracy}")

```

Listing 7.10: Complete code to build, train, and evaluate an MLP model

You can copy all the code into your Python file and save it as “pytorch_network.py” in the same directory as your data file “pima-indians-diabetes.csv”. You can then run the Python file as a script from your command line.

Running this example, you should see that the training loop progresses on each epoch with the loss with the final accuracy printed last. Ideally, you would like the loss to go to zero and the accuracy to go to 1.0 (e.g., 100%). This is not possible for any but the most trivial machine learning problems. Instead, you will always have some error in your model. The goal is to choose a model configuration and training configuration that achieves the lowest loss and highest accuracy possible for a given dataset.

Neural networks are stochastic algorithms, meaning that the same algorithm on the same data can train a different model with different skill each time the code is run. This is a feature, not a bug. The variance in the performance of the model means that to get a reasonable approximation of how well your model is performing, you may need to fit it many times and calculate the average of the accuracy scores. For example, below are the accuracy scores from re-running the example five times:

```

Accuracy: 0.7604166865348816
Accuracy: 0.7838541865348816
Accuracy: 0.7669270634651184
Accuracy: 0.7721354365348816
Accuracy: 0.7669270634651184

```

Output 7.5: Various accuracy obtained by re-running the program

You can see that all accuracy scores are around 77%, roughly.

7.6 Make Predictions

You can adapt the above example and use it to generate predictions on the training dataset, pretending it is a new dataset you have not seen before. Making predictions is as easy as calling the model as if it is a function. You are using a sigmoid activation function on the output layer so that the predictions will be a probability in the range between 0 and 1. You can easily convert them into a crisp binary prediction for this classification task by rounding them. For example:

```
...

# make probability predictions with the model
predictions = model(X)
# round predictions
rounded = predictions.round()
```

Listing 7.11: Making prediction from the trained model

Alternately, you can convert the probability into 0 or 1 to predict crisp classes directly; for example:

```
...

# make class predictions with the model
predictions = (model(X) > 0.5).int()
```

Listing 7.12: Converting a probability output into the class label

The complete example below makes predictions for each example in the dataset, then prints the input data, predicted class, and expected class for the first five examples in the dataset.

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

# load the dataset, split into input (X) and output (y) variables
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
X = dataset[:,0:8]
y = dataset[:,8]

X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# define the model
class PimaClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden1 = nn.Linear(8, 12)
        self.act1 = nn.ReLU()
        self.hidden2 = nn.Linear(12, 8)
        self.act2 = nn.ReLU()
        self.output = nn.Linear(8, 1)
```

```

        self.act_output = nn.Sigmoid()

    def forward(self, x):
        x = self.act1(self.hidden1(x))
        x = self.act2(self.hidden2(x))
        x = self.act_output(self.output(x))
        return x

model = PimaClassifier()
print(model)

# train the model
loss_fn = nn.BCELoss() # binary cross-entropy
optimizer = optim.Adam(model.parameters(), lr=0.001)

n_epochs = 100
batch_size = 10

for epoch in range(n_epochs):
    for i in range(0, len(X), batch_size):
        Xbatch = X[i:i+batch_size]
        y_pred = model(Xbatch)
        ybatch = y[i:i+batch_size]
        loss = loss_fn(y_pred, ybatch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

# compute accuracy
y_pred = model(X)
accuracy = (y_pred.round() == y).float().mean()
print(f"Accuracy {accuracy}")

# make class predictions with the model
predictions = (model(X) > 0.5).int()
for i in range(5):
    print('%s => %d (expected %d)' % (X[i].tolist(), predictions[i], y[i]))

```

Listing 7.13: Complete code to use the model for prediction

This code uses a different way of building the model but should functionally be the same as before. After the model is trained, predictions are made for all examples in the dataset, and the input rows and predicted class value for the first five examples are printed and compared to the expected class value. You can see that most rows are correctly predicted. In fact, you can expect about 77% of the rows to be correctly predicted based on your estimated performance of the model in the previous section.

```

[6.0, 148.0, 72.0, 35.0, 0.0, 33.599998474121094, 0.6269999742507935, 50.0] => 1 (expected 1)
[1.0, 85.0, 66.0, 29.0, 0.0, 26.600000381469727, 0.35100001096725464, 31.0] => 0 (expected 0)
[8.0, 183.0, 64.0, 0.0, 0.0, 23.299999237060547, 0.671999990940094, 32.0] => 1 (expected 1)
[1.0, 89.0, 66.0, 23.0, 94.0, 28.100000381469727, 0.16699999570846558, 21.0] => 0 (expected 0)
[0.0, 137.0, 40.0, 35.0, 168.0, 43.099998474121094, 2.2880001068115234, 33.0] => 1 (expected 1)

```

Output 7.6: Inputs to the model and its predictions

7.7 Further Reading

To learn more about deep learning and PyTorch, take a look at some of these:

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

<https://www.amazon.com/dp/0262035618>

(Online version at <http://www.deeplearningbook.org>).

PyTorch documentation.

<https://pytorch.org/docs/stable/index.html>

Build the Neural Network. PyTorch tutorials.

https://pytorch.org/tutorials/beginner/basics/buildmodel_tutorial.html

Optimizing Model Parameters. PyTorch tutorials.

https://pytorch.org/tutorials/beginner/basics/optimization_tutorial.html

7.8 Summary

In this chapter, you discovered how to create your first neural network model using PyTorch. Specifically, you learned the key steps in using PyTorch to create a neural network or deep learning model step by step, including:

- ▷ How to load data
- ▷ How to define a neural network in PyTorch
- ▷ How to train a model on data
- ▷ How to evaluate a model
- ▷ How to make predictions with the model

In this chapter, a simple workflow is presented but every step can be deep-dived. In the next chapter, you will look into the training loop and learn how to polish it.

Creating a Training Loop for Your Models

PyTorch provides a lot of building blocks for a deep learning model, but a training loop is not part of them. It is a flexibility that allows you to do whatever you want during training, but some basic structure is universal across most use cases.

In this chapter, you will see how to make a training loop that provides essential information for your model training, with the option to allow any information to be displayed. After completing this chapter, you will know:

- ▷ The basic building block of a training loop
- ▷ How to use `tqdm` to display training progress

Let's get started.

Overview

This chapter is in three parts; they are:

- ▷ Elements of Training a Deep Learning Model
- ▷ Collecting Statistics During Training
- ▷ Using `tqdm` to Report the Training Progress

8.1 Elements of Training a Deep Learning Model

As with all machine learning models, the model design specifies the algorithm to manipulate an input and produce an output. But in the model, there are parameters that you need to fine-tune to achieve that. These model parameters are also called the weights, biases, kernels, or other names depending on the particular model and layers. Training is to feed in the sample data to the model so that an optimizer can fine-tune these parameters.

When you train a model, you usually start with a dataset. Each dataset is a fairly large number of data samples. When you get a dataset, it is recommended to split it into two portions: the training set and the test set. The training set is further split into batches and used in the training loop to drive the gradient descent algorithms. The test set, however, is used as a benchmark to tell how good your model is. Usually, you do not use the training set

as a metric but take the test set, which is not seen by the gradient descent algorithm, so you can tell if your model fits well to the unseen data.

Overfitting is when the model fits too well to the training set (i.e., at very high accuracy) but performs significantly worse in the test set. Underfitting is when the model cannot even fit well to the training set. Naturally, you don't want to see either on a good model.

Training of a neural network model is in epochs. Usually, one epoch means you run through the entire training set once, although you only feed one batch at a time. It is also customary to do some housekeeping tasks at the end of each epoch, such as benchmarking the partially trained model with the test set, checkpointing the model, deciding if you want to stop the training early, and collecting training statistics, and so on.

In each epoch, you feed data samples into the model in batches and run a gradient descent algorithm. This is one step in the training loop because you run the model in one forward pass (i.e., providing input and capturing output), and one backward pass (evaluating the loss metric from the output and deriving the gradient of each parameter all the way back to the input layer). The backward pass computes the gradient using automatic differentiation. Then, this gradient is used by the gradient descent algorithm to adjust the model parameters. There are multiple steps in one epoch.

Reusing the examples in Chapter 7, you can download the dataset¹ and split the dataset into two as follows:

```
import numpy as np
import torch

# load the dataset
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
X = dataset[:,0:8]
y = dataset[:,8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# split the dataset into training and test sets
Xtrain = X[:700]
ytrain = y[:700]
Xtest = X[700:]
ytest = y[700:]
```

Listing 8.1: Loading data from a CSV file

This dataset is small — only 768 samples. Here, it takes the first 700 as the training set and the rest as the test set.

It is not the focus of this chapter, but you can reuse the model, the loss function, and the optimizer from a previous chapter:

```
import torch.nn as nn
import torch.optim as optim
```

¹<https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv>

```

model = nn.Sequential(
    nn.Linear(8, 12),
    nn.ReLU(),
    nn.Linear(12, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
    nn.Sigmoid()
)
print(model)

# loss function and optimizer
loss_fn = nn.BCELoss() # binary cross entropy
optimizer = optim.Adam(model.parameters(), lr=0.001)

```

Listing 8.2: Setting up a model, the loss function, and the optimizer

With the data and the model, this is the minimal training loop, with the forward and backward pass in each step:

```

n_epochs = 50 # number of epochs to run
batch_size = 10 # size of each batch
batches_per_epoch = len(Xtrain) // batch_size

for epoch in range(n_epochs):
    for i in range(batches_per_epoch):
        start = i * batch_size
        # take a batch
        Xbatch = Xtrain[start:start+batch_size]
        ybatch = ytrain[start:start+batch_size]
        # forward pass
        y_pred = model(Xbatch)
        loss = loss_fn(y_pred, ybatch)
        # backward pass
        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()

```

Listing 8.3: Training the model in a loop

In the inner for-loop, you take each batch in the dataset and evaluate the loss. The `loss` is a PyTorch tensor that remembers how it comes up with its value. Then you zero out all gradients that the optimizer manages and call `loss.backward()` to run the backpropagation algorithm. The result sets up the gradients of all the tensors that the tensor `loss` depends on directly and indirectly. Afterward, upon calling `step()`, the optimizer will check each parameter that it manages and update them.

After everything is done, you can run the model with the test set to evaluate its performance. The evaluation can be based on a different function than the loss function. For example, this classification problem uses accuracy:

```
...

# evaluate trained model with test set
with torch.no_grad():
    y_pred = model(X)
accuracy = (y_pred.round() == y).float().mean()
print("Accuracy {:.2f}".format(accuracy * 100))
```

Listing 8.4: Evaluating the trained model

Putting everything together, this is the complete code:

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

# load the dataset
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
X = dataset[:,0:8]
y = dataset[:,8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# split the dataset into training and test sets
Xtrain = X[:700]
ytrain = y[:700]
Xtest = X[700:]
ytest = y[700:]

model = nn.Sequential(
    nn.Linear(8, 12),
    nn.ReLU(),
    nn.Linear(12, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
    nn.Sigmoid()
)
print(model)

# loss function and optimizer
loss_fn = nn.BCELoss() # binary cross entropy
optimizer = optim.Adam(model.parameters(), lr=0.001)

n_epochs = 50 # number of epochs to run
batch_size = 10 # size of each batch
batches_per_epoch = len(Xtrain) // batch_size

for epoch in range(n_epochs):
    for i in range(batches_per_epoch):
        start = i * batch_size
        # take a batch
        Xbatch = Xtrain[start:start+batch_size]
        ybatch = ytrain[start:start+batch_size]
```

```

    # forward pass
    y_pred = model(Xbatch)
    loss = loss_fn(y_pred, ybatch)
    # backward pass
    optimizer.zero_grad()
    loss.backward()
    # update weights
    optimizer.step()

# evaluate trained model with test set
with torch.no_grad():
    y_pred = model(X)
accuracy = (y_pred.round() == y).float().mean()
print("Accuracy {:.2f}".format(accuracy * 100))

```

Listing 8.5: Creating, training, and evaluating a model

8.2 Collecting Statistics During Training

The training loop above should work well with small models that can finish training in a few seconds. But for a larger model or a larger dataset, you will find that it takes significantly longer to train. While you're waiting for the training to complete, you may want to see how it's going as you may want to interrupt the training if any mistake is made.

Usually, during training, you would like to see the following:

- ▷ In each step, you would like to know the loss metrics, and you are expecting the loss to go down
- ▷ In each step, you would like to know other metrics, such as accuracy on the training set, that are of interest but not involved in the gradient descent
- ▷ At the end of each epoch, you would like to evaluate the partially-trained model with the test set and report the evaluation metric
- ▷ At the end of the training, you would like to be able to visualize the above metrics

These all are possible, but you need to add more code into the training loop, as follows:

```

n_epochs = 50    # number of epochs to run
batch_size = 10  # size of each batch
batches_per_epoch = len(Xtrain) // batch_size

# collect statistics
train_loss = []
train_acc = []
test_acc = []

for epoch in range(n_epochs):
    for i in range(batches_per_epoch):
        start = i * batch_size
        # take a batch
        Xbatch = Xtrain[start:start+batch_size]

```



```

ybatch = ytrain[start:start+batch_size]
# forward pass
y_pred = model(Xbatch)
loss = loss_fn(y_pred, ybatch)
acc = (y_pred.round() == ybatch).float().mean()
# store metrics
train_loss.append(float(loss))
train_acc.append(float(acc))
# backward pass
optimizer.zero_grad()
loss.backward()
# update weights
optimizer.step()
# print progress
print(f"epoch {epoch} step {i} loss {loss} accuracy {acc}")
# evaluate model at end of epoch
y_pred = model(Xtest)
acc = (y_pred.round() == ytest).float().mean()
test_acc.append(float(acc))
print(f"End of {epoch}, accuracy {acc}")

```

Listing 8.6: The training loop expanded to collect more statistics

As you collect the loss and accuracy in the list, you can plot them using matplotlib. But be careful that you collected training set statistics at each step, whereas the test set accuracy only at the end of the epoch. Thus you would like to show the *average accuracy* from the training loop in each epoch, so they are comparable to each other.

```

import matplotlib.pyplot as plt

# Plot the loss metrics, set the y-axis to start from 0
plt.plot(train_loss)
plt.xlabel("steps")
plt.ylabel("loss")
plt.ylim(0)
plt.show()

# plot the accuracy metrics
avg_train_acc = []
for i in range(n_epochs):
    start = i * batch_size
    average = sum(train_acc[start:start+batches_per_epoch]) / batches_per_epoch
    avg_train_acc.append(average)

plt.plot(avg_train_acc, label="train")
plt.plot(test_acc, label="test")
plt.xlabel("epochs")
plt.ylabel("accuracy")
plt.ylim(0)
plt.show()

```

Listing 8.7: Visualizing accuracy metric during the training of a model

Putting everything together, below is the complete code:

```

import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

# load the dataset
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
# split into input (X) and output (y) variables
X = dataset[:,0:8]
y = dataset[:,8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# split the dataset into training and test sets
Xtrain = X[:700]
ytrain = y[:700]
Xtest = X[700:]
ytest = y[700:]

model = nn.Sequential(
    nn.Linear(8, 12),
    nn.ReLU(),
    nn.Linear(12, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
    nn.Sigmoid()
)
print(model)

# loss function and optimizer
loss_fn = nn.BCELoss() # binary cross entropy
optimizer = optim.Adam(model.parameters(), lr=0.0001)

n_epochs = 50 # number of epochs to run
batch_size = 10 # size of each batch
batches_per_epoch = len(Xtrain) // batch_size

# collect statistics
train_loss = []
train_acc = []
test_acc = []

for epoch in range(n_epochs):
    for i in range(batches_per_epoch):
        # take a batch
        start = i * batch_size
        Xbatch = Xtrain[start:start+batch_size]
        ybatch = ytrain[start:start+batch_size]
        # forward pass
        y_pred = model(Xbatch)
        loss = loss_fn(y_pred, ybatch)
        acc = (y_pred.round() == ybatch).float().mean()
        # store metrics

```

```

    train_loss.append(float(loss))
    train_acc.append(float(acc))
    # backward pass
    optimizer.zero_grad()
    loss.backward()
    # update weights
    optimizer.step()
    # print progress
    print(f"epoch {epoch} step {i} loss {loss} accuracy {acc}")
# evaluate model at end of epoch
y_pred = model(Xtest)
acc = (y_pred.round() == ytest).float().mean()
test_acc.append(float(acc))
print(f"End of {epoch}, accuracy {acc}")

# Plot the loss metrics
plt.plot(train_loss)
plt.xlabel("steps")
plt.ylabel("loss")
plt.ylim(0)
plt.show()

# plot the accuracy metrics
avg_train_acc = []
for i in range(n_epochs):
    start = i * batch_size
    average = sum(train_acc[start:start+batches_per_epoch]) / batches_per_epoch
    avg_train_acc.append(average)

plt.plot(avg_train_acc, label="train")
plt.plot(test_acc, label="test")
plt.xlabel("epochs")
plt.ylabel("accuracy")
plt.ylim(0)
plt.show()

```

Listing 8.8: Complete code to build and train a model with visualized accuracy metric

The story does not end here. Indeed, you can add more code to the training loop, especially in dealing with a more complex model. One example is checkpointing. You may want to save your model (e.g., using pickle) so that, if for any reason, your program stops, you can restart the training loop from the middle. Another example is early stopping, which lets you monitor the accuracy you obtained with the test set at the end of each epoch and interrupt the training if you don't see the model improving for a while. This is because you probably can't go further, given the design of the model, and you do not want to overfit. Chapter 21 will cover more on this topic.

8.3 Using `tqdm` to Report the Training Progress

If you run the above code, you will find that there are a lot of lines printed on the screen while the training loop is running. Your screen may be cluttered. And you may also want to

see an animated progress bar to better tell you how far you are in the training progress. The library `tqdm` is the popular tool for creating the progress bar. Converting the above code to use `tqdm` cannot be easier:

```
for epoch in range(n_epochs):
    with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for i in bar:
            # take a batch
            start = i * batch_size
            Xbatch = Xtrain[start:start+batch_size]
            ybatch = ytrain[start:start+batch_size]
            # forward pass
            y_pred = model(Xbatch)
            loss = loss_fn(y_pred, ybatch)
            acc = (y_pred.round() == ybatch).float().mean()
            # store metrics
            train_loss.append(float(loss))
            train_acc.append(float(acc))
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
            # print progress
            bar.set_postfix(
                loss=float(loss),
                acc=f"{float(acc)*100:.2f}%"
            )
        # evaluate model at end of epoch
        y_pred = model(Xtest)
        acc = (y_pred.round() == ytest).float().mean()
        test_acc.append(float(acc))
        print(f"End of {epoch}, accuracy {acc}")
```

Listing 8.9: A training loop with `tqdm` progress bar

The usage of `tqdm` creates an iterator using `trange()` just like Python's `range()` function, and you can read the number in a loop. You can access the progress bar by updating its description or “postfix” data, but you have to do that before it exhausts its content. The `set_postfix()` function is powerful as it can show you anything.

In fact, there is a `tqdm()` function besides `trange()` that iterates over an existing list. You may find it easier to use, and you can rewrite the above loop as follows:

```
starts = [i*batch_size for i in range(batches_per_epoch)]

for epoch in range(n_epochs):
    with tqdm.tqdm(starts, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for start in bar:
            # take a batch
            Xbatch = Xtrain[start:start+batch_size]
```

```

        ybatch = ytrain[start:start+batch_size]
        # forward pass
        y_pred = model(Xbatch)
        loss = loss_fn(y_pred, ybatch)
        acc = (y_pred.round() == ybatch).float().mean()
        # store metrics
        train_loss.append(float(loss))
        train_acc.append(float(acc))
        # backward pass
        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()
        # print progress
        bar.set_postfix(
            loss=float(loss),
            acc=f"{float(acc)*100:.2f}%"
        )

    # evaluate model at end of epoch
    y_pred = model(Xtest)
    acc = (y_pred.round() == ytest).float().mean()
    test_acc.append(float(acc))
    print(f"End of {epoch}, accuracy {acc}")

```

Listing 8.10: Using `tqdm` to iterate over a list

The following is the complete code (without the matplotlib plotting):

```

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import tqdm

# load the dataset
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
# split into input (X) and output (y) variables
X = dataset[:,0:8]
y = dataset[:,8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# split the dataset into training and test sets
Xtrain = X[:700]
ytrain = y[:700]
Xtest = X[700:]
ytest = y[700:]

model = nn.Sequential(
    nn.Linear(8, 12),
    nn.ReLU(),
    nn.Linear(12, 8),
    nn.ReLU(),
    nn.Linear(8, 1),

```

```

    nn.Sigmoid()
)
print(model)

# loss function and optimizer
loss_fn = nn.BCELoss() # binary cross entropy
optimizer = optim.Adam(model.parameters(), lr=0.0001)

n_epochs = 50 # number of epochs to run
batch_size = 10 # size of each batch
batches_per_epoch = len(Xtrain) // batch_size

# collect statistics
train_loss = []
train_acc = []
test_acc = []

for epoch in range(n_epochs):
    with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for i in bar:
            # take a batch
            start = i * batch_size
            Xbatch = Xtrain[start:start+batch_size]
            ybatch = ytrain[start:start+batch_size]
            # forward pass
            y_pred = model(Xbatch)
            loss = loss_fn(y_pred, ybatch)
            acc = (y_pred.round() == ybatch).float().mean()
            # store metrics
            train_loss.append(float(loss))
            train_acc.append(float(acc))
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
            # print progress
            bar.set_postfix(
                loss=float(loss),
                acc=f"{float(acc)*100:.2f}%"
            )
        # evaluate model at end of epoch
        y_pred = model(Xtest)
        acc = (y_pred.round() == ytest).float().mean()
        test_acc.append(float(acc))
    print(f"End of {epoch}, accuracy {acc}")

```

Listing 8.11: Complete code to train a model with progress printed using *tqdm*

8.4 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Software

tqdm.

<https://github.com/tqdm/tqdm>

Articles

torch.optim. PyTorch API.

<https://pytorch.org/docs/stable/optim.html>

Optimizing Model Parameters. PyTorch tutorials.

https://pytorch.org/tutorials/beginner/basics/optimization_tutorial.html

8.5 Summary

In this chapter, you looked in detail at how to properly set up a training loop for a PyTorch model. In particular, you saw:

- ▷ What are the elements needed to implement in a training loop
- ▷ How a training loop connects the training data to the gradient descent optimizer
- ▷ How to collect information in the training loop and display them

In the next chapter, you will learn how to evaluate and compare deep learning models.

Evaluating PyTorch Models

Designing a deep learning model is sometimes an art. There are a lot of decision points, and it is not easy to tell what is the best. One way to come up with a design is by trial and error and evaluating the result on real data. Therefore, it is important to have a scientific method to evaluate the performance of your neural network and deep learning models. In fact, it is also the same method to compare any kind of machine learning models on a particular usage.

In this chapter, you will discover the received workflow to robustly evaluate model performance. We use PyTorch to build our models as examples, but the method can also be applied to other models. After completing this chapter, you will know:

- ▷ How to evaluate a PyTorch model using a verification dataset
- ▷ How to evaluate a PyTorch model with k -fold cross-validation

Let's get started.

Overview

This chapter is in four parts; they are:

- ▷ Empirical Evaluation of Models
- ▷ Data Splitting
- ▷ Training a PyTorch Model with Validation
- ▷ k -Fold Cross-Validation

9.1 Empirical Evaluation of Models

In designing and configuring a deep learning model from scratch, there are a lot of decisions to make. This includes design decisions such as how many layers to use in a deep learning model, how big is each layer, and what kind of layers or activation functions to use. It can also be the choice of the loss function, optimization algorithm, number of epochs to train, and the interpretation of the model output. Luckily, sometimes, you can copy the structure of other people's networks. Sometimes, you can just make up your choice using some heuristics.

To tell if you made a good choice or not, the best way is to compare multiple alternatives by empirically evaluating them with actual data.

Deep learning is often used on problems that have very large datasets. That is tens of thousands or hundreds of thousands of data samples. This provides ample data for testing. But you need to have a robust test strategy to estimate the performance of your model on unseen data. Based on that, you can have a metric to compare among different model configurations.

9.2 Data Splitting

If you have a dataset of tens of thousands of samples or even more, you don't always need to give everything to your model for training. This will unnecessarily increase the complexity and lengthen the training time. More is not always better. You may not get the best result.

When you have a large amount of data, you should take a portion of it as the *training set* that is fed into the model for training. Another portion is kept as a *test set* to hold back from the training but verified with a trained or partially trained model as an evaluation. This step is usually called “train-test split.”

Let's consider the Pima Indians Diabetes dataset¹. You can load the data using NumPy:

```
import numpy as np
data = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
```

Listing 9.1: Loading a dataset from CSV file

There are 768 data samples. It is not a lot but is enough to demonstrate the split. Let's consider the first 66% as the training set and the remaining as the test set. The easiest way to do so is by slicing an array:

```
# find the boundary at 66% of total samples
count = len(data)
n_train = int(count * 0.66)
# split the data at the boundary
train_data = data[:n_train]
test_data = data[n_train:]
```

Listing 9.2: Splitting the dataset into training and test sets

The choice of 66% is arbitrary, but you do not want the training set too small. Sometimes you may use 70%–30% split. But if the dataset is huge, you may even use 30%–70% split if 30% of training data is large enough.

If you split the data in this way, you're suggesting the datasets are shuffled so that the training set and the test set are equally diverse. If you find the original dataset is sorted and take the test set only at the end, you may find you have all the test data belonging to the same class or carrying the same value in one of the input features. That's not ideal.

Of course, you can call `np.random.shuffle(data)` before the split to avoid that. But many machine learning engineers usually use scikit-learn for this. See this example:

¹<https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.csv>

```
import numpy as np
from sklearn.model_selection import train_test_split

data = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
train_data, test_data = train_test_split(data, test_size=0.33)
```

Listing 9.3: Splitting dataset using scikit-learn

But more commonly, it is done after you separate the input feature and output labels. Note that this function from scikit-learn can work not only on NumPy arrays but also on PyTorch tensors:

```
import numpy as np
import torch
from sklearn.model_selection import train_test_split

data = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
X = data[:, 0:8]
y = data[:, 8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33)
```

Listing 9.4: Splitting PyTorch tensors using scikit-learn

9.3 Training a PyTorch Model with Validation

Let's revisit the code for building and training a deep learning model on this dataset:

```
import torch
import torch.nn as nn
import torch.optim as optim
import tqdm

...

model = nn.Sequential(
    nn.Linear(8, 12),
    nn.ReLU(),
    nn.Linear(12, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
    nn.Sigmoid()
)

# loss function and optimizer
loss_fn = nn.BCELoss() # binary cross-entropy
optimizer = optim.Adam(model.parameters(), lr=0.0001)

n_epochs = 50 # number of epochs to run
batch_size = 10 # size of each batch
batches_per_epoch = len(X_train) // batch_size
```

```

for epoch in range(n_epochs):
    with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for i in bar:
            # take a batch
            start = i * batch_size
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            # forward pass
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
            # print progress
            bar.set_postfix(
                loss=float(loss)
            )

```

Listing 9.5: Building and training a PyTorch model

In this code, one batch is extracted from the training set in each iteration and sent to the model in the forward pass. Then you compute the gradient in the backward pass and update the weights.

While, in this case, you used binary cross-entropy as the loss metric in the training loop, you may be more concerned with the prediction accuracy. Calculating accuracy is easy. You round off the output (in the range of 0 to 1) to the nearest integer so you can get a binary value of 0 or 1. Then you count how much percentage your prediction matched the label; this gives you the accuracy.

But what is your prediction? It is `y_pred` above, which is the prediction by your current model on `X_batch`. Adding accuracy to the training loop becomes this:

```

for epoch in range(n_epochs):
    with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for i in bar:
            # take a batch
            start = i * batch_size
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            # forward pass
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()

```

```

# print progress, with accuracy
acc = (y_pred.round() == y_batch).float().mean()
bar.set_postfix(
    loss=float(loss)
    acc=float(acc)
)

```

Listing 9.6: Training loop with accuracy metric reported

However, the `X_batch` and `y_batch` is used by the optimizer, and the optimizer will fine-tune your model so that it can predict `y_batch` from `X_batch`. And now you're using accuracy to check if `y_pred` match with `y_batch`. It is like cheating because if your model somehow remembers the solution, it can just report to you the `y_pred` and get perfect accuracy without actually inferring `y_pred` from `X_batch`.

Indeed, a deep learning model can be so convoluted that you cannot know if your model simply remembers the answer or is inferring the answer. Therefore, the best way is *not* to calculate accuracy from `X_batch` or anything from `X_train` but from something else: your test set. Let's add an accuracy measurement *after* each epoch using `X_test`:

```

for epoch in range(n_epochs):
    with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for i in bar:
            # take a batch
            start = i * batch_size
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            # forward pass
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
            # print progress
            acc = (y_pred.round() == y_batch).float().mean()
            bar.set_postfix(
                loss=float(loss),
                acc=float(acc)
            )
        # evaluate model at end of epoch
        y_pred = model(X_test)
        acc = (y_pred.round() == y_test).float().mean()
        acc = float(acc)
        print(f"End of {epoch}, accuracy {acc}")

```

Listing 9.7: Training loop with accuracy measurement after each epoch

In this case, the `acc` in the inner for-loop is just a metric showing the progress. Not much difference in displaying the loss metric, except it is not involved in the gradient descent algorithm. And you expect the accuracy to improve as the loss metric also improves.

In the outer for-loop, at the end of each epoch, you calculate the accuracy from `X_test`. The workflow is similar: You give the test set to the model and ask for its prediction, then count the number of matched results with your test set labels. But this accuracy is the one you should care about. It should improve as the training progresses, but if you do not see it improve (i.e., accuracy increase) or even deteriorates, you have to interrupt the training as it seems to start overfitting. Overfitting is when the model started to remember the training set rather than learning to infer the prediction from it. A sign of that is the accuracy from the training set keeps increasing while the accuracy from the test set decreases.

The following is the complete code to implement everything above, from data splitting to validation using the test set:

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import tqdm
from sklearn.model_selection import train_test_split

data = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
X = data[:, 0:8]
y = data[:, 8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33)

model = nn.Sequential(
    nn.Linear(8, 12),
    nn.ReLU(),
    nn.Linear(12, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
    nn.Sigmoid()
)

# loss function and optimizer
loss_fn = nn.BCELoss() # binary cross entropy
optimizer = optim.Adam(model.parameters(), lr=0.0001)

n_epochs = 50 # number of epochs to run
batch_size = 10 # size of each batch
batches_per_epoch = len(X_train) // batch_size

for epoch in range(n_epochs):
    with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for i in bar:
            # take a batch
            start = i * batch_size
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            # forward pass
```

```

        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        # backward pass
        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()
        # print progress
        acc = (y_pred.round() == y_batch).float().mean()
        bar.set_postfix(
            loss=float(loss),
            acc=float(acc)
        )
    # evaluate model at end of epoch
    y_pred = model(X_test)
    acc = (y_pred.round() == y_test).float().mean()
    acc = float(acc)
    print(f"End of {epoch}, accuracy {acc}")

```

Listing 9.8: Complete code to train a model with evaluation using a test set

The code above will print the following:

```

End of 0, accuracy 0.5787401795387268
End of 1, accuracy 0.6102362275123596
End of 2, accuracy 0.6220472455024719
End of 3, accuracy 0.6220472455024719
...
End of 17, accuracy 0.6811023354530334
End of 18, accuracy 0.6850393414497375
End of 19, accuracy 0.6889764070510864
End of 20, accuracy 0.6850393414497375
End of 21, accuracy 0.6889764070510864
End of 22, accuracy 0.6889764070510864
End of 23, accuracy 0.6889764070510864
End of 24, accuracy 0.6889764070510864
End of 25, accuracy 0.6850393414497375
End of 26, accuracy 0.6811023354530334
End of 27, accuracy 0.6771653294563293
End of 28, accuracy 0.6771653294563293
End of 29, accuracy 0.6692913174629211
End of 30, accuracy 0.6732283234596252
End of 31, accuracy 0.6692913174629211
End of 32, accuracy 0.6692913174629211
End of 33, accuracy 0.6732283234596252
End of 34, accuracy 0.6771653294563293
End of 35, accuracy 0.6811023354530334
End of 36, accuracy 0.6811023354530334
End of 37, accuracy 0.6811023354530334
End of 38, accuracy 0.6811023354530334
End of 39, accuracy 0.6811023354530334
End of 40, accuracy 0.6811023354530334
End of 41, accuracy 0.6771653294563293
End of 42, accuracy 0.6771653294563293

```

```

End of 43, accuracy 0.6771653294563293
End of 44, accuracy 0.6771653294563293
End of 45, accuracy 0.6771653294563293
End of 46, accuracy 0.6771653294563293
End of 47, accuracy 0.6732283234596252
End of 48, accuracy 0.6732283234596252
End of 49, accuracy 0.6732283234596252

```

Output 9.1: Accuracy metric on the test set printed from the training loop

9.4 *k*-Fold Cross-Validation

In the above example, you calculated the accuracy from the test set. It is used as a *score* for the model as you progressed in the training. You want to stop at the point where this score is at its maximum. In fact, by merely comparing the score from this test set, you know your model works best after epoch 21 and start to overfit afterwards. Is that right?

If you built two models of different designs, should you just compare these models' accuracy on the same test set and claim one is better than another?

You have a test set to evaluate your model. But you may argue that, by chance, one model fits better to this particular test set but not always better. To make a stronger argument on which model is better independent of the selection of the test set, you can try *multiple test sets* and average the accuracy.

This is what a *k*-fold cross-validation does. It is a procedure to decide on which *design* works better. It works by repeating the training process from scratch for *k* times, each with a different composition of the training and test set. Because of that, you will have *k* models and *k* accuracy scores from their respective test set. You are not only interested in the average accuracy, but also the standard deviation. The standard deviation tells whether the accuracy score is consistent or some test set is particularly good or bad to a model.

Since *k*-fold cross-validation is to train the model from scratch a few times, it is best to wrap around the training loop in a function:

```

def model_train(X_train, y_train, X_test, y_test):
    # create new model
    model = nn.Sequential(
        nn.Linear(8, 12),
        nn.ReLU(),
        nn.Linear(12, 8),
        nn.ReLU(),
        nn.Linear(8, 1),
        nn.Sigmoid()
    )

    # loss function and optimizer
    loss_fn = nn.BCELoss() # binary cross-entropy
    optimizer = optim.Adam(model.parameters(), lr=0.0001)

    n_epochs = 25    # number of epochs to run

```

```

batch_size = 10 # size of each batch
batches_per_epoch = len(X_train) // batch_size

for epoch in range(n_epochs):
    with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0, disable=True) as bar:
        bar.set_description(f"Epoch {epoch}")
        for i in bar:
            # take a batch
            start = i * batch_size
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            # forward pass
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
            # print progress
            acc = (y_pred.round() == y_batch).float().mean()
            bar.set_postfix(
                loss=float(loss),
                acc=float(acc)
            )
        # evaluate accuracy at end of training
        y_pred = model(X_test)
        acc = (y_pred.round() == y_test).float().mean()
    return float(acc)

```

Listing 9.9: A function to create a PyTorch model

The code above is deliberately not printing anything (with `disable=True` in `tqdm`) to keep the screen less cluttered.

Also from `scikit-learn`, you have a function for *k*-fold cross-validation. You can make use of it to produce a robust estimate of model accuracy:

```

from sklearn.model_selection import StratifiedKFold

# define 5-fold cross-validation test harness
kfold = StratifiedKFold(n_splits=5, shuffle=True)
cv_scores = []
for train, test in kfold.split(X, y):
    # create model, train, and get accuracy
    acc = model_train(X[train], y[train], X[test], y[test])
    print("Accuracy: %.2f" % acc)
    cv_scores.append(acc)
# evaluate the model
print("%.2f%% (+/- %.2f%%)" % (np.mean(cv_scores)*100, np.std(cv_scores)*100))

```

Listing 9.10: *k*-fold cross-validation on a PyTorch model

Running this prints:


```

Accuracy: 0.64
Accuracy: 0.67
Accuracy: 0.68
Accuracy: 0.63
Accuracy: 0.59
64.05% (+/- 3.30%)

```

Output 9.2: Result from *k*-fold cross-validation

In scikit-learn, there are multiple *k*-fold cross-validation functions, and the one used here is stratified *k*-fold. It assumes *y* are class labels and takes into account of their values such that it will provide a balanced class representation in the splits.

The code above used $k = 5$ or five splits. It means splitting the dataset into five equal portions, picking one of them as the test set and combining the rest into a training set. There are five ways of doing that, so the for-loop above will have five iterations. In each iteration, you call the `model_train()` function and obtain the accuracy score in return. Then you save it into a list, which will be used to calculate the mean and standard deviation at the end.

The `kfold` object will return to you the *indices*. Hence you do not need to run the train-test split in advance but use the indices provided to extract the training set and test set on the fly when you call the `model_train()` function.

The result above shows the model is moderately good, at 64% average accuracy. And this score is stable since the standard deviation is at 3%. This means that most of the time, you expect the model accuracy to be 61% to 67%. You may try to change the model above, such as adding or removing a layer, and see how much change you have in the mean and standard deviation. You may also try to increase the number of epochs used in training and observe the result.

The mean and standard deviation from the *k*-fold cross-validation is what you should use to benchmark a model design.

Tying it all together, below is the complete code for *k*-fold cross-validation:

```

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import tqdm
from sklearn.model_selection import StratifiedKFold

data = np.loadtxt("pima-indians-diabetes.csv", delimiter=",")
X = data[:, 0:8]
y = data[:, 8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

def model_train(X_train, y_train, X_test, y_test):
    # create new model
    model = nn.Sequential(
        nn.Linear(8, 12),
        nn.ReLU(),

```

```

        nn.Linear(12, 8),
        nn.ReLU(),
        nn.Linear(8, 1),
        nn.Sigmoid()
    )

    # loss function and optimizer
    loss_fn = nn.BCELoss() # binary cross entropy
    optimizer = optim.Adam(model.parameters(), lr=0.0001)

    n_epochs = 25 # number of epochs to run
    batch_size = 10 # size of each batch
    batches_per_epoch = len(X_train) // batch_size

    for epoch in range(n_epochs):
        with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0, disable=True) as bar:
            bar.set_description(f"Epoch {epoch}")
            for i in bar:
                # take a batch
                start = i * batch_size
                X_batch = X_train[start:start+batch_size]
                y_batch = y_train[start:start+batch_size]
                # forward pass
                y_pred = model(X_batch)
                loss = loss_fn(y_pred, y_batch)
                # backward pass
                optimizer.zero_grad()
                loss.backward()
                # update weights
                optimizer.step()
                # print progress
                acc = (y_pred.round() == y_batch).float().mean()
                bar.set_postfix(
                    loss=float(loss),
                    acc=float(acc)
                )
            # evaluate accuracy at end of training
            y_pred = model(X_test)
            acc = (y_pred.round() == y_test).float().mean()
            return float(acc)

# define 5-fold cross validation test harness
kfold = StratifiedKFold(n_splits=5, shuffle=True)
cv_scores = []
for train, test in kfold.split(X, y):
    # create model, train, and get accuracy
    acc = model_train(X[train], y[train], X[test], y[test])
    print("Accuracy: %.2f" % acc)
    cv_scores.append(acc)

# evaluate the model
print("%.2f%% (+/- %.2f%%)" % (np.mean(cv_scores)*100, np.std(cv_scores)*100))

```

Listing 9.11: Evaluating a model using *k*-fold cross-validation

9.5 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Articles

Stratified K-Folds cross-validator. scikit-learn documentation.

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html

KFold cross-validator. scikit-learn documentation.

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.KFold.html

9.6 Summary

In this chapter, you discovered the importance of having a robust way to estimate the performance of your deep learning models on unseen data, and you learned how to do that. You saw:

- ▷ How to split data into training and test sets using scikit-learn
- ▷ How to do k -fold cross-validation with the help of scikit-learn
- ▷ How to modify the training loop in a PyTorch model to incorporate test set validation and cross-validation

In the next few chapters, you will put everything you learned so far into practice by finishing some projects.

Project: Building a Multiclass Classification Model in PyTorch

10

The PyTorch library is for deep learning. Some applications of deep learning models are used to solve regression or classification problems. In this chapter, you will discover how to use PyTorch to develop and evaluate neural network models for multiclass classification problems.

After completing this step-by-step tutorial, you will know:

- ▷ How to load data from CSV and make it available to PyTorch
- ▷ How to prepare multiclass classification data for modeling with neural networks
- ▷ How to use cross-validation to evaluate a PyTorch neural network model

Let's get started.

10.1 Problem Description

In this chapter, you will use a standard machine learning dataset called the iris flowers dataset. It is a well-studied dataset and good for practicing machine learning. It has four input variables; all are numeric and length measurements in centimeters. Therefore, they are on a similar scale. Each data sample describes the properties of an observed iris flower. Its goal is to use the measurements (input features) to classify the iris species (output label).

There are three iris species in the dataset. Therefore, it is a multiclass classification problem. Multiclass classification problems are special because they require special handling to specify a class.

This dataset came from Sir Ronald Fisher, the father of modern statistics. It is the best-known dataset for pattern recognition, and you can achieve a model accuracy in the range of 95% to 97%. You can make this your target in developing the deep learning model.

You can download the iris flowers dataset from the UCI Machine Learning repository and place it in your current working directory with the filename “iris.csv”. You can also download the dataset here:

- ▷ <https://raw.githubusercontent.com/jbrownlee/Datasets/master/iris.csv>

10.2 Load the Dataset

There are multiple ways to read a CSV file. The easiest way is probably to use a pandas library. After reading the dataset, you want to split it into features and labels as you need to further process the labels before use. Unlike NumPy or PyTorch tensors, pandas DataFrame can do slicing by indices only through `iloc`:

```
import pandas as pd
data = pd.read_csv("iris.csv", header=None)
X = data.iloc[:, 0:4]
y = data.iloc[:, 4:]
```

Listing 10.1: Loading a dataset from CSV file

Now, you have loaded the dataset and split the attributes (i.e., input features, columns in the DataFrame) as `X` and the output variables (i.e., species labels) as a single-column DataFrame `y`.

10.3 Encode the Categorical Variable

The species labels are strings, but you want them in numbers. It is because numerical data are easier to use. In this dataset, the three class labels are *Iris-setosa*, *Iris-versicolor*, and *Iris-virginica*. One way to convert these labels into a number (i.e., encode them) is simply to assign an integer value such as 0, 1, or 2 to replace these labels. But there is a problem: You do not want the model to think that *Iris-virginica* it is the sum of *Iris-setosa* and *Iris-versicolor*. In fact, in statistics, there are levels of measurement:

- ▷ Nominal numbers: Those numbers are, in fact, names. Operations on them do not make sense
- ▷ Ordinal numbers: They are orders of something. Comparing for greater or less than makes sense, but addition or subtraction does not
- ▷ Interval numbers: They are measurements, such as the year today, so that subtraction bears meaning (e.g., how old are you), but the zero value is arbitrary and not special
- ▷ Ratio numbers: Like interval, but the zero is meaningful, such as the measure of length or time. In this case, both subtraction and division bear meaning, and you can say that something is twice as long.

The encoded label is nominal. You do not want to mistake it as interval or ratio data, but your model would not know. One way to avoid such a mistake is with *one-hot encoding*, which instead of converting a label into an integer, converts a label into a *one-hot vector*. A one-hot vector is a vector of integers, but only one of them is 1 and the rest are all zero. In this case, you convert the labels into the following:

<i>Iris-setosa</i>	1	0	0
<i>Iris-versicolor</i>	0	1	0
<i>Iris-virginica</i>	0	0	1

Output 10.1: One-hot labels to the iris dataset

The above is a one-hot encoded binary matrix. You don't need to create it manually. You can encode the strings consistently to integers using the scikit-learn class `LabelEncoder` or into one-hot encoding vectors using the class `OneHotEncoder`:

```
...
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False).fit(y)
print(ohe.categories_)

y = ohe.transform(y)
print(y)
```

Listing 10.2: Transforming textual labels into one-hot encoding

From these, you can see the `OneHotEncoder` learned the three categories:

```
[array(['Iris-setosa', 'Iris-versicolor', 'Iris-virginica'], dtype=object)]
```

Output 10.2: The categories as from `ohe.categories_`

and the string labels are transformed into one-hot vectors like the following:

```
[[1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 ...
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 0. 1.]]
```

Output 10.3: The one-hot encoded label in `y`

10.4 Define the Neural Network Model

Now you need to have a model that can take the input and predict the output, ideally in the form of one-hot vectors. There is no science behind the design of a perfect neural network model. But know one thing — it has to take in a vector of 4 features and output a vector of 3 values. The 4 features correspond to what you have in the dataset. The 3-value output is because we know the one-hot vector has 3 elements. Anything can be in between, known as the “hidden layers,” since they are neither input nor output.

The simplest is to have only one hidden layer. Let's make one like this:

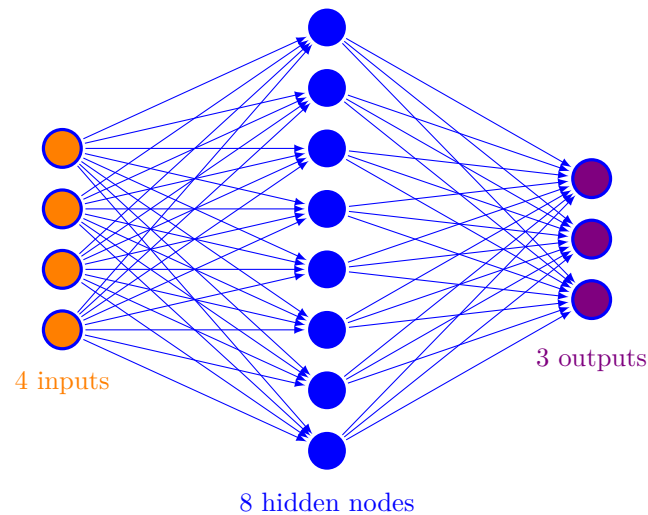


Figure 10.1: Example network structure

Such a design is called the network topology. You should use a *softmax* activation at the output layer. In the formula, it means:

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^3 e^{z_j}}$$

This normalizes the values (z_1, z_2, z_3) and applies a nonlinear function such that the sum of all 3 outputs will be 1, and each of them is in the range of 0 to 1. This makes the output look like a vector of probabilities. The use of the softmax function at the output is the signature of a multiclass classification model. But in PyTorch, you can skip this if you combine it with an appropriate loss function.

In PyTorch, you can build such a model as follows:

```
import torch.nn as nn

class Multiclass(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(4, 8)
        self.act = nn.ReLU()
        self.output = nn.Linear(8, 3)

    def forward(self, x):
        x = self.act(self.hidden(x))
        x = self.output(x)
        return x

model = Multiclass()
```

Listing 10.3: The model in PyTorch code

The output of this model is the *weight* of the three classes. Ideally, the model output will be such that only one of the elements is positive infinity, and the rest are negative infinity, making an extreme contrast with absolute confidence to which of the three classes the input

features belong. In an imperfect situation, as always happens, you can expect a good model to tell you that one of the values is very positive, and the others are very negative. Or if you transform these values using a sigmoid function or softmax function, one is very close to 1, and the others are very close to 0.

In this case, the loss metric for the output can simply be measuring how close the output is to the one-hot vector you transformed from the label. But usually, in multiclass classification, you use categorical cross-entropy as the loss metric. In the formula, it is:

$$H(p, q) = - \sum_x p(x) \log q(x)$$

This means, given the real probability vector $p(x)$ and the predicted probability vector $q(x)$, the similarity is the sum of the product of $p(x)$ and $\log q(x)$ for each element x . The one-hot vector is considered as the probability vector $p(x)$, and the model output is $q(x)$. Since it is a one-hot vector, only the actual class has $p(x) = 1$, and the other classes have $p(x) = 0$. The sum above is essentially $-\log q(x)$ of the actual class x . The value will be 0 when $q(x) = 1$, and as $q(x)$ approaches 0 (the minimum value softmax can produce), $-\log q(x)$ approaches infinity.

Below is how you can define the loss metric. The `CrossEntropyLoss` function in PyTorch combines the softmax function with the cross-entropy calculation, so you don't need any activation function at the output layer of your model. You also need an optimizer, and Adam is chosen below.

```
import torch.optim as optim

loss_fn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

Listing 10.4: Loss function and optimizer

Note that when you define the optimizer, you need to tell it the model parameters as well since these are what the optimizer is going to update.

Now you need to run the training loop to train your model. Minimally, you need to put three steps in the loop: a forward pass, a backward pass, and the weight update. The forward pass provides the input to the model and takes the output. The backward pass starts with the loss metric, which is based on the model output, and propagates the gradient back to the input. The weight update is based on the gradient used to update the weights.

A minimal training loop can be implemented using a for-loop. But you can make use of `tqdm` to create a progress bar visualization:

```
import tqdm

# convert pandas DataFrame (X) and numpy array (y) into PyTorch tensors
X = torch.tensor(X.values, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32)

# training parameters
n_epochs = 200
batch_size = 5
```



```

batches_per_epoch = len(X) // batch_size

for epoch in range(n_epochs):
    with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for i in bar:
            # take a batch
            start = i * batch_size
            X_batch = X[start:start+batch_size]
            y_batch = y[start:start+batch_size]
            # forward pass
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()

```

Listing 10.5: Training loop

10.5 Benchmark the Model

The goal of a model is never to match the dataset per se. The reason you want to build a machine learning model is to prepare for the data you will encounter in the future, which is unseen yet. How can you know the model can do that? You need a test set. It is a dataset that is the same structure as the one used in training but is separate. So it is like unseen data from the training process, and you can take that as a benchmark. This technique of evaluating a model is called *cross-validation*.

Usually, you do not add a test set but split the data set you obtained into a training set and a test set. Then you use the test set to evaluate the model at the end. Such a benchmark serves another purpose: You do not want your model to overfit. It means the model learned too much about the training set and failed to generalize. If this happens, you will see that the model does not give a good result on the test set.

Splitting the data into a training set and a test set can be easily done with scikit-learn. The workflow from loading data to one-hot encoding and splitting is as follows:

```

import pandas as pd
import torch
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder

# read data and apply one-hot encoding
data = pd.read_csv("iris.csv", header=None)
X = data.iloc[:, 0:4]
y = data.iloc[:, 4:]
ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False).fit(y)
y = ohe.transform(y)

```

```
# convert pandas DataFrame (X) and numpy array (y) into PyTorch tensors
X = torch.tensor(X.values, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32)

# split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)
```

Listing 10.6: Splitting data into training and test sets

The arguments `train_size=0.7` with `shuffle=True` means to randomly select 70% of the samples in the dataset into the training set, while the rest will become the test set.

Once you do that, you need to modify the training loop to use the training set in training and use the test set at the end of each epoch for benchmarking:

```
n_epochs = 200
batch_size = 5
batches_per_epoch = len(X_train) // batch_size

for epoch in range(n_epochs):
    with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for i in bar:
            # take a batch
            start = i * batch_size
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            # forward pass
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
        y_pred = model(X_test)
        ce = loss_fn(y_pred, y_test)
        acc = (torch.argmax(y_pred, 1) == torch.argmax(y_test, 1)).float().mean()
        print(f"Epoch {epoch} validation: Cross-entropy={float(ce)}, Accuracy={float(acc)}")
```

Listing 10.7: Benchmarking the model in the training loop

That's almost everything you need to finish a deep learning model in PyTorch. But you may also want to do a bit more: Firstly, after all the training epochs, you may want to roll back the model to the best you ever achieved rather than the last one. Secondly, you may want to produce a graph to visualize how the cross-entropy and accuracy progressed.

It is not difficult to do. In the training loop, you keep track of the accuracy on the test set and keep a copy of the model whenever this accuracy is higher. And at the same time, remember the metrics you calculated in a list. Then at the end of the training loop, you restore the best model you ever saw and plot the metrics as a time series.

In code, this is how the training loop is modified into:

```

import copy
import tqdm
import numpy as np

n_epochs = 200
batch_size = 5
batches_per_epoch = len(X_train) // batch_size

best_acc = - np.inf  # init to negative infinity
best_weights = None
train_loss_hist = []
train_acc_hist = []
test_loss_hist = []
test_acc_hist = []

for epoch in range(n_epochs):
    epoch_loss = []
    epoch_acc = []
    # set model in training mode and run through each batch
    model.train()
    with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
        bar.set_description(f"Epoch {epoch}")
        for i in bar:
            # take a batch
            start = i * batch_size
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            # forward pass
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
            # compute and store metrics
            acc = (torch.argmax(y_pred, 1) == torch.argmax(y_batch, 1)).float().mean()
            epoch_loss.append(float(loss))
            epoch_acc.append(float(acc))
            bar.set_postfix(
                loss=float(loss),
                acc=float(acc)
            )
    # set model in evaluation mode and run through the test set
    model.eval()
    y_pred = model(X_test)
    ce = loss_fn(y_pred, y_test)
    acc = (torch.argmax(y_pred, 1) == torch.argmax(y_test, 1)).float().mean()
    ce = float(ce)
    acc = float(acc)
    train_loss_hist.append(np.mean(epoch_loss))
    train_acc_hist.append(np.mean(epoch_acc))
    test_loss_hist.append(ce)
    test_acc_hist.append(acc)

```

```

if acc > best_acc:
    best_acc = acc
    best_weights = copy.deepcopy(model.state_dict())
print(f"Epoch {epoch} validation: Cross-entropy={ce}, Accuracy={acc}")

model.load_state_dict(best_weights)

```

Listing 10.8: The training loop with the evaluation metric collected and best model restored

When you run it, you will see something like this, where you see the accuracy and cross-entropy loss at each epoch:

```

Epoch 0: 100%|██████████| 21/21 [00:00<00:00, 850.82batch/s, acc=0.4, loss=1.23]
Epoch 0 validation: Cross-entropy=1.10, Accuracy=60.0%
Epoch 1: 100%|██████████| 21/21 [00:00<00:00, 3155.53batch/s, acc=0.4, loss=1.24]
Epoch 1 validation: Cross-entropy=1.08, Accuracy=57.8%
Epoch 2: 100%|██████████| 21/21 [00:00<00:00, 3489.58batch/s, acc=0.4, loss=1.24]
Epoch 2 validation: Cross-entropy=1.07, Accuracy=60.0%
Epoch 3: 100%|██████████| 21/21 [00:00<00:00, 3312.79batch/s, acc=0.4, loss=1.22]
Epoch 3 validation: Cross-entropy=1.06, Accuracy=62.2%
...
Epoch 197: 100%|██████████| 21/21 [00:00<00:00, 3529.57batch/s, acc=1, loss=0.563]
Epoch 197 validation: Cross-entropy=0.61, Accuracy=97.8%
Epoch 198: 100%|██████████| 21/21 [00:00<00:00, 3479.10batch/s, acc=1, loss=0.563]
Epoch 198 validation: Cross-entropy=0.61, Accuracy=97.8%
Epoch 199: 100%|██████████| 21/21 [00:00<00:00, 3337.52batch/s, acc=1, loss=0.563]
Epoch 199 validation: Cross-entropy=0.61, Accuracy=97.8%

```

Output 10.4: Output from the training loop

You added more lines into the training loop but for a good reason. It is a good practice to switch the model between training mode and evaluation mode when you are switching between the training set and test set. In this particular model, nothing is changed. But for some other models, it will affect the model's behavior.

You collected metrics in Python lists. You need to be careful to convert the PyTorch tensor (even if it is a scalar value) into a Python float. The purpose of this conversion is to make a copy of the number so that PyTorch will not silently mutate it (e.g., through the optimizer).

After each epoch, you calculated the accuracy based on the test set, and the model weight is stored if you saw the accuracy is higher. However, when you take out the model weight, you should make a deep copy; otherwise, you lost them when the model changes its weights in the next epoch.

Finally, you can plot the loss and accuracy across epochs using matplotlib as follows:

```

import matplotlib.pyplot as plt

plt.plot(train_loss_hist, label="train")
plt.plot(test_loss_hist, label="test")
plt.xlabel("epochs")
plt.ylabel("cross entropy")

```

```
plt.legend()
plt.show()

plt.plot(train_acc_hist, label="train")
plt.plot(test_acc_hist, label="test")
plt.xlabel("epochs")
plt.ylabel("accuracy")
plt.legend()
plt.show()
```

Listing 10.9: Plotting the loss and accuracy

A typical result is as follows:

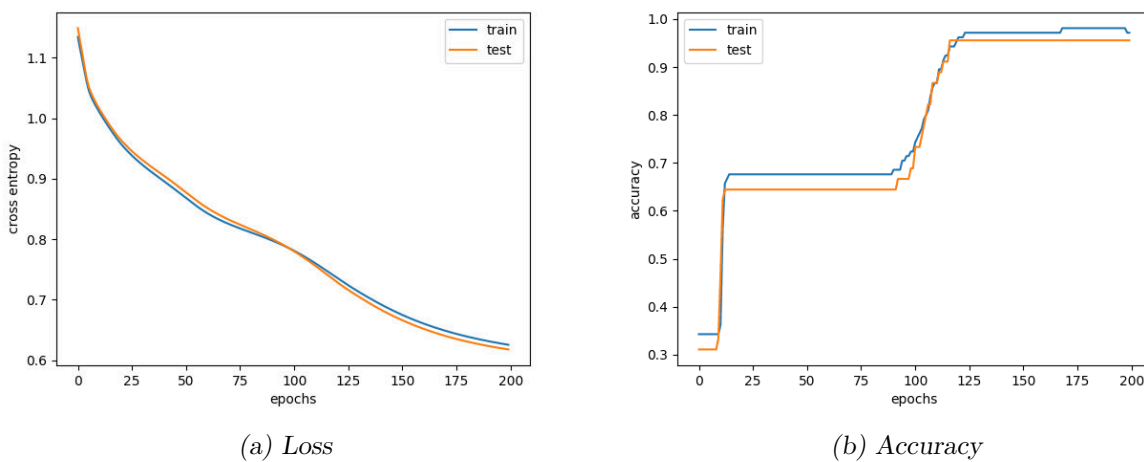


Figure 10.2: Training and validation loss and accuracy

You can see from the graph that at the beginning, both training and test accuracy are very low. This is when your model is underfitting and is performing horribly. As you keep training the model, the accuracy increases, and the cross-entropy loss decreases. But at a certain point, the training accuracy is higher than the test accuracy, and in fact, even when the training accuracy improves, the test accuracy flattened or even lowered. This is when the model overfitted, and you do not want to use such a model. That's why you want to keep track of the test accuracy and restore the model weight to the best result based on the test set.

10.6 Complete Example

Putting everything together, the following is the complete code:

```
import copy

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
```

```

import torch.nn as nn
import torch.optim as optim
import tqdm
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder

# read data and apply one-hot encoding
data = pd.read_csv("iris.csv", header=None)
X = data.iloc[:, 0:4]
y = data.iloc[:, 4:]
ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False).fit(y)
y = ohe.transform(y)

# convert pandas DataFrame (X) and numpy array (y) into PyTorch tensors
X = torch.tensor(X.values, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32)

# split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

class Multiclass(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(4, 8)
        self.act = nn.ReLU()
        self.output = nn.Linear(8, 3)

    def forward(self, x):
        x = self.act(self.hidden(x))
        x = self.output(x)
        return x

# loss metric and optimizer
model = Multiclass()
loss_fn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# prepare model and training parameters
n_epochs = 200
batch_size = 5
batches_per_epoch = len(X_train) // batch_size

best_acc = - np.inf # init to negative infinity
best_weights = None
train_loss_hist = []
train_acc_hist = []
test_loss_hist = []
test_acc_hist = []

# training loop
for epoch in range(n_epochs):
    epoch_loss = []
    epoch_acc = []
    # set model in training mode and run through each batch

```

```

model.train()
with tqdm.trange(batches_per_epoch, unit="batch", mininterval=0) as bar:
    bar.set_description(f"Epoch {epoch}")
    for i in bar:
        # take a batch
        start = i * batch_size
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        # forward pass
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        # backward pass
        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()
        # compute and store metrics
        acc = (torch.argmax(y_pred, 1) == torch.argmax(y_batch, 1)).float().mean()
        epoch_loss.append(float(loss))
        epoch_acc.append(float(acc))
        bar.set_postfix(
            loss=float(loss),
            acc=float(acc)
        )

    # set model in evaluation mode and run through the test set
    model.eval()
    y_pred = model(X_test)
    ce = loss_fn(y_pred, y_test)
    acc = (torch.argmax(y_pred, 1) == torch.argmax(y_test, 1)).float().mean()
    ce = float(ce)
    acc = float(acc)
    train_loss_hist.append(np.mean(epoch_loss))
    train_acc_hist.append(np.mean(epoch_acc))
    test_loss_hist.append(ce)
    test_acc_hist.append(acc)
    if acc > best_acc:
        best_acc = acc
        best_weights = copy.deepcopy(model.state_dict())
    print(f"Epoch {epoch} validation: Cross-entropy={ce:.2f}, Accuracy={acc*100:.1f}%")

# Restore best model
model.load_state_dict(best_weights)

# Plot the loss and accuracy
plt.plot(train_loss_hist, label="train")
plt.plot(test_loss_hist, label="test")
plt.xlabel("epochs")
plt.ylabel("cross entropy")
plt.legend()
plt.show()

plt.plot(train_acc_hist, label="train")
plt.plot(test_acc_hist, label="test")
plt.xlabel("epochs")

```

```
plt.ylabel("accuracy")
plt.legend()
plt.show()
```

Listing 10.10: An example of multiclass classification in PyTorch

10.7 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Articles

Iris Data Set. UCI Machine Learning Repository.

<https://archive.ics.uci.edu/ml/datasets/Iris>

Softmax. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.Softmax.html>

CrossEntropyLoss. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html>

One-Hot encoder. scikit-learn documentation.

<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html>

10.8 Summary

In this chapter, you discovered how to develop and evaluate a neural network for multiclass classification using PyTorch.

By completing this chapter, you learned:

- ▷ How to load data and convert them to PyTorch tensors
- ▷ How to prepare multiclass classification data for modeling using one-hot encoding
- ▷ How to define a deep learning model with PyTorch for multiclass classification
- ▷ How to measure the likeness of model output to the expected result of a multiclass classification
- ▷ How to run a training loop on a PyTorch model and collect evaluation metrics

In the next chapter, you will do another classification project.

Project: Building a Binary Classification Model in PyTorch

11

PyTorch library is for deep learning. Some applications of deep learning models are to solve regression or classification problems. In this chapter, you will discover how to use PyTorch to develop and evaluate neural network models for binary classification problems.

After completing this chapter, you will know:

- ▷ How to load training data and make it available to PyTorch
- ▷ How to design and train a neural network
- ▷ How to evaluate the performance of a neural network model using k -fold cross-validation
- ▷ How to run a model in inference mode
- ▷ How to create receiver operating characteristics curve for a binary classification model

Let's get started.

11.1 Description of the Dataset

The dataset you will use in this chapter is the Sonar dataset.

This is a dataset that describes sonar chirp returns bouncing off different services. The 60 input variables are the strength of the returns at different angles. It is a binary classification problem that requires a model to differentiate rocks from metal cylinders.

You can learn more about this dataset on the UCI Machine Learning repository. You can download the dataset for free and place it in your working directory with the filename `sonar.csv`:

- ▷ <https://archive.ics.uci.edu/ml/machine-learning-databases/undocumented/connectionist-bench/sonar/sonar.all-data>

It is a well-understood dataset. All the variables are continuous and generally in the range of 0 to 1. The output variable is a string “M” for mine and “R” for rock, which will need to be converted to integers 1 and 0.

A benefit of using this dataset is that it is a standard benchmark problem. This means that we have some idea of the expected skill of a good model. Using cross-validation, a neural network should be able to achieve a performance¹ of 84% to 88% accuracy.

11.2 Load the Dataset

If you have downloaded the dataset in CSV format and saved it as `sonar.csv` in the local directory, you can load the dataset using pandas. There are 60 input variables (x) and one output variable (y). Because the file contains mixed data of strings and numbers, it is easier to read them using pandas rather than other tools such as NumPy.

Data can be read as follows:

```
import pandas as pd

# Read data
data = pd.read_csv("sonar.csv", header=None)
X = data.iloc[:, 0:60]
y = data.iloc[:, 60]
```

Listing 11.1: Loading the dataset

It is a binary classification dataset. You would prefer a numeric label over a string label. You can do such conversion with `LabelEncoder` in scikit-learn. The `LabelEncoder` is to map each label to an integer. In this case, there are only two labels and they will become 0 and 1.

Using it, you need to first call the `fit()` function to make it learn what labels are available. Then call `transform()` to do the actual conversion. Below is how you use `LabelEncoder` to convert y from strings into 0 and 1:

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)
```

Listing 11.2: Encode the dataset label

You can see the labels using:

```
print(encoder.classes_)
```

Listing 11.3: Show the classes as from the CSV file

which outputs:

```
['M' 'R']
```

Output 11.1: The class labels from the CSV file

and if you run `print(y)`, you would see the following

¹<http://www.is.umk.pl/projects/datasets.html#Sonar>


```
x = self.sigmoid(self.output(x))
return x
```

Listing 11.5: Building a wide model

Because it is a binary classification problem, the output have to be a vector of length 1. Then you also want the output to be between 0 and 1 so you can consider that as probability or the model's confidence of prediction that the input corresponds to the “positive” class.

A model with more layer is called a deeper model. Considering that the previous model has one layer with 180 neurons, you can try one with three layers of 60 neurons each instead. Such model can be built using PyTorch:

```
import torch.nn as nn

class Deep(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(60, 60)
        self.act1 = nn.ReLU()
        self.layer2 = nn.Linear(60, 60)
        self.act2 = nn.ReLU()
        self.layer3 = nn.Linear(60, 60)
        self.act3 = nn.ReLU()
        self.output = nn.Linear(60, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.act1(self.layer1(x))
        x = self.act2(self.layer2(x))
        x = self.act3(self.layer3(x))
        x = self.sigmoid(self.output(x))
        return x
```

Listing 11.6: Building a deep model

You can confirm that these two models are having similar number of parameters, as follows:

```
# Compare model sizes
model1 = Wide()
model2 = Deep()
print(sum([x.reshape(-1).shape[0] for x in model1.parameters()])) # 11161
print(sum([x.reshape(-1).shape[0] for x in model2.parameters()])) # 11041
```

Listing 11.7: Comparing the size of the two models

There will be all the model's parameters returned by `model1.parameters()` and each is a PyTorch tensor. Then you can reformat each tensor into a vector and count the length of the vector, using `x.reshape(-1).shape[0]`. So the above sum up the total number of parameters in each model.

11.4 Comparing Models with Cross-Validation

Should you use a wide model or a deep model? One way to tell is to use cross-validation to compare them.

It is a technique that, use a *training set* of data to train the model and then use a *test set* of data to see how accurate the model can predict. The result from test set is what you should focus on. But you do not want to test a model once because if you see an extremely good or bad result, it may be by chance. You want to run this process k times with different training and test sets, such that you are ensured that you are comparing the *model design*, not the result of a particular training.

The technique that you can use here is called k -fold cross-validation (more in Chapter 19). It is to split a larger dataset into k portions and take one portion as the test set while the $k - 1$ portions are combined as the training set. There are k different such combinations. Therefore you can repeat the experiment for k times and take the average result.

In scikit-learn, you have a function for stratified k -fold. Stratified means that when the data is split into k portions, the algorithm will look at the labels (i.e., the positive and negative classes in a binary classification problem) to ensure it is split in such a way that each portion contains equal number of either classes.

Running k -fold cross-validation is trivial, such as the following:

```
# define 5-fold cross-validation test harness
kfold = StratifiedKFold(n_splits=5, shuffle=True)
cv_scores = []
for train, test in kfold.split(X, y):
    # create model, train, and get accuracy
    model = Wide()
    acc = model_train(model, X[train], y[train], X[test], y[test])
    print("Accuracy (wide): %.2f" % acc)
    cv_scores.append(acc)

# evaluate the model
acc = np.mean(cv_scores)
std = np.std(cv_scores)
print("Model accuracy: %.2f%% (+/- %.2f%%)" % (acc*100, std*100))
```

Listing 11.8: Running k -fold cross-validation

Simply speaking, you use `StratifiedKFold()` from scikit-learn to split the dataset. This function returns to you the indices. Hence you can create the split dataset using `X[train]` and `X[test]` and named them training set and validation set (so it is not confused with “test set” which will be used later, after we picked our model design). You assume to have a function that runs the training loop on a model and give you the accuracy on the validation set. You can then find the mean and standard deviation of this score as the performance metric of such model design. Note that you need to create a new model every time in the for-loop above because you should not re-train a trained model in the k -fold cross-validation.

The training loop can be defined as follows:

```

import copy
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import tqdm

def model_train(model, X_train, y_train, X_val, y_val):
    # loss function and optimizer
    loss_fn = nn.BCELoss() # binary cross entropy
    optimizer = optim.Adam(model.parameters(), lr=0.0001)

    n_epochs = 250 # number of epochs to run
    batch_size = 10 # size of each batch
    batch_start = torch.arange(0, len(X_train), batch_size)

    # Hold the best model
    best_acc = - np.inf # init to negative infinity
    best_weights = None

    for epoch in range(n_epochs):
        model.train()
        with tqdm.tqdm(batch_start, unit="batch", mininterval=0, disable=True) as bar:
            bar.set_description(f"Epoch {epoch}")
            for start in bar:
                # take a batch
                X_batch = X_train[start:start+batch_size]
                y_batch = y_train[start:start+batch_size]
                # forward pass
                y_pred = model(X_batch)
                loss = loss_fn(y_pred, y_batch)
                # backward pass
                optimizer.zero_grad()
                loss.backward()
                # update weights
                optimizer.step()
                # print progress
                acc = (y_pred.round() == y_batch).float().mean()
                bar.set_postfix(
                    loss=float(loss),
                    acc=float(acc)
                )
            # evaluate accuracy at end of each epoch
            model.eval()
            y_pred = model(X_val)
            acc = (y_pred.round() == y_val).float().mean()
            acc = float(acc)
            if acc > best_acc:
                best_acc = acc
                best_weights = copy.deepcopy(model.state_dict())
    # restore model and return best accuracy
    model.load_state_dict(best_weights)
    return best_acc

```

Listing 11.9: Function to train a PyTorch model with the given dataset

The training loop above contains the usual elements: The forward pass, the backward pass, and the gradient descent weight updates. But it is extended to have an evaluation step after each epoch: You run the model at evaluation mode and check how the model predicts the *validation set*. The accuracy on the validation set is remembered along with the model weight. At the end of the training, the best weight is restored to the model and the best accuracy is returned. This returned value is the best you ever encountered during the many epochs of training and it is based on the validation set.

Note that you set `disable=True` in the `tqdm` above. You can set it to `False` to see the training set loss and accuracy as you progress in the training.

Remind that the goal is to pick the best design and train the model again, which in the training, you want to have an evaluation score so you know what to expect in production. Thus you should split the entire dataset you obtained into a training set and test set. Then you further split the training set in k -fold cross-validation.

With these, here is how you can compare the two model designs: By running k -fold cross-validation on each and compare the accuracy:

```
from sklearn.model_selection import StratifiedKFold, train_test_split

# train-test split: Hold out the test set for final model evaluation
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# define 5-fold cross-validation test harness
kfold = StratifiedKFold(n_splits=5, shuffle=True)
cv_scores_wide = []
for train, test in kfold.split(X_train, y_train):
    # create model, train, and get accuracy
    model = Wide()
    acc = model_train(model, X_train[train], y_train[train], X_train[test], y_train[test])
    print("Accuracy (wide): %.2f" % acc)
    cv_scores_wide.append(acc)
cv_scores_deep = []
for train, test in kfold.split(X_train, y_train):
    # create model, train, and get accuracy
    model = Deep()
    acc = model_train(model, X_train[train], y_train[train], X_train[test], y_train[test])
    print("Accuracy (deep): %.2f" % acc)
    cv_scores_deep.append(acc)

# evaluate the model
wide_acc = np.mean(cv_scores_wide)
wide_std = np.std(cv_scores_wide)
deep_acc = np.mean(cv_scores_deep)
deep_std = np.std(cv_scores_deep)
print("Wide: %.2f%% (+/- %.2f%%)" % (wide_acc*100, wide_std*100))
print("Deep: %.2f%% (+/- %.2f%%)" % (deep_acc*100, deep_std*100))
```

Listing 11.10: Compare models with k -fold cross-validation

You may see the output of above as follows:

```

Accuracy (wide): 0.72
Accuracy (wide): 0.66
Accuracy (wide): 0.83
Accuracy (wide): 0.76
Accuracy (wide): 0.83
Accuracy (deep): 0.90
Accuracy (deep): 0.72
Accuracy (deep): 0.93
Accuracy (deep): 0.69
Accuracy (deep): 0.76
Wide: 75.86% (+/- 6.54%)
Deep: 80.00% (+/- 9.61%)

```

Output 11.3: Result of the k -fold cross-validation

So you found that the deeper model is better than the wider model, in the sense that the mean accuracy is higher.

11.5 Retrain the Final Model

Now you know which design to pick, you want to rebuild the model and retrain it. Usually in k -fold cross-validation, you will use a smaller dataset to make the training faster. The final accuracy is not an issue because the gold of k -fold cross-validation is to tell which design is better. In the final model, you want to provide more data and produce a better model, since this is what you will use in production.

As you already split the data into training and test set, these are what you will use. In Python code,

```

# rebuild model with full set of training data
if wide_acc > deep_acc:
    print("Retrain a wide model")
    model = Wide()
else:
    print("Retrain a deep model")
    model = Deep()
acc = model_train(model, X_train, y_train, X_test, y_test)
print(f"Final model accuracy: {acc*100:.2f}%")

```

Listing 11.11: Retrain the model of choice after cross-validation

You can reuse the `model_train()` function as it is doing all the required training and validation. This is because the training procedure doesn't change for the final model or during k -fold cross-validation.

This model is what you can use in production. Usually it is unlike training, prediction is one data sample at a time in production. The following is how we demonstrate using the model for inference by running five samples from the test set:


```

model.eval()
with torch.no_grad():
    # Test out inference with 5 samples
    for i in range(5):
        y_pred = model(X_test[i:i+1])
        print(f"{X_test[i].numpy()} -> {y_pred[0].numpy()} " +
              f"(expected {y_test[i].numpy()})")

```

Listing 11.12: Evaluation of a model

Its output should look like the following:

```

[0.0265 0.044 0.0137 0.0084 0.0305 0.0438 0.0341 0.078 0.0844 0.0779
 0.0327 0.206 0.1908 0.1065 0.1457 0.2232 0.207 0.1105 0.1078 0.1165
 0.2224 0.0689 0.206 0.2384 0.0904 0.2278 0.5872 0.8457 0.8467 0.7679
 0.8055 0.626 0.6545 0.8747 0.9885 0.9348 0.696 0.5733 0.5872 0.6663
 0.5651 0.5247 0.3684 0.1997 0.1512 0.0508 0.0931 0.0982 0.0524 0.0188
 0.01 0.0038 0.0187 0.0156 0.0068 0.0097 0.0073 0.0081 0.0086 0.0095] -> [0.9583146] (expected [1.])
...

[0.034 0.0625 0.0381 0.0257 0.0441 0.1027 0.1287 0.185 0.2647 0.4117
 0.5245 0.5341 0.5554 0.3915 0.295 0.3075 0.3021 0.2719 0.5443 0.7932
 0.8751 0.8667 0.7107 0.6911 0.7287 0.8792 1. 0.9816 0.8984 0.6048
 0.4934 0.5371 0.4586 0.2908 0.0774 0.2249 0.1602 0.3958 0.6117 0.5196
 0.2321 0.437 0.3797 0.4322 0.4892 0.1901 0.094 0.1364 0.0906 0.0144
 0.0329 0.0141 0.0019 0.0067 0.0099 0.0042 0.0057 0.0051 0.0033 0.0058] -> [0.01937182] (expected [0.])

```

Output 11.4: Result of evaluation

You run the code under `torch.no_grad()` context because you sure there's no need to run the optimizer on the result. Hence you want to relieve the tensors involved from remembering how the values are computed.

The output of a binary classification neural network is between 0 and 1 (because of the sigmoid function at the end). From `encoder.classes_`, you can see that 0 means “M” and 1 means “R”. For a value between 0 and 1, you can simply round it to the nearest integer and interpret the 0-1 result, i.e.,

```

y_pred = model(X_test[i:i+1])
y_pred = y_pred.round() # 0 or 1

```

Listing 11.13: Prediction using the trained model

or use any other threshold to quantize the value into 0 or 1, i.e.,

```

threshold = 0.68
y_pred = model(X_test[i:i+1])
y_pred = (y_pred > threshold).float() # 0.0 or 1.0

```

Listing 11.14: Prediction with a threshold

Indeed, round to the nearest integer is equivalent to using 0.5 as the threshold. A good model should be robust to the choice of threshold. It is when the model output exactly 0 or 1. Otherwise, you would prefer a model that seldom returns values in the middle but often returns values close to 0 or close to 1. To see if your model is good, you can use *receiver*

operating characteristic curve (ROC), which is to plot the true positive rate against the false positive rate of the model under various threshold. You can make use of scikit-learn and matplotlib to plot the ROC:

```
from sklearn.metrics import roc_curve
import matplotlib.pyplot as plt

with torch.no_grad():
    # Plot the ROC curve
    y_pred = model(X_test)
    fpr, tpr, thresholds = roc_curve(y_test, y_pred)
    plt.plot(fpr, tpr) # ROC curve = TPR vs FPR
    plt.title("Receiver Operating Characteristics")
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.show()
```

Listing 11.15: Plotting the ROC curve of a binary classification model

You may see the following. The curve is always starting from the lower left corner and ends at upper right corner. The closer the curve to the upper left corner, the better your model is.

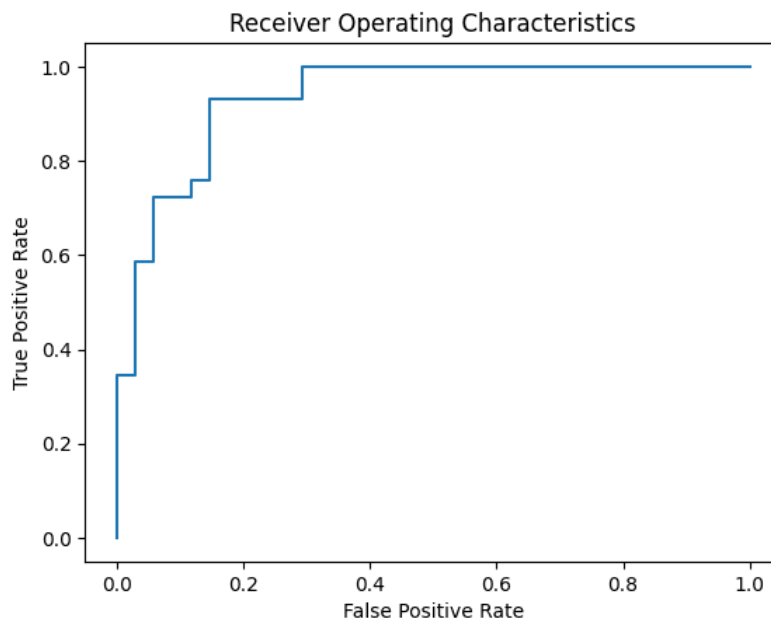


Figure 11.1: The ROC curve from the binary classification model

11.6 Complete Code

Putting everything together, the following is the complete code of the above:

```

import copy

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
import tqdm
from sklearn.metrics import roc_curve
from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.preprocessing import LabelEncoder

# Read data
data = pd.read_csv("sonar.csv", header=None)
X = data.iloc[:, 0:60]
y = data.iloc[:, 60]

# Binary encoding of labels
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# Convert to 2D PyTorch tensors
X = torch.tensor(X.values, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# Define two models
class Wide(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(60, 180)
        self.relu = nn.ReLU()
        self.output = nn.Linear(180, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.relu(self.hidden(x))
        x = self.sigmoid(self.output(x))
        return x

class Deep(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(60, 60)
        self.act1 = nn.ReLU()
        self.layer2 = nn.Linear(60, 60)
        self.act2 = nn.ReLU()
        self.layer3 = nn.Linear(60, 60)
        self.act3 = nn.ReLU()
        self.output = nn.Linear(60, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):

```

```

        x = self.act1(self.layer1(x))
        x = self.act2(self.layer2(x))
        x = self.act3(self.layer3(x))
        x = self.sigmoid(self.output(x))
        return x

# Compare model sizes
model1 = Wide()
model2 = Deep()
print(sum([x.reshape(-1).shape[0] for x in model1.parameters()])) # 11161
print(sum([x.reshape(-1).shape[0] for x in model2.parameters()])) # 11041

# Helper function to train one model
def model_train(model, X_train, y_train, X_val, y_val):
    # loss function and optimizer
    loss_fn = nn.BCELoss() # binary cross entropy
    optimizer = optim.Adam(model.parameters(), lr=0.0001)

    n_epochs = 300 # number of epochs to run
    batch_size = 10 # size of each batch
    batch_start = torch.arange(0, len(X_train), batch_size)

    # Hold the best model
    best_acc = - np.inf # init to negative infinity
    best_weights = None

    for epoch in range(n_epochs):
        model.train()
        with tqdm.tqdm(batch_start, unit="batch", mininterval=0, disable=True) as bar:
            bar.set_description(f"Epoch {epoch}")
            for start in bar:
                # take a batch
                X_batch = X_train[start:start+batch_size]
                y_batch = y_train[start:start+batch_size]
                # forward pass
                y_pred = model(X_batch)
                loss = loss_fn(y_pred, y_batch)
                # backward pass
                optimizer.zero_grad()
                loss.backward()
                # update weights
                optimizer.step()
                # print progress
                acc = (y_pred.round() == y_batch).float().mean()
                bar.set_postfix(
                    loss=float(loss),
                    acc=float(acc)
                )
            # evaluate accuracy at end of each epoch
            model.eval()
            y_pred = model(X_val)
            acc = (y_pred.round() == y_val).float().mean()
            acc = float(acc)
            if acc > best_acc:

```

```

        best_acc = acc
        best_weights = copy.deepcopy(model.state_dict())
        # restore model and return best accuracy
        model.load_state_dict(best_weights)
        return best_acc

# train-test split: Hold out the test set for final model evaluation
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# define 5-fold cross validation test harness
kfold = StratifiedKFold(n_splits=5, shuffle=True)
cv_scores_wide = []
for train, test in kfold.split(X_train, y_train):
    # create model, train, and get accuracy
    model = Wide()
    acc = model_train(model, X_train[train], y_train[train], X_train[test], y_train[test])
    print("Accuracy (wide): %.2f" % acc)
    cv_scores_wide.append(acc)
cv_scores_deep = []
for train, test in kfold.split(X_train, y_train):
    # create model, train, and get accuracy
    model = Deep()
    acc = model_train(model, X_train[train], y_train[train], X_train[test], y_train[test])
    print("Accuracy (deep): %.2f" % acc)
    cv_scores_deep.append(acc)

# evaluate the model
wide_acc = np.mean(cv_scores_wide)
wide_std = np.std(cv_scores_wide)
deep_acc = np.mean(cv_scores_deep)
deep_std = np.std(cv_scores_deep)
print("Wide: %.2f%% (+/- %.2f%%)" % (wide_acc*100, wide_std*100))
print("Deep: %.2f%% (+/- %.2f%%)" % (deep_acc*100, deep_std*100))

# rebuild model with full set of training data
if wide_acc > deep_acc:
    print("Retrain a wide model")
    model = Wide()
else:
    print("Retrain a deep model")
    model = Deep()
acc = model_train(model, X_train, y_train, X_test, y_test)
print(f"Final model accuracy: {acc*100:.2f}%")

model.eval()
with torch.no_grad():
    # Test out inference with 5 samples
    for i in range(5):
        y_pred = model(X_test[i:i+1])
        print(f"{X_test[i].numpy()} -> {y_pred[0].numpy()} " +
              f"(expected {y_test[i].numpy()})")

# Plot the ROC curve
y_pred = model(X_test)

```

```
fpr, tpr, thresholds = roc_curve(y_test, y_pred)
plt.plot(fpr, tpr) # ROC curve = TPR vs FPR
plt.title("Receiver Operating Characteristics")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.show()
```

Listing 11.16: Complete code for the binary classification problem

11.7 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Articles

Connectionist Bench (Snoar, Mines vs. Rocks) Data Set. UCI Machine Learning Repository.

[https://archive.ics.uci.edu/ml/datasets/Connectionist+Bench+\(Sonar,+Mines+vs.+Rocks\)](https://archive.ics.uci.edu/ml/datasets/Connectionist+Bench+(Sonar,+Mines+vs.+Rocks))

BCELoss. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.BCELoss.html>

Label encoder. scikit-learn documentation.

<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html>

ROC curve. scikit-learn documentation.

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_curve.html

Multiclass Receiver Operating Characteristic (ROC). scikit-learn examples.

https://scikit-learn.org/stable/auto_examples/model_selection/plot_roc.html

11.8 Summary

In this chapter, you discovered the use of PyTorch to build a binary classification model.

You learned how you can work through a binary classification problem step-by-step with PyTorch, specifically:

- ▷ How to load and prepare data for use in PyTorch
- ▷ How to create neural network models and use k -fold cross-validation to compare them
- ▷ How to train a binary classification model and obtain the receiver operating characteristics curve for it

In the next chapter, you will do a regression project.

Project: Building a Regression Model in PyTorch

12

Regression problems are usually easier to solve compared to classification problems. Many models can handle regression problems well and deep learning is one of them. In this chapter, you will discover how to use PyTorch to develop and evaluate neural network models for regression problems.

After completing this chapter, you will know:

- ▷ How to load data from scikit-learn and adapt it for PyTorch models
- ▷ How to create a neural network for regression problem using PyTorch
- ▷ How to improve model performance with data preparation techniques

Let's get started.

12.1 Description of the Dataset

The dataset you will use in this chapter is the California housing dataset.

This is a dataset that describes the median house value for California districts. Each data sample is a census block group. The target variable is the median house value in multiples of USD 100,000 in 1990 and there are 8 input features, each describing something about the house. They are, namely,

- ▷ MedInc: median income in block group
- ▷ HouseAge: median house age in block group
- ▷ AveRooms: average number of rooms per household
- ▷ AveBedrms: average number of bedrooms per household
- ▷ Population: block group population
- ▷ AveOccup: average number of household members
- ▷ Latitude: block group centroid latitude
- ▷ Longitude: block group centroid longitude

This data is special because the input data is in vastly different scale. For example, the number of rooms per house is usually small but the population per block group is usually

large. Moreover, most features should be positive but the longitude must be negative (because that's about California). Handling such diversity of data is a challenge to some machine learning models.

You can get the dataset from scikit-learn, which in turn, is downloaded from the Internet at realtime:

```
from sklearn.datasets import fetch_california_housing

data = fetch_california_housing()
print(data.feature_names)

X, y = data.data, data.target
```

Listing 12.1: Loading the dataset using scikit-learn

12.2 Building a Model and Train

This is a regression problem. Unlike classification problems, the output variable is a continuous value. In case of neural networks, you usually use linear activation at the output layer (i.e., no activation) such that the output range theoretically can be anything from negative infinity to positive infinity.

Also for regression problems, you should never expect the model to predict the values perfectly. Therefore, you should care about how close the prediction is to the actual value. The loss metric that you can use for this is the mean square error (MSE) or mean absolute error (MAE). But you may also be interested in the root mean squared error (RMSE) because that's a metric in the same unit as your output variable.

Let's try the traditional design of a neural network, namely, the pyramid structure. A pyramid structure is to have the number of neurons in each layer decreasing as the network progresses to the output. The number of input features is fixed, but you set a large number of neurons on the first hidden layer and gradually reduce the number in the subsequent layers. Because you have only one target in this dataset, the final layer should output only one value.

One design is as follows:

```
import torch.nn as nn

# Define the model
model = nn.Sequential(
    nn.Linear(8, 24),
    nn.ReLU(),
    nn.Linear(24, 12),
    nn.ReLU(),
    nn.Linear(12, 6),
    nn.ReLU(),
    nn.Linear(6, 1)
)
```

Listing 12.2: A model built in PyTorch

To train this network, you need to define a loss function. MSE is a reasonable choice. You also need an optimizer, such as Adam.

```
import torch.nn as nn
import torch.optim as optim

# loss function and optimizer
loss_fn = nn.MSELoss() # mean square error
optimizer = optim.Adam(model.parameters(), lr=0.0001)
```

Listing 12.3: Creating the loss function and optimizer

To train this model, you can use your usual training loop. In order to obtain an evaluation score so you are confident that the model works, you need to split the data into training and test sets. You may also want to avoid overfitting by keeping track on the test set MSE. The following is the training loop with the train-test split:

```
import copy
import numpy as np
import torch
import tqdm
from sklearn.model_selection import train_test_split

# train-test split of the dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)
X_train = torch.tensor(X_train, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1, 1)

# training parameters
n_epochs = 100 # number of epochs to run
batch_size = 10 # size of each batch
batch_start = torch.arange(0, len(X_train), batch_size)

# Hold the best model
best_mse = np.inf # init to infinity
best_weights = None
history = []

# training loop
for epoch in range(n_epochs):
    model.train()
    with tqdm.tqdm(batch_start, unit="batch", mininterval=0, disable=True) as bar:
        bar.set_description(f"Epoch {epoch}")
        for start in bar:
            # take a batch
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            # forward pass
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            # backward pass
```

```

        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()
        # print progress
        bar.set_postfix(mse=float(loss))
    # evaluate accuracy at end of each epoch
    model.eval()
    y_pred = model(X_test)
    mse = loss_fn(y_pred, y_test)
    mse = float(mse)
    history.append(mse)
    if mse < best_mse:
        best_mse = mse
        best_weights = copy.deepcopy(model.state_dict())

# restore model and return best accuracy
model.load_state_dict(best_weights)

```

Listing 12.4: Training loop for the regression model

In the training loop, `tqdm` is used to set up a progress bar and in each iteration step, MSE is calculated and reported. You can see how the MSE changed by setting the `tqdm` parameter `disable` above to `False`.

Note that in the training loop, each epoch is to run the forward and backward steps with the training set a few times to optimize the model weights, and at the end of the epoch, the model is evaluated using the test set. It is the MSE from the test set that is remembered in the list `history`. It is also the metric to evaluate a model, which the best one is stored in the variable `best_weights`.

After you run this, you will have the best model restored and the best MSE stored in the variable `best_mse`. Note that the mean square error is the average of the square of the difference between the predicted value and the actual value. The square root of it, RMSE, can be regarded as the average difference and it is numerically more useful.

In below, you can show the MSE and RMSE, and plot the history of MSE. It should be decreasing with the epochs.

```

print("MSE: %.2f" % best_mse)
print("RMSE: %.2f" % np.sqrt(best_mse))
plt.plot(history)
plt.show()

```

Listing 12.5: Plotting the MSE history

This model produced:

```

MSE: 0.47
RMSE: 0.68

```

Output 12.1: The best MSE and RMSE achieved

The MSE graph would like the following.

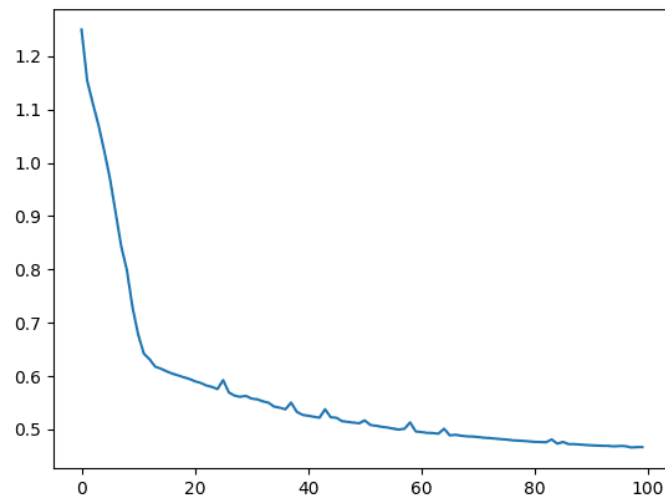


Figure 12.1: The plot of the MSE history

Putting everything together, the following is the complete code.

```
import copy

import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import tqdm
from sklearn.model_selection import train_test_split
from sklearn.datasets import fetch_california_housing

# Read data
data = fetch_california_housing()
X, y = data.data, data.target

# train-test split for model evaluation
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# Convert to 2D PyTorch tensors
X_train = torch.tensor(X_train, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1, 1)

# Define the model
model = nn.Sequential(
    nn.Linear(8, 24),
    nn.ReLU(),
    nn.Linear(24, 12),
    nn.ReLU(),
    nn.Linear(12, 6),
```

```

        nn.ReLU(),
        nn.Linear(6, 1)
    )

    # loss function and optimizer
    loss_fn = nn.MSELoss() # mean square error
    optimizer = optim.Adam(model.parameters(), lr=0.0001)

    n_epochs = 100 # number of epochs to run
    batch_size = 10 # size of each batch
    batch_start = torch.arange(0, len(X_train), batch_size)

    # Hold the best model
    best_mse = np.inf # init to infinity
    best_weights = None
    history = []

    for epoch in range(n_epochs):
        model.train()
        with tqdm.tqdm(batch_start, unit="batch", mininterval=0, disable=True) as bar:
            bar.set_description(f"Epoch {epoch}")
            for start in bar:
                # take a batch
                X_batch = X_train[start:start+batch_size]
                y_batch = y_train[start:start+batch_size]
                # forward pass
                y_pred = model(X_batch)
                loss = loss_fn(y_pred, y_batch)
                # backward pass
                optimizer.zero_grad()
                loss.backward()
                # update weights
                optimizer.step()
                # print progress
                bar.set_postfix(mse=float(loss))
            # evaluate accuracy at end of each epoch
            model.eval()
            y_pred = model(X_test)
            mse = loss_fn(y_pred, y_test)
            mse = float(mse)
            history.append(mse)
            if mse < best_mse:
                best_mse = mse
                best_weights = copy.deepcopy(model.state_dict())

    # restore model and return best accuracy
    model.load_state_dict(best_weights)
    print("MSE: %.2f" % best_mse)
    print("RMSE: %.2f" % np.sqrt(best_mse))
    plt.plot(history)
    plt.show()

```

Listing 12.6: Solving the regression problem with a PyTorch model

12.3 Improving the Model with Preprocessing

In the above, you see the RMSE is 0.68. Indeed, it is easy to improve the RMSE by polishing the data before training. The problem of this dataset is the diversity of the features: Some are with a narrow range and some are wide. And some are small but positive while some are very negative. This indeed is not very nice to most of the machine learning model.

One way to improve this is to apply a *standard scaler*. It is to convert each feature into their standard score. In other words, for each feature x , you replace it with

$$z = \frac{x - \bar{x}}{\sigma_x}$$

Where $\bar{x}$ is the mean of x and σ_x is the standard deviation. This way, every transformed feature is centered around 0 and in a narrow range that around 70% of the samples are between -1 to $+1$. This can help the machine learning model to converge.

You can apply the standard scaler from scikit-learn. The following is how you should modify the data preparation part of the above code:

```
import torch
from sklearn.model_selection import train_test_split
from sklearn.datasets import fetch_california_housing
from sklearn.preprocessing import StandardScaler

# Read data
data = fetch_california_housing()
X, y = data.data, data.target

# train-test split for model evaluation
X_train_raw, X_test_raw, y_train, y_test = train_test_split(X, y, train_size=0.7,
                                                            shuffle=True)

# Standardizing data
scaler = StandardScaler()
scaler.fit(X_train_raw)
X_train = scaler.transform(X_train_raw)
X_test = scaler.transform(X_test_raw)

# Convert to 2D PyTorch tensors
X_train = torch.tensor(X_train, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1, 1)
```

Listing 12.7: Scaling the dataset as a preprocessing step

Note that standard scaler is applied after train-test split. The `StandardScaler` above is fitted on the training set but applied on both the training and test set. You must not fit the standard scaler to all data because nothing from the test set should be hinted to the model. Otherwise you are introducing *data leakage*.

Other than that, virtually nothing shall be changed: You still have 8 features (only they are not the same in value). You still use the same training loop. If you train the model with the scaled data, you should see the RMSE improved, e.g.,

```
MSE: 0.29
RMSE: 0.54
```

Output 12.2: The MSE improved by preprocessing data

While the MSE history is in a similar falling shape, the y -axis shows it is indeed better after scaling:

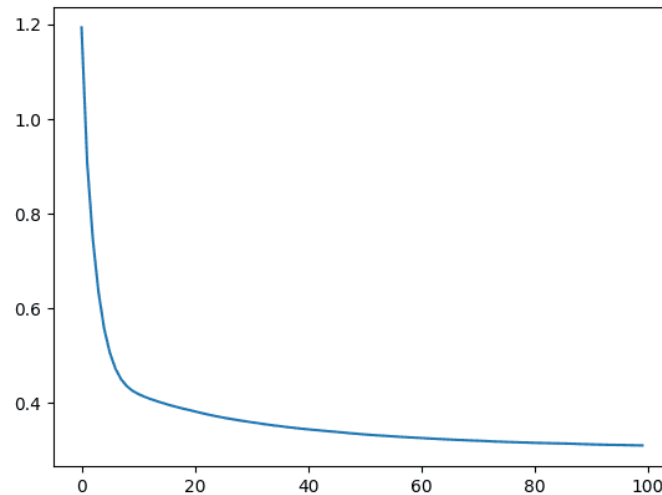


Figure 12.2: The MSE history

However, you need to be careful at the end: When you use the trained model and apply to new data, you should apply the scaler to the input data before feed into the model. That is, inference should be done as follows:

```
model.eval()
with torch.no_grad():
    # Test out inference with 5 samples from the original test set
    for i in range(5):
        X_sample = X_test_raw[i: i+1]
        X_sample = scaler.transform(X_sample)
        X_sample = torch.tensor(X_sample, dtype=torch.float32)
        y_pred = model(X_sample)
        print(f"{X_test_raw[i]} -> {y_pred[0].numpy()} (expected {y_test[i].numpy()}")
```

Listing 12.8: Using a model that expects scaled dataset

The following is the complete code:

```
import copy

import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import tqdm
from sklearn.model_selection import train_test_split
from sklearn.datasets import fetch_california_housing
from sklearn.preprocessing import StandardScaler

# Read data
data = fetch_california_housing()
X, y = data.data, data.target

# train-test split for model evaluation
X_train_raw, X_test_raw, y_train, y_test = train_test_split(X, y, train_size=0.7,
                                                            shuffle=True)

# Standardizing data
scaler = StandardScaler()
scaler.fit(X_train_raw)
X_train = scaler.transform(X_train_raw)
X_test = scaler.transform(X_test_raw)

# Convert to 2D PyTorch tensors
X_train = torch.tensor(X_train, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1, 1)

# Define the model
model = nn.Sequential(
    nn.Linear(8, 24),
    nn.ReLU(),
    nn.Linear(24, 12),
    nn.ReLU(),
    nn.Linear(12, 6),
    nn.ReLU(),
    nn.Linear(6, 1)
)

# loss function and optimizer
loss_fn = nn.MSELoss() # mean square error
optimizer = optim.Adam(model.parameters(), lr=0.0001)

n_epochs = 100 # number of epochs to run
batch_size = 10 # size of each batch
batch_start = torch.arange(0, len(X_train), batch_size)

# Hold the best model
best_mse = np.inf # init to infinity
best_weights = None
```

```

history = []

for epoch in range(n_epochs):
    model.train()
    with tqdm.tqdm(batch_start, unit="batch", mininterval=0, disable=True) as bar:
        bar.set_description(f"Epoch {epoch}")
        for start in bar:
            # take a batch
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            # forward pass
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
            # print progress
            bar.set_postfix(mse=float(loss))
        # evaluate accuracy at end of each epoch
        model.eval()
        y_pred = model(X_test)
        mse = loss_fn(y_pred, y_test)
        mse = float(mse)
        history.append(mse)
        if mse < best_mse:
            best_mse = mse
            best_weights = copy.deepcopy(model.state_dict())

# restore model and return best accuracy
model.load_state_dict(best_weights)
print("MSE: %.2f" % best_mse)
print("RMSE: %.2f" % np.sqrt(best_mse))
plt.plot(history)
plt.show()

model.eval()
with torch.no_grad():
    # Test out inference with 5 samples
    for i in range(5):
        X_sample = X_test_raw[i: i+1]
        X_sample = scaler.transform(X_sample)
        X_sample = torch.tensor(X_sample, dtype=torch.float32)
        y_pred = model(X_sample)
        print(f"{X_test_raw[i]} -> {y_pred[0].numpy()} (expected {y_test[i].numpy()})")

```

Listing 12.9: A regression model with the input data scaled

Of course, there is still room to improve the model. One way is to present the target in log scale or, equivalently, use mean absolute percentage error (MAPE) as the loss function. This is because the target variable is the value of houses and it is in a wide range. For the same error magnitude, it is more significant in percentage for low-valued houses. It is your exercise to modify the above code to produce a better prediction.

12.4 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Articles

California housing dataset. scikit-learn documentation.

https://scikit-learn.org/stable/datasets/real_world.html#california-housing-dataset

Train test split. scikit-learn documentation.

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

Standard scaler. scikit-learn documentation.

<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>

MSELoss. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.MSELoss.html>

12.5 Summary

In this chapter, you discovered the use of PyTorch to build a regression model.

You learned how you can work through a regression problem step-by-step with PyTorch, specifically:

- ▷ How to load and prepare data for use in PyTorch
- ▷ How to create neural network models and choose a loss function for regression
- ▷ How to improve model accuracy by applying standard scaler

Starting from next chapter, you will learn various techniques to fine tune a deep learning model.

Techniques for Better Deep Learning Models

Save and Load Your PyTorch Models

A deep learning model is a mathematical abstraction of data, in which a lot of parameters are involved. Training these parameters can take hours, days, and even weeks but afterward, you can make use of the result to apply on new data. This is called inference in machine learning. It is important to know how we can preserve the trained model in disk and later, load it for use in inference. In this chapter, you will discover how to save your PyTorch models to files and load them up again to make predictions. After reading this chapter, you will know:

- ▷ What are states and parameters in a PyTorch model
- ▷ How to save model states
- ▷ How to load model states

Let's get started.

Overview

This chapter is in three parts; they are

- ▷ Build an Example Model
- ▷ What's Inside a PyTorch Model
- ▷ Accessing `state_dict` of a Model

13.1 Build an Example Model

Let's start with a very simple model in PyTorch. It is a model based on the iris dataset (see Chapter 10). You will load the dataset using scikit-learn (which the targets are integer labels 0, 1, and 2) and train a neural network for this multiclass classification problem. In this model, you used log softmax as the output activation so you can combine with the negative log likelihood loss function. It is equivalent to no output activation combined with cross-entropy loss function.

```

import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load data into NumPy arrays
data = load_iris()
X, y = data["data"], data["target"]

# convert NumPy array into PyTorch tensors
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.long)

# split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# PyTorch model
class Multiclass(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(4, 8)
        self.act = nn.ReLU()
        self.output = nn.Linear(8, 3)
        self.logsoftmax = nn.LogSoftmax(dim=1)

    def forward(self, x):
        x = self.act(self.hidden(x))
        x = self.logsoftmax(self.output(x))
        return x

model = Multiclass()

# loss metric and optimizer
loss_fn = nn.NLLLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# prepare model and training parameters
n_epochs = 100
batch_size = 5
batch_start = torch.arange(0, len(X), batch_size)

# training loop
for epoch in range(n_epochs):
    for start in batch_start:
        # take a batch
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        # forward pass
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        # backward pass
        optimizer.zero_grad()
        loss.backward()

```

```
# update weights
optimizer.step()
```

Listing 13.1: An example model trained with negative log-likelihood loss function

With such a simple model and small dataset, it shouldn't take a long time to finish training. Afterwards, we can confirm that this model works, by evaluating it with the test set:

```
...
y_pred = model(X_test)
acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
print("Accuracy: %.2f" % acc)
```

Listing 13.2: Evaluating a model with test set

It prints, for example,

```
Accuracy: 0.96
```

Output 13.1: Accuracy as evaluated on the test set

13.2 What's Inside a PyTorch Model

PyTorch model is an object in Python. It holds some deep learning building blocks such as various kinds of layers and activation functions. It also knows how to connect them so it can produce you an output from your input tensors. The algorithm of a model is fixed at the time you created it, however, it has trainable parameters that is supposed to be modified during training loop so the model can be more accurate.

You saw how to get the model parameters when you setup the optimizer for your training loop, namely,

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

Listing 13.3: Creating an optimizer

The function `model.parameters()` give you a generator that reference to each layers' trainable parameters in turn in the form of PyTorch tensors. Therefore, it is possible for you to make a copy of them or overwrite them, for example:

```
# create a new model
newmodel = Multiclass()
# ask PyTorch to ignore autograd on update and overwrite parameters
with torch.no_grad():
    for newtensor, oldtensor in zip(newmodel.parameters(), model.parameters()):
        newtensor.copy_(oldtensor)
# test with new model using copied tensor
y_pred = newmodel(X_test)
acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
print("Accuracy: %.2f" % acc)
```

Listing 13.4: Copying weights from one model to another

Which the result should be exactly the same as before since you essentially made the two models identical by copying the parameters.

However, this is not always the case. Some models has *non-trainable parameters*. One example is the batch normalization layer that is common in many convolution neural networks. What it does is to apply normalization on tensors that produced by its previous layer and pass on the normalized tensor to its next layer. It has two parameters: The mean and standard deviation, which are learned from your input data during training loop but not trainable by the optimizer. Therefore these are not part of `model.parameters()` but equally important.

13.3 Accessing `state_dict` of a Model

To access all parameters of a model, trainable or not, you can get it from `state_dict()` function. From the model above, this is what you can get:

```
import pprint
pp = pprint.PrettyPrinter(indent=4)
pp.pprint(model.state_dict())
```

Listing 13.5: Pretty-print all parameters from a model

The model above produces the following:

```
OrderedDict([ ( 'hidden.weight',
               tensor([[ 0.1480,  0.0336,  0.3425,  0.2832],
                        [ 0.5265,  0.8587, -0.7023, -1.1149],
                        [ 0.1620,  0.8440, -0.6189, -0.6513],
                        [-0.1559,  0.0393, -0.4701,  0.0825],
                        [ 0.6364, -0.6622,  1.1150,  0.9162],
                        [ 0.2081, -0.0958, -0.2601, -0.3148],
                        [-0.0804,  0.1027,  0.7363,  0.6068],
                        [-0.4101, -0.3774, -0.1852,  0.1524]])),
              ( 'hidden.bias',
               tensor([ 0.2057,  0.7998, -0.0578,  0.1041, -0.3903, -0.4521, -0.5307, -0.1532])),
              ( 'output.weight',
               tensor([[ -0.0954,  0.8683,  1.0667,  0.2382, -0.4245, -0.0409, -0.2587, -0.0745],
                        [-0.0829,  0.8642, -1.6892, -0.0188,  0.0420, -0.1020,  0.0344, -0.1210],
                        [-0.0176, -1.2809, -0.3040,  0.1985,  0.2423,  0.3333,  0.4523, -0.1928]])),
              ('output.bias', tensor([ 0.0998,  0.6360, -0.2990]))])
```

Output 13.2: The `state_dict` from a model

It is called `state_dict` because all state variables of a model are here. It is an `OrderedDict` object from Python's built-in `collections` module. All components from a PyTorch model has a name and so as the parameters therein. The `OrderedDict` object allows you to map the weights back to the parameters correctly by matching their names.

This is how you should save and load the model: Fetch the model states into an `OrderedDict`, serialize and save it to disk. For inference, you create a model first (without training), and load the states. In Python, the native format for serialization is pickle:

```
import pickle

# Save model
with open("iris-model.pickle", "wb") as fp:
    pickle.dump(model.state_dict(), fp)

# Create new model and load states
newmodel = Multiclass()
with open("iris-model.pickle", "rb") as fp:
    newmodel.load_state_dict(pickle.load(fp))

# test with new model using copied tensor
y_pred = newmodel(X_test)
acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
print("Accuracy: %.2f" % acc)
```

Listing 13.6: Serialize model parameters using pickle

You know it works because the model you didn't train produced the same result as the one you trained.

Indeed, the recommended way is to use the PyTorch API to save and load the states, instead of using pickle manually:

```
# Save model
torch.save(model.state_dict(), "iris-model.pth")

# Create new model and load states
newmodel = Multiclass()
newmodel.load_state_dict(torch.load("iris-model.pth"))

# test with new model using copied tensor
y_pred = newmodel(X_test)
acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
print("Accuracy: %.2f" % acc)
```

Listing 13.7: Serialize model parameters using PyTorch save and load functions

The `*.pth` file is indeed a zip file of some pickle files created by PyTorch. It is recommended because PyTorch can store additional information in it. Note that you stored only the states but not the model. You still need to create the model using Python code and load the states into it. If you wish to store the model as well, you can pass in the entire model instead of the states:

```
# Save model
torch.save(model, "iris-model-full.pth")

# Load model
newmodel = torch.load("iris-model-full.pth")

# test with new model using copied tensor
y_pred = newmodel(X_test)
acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
print("Accuracy: %.2f" % acc)
```

Listing 13.8: Saving and loading a PyTorch model

But remember, due to the nature of Python language, doing so does not relieve you from keeping the code of the model. The `newmodel` object above is an instance of `Multiclass` class that you defined before. When you load the model from disk, Python need to know in detail how this class is defined. If you run a script with just the line `torch.load()`, you will see the following error message:

```
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
File "/.../torch/serialization.py", line 789, in load
    return _load(opened_zipfile, map_location, pickle_module, **pickle_load_args)
File "/.../torch/serialization.py", line 1131, in _load
    result = unpickler.load()
File "/.../torch/serialization.py", line 1124, in find_class
    return super().find_class(mod_name, name)
AttributeError: Can't get attribute 'Multiclass' on <module '__main__' (built-in)>
```

Output 13.3: Error message for unable to find the code of a model class

That's why it is recommended to save only the state dict rather than the entire model.

Putting everything together, the following is the complete code to demonstrate how to create a model, train it, and save to disk:

```
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load data into NumPy arrays
data = load_iris()
X, y = data["data"], data["target"]

# convert NumPy array into PyTorch tensors
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.long)

# split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# PyTorch model
class Multiclass(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(4, 8)
        self.act = nn.ReLU()
        self.output = nn.Linear(8, 3)
        self.logsoftmax = nn.LogSoftmax(dim=1)

    def forward(self, x):
        x = self.act(self.hidden(x))
        x = self.logsoftmax(self.output(x))
        return x
```



```

model = Multiclass()

# loss metric and optimizer
loss_fn = nn.NLLLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# prepare model and training parameters
n_epochs = 100
batch_size = 5
batch_start = torch.arange(0, len(X), batch_size)

# training loop
for epoch in range(n_epochs):
    for start in batch_start:
        # take a batch
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        # forward pass
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        # backward pass
        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()

# Save model
torch.save(model.state_dict(), "iris-model.pth")

```

Listing 13.9: Create, train, and save a model

And the following is how to load the model from disk and run it for inference:

```

import torch
import torch.nn as nn
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load data into NumPy arrays
data = load_iris()
X, y = data["data"], data["target"]

# convert NumPy array into PyTorch tensors
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.long)

# split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# PyTorch model
class Multiclass(nn.Module):
    def __init__(self):
        super().__init__()

```

```

        self.hidden = nn.Linear(4, 8)
        self.act = nn.ReLU()
        self.output = nn.Linear(8, 3)
        self.logsoftmax = nn.LogSoftmax(dim=1)

    def forward(self, x):
        x = self.act(self.hidden(x))
        x = self.logsoftmax(self.output(x))
        return x

# Create new model and load states
model = Multiclass()
model.load_state_dict(torch.load("iris-model.pth"))

# Run model for inference
y_pred = model(X_test)
acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
print("Accuracy: %.2f" % acc)

```

Listing 13.10: Load a saved model for inference

13.4 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Articles

Iris Data Set. UCI Machine Learning Repository.

<https://archive.ics.uci.edu/ml/datasets/Iris>

Load and return the iris dataset. scikit-learn documentation.

https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_iris.html

Saving and loading models. PyTorch tutorial.

https://pytorch.org/tutorials/beginner/saving_loading_models.html

NLLLoss. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.NLLLoss.html>

LogSoftmax. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.LogSoftmax.html>

13.5 Summary

In this chapter, you learned how to keep a copy of your trained PyTorch model in disk and how to reuse it. In particular, you learned

- ▷ What are parameters and states in a PyTorch model
- ▷ How to save all necessary states from a model to disk
- ▷ How to rebuild a working model from the saved states

In the next chapter, you will learn how to select activation functions for your model.

Using Activation Functions in Deep Learning Models

14

A deep learning model in its simplest form are layers of perceptrons connected in tandem. Without any activation functions, they are just matrix multiplications with limited power, regardless how many of them. Activation is the magic why neural network can be an approximation to a wide variety of nonlinear function. In PyTorch, there are many activation functions available for use in your deep learning models. In this chapter, you will see how the choice of activation functions can impact the model. Specifically,

- ▷ What are the common activation functions
- ▷ What are the nature of activation functions
- ▷ How the different activation functions impact the learning rate
- ▷ How the selection of activation function can solve the vanishing gradient problem

Let's get started.

Overview

This chapter is in three parts; they are

- ▷ A Toy Model of Binary Classification
- ▷ Why Nonlinear Functions?
- ▷ The Effect of Activation Functions

14.1 A Toy Model of Binary Classification

Let's start with a simple example of binary classification. Here you use the `make_circle()` function from scikit-learn to create a synthetic dataset for binary classification. This dataset has two features: The x - and y -coordinate of points. Each point belongs to one of the two classes. You can generate 1000 data points and visualize them as below:

```

from sklearn.datasets import make_circles
import matplotlib.pyplot as plt
import torch

# Make data: Two circles on x-y plane as a classification problem
X, y = make_circles(n_samples=1000, factor=0.5, noise=0.1)
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y.reshape(-1, 1), dtype=torch.float32)

plt.figure(figsize=(8,6))
plt.scatter(X[:,0], X[:,1], c=y)
plt.show()

```

Listing 14.1: Creating a synthetic dataset for binary classification

The dataset is visualized as follows:

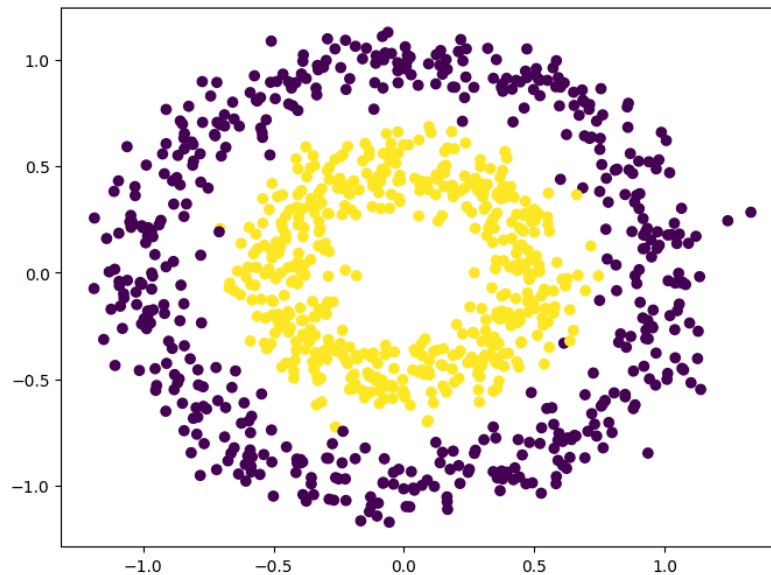


Figure 14.1: The synthetic dataset

This dataset is special because it is simple but not linearly separable: It is impossible to find a straight line to separate two classes. How can your neural network figure out there's a circle boundary between the classes is a challenge.

Let's create a deep learning model for this problem. To make things simple, you do not do cross-validation. You may find the neural network overfit the data but it doesn't affect the discussion below. The model has 4 hidden layers and the output layer gives a sigmoidal value (0 to 1) for binary classification. The model accepts a parameter at its constructor to specify what is the activation to use in the hidden layers. You implement the training loop in a function as you will run this for several times.

The implementation is as follows:

```

class Model(nn.Module):
    def __init__(self, activation=nn.ReLU):
        super().__init__()
        self.layer0 = nn.Linear(2,5)
        self.act0 = activation()
        self.layer1 = nn.Linear(5,5)
        self.act1 = activation()
        self.layer2 = nn.Linear(5,5)
        self.act2 = activation()
        self.layer3 = nn.Linear(5,5)
        self.act3 = activation()
        self.layer4 = nn.Linear(5,1)
        self.act4 = nn.Sigmoid()

    def forward(self, x):
        x = self.act0(self.layer0(x))
        x = self.act1(self.layer1(x))
        x = self.act2(self.layer2(x))
        x = self.act3(self.layer3(x))
        x = self.act4(self.layer4(x))
        return x

def train_loop(model, X, y, n_epochs=300, batch_size=32):
    loss_fn = nn.BCELoss()
    optimizer = optim.Adam(model.parameters(), lr=0.0001)
    batch_start = torch.arange(0, len(X), batch_size)

    bce_hist = []
    acc_hist = []

    for epoch in range(n_epochs):
        # train model with optimizer
        model.train()
        for start in batch_start:
            X_batch = X[start:start+batch_size]
            y_batch = y[start:start+batch_size]
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
        # evaluate BCE and accuracy at end of each epoch
        model.eval()
        with torch.no_grad():
            y_pred = model(X)
            bce = float(loss_fn(y_pred, y))
            acc = float((y_pred.round() == y).float().mean())
        bce_hist.append(bce)
        acc_hist.append(acc)
        # print metrics every 10 epochs
        if (epoch+1) % 10 == 0:
            print("Before epoch %d: BCE=%.4f, Accuracy=%.2f%%" % (epoch+1, bce, acc*100))
    return bce_hist, acc_hist

```

Listing 14.2: A PyTorch model for binary classification and a training loop function

At the end of each epoch in the training function, you evaluate the model with the entire dataset. The evaluation result is returned when the training finished. In the following, you create a model, train it, and plot the training history. The activation function you use is *rectified linear unit* or ReLU, which is the most common activation function nowadays:

```
activation = nn.ReLU
model = Model(activation=activation)
bce_hist, acc_hist = train_loop(model, X, y)
plt.plot(bce_hist, label="BCE")
plt.plot(acc_hist, label="Accuracy")
plt.xlabel("Epochs")
plt.ylim(0, 1)
plt.title(str(activation))
plt.legend()
plt.show()
```

Listing 14.3: Creating and training a model, with the training history visualized

Running this give you the following:

```
Before epoch 10: BCE=0.7025, Accuracy=50.00%
Before epoch 20: BCE=0.6990, Accuracy=50.00%
Before epoch 30: BCE=0.6959, Accuracy=50.00%
...
Before epoch 280: BCE=0.3051, Accuracy=96.30%
Before epoch 290: BCE=0.2785, Accuracy=96.90%
Before epoch 300: BCE=0.2543, Accuracy=97.00%
```

Output 14.1: Training history

and this plot:

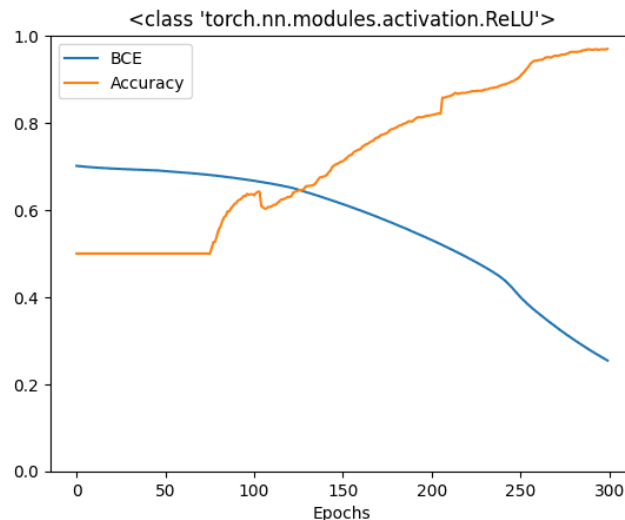


Figure 14.2: Training history visualized

This model works great. After 300 epochs, it can achieve 90% accuracy. However, ReLU is not the only activation function. Historically, sigmoid function and hyperbolic tangents were

common in neural networks literature. If you're curious, below are how you can compare these three activation functions, using matplotlib:

```
import matplotlib.pyplot as plt
import torch
import torch.nn as nn

x = torch.linspace(-4, 4, 200)
relu = nn.ReLU()(x)
tanh = nn.Tanh()(x)
sigmoid = nn.Sigmoid()(x)

plt.plot(x, sigmoid, label="sigmoid")
plt.plot(x, tanh, label="tanh")
plt.plot(x, relu, label="ReLU")
plt.ylim(-1.5, 2)
plt.legend()
plt.show()
```

Listing 14.4: Comparing different activation functions

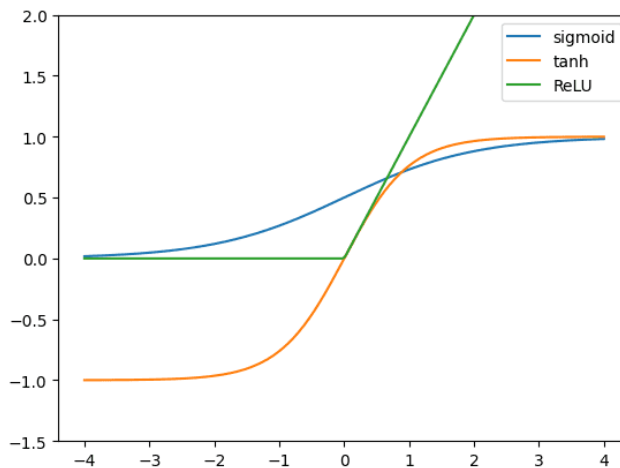


Figure 14.3: Plot and formula of different activation functions

$$\text{ReLU}(x) = \max(0, x)$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

ReLU is called rectified linear unit because it is a linear function $y = x$ at positive x but remains zero if x is negative. Mathematically, it is $y = \max(0, x)$. Hyperbolic tangent ($y = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$) goes from -1 to $+1$ smoothly while sigmoid function ($y = \sigma(x) = 1/(1 + e^{-x})$) goes from 0 to $+1$.

If you try to differentiate these functions, you will find that ReLU is the easiest: The gradient is 1 at positive region and 0 otherwise. Hyperbolic tangent has a steeper slope therefore its gradient is greater than that of sigmoid function.

All these functions are increasing. Therefore, their gradients are never negative. This is one of the criteria for an activation function suitable to use in neural networks.

14.2 Why Nonlinear Functions?

You might be wondering, why all this hype about nonlinear activation functions? Or why can't we just use an identity function after the weighted linear combination of activations from the previous layer? Using multiple linear layers is basically the same as using a single linear layer. This can be seen through a simple example. Let's say you have a one hidden layer neural network, each with two hidden neurons.

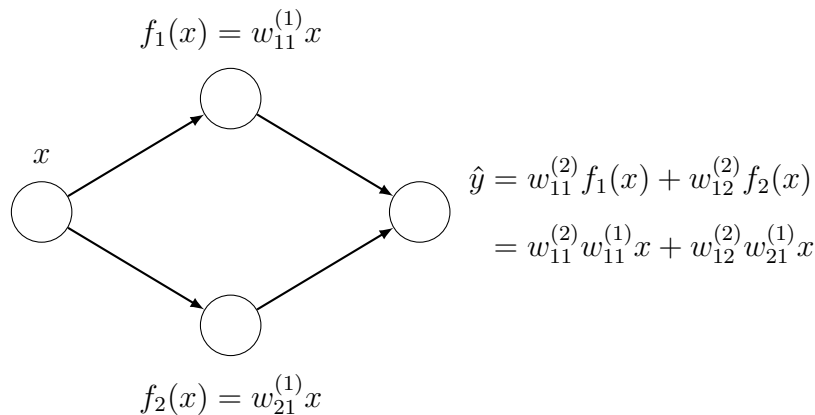


Figure 14.4: Single hidden layer neural network with linear layers

You can then rewrite the output layer as a linear combination of the original input variable if you used a linear hidden layer. If you had more neurons and weights, the equation would be a lot longer with more nesting and more multiplications between successive layer weights. However, the idea remains the same: You can represent the entire network as a single linear layer. To make the network represent more complex functions, you would need nonlinear activation functions.

14.3 The Effect of Activation Functions

To explain how much impact the activation function can bring to your model, let's modify the training loop function to capture more data: The gradients in each training step. Your model has four hidden layers and one output layer. In each step, the backward pass calculates the gradient of the weights of each layer and the weight update is done by the optimizer based on the result of the backward pass. You should observe how the gradient changes as the training progressed. Therefore, the training loop function is modified to collect the mean absolute value of the gradient in each layer in each step, as follows:

```
def train_loop(model, X, y, n_epochs=300, batch_size=32):
    loss_fn = nn.BCELoss()
    optimizer = optim.Adam(model.parameters(), lr=0.0001)
    batch_start = torch.arange(0, len(X), batch_size)

    bce_hist = []
    acc_hist = []
    grad_hist = [[], [], [], [], []]
```

```

for epoch in range(n_epochs):
    # train model with optimizer
    model.train()
    layer_grad = [[]] * 5
    for start in batch_start:
        X_batch = X[start:start+batch_size]
        y_batch = y[start:start+batch_size]
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        # collect mean absolute value of gradients
        layers = [model.layer0, model.layer1, model.layer2, model.layer3,
                  model.layer4]
        for n, layer in enumerate(layers):
            mean_grad = float(layer.weight.grad.abs().mean())
            layer_grad[n].append(mean_grad)
    # evaluate BCE and accuracy at end of each epoch
    model.eval()
    with torch.no_grad():
        y_pred = model(X)
        bce = float(loss_fn(y_pred, y))
        acc = float((y_pred.round() == y).float().mean())
    bce_hist.append(bce)
    acc_hist.append(acc)
    for n, grads in enumerate(layer_grad):
        grad_hist[n].append(sum(grads)/len(grads))
    # print metrics every 10 epochs
    if epoch % 10 == 9:
        print("Epoch %d: BCE=%.4f, Accuracy=%.2f%%" % (epoch, bce, acc*100))
    return bce_hist, acc_hist, layer_grad

```

Listing 14.5: Training loop function to collect statistics of the computed gradient

At the end of the inner for-loop, the gradients of the layer weights are computed by the backward process earlier and you can access to the gradient using `model.layer0.weight.grad`. Like the weights, the gradients are tensors. You take the absolute value of each element and then compute the mean over all elements. This value depends on the batch and can be very noisy. Thus you summarize all such mean absolute value over the same epoch at the end.

Note that you have five layers in the neural network (hidden and output layers combined). So you can see the pattern of each layer's gradient across the epochs if you visualize them. In below, you run the training loop as before and plot both the cross-entropy and accuracy as well as the mean absolute gradient of each layer:

```

activation = nn.ReLU
model = Model(activation=activation)
bce_hist, acc_hist, grad_hist = train_loop(model, X, y)

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].plot(bce_hist, label="BCE")

```

```

ax[0].plot(acc_hist, label="Accuracy")
ax[0].set_xlabel("Epochs")
ax[0].set_ylim(0, 1)
for n, grads in enumerate(grad_hist):
    ax[1].plot(grads, label="layer"+str(n))
ax[1].set_xlabel("Epochs")
fig.suptitle(str(activation))
ax[0].legend()
ax[1].legend()
plt.show()

```

Listing 14.6: Plotting the mean absolute gradient

Running the above produces the following plot:

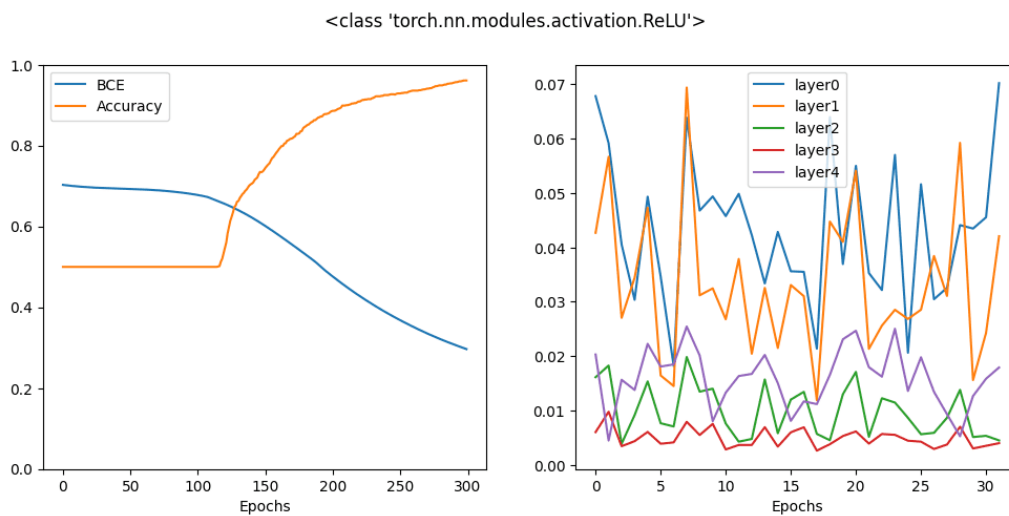


Figure 14.5: The mean absolute gradient of each layer

In the plot above, you can see how the accuracy increases and the cross-entropy loss decreases. At the same time, you can see the gradient of each layer is fluctuating in a similar range, especially you should pay attention to the line corresponding to the first layer and the last layer. This behavior is ideal.

Let's repeat the same with a sigmoid activation:

```

activation = nn.Sigmoid
model = Model(activation=activation)
bce_hist, acc_hist, grad_hist = train_loop(model, X, y)

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].plot(bce_hist, label="BCE")
ax[0].plot(acc_hist, label="Accuracy")
ax[0].set_xlabel("Epochs")
ax[0].set_ylim(0, 1)
for n, grads in enumerate(grad_hist):
    ax[1].plot(grads, label="layer"+str(n))
ax[1].set_xlabel("Epochs")
fig.suptitle(str(activation))

```

```
ax[0].legend()
ax[1].legend()
plt.show()
```

Listing 14.7: Repeat the training with a model using sigmoid activation

which the plot is as follows:

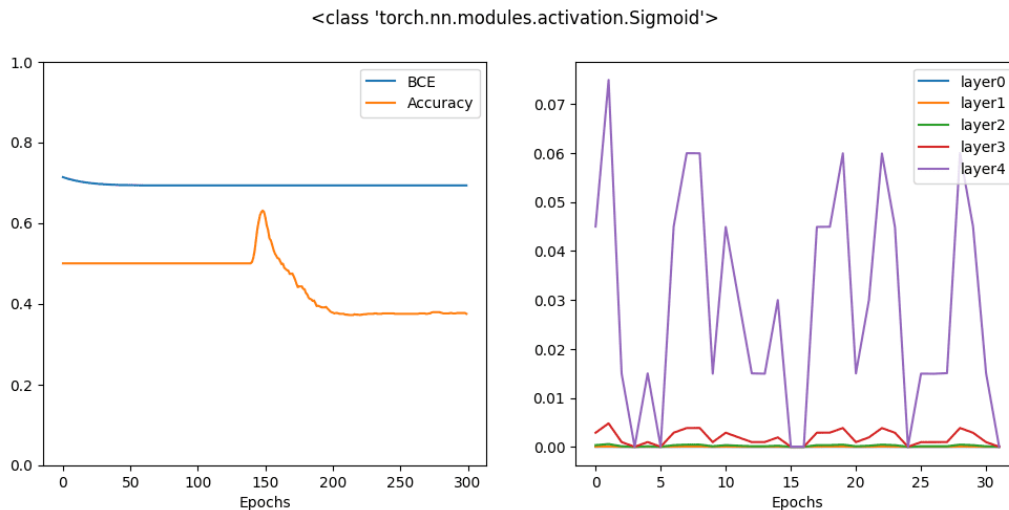


Figure 14.6: Training progress on a model using sigmoid activation

You can see that after 300 epochs, the final result is much worse than ReLU activation. Indeed, you may need much more epochs for this model to converge. The reason can be easily found on the graph at right, which you can see the gradient is significant only for the output layer while all the hidden layers' gradients are virtually zero. This is the *vanishing gradient effect* which is the problem of many neural network models with sigmoid activation function.

The hyperbolic tangent function has a similar shape as sigmoid function but its curve is steeper. Let's see how it behaves:

```
activation = nn.Tanh
model = Model(activation=activation)
bce_hist, acc_hist, grad_hist = train_loop(model, X, y)

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].plot(bce_hist, label="BCE")
ax[0].plot(acc_hist, label="Accuracy")
ax[0].set_xlabel("Epochs")
ax[0].set_ylim(0, 1)
for n, grads in enumerate(grad_hist):
    ax[1].plot(grads, label="layer"+str(n))
ax[1].set_xlabel("Epochs")
fig.suptitle(str(activation))
ax[0].legend()
ax[1].legend()
plt.show()
```

Listing 14.8: Repeat the training with a model using hyperbolic tangent activation

Which is:

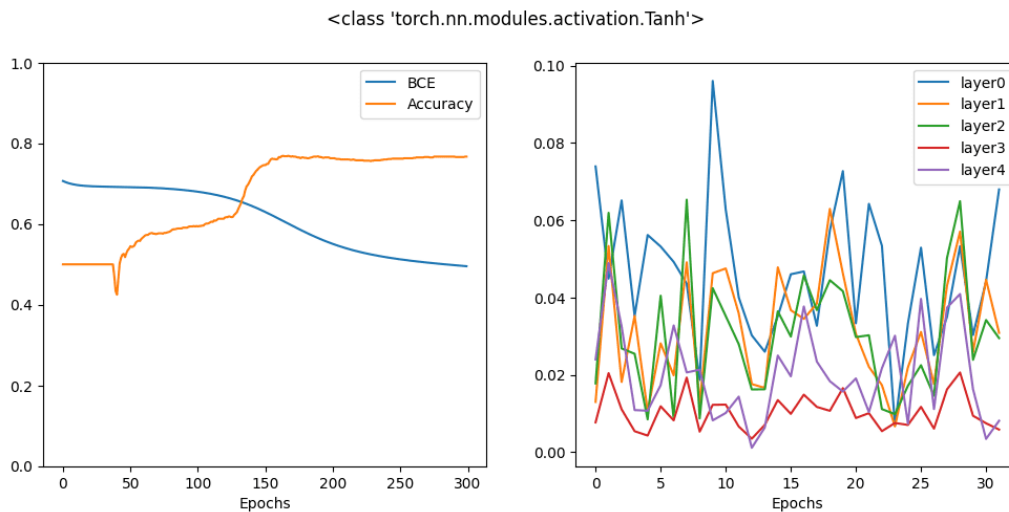


Figure 14.7: Training progress on a model using hyperbolic tangent activation

The result looks better than sigmoid activation but still worse than ReLU. From the gradient plot, you can notice that the gradients at the hidden layers are significant. But in fact, the backward process is often not very effective at propagating the gradient to the input due to the large flat region in tanh function.

This is the reason you see ReLU activation in every neural network model today. Not only because ReLU is simpler and computing the differentiation of it is much faster than the other activation function, but also because it can make the model converge faster.

Indeed, you can do better than ReLU sometimes. In PyTorch, you have a number of ReLU variations. Let's look at two of them. You can compare these three variations of ReLU as follows:

```
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

x = torch.linspace(-8,8,200)
relu = nn.ReLU()(x)
relu6 = nn.ReLU6()(x)
leaky = nn.LeakyReLU()(x)

plt.plot(x, relu, c="purple", lw=2, ls=":", label="ReLU")
plt.plot(x, relu6, c="orange", lw=2, ls="--", alpha=0.5, label="ReLU6")
plt.plot(x, leaky, c="darkblue", lw=2, alpha=0.5, label="LeakyReLU")
plt.legend()
plt.show()
```

Listing 14.9: Plotting three variations of ReLU function

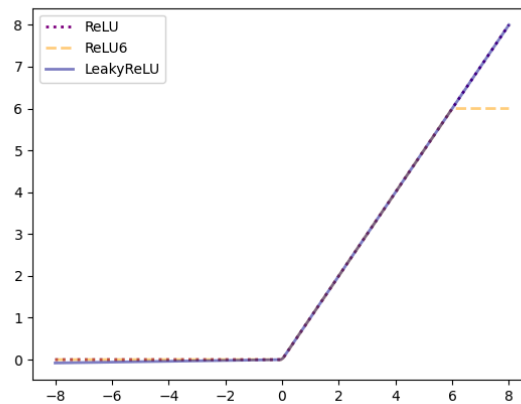


Figure 14.8: The plot of three variations of ReLU function

First is the ReLU6, which is ReLU but cap the function at 6.0 if the input to the function is more than 6.0:

```
activation = nn.ReLU6
model = Model(activation=activation)
bce_hist, acc_hist, grad_hist = train_loop(model, X, y)

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].plot(bce_hist, label="BCE")
ax[0].plot(acc_hist, label="Accuracy")
ax[0].set_xlabel("Epochs")
ax[0].set_ylim(0, 1)
for n, grads in enumerate(grad_hist):
    ax[1].plot(grads, label="layer"+str(n))
ax[1].set_xlabel("Epochs")
fig.suptitle(str(activation))
ax[0].legend()
ax[1].legend()
plt.show()
```

Listing 14.10: Repeat the training with a model using ReLU6 activation

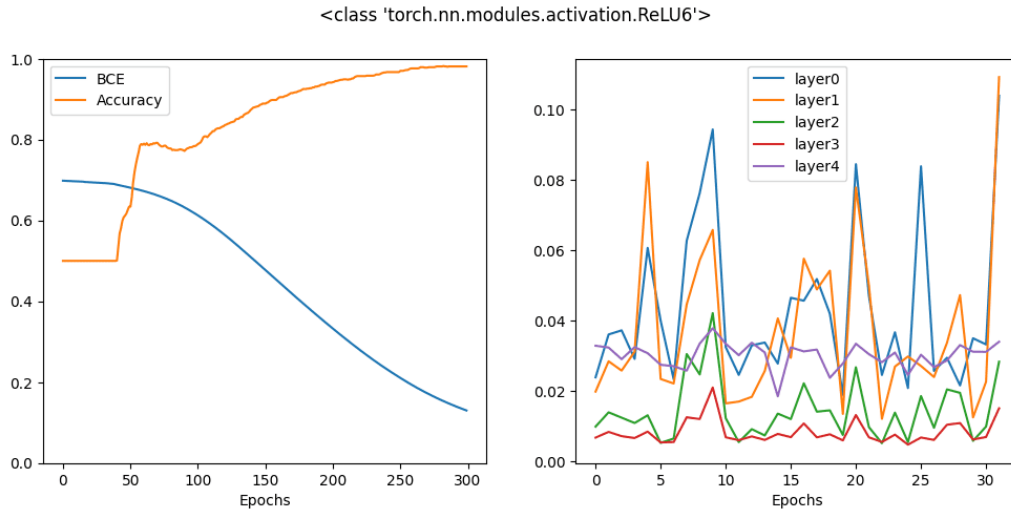


Figure 14.9: Training progress on a model using ReLU6 activation

Next is leaky ReLU, which the negative half of ReLU is no longer flat but a gently slanted line. The rationale behind is to keep a small positive slope at that region.

```
activation = nn.LeakyReLU
model = Model(activation=activation)
bce_hist, acc_hist, grad_hist = train_loop(model, X, y)

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].plot(bce_hist, label="BCE")
ax[0].plot(acc_hist, label="Accuracy")
ax[0].set_xlabel("Epochs")
ax[0].set_ylim(0, 1)
for n, grads in enumerate(grad_hist):
    ax[1].plot(grads, label="layer"+str(n))
ax[1].set_xlabel("Epochs")
fig.suptitle(str(activation))
ax[0].legend()
ax[1].legend()
plt.show()
```

Listing 14.11: Repeat the training with a model using leaky ReLU activation

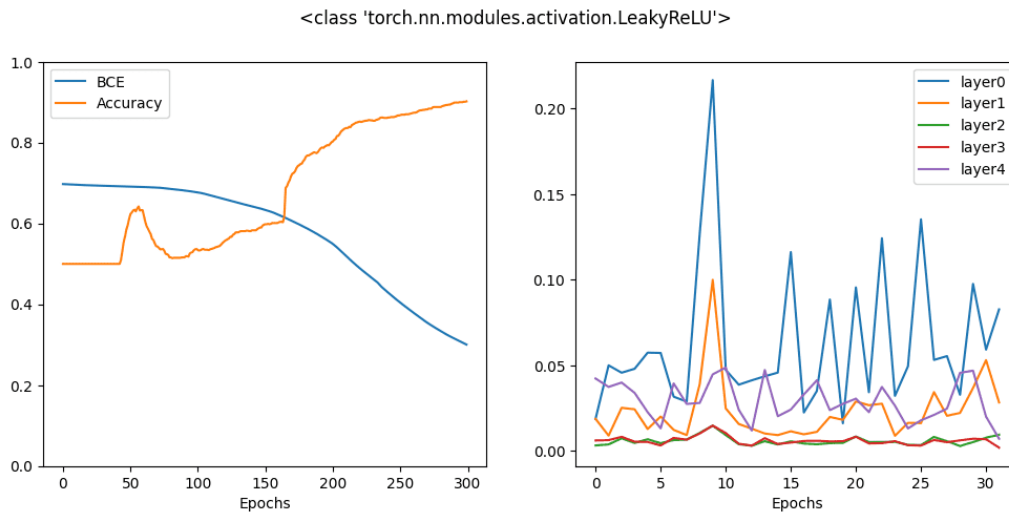


Figure 14.10: Training progress on a model using leaky ReLU activation

You can see that all these variations can give you similar accuracy after 300 epochs but from the history curve, you know some are faster to reach a high accuracy than another. This is because of the interaction between the gradient of an activation function with the optimizer. There's no golden rule that a single activation function works best but the design helps:

- ▷ In backpropagation, passing the loss metric from the output layer all the way to the input layer
- ▷ Maintaining stable gradient calculation under specific condition, e.g., limiting floating point precision. Remember that all floating point calculation involves some round off and the activation function must be robust to such rounding.
- ▷ Providing enough contrast on different inputs such that the backward pass can determine accurate adjustment to the parameter. You can see from Figure 14.3 that sigmoid is just a flattened tanh function (since $\tanh(x) = 2\sigma(2x) - 1$) but you saw how different are the result.

The following is the complete code to generate all the plots above:

```
from sklearn.datasets import make_circles
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim

# Make data: Two circles on x-y plane as a classification problem
X, y = make_circles(n_samples=1000, factor=0.5, noise=0.1)
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y.reshape(-1, 1), dtype=torch.float32)

# Binary classification model
class Model(nn.Module):
    def __init__(self, activation=nn.ReLU):
        super().__init__()
        self.layer0 = nn.Linear(2,5)
```



```

        self.act0 = activation()
        self.layer1 = nn.Linear(5,5)
        self.act1 = activation()
        self.layer2 = nn.Linear(5,5)
        self.act2 = activation()
        self.layer3 = nn.Linear(5,5)
        self.act3 = activation()
        self.layer4 = nn.Linear(5,1)
        self.act4 = nn.Sigmoid()

    def forward(self, x):
        x = self.act0(self.layer0(x))
        x = self.act1(self.layer1(x))
        x = self.act2(self.layer2(x))
        x = self.act3(self.layer3(x))
        x = self.act4(self.layer4(x))
        return x

# train the model and produce history
def train_loop(model, X, y, n_epochs=300, batch_size=32):
    loss_fn = nn.BCELoss()
    optimizer = optim.Adam(model.parameters(), lr=0.0001)
    batch_start = torch.arange(0, len(X), batch_size)

    bce_hist = []
    acc_hist = []
    grad_hist = [[], [], [], [], []]

    for epoch in range(n_epochs):
        # train model with optimizer
        model.train()
        layer_grad = [[], [], [], [], []]
        for start in batch_start:
            X_batch = X[start:start+batch_size]
            y_batch = y[start:start+batch_size]
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            # collect mean absolute value of gradients
            layers = [model.layer0, model.layer1, model.layer2, model.layer3,
                      model.layer4]
            for n, layer in enumerate(layers):
                mean_grad = float(layer.weight.grad.abs().mean())
                layer_grad[n].append(mean_grad)
        # evaluate BCE and accuracy at end of each epoch
        model.eval()
        with torch.no_grad():
            y_pred = model(X)
            bce = float(loss_fn(y_pred, y))
            acc = float((y_pred.round() == y).float().mean())
        bce_hist.append(bce)
        acc_hist.append(acc)

```

```

    for n, grads in enumerate(layer_grad):
        grad_hist[n].append(sum(grads)/len(grads))
    # print metrics every 10 epochs
    if epoch % 10 == 9:
        print("Epoch %d: BCE=%.4f, Accuracy=%.2f%%" % (epoch, bce, acc*100))
    return bce_hist, acc_hist, layer_grad

# pick different activation functions and compare the result visually
for activation in [nn.Sigmoid, nn.Tanh, nn.ReLU, nn.ReLU6, nn.LeakyReLU]:
    model = Model(activation=activation)
    bce_hist, acc_hist, grad_hist = train_loop(model, X, y)

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].plot(bce_hist, label="BCE")
ax[0].plot(acc_hist, label="Accuracy")
ax[0].set_xlabel("Epochs")
ax[0].set_ylim(0, 1)
for n, grads in enumerate(grad_hist):
    ax[1].plot(grads, label="layer"+str(n))
ax[1].set_xlabel("Epochs")
fig.suptitle(str(activation))
ax[0].legend()
ax[1].legend()
plt.show()

```

Listing 14.12: Experiment on different activation functions

14.4 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Articles

Vanishing gradient problem. Wikipedia.

https://en.wikipedia.org/wiki/Vanishing_gradient_problem

Sigmoid. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.Sigmoid.html>

Tanh. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.Tanh.html>

ReLU. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.ReLU.html>

ReLU6. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.ReLU6.html>

LeakyReLU. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.LeakyReLU.html>

14.5 Summary

In this chapter, you discovered how to select activation functions for your PyTorch model. You learned:

- ▷ What are the common activation functions and how they look like
- ▷ How to use activation functions in your PyTorch model
- ▷ What is vanishing gradient problem
- ▷ The impact of activation function to the performance of your model

In the next chapter, you will learn about loss function, another key component in building a deep learning model.

Loss Functions in PyTorch Models

15

The loss metric is very important for neural networks. As all machine learning models are one optimization problem or another, the loss is the objective function to minimize. In neural networks, the optimization is done with gradient descent and backpropagation. But what are loss functions, and how are they affecting your neural networks?

In this chapter, you will learn what loss functions are and delve into some commonly used loss functions and how you can apply them to your neural networks. After reading this chapter, you will learn:

- ▷ What are loss functions, and their role in training neural network models
- ▷ Common loss functions for regression and classification problems
- ▷ How to use loss functions in your PyTorch model

Let's get started!

Overview

This chapter is divided into four sections; they are:

- ▷ What Are Loss Functions?
- ▷ Loss Functions for Regression
- ▷ Loss Functions for Classification
- ▷ Custom Loss Function in PyTorch

15.1 What Are Loss Functions?

In neural networks, loss functions help optimize the performance of the model. They are usually used to measure some penalty that the model incurs on its predictions, such as the deviation of the prediction away from the ground truth label. Loss functions are usually differentiable across their domain (but it is allowed that the gradient is undefined only for very specific points, such as $x = 0$, which is basically ignored in practice). In the training

loop, they are differentiated with respect to parameters, and these gradients are used for your backpropagation and gradient descent steps to optimize your model on the training set.

Loss functions are also slightly different from metrics. While loss functions can tell you the performance of our model, they might not be of direct interest or easily explainable by humans. This is where metrics come in. Metrics such as accuracy are much more useful for humans to understand the performance of a neural network even though they might not be good choices for loss functions since they might not be differentiable.

In the following, let's explore some common loss functions, for regression problems and for classification problems.

15.2 Loss Functions for Regression

In regression problems, the model is to predict a value in a continuous range. Too good to be true that your model can predict the exact value all the time, but it is good enough if the value is close enough. Therefore, you need a loss function to measure how close it is. The farther away from the exact value, the more the loss is your prediction.

One simple function is just to measure the difference between the prediction and the target value. You do not care the value is greater than or less than the target value in finding the difference. Hence, in mathematics, we find $\frac{1}{m} \sum_{i=1}^m |\hat{y}_i - y_i|$ with m the number of training examples whereas y_i and $\hat{y}_i$ are the target and predicted values, respectively, averaged over all training examples. This is the mean absolute error (MAE).

The MAE is never negative and would be zero only if the prediction matched the ground truth perfectly. It is an intuitive loss function and might also be used as one of your metrics, specifically for regression problems, since you want to minimize the error in your predictions.

However, absolute value is not differentiable at 0. It is not really a problem because you rarely hit that value. But sometimes people would prefer to use mean square error (MSE) instead. MSE equals to $\frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$, which is similar to MAE but use square function in place of absolute value.

It also measures the deviation of the predicted value from the target value. However, the MSE squares this difference (always non-negative since squares of real numbers are always non-negative), which gives it slightly different properties. One property is that the mean squared error tolerates a large number of small errors more than a small number of large errors, which leads to models with fewer outliers or at least outliers that are less severe than models trained with a MAE. This is because the square function amplified large errors, and consequently, the gradient of the resulting MSE is more significant.

Let's look at what the mean absolute error and mean square error loss function looks like graphically:

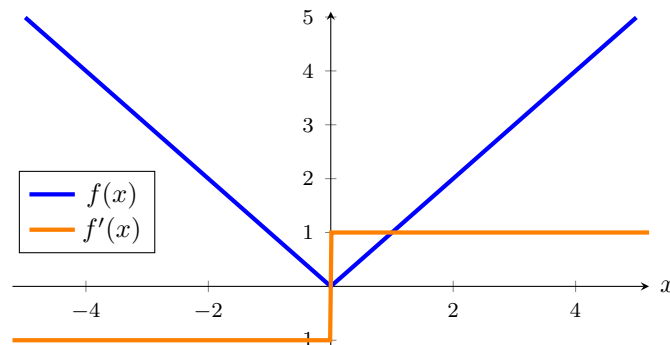


Figure 15.1: Mean absolute error loss function (blue) and gradient (orange)

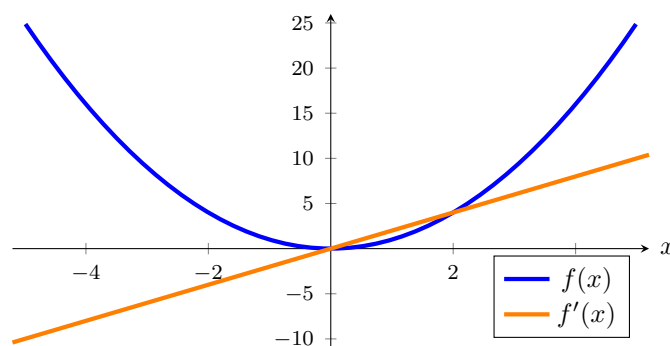


Figure 15.2: Mean square error loss function (blue) and gradient (orange)

Similar to activation functions, you might also be interested in what the gradient of the loss function looks like since you are using the gradient later to do backpropagation to train your model's parameters. You should see that in MSE, larger errors would lead to a larger magnitude for the gradient and a larger loss. Hence, for example, two training examples that deviate from their ground truths by 1 unit would lead to a loss of 2, while a single training example that deviates from its ground truth by 2 units would lead to a loss of 4, hence having a larger impact. This is not the case in MAE.

In PyTorch, you can create MAE and MSE as loss functions using `nn.L1Loss()` and `nn.MSELoss()` respectively. It is named as L1 because the computation of MAE is also called the L1-norm in mathematics. Below is an example of computing the MAE and MSE between two vectors:

```
import torch
import torch.nn as nn

mae = nn.L1Loss()
mse = nn.MSELoss()

predict = torch.tensor([0., 3.])
target = torch.tensor([1., 0.])
```

```
print("MAE: %.3f" % mae(predict, target))
print("MSE: %.3f" % mse(predict, target))
```

Listing 15.1: Computing MSE and MAE

You should get

```
MAE: 2.000
MSE: 5.000
```

Output 15.1: MAE and MSE as computed

MAE is 2.0 because $\frac{1}{2}(|0 - 1| + |3 - 0|) = \frac{1}{2}(1 + 3) = 2$ whereas MSE is 5.0 because $\frac{1}{2}[(0 - 1)^2 + (3 - 0)^2] = \frac{1}{2}(1 + 9) = 5$. Notice that in MSE, the second example with a predicted value of 3 and actual value of 0 contributes 90% of the error under the mean squared error vs. 75% under the mean absolute error.

Sometimes, you may see people use root mean squared error (RMSE) as a metric. This will take the square root of MSE. From the perspective of a loss function, MSE and RMSE are equivalent. But from the perspective of the value, the RMSE is in the same unit as the predicted values. If your prediction is money in dollars, both MAE and RMSE give you how much your prediction is away from the true value in dollars in average. But MSE is in unit of squared dollars, which its physical meaning is not intuitive.

15.3 Loss Functions for Classification

For classification problems, there is a small, discrete set of numbers that the output could take. Furthermore, the number used to label-encode the classes is arbitrary and with no semantic meaning (e.g., using the labels 0 for cat, 1 for dog, and 2 for horse does not represent that a dog is half cat and half horse). Therefore, it should not have an impact on the performance of the model.

In a classification problem, the model's output is usually a vector of probability for each category. Often, this vector is usually expected to be *logits*, i.e., real numbers to be transformed to probability using the softmax function, or the output of a softmax activation function.

The cross-entropy between two probability distributions is a measure of the difference between the two probability distributions. Precisely, it is $-\sum_i P(X = x_i) \log Q(X = x_i)$ for probability P and Q . In machine learning, we usually have the probability P provided by the training data and Q predicted by the model, which P is 1 for the correct class and 0 for every other class. The predicted probability Q , however, is usually a floating point valued between 0 and 1. Hence when used for classification problems in machine learning, this formula can be simplified into:

$$\text{categorical cross-entropy} = -\log p_{\text{target}}$$

where p_{target} is the model-predicted probability of the ground-truth class for that particular sample.

Cross-entropy metrics have a negative sign because $\log(x)$ tends to negative infinity as x tends to zero. We want a higher loss when the probability approaches 0 and a lower loss when the probability approaches 1. Graphically,

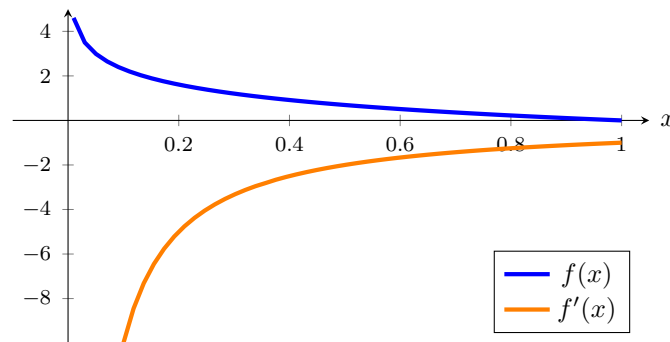


Figure 15.3: Categorical cross-entropy loss function (blue) and gradient (orange)

Notice that the loss is exactly 0 if the probability of the ground truth class is 1 as desired. Also, as the probability of the ground truth class tends to 0, the loss tends to positive infinity as well, hence substantially penalizing bad predictions. You might recognize this loss function for logistic regression, which is similar except the logistic regression loss is specific to the case of binary classes.

Looking at the gradient, you can see that the gradient is generally negative, which is also expected since, to decrease this loss, you would want the probability on the ground truth class to be as high as possible. Recall that gradient descent goes in the opposite direction of the gradient.

In PyTorch, the cross-entropy function is provided by `nn.CrossEntropyLoss()`. It takes the predicted logits and the target as parameter and compute the categorical cross-entropy. Remind that inside the `CrossEntropyLoss()` function, softmax will be applied to the logits hence you should not use softmax activation function at the output layer. Example of using the cross-entropy loss function from PyTorch is as follows:

```
import torch
import torch.nn as nn

ce = nn.CrossEntropyLoss()

logits = torch.tensor([[-1.90, -0.29, -2.30], [-0.29, -1.90, -2.30]])
target = torch.tensor([[0., 1., 0.], [1., 0., 0.]])
print("Cross entropy: %.3f" % ce(logits, target))
```

Listing 15.2: Creating and using cross-entropy loss function

It prints:

```
Cross entropy: 0.288
```

Output 15.2: Cross-entropy as computed

Note the first argument to the cross-entropy loss function is logit, not probabilities. Hence each row does not sum to 1. The second argument, however, is a tensor containing rows of probabilities. If you convert the `logits` tensor above into probability using softmax function, it would be:


```
probs = torch.tensor([[0.15, 0.75, 0.1], [0.75, 0.15, 0.1]])
```

Listing 15.3: Softmax equivalent of the logits

which each row sums to 1.0. This tensor also reveals why the cross-entropy above calculated to be 0.288, which is $-\log(0.75)$.

The other way of calculating the cross-entropy in PyTorch is not to use one-hot encoding in the target but to use the integer indices label:

```
import torch
import torch.nn as nn

ce = nn.CrossEntropyLoss()

logits = torch.tensor([[−1.90, −0.29, −2.30], [−0.29, −1.90, −2.30]])
indices = torch.tensor([1, 0])
print("Cross entropy: %.3f" % ce(logits, indices))
```

Listing 15.4: Cross-entropy loss function on integer labels

This gives you the same cross-entropy of 0.288. Note that,

```
import torch

target = torch.tensor([[0., 1., 0.], [1., 0., 0.]])
indices = torch.argmax(target, dim=1)
print(indices)
```

Listing 15.5: Converting one-hot encoding to integer indices

gives you:

```
tensor([1, 0])
```

Output 15.3: Integer indices as computed using `argmax()` function

This is how PyTorch interprets your target tensor. It is also called *sparse cross-entropy* function in other libraries, to make a distinction that it does not expect a one-hot vector.

Note in PyTorch, you can use `nn.LogSoftmax()` as an activation function. It is to apply softmax on the output of a layer and then take the logarithm on each element. If this is your output layer, you should use `nn.NLLLoss()` (negative log likelihood) as the loss function. Mathematically this duo is same as cross-entropy loss. You can confirm this by checking the code below produced the same output:

```
import torch
import torch.nn as nn

ce = nn.NLLLoss()

# softmax to apply on dimension 1, i.e. per row
logsoftmax = nn.LogSoftmax(dim=1)
```

```
logits = torch.tensor([[ -1.90, -0.29, -2.30], [-0.29, -1.90, -2.30]])
pred = logsoftmax(logits)
indices = torch.tensor([1, 0])
print("Cross entropy: %.3f" % ce(pred, indices))
```

Listing 15.6: Using negative log-likelihood function

In case of a classification problem with only two classes, it becomes binary classification. It is special because the model is now a logistic regression model in which there can be only one output instead of a vector of two values. You can still implement binary classification as multiclass classification and use the same cross-entropy function. But if you output x as the probability (between 0 and 1) for the “positive class”, it is known that the probability for the “negative class” must be $1 - x$.

In PyTorch, you have `nn.BCELoss()` for binary cross-entropy. It is specialized for binary case. For example:

```
import torch
import torch.nn as nn

bce = nn.BCELoss()

pred = torch.tensor([0.75, 0.25])
target = torch.tensor([1., 0.])
print("Binary cross entropy: %.3f" % bce(pred, target))
```

Listing 15.7: Using binary cross-entropy loss function

This gives you:

```
Binary cross entropy: 0.288
```

Output 15.4: Binary cross-entropy as computed

It is because:

$$\frac{1}{2}[-\log(0.75) + (-\log(1 - 0.25))] = -\log(0.75) = 0.288$$

Note that in PyTorch, the target label 1 is taken as the “positive class” and label 0 is the “negative class”. There should not be other values in the target tensor.

15.4 Custom Loss Function in PyTorch

Notice in above, the loss metric is calculated using an object from `torch.nn` module. The loss metric computed is a PyTorch tensor, so you can differentiate it and start the backpropagation. Therefore, nothing forbid you from creating your own loss function as long as you can compute a tensor based on the model’s output.

PyTorch does not give you all the possible loss metrics. For example, mean absolute percentage error is not included. It is like MAE, defined as:

$$\text{MAPE} = \frac{1}{m} \sum_{i=1}^m \left| \frac{\hat{y}_i - y_i}{y_i} \right|$$

Sometimes you may prefer to use MAPE. Recall the example on regression on the California housing dataset, the prediction is on the house price. It may make more sense to consider prediction accurate based on percentage difference rather than dollar difference. You can define your MAPE function, just remember to use PyTorch functions to compute, and return a PyTorch tensor.

See the full example below:

```
import copy

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
from sklearn.datasets import fetch_california_housing
from sklearn.preprocessing import StandardScaler

# Read data
data = fetch_california_housing()
X, y = data.data, data.target

# train-test split for model evaluation
X_train_raw, X_test_raw, y_train, y_test = train_test_split(X, y, train_size=0.7,
                                                            shuffle=True)

# Standardizing data
scaler = StandardScaler()
scaler.fit(X_train_raw)
X_train = scaler.transform(X_train_raw)
X_test = scaler.transform(X_test_raw)

# Convert to 2D PyTorch tensors
X_train = torch.tensor(X_train, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1, 1)

# Define the model
model = nn.Sequential(
    nn.Linear(8, 24),
    nn.ReLU(),
    nn.Linear(24, 12),
    nn.ReLU(),
    nn.Linear(12, 6),
    nn.ReLU(),
    nn.Linear(6, 1)
)

# loss function and optimizer
def loss_fn(output, target):
    # MAPE loss
    return torch.mean(torch.abs((target - output) / target))
```

```

optimizer = optim.Adam(model.parameters(), lr=0.0001)

n_epochs = 100  # number of epochs to run
batch_size = 10 # size of each batch
batch_start = torch.arange(0, len(X_train), batch_size)

# Hold the best model
best_mape = np.inf # init to infinity
best_weights = None

for epoch in range(n_epochs):
    model.train()
    for start in batch_start:
        # take a batch
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        # forward pass
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        # backward pass
        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()
    # evaluate accuracy at end of each epoch
    model.eval()
    y_pred = model(X_test)
    mape = float(loss_fn(y_pred, y_test))
    if mape < best_mape:
        best_mape = mape
        best_weights = copy.deepcopy(model.state_dict())

# restore model and return best accuracy
model.load_state_dict(best_weights)
print("MAPE: %.2f" % best_mape)

model.eval()
with torch.no_grad():
    # Test out inference with 5 samples
    for i in range(5):
        X_sample = X_test_raw[i: i+1]
        X_sample = scaler.transform(X_sample)
        X_sample = torch.tensor(X_sample, dtype=torch.float32)
        y_pred = model(X_sample)
        print(f"{X_test_raw[i]} -> {y_pred[0].numpy()} (expected {y_test[i].numpy()})")

```

Listing 15.8: Training a model using MAPE as custom loss function

Compare to the example in the other chapter, you can see that `loss_fn` now is defined as a custom function. Otherwise, everything is just the same.

15.5 Further Readings

Below are the documentations from PyTorch that give you more details on how the various loss functions are implemented:

APIs

L1Loss. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.L1Loss.html>

MSELoss. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.MSELoss.html>

CrossEntropyLoss. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html>

BCELoss. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.BCELoss.html>

NLLLoss. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.NLLLoss.html>

15.6 Summary

In this chapter, you have seen loss functions and the role that they play in a neural network. You have also seen some popular loss functions used in regression and classification models, as well as how to implement your own loss function for your PyTorch model. Specifically, you learned:

- ▷ What are loss functions, and why they are important in training
- ▷ Common loss functions for regression and classification problems
- ▷ How to use loss functions in your PyTorch model

In the next chapter, you will learn about the most common regularization technique in deep learning.

Using Dropout Regularization in PyTorch Models

16

Dropout is a simple and powerful regularization technique for neural networks and deep learning models.

In this chapter, you will discover the Dropout regularization technique and how to apply it to your models in PyTorch models.

After reading this chapter, you will know:

- ▷ How the Dropout regularization technique works
- ▷ How to use Dropout on your input layers
- ▷ How to use Dropout on your hidden layers
- ▷ How to tune the dropout level on your problem

Let's get started.

Overview

This chapter is divided into six parts; they are

- ▷ Dropout Regularization for Neural Networks
- ▷ Dropout Regularization in PyTorch
- ▷ Using Dropout on the Input Layer
- ▷ Using Dropout on the Hidden Layers
- ▷ Dropout in Evaluation Mode
- ▷ Tips for Using Dropout

16.1 Dropout Regularization for Neural Networks

Dropout is a regularization technique for neural network models proposed around 2012 to 2014. It is a layer in the neural network. During training of a neural network model, it will take the output from its previous layer, randomly select some neurons and zero them out before passing to the next layer, effectively ignoring them. This means that their contribution

to the activation of downstream neurons is temporally removed on the forward pass, and any weight updates are not applied to the neuron on the backward pass. To compensate the dropped neurons, the other neurons will be scaled up in the same percentage as the dropout probability.

When the model is used for inference, dropout layer has no effect, i.e., just pass everything as is to the next layer like an identity function.

Dropout is destructive in training but surprisingly can improve the model's accuracy. As a neural network learns, neuron weights settle into their context within the network. If the model learned a specific pattern and in reality, such pattern is not exactly as seen in training data, the model would be too fragile so failed to predict correctly. Such reliance on a specific context is referred to as complex co-adaptions.

If there are multiple patterns to give the same clue, and the model learned all of them, it should perform better. You can imagine that if neurons are randomly dropped out of the network during training, other neurons will have to step in and handle the representation required to make predictions for the missing neurons. This is believed to result in multiple independent internal representations being learned by the network.

The effect is that the network becomes less sensitive to a specific pattern. This, in turn, results in a network capable of better generalization and less likely to overfit the training data.

16.2 Dropout Regularization in PyTorch

You do not need to randomly select elements from a PyTorch tensor to implement dropout manually. The `nn.Dropout()` layer from PyTorch can be introduced into your model. It is implemented by randomly selecting nodes to be dropped out with a given probability p (e.g., 20%) while in the training loop. In PyTorch, the dropout layer further scale the resulting tensor by a factor of $\frac{1}{1-p}$ so the average tensor value is maintained. Thanks to this scaling, the dropout layer operates at inference will be an identify function (i.e., no effect, simply copy over the input tensor as output tensor). You should make sure to turn the model into inference mode when evaluating the model.

Let's see how to use `nn.Dropout()` in a PyTorch model.

The examples will use the Sonar dataset. This is a binary classification problem that aims to correctly identify rocks and mock-mines from sonar chirp returns. It is a good test dataset for neural networks because all the input values are numerical and have the same scale.

The dataset can be downloaded from the UCI Machine Learning repository¹. You can place the sonar dataset in your current working directory with the file name *sonar.csv*.

You will evaluate the developed models using scikit-learn with 10-fold cross-validation in order to tease out differences in the results better.

¹<http://archive.ics.uci.edu/ml/machine-learning-databases/undocumented/connectionist-bench/sonar/sonar.all-data>

There are 60 input values and a single output value. The input values are standardized before being used in the network. The baseline neural network model has two hidden layers, the first with 60 units and the second with 30. Stochastic gradient descent is used to train the model with a relatively low learning rate and momentum.

The full baseline model is listed below:

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import StratifiedKFold

# Read data
data = pd.read_csv("sonar.csv", header=None)
X = data.iloc[:, 0:60]
y = data.iloc[:, 60]

# Label encode the target from string to integer
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# Convert to 2D PyTorch tensors
X = torch.tensor(X.values, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# Define PyTorch model
class SonarModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(60, 60)
        self.act1 = nn.ReLU()
        self.layer2 = nn.Linear(60, 30)
        self.act2 = nn.ReLU()
        self.output = nn.Linear(30, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.act1(self.layer1(x))
        x = self.act2(self.layer2(x))
        x = self.sigmoid(self.output(x))
        return x

# Helper function to train the model and return the validation result
def model_train(model, X_train, y_train, X_val, y_val,
                n_epochs=300, batch_size=16):
    loss_fn = nn.BCELoss()
    optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.8)
    batch_start = torch.arange(0, len(X_train), batch_size)
```



```

model.train()
for epoch in range(n_epochs):
    for start in batch_start:
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    # evaluate accuracy after training
    model.eval()
    y_pred = model(X_val)
    acc = (y_pred.round() == y_val).float().mean()
    acc = float(acc)
    return acc

# run 10-fold cross validation
kfold = StratifiedKFold(n_splits=10, shuffle=True)
accuracies = []
for train, test in kfold.split(X, y):
    # create model, train, and get accuracy
    model = SonarModel()
    acc = model_train(model, X[train], y[train], X[test], y[test])
    print("Accuracy: %.2f" % acc)
    accuracies.append(acc)

# evaluate the model
mean = np.mean(accuracies)
std = np.std(accuracies)
print("Baseline: %.2f%% (+/- %.2f%%)" % (mean*100, std*100))

```

Listing 16.1: Baseline model for binary classification on the sonar data

Running the example generates an estimated classification accuracy of 82%.

```

Accuracy: 0.81
Accuracy: 0.81
Accuracy: 0.76
Accuracy: 0.86
Accuracy: 0.81
Accuracy: 0.90
Accuracy: 0.86
Accuracy: 0.95
Accuracy: 0.65
Accuracy: 0.80
Baseline: 82.12% (+/- 7.78%)

```

Output 16.1: Result of 10-fold cross-validation

16.3 Using Dropout on the Input Layer

Dropout can be applied to input neurons called the visible layer.

In the example below, a new Dropout layer between the input and the first hidden layer was added. The dropout rate is set to 20%, meaning one in five inputs will be randomly excluded from each update cycle.

Continuing from the baseline example above, the code below exercises the same network with input dropout:

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import StratifiedKFold

# Read data
data = pd.read_csv("sonar.csv", header=None)
X = data.iloc[:, 0:60]
y = data.iloc[:, 60]

# Label encode the target from string to integer
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# Convert to 2D PyTorch tensors
X = torch.tensor(X.values, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# Define PyTorch model, with dropout at input
class SonarModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.dropout = nn.Dropout(0.2)
        self.layer1 = nn.Linear(60, 60)
        self.act1 = nn.ReLU()
        self.layer2 = nn.Linear(60, 30)
        self.act2 = nn.ReLU()
        self.output = nn.Linear(30, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.dropout(x)
        x = self.act1(self.layer1(x))
        x = self.act2(self.layer2(x))
        x = self.sigmoid(self.output(x))
        return x

# Helper function to train the model and return the validation result
```

```

def model_train(model, X_train, y_train, X_val, y_val,
                n_epochs=300, batch_size=16):
    loss_fn = nn.BCELoss()
    optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.8)
    batch_start = torch.arange(0, len(X_train), batch_size)

    model.train()
    for epoch in range(n_epochs):
        for start in batch_start:
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

        # evaluate accuracy after training
        model.eval()
        y_pred = model(X_val)
        acc = (y_pred.round() == y_val).float().mean()
        acc = float(acc)
        return acc

# run 10-fold cross validation
kfold = StratifiedKFold(n_splits=10, shuffle=True)
accuracies = []
for train, test in kfold.split(X, y):
    # create model, train, and get accuracy
    model = SonarModel()
    acc = model_train(model, X[train], y[train], X[test], y[test])
    print("Accuracy: %.2f" % acc)
    accuracies.append(acc)

# evaluate the model
mean = np.mean(accuracies)
std = np.std(accuracies)
print("Baseline: %.2f%% (+/- %.2f%%)" % (mean*100, std*100))

```

Listing 16.2: Model with dropout layer

Running the example provides a slight drop in classification accuracy, at least on a single test run.

```

Accuracy: 0.62
Accuracy: 0.90
Accuracy: 0.76
Accuracy: 0.62
Accuracy: 0.67
Accuracy: 0.86
Accuracy: 0.90
Accuracy: 0.86
Accuracy: 0.90

```

```
Accuracy: 0.85  
Baseline: 79.40% (+/- 11.20%)
```

Output 16.2: Result of 10-fold cross-validation

16.4 Using Dropout on Hidden Layers

Dropout can be applied to hidden neurons in the body of your network model. This is more common.

In the example below, Dropout is applied between the two hidden layers and between the last hidden layer and the output layer. Again a dropout rate of 20% is used:

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import StratifiedKFold

# Read data
data = pd.read_csv("sonar.csv", header=None)
X = data.iloc[:, 0:60]
y = data.iloc[:, 60]

# Label encode the target from string to integer
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# Convert to 2D PyTorch tensors
X = torch.tensor(X.values, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# Define PyTorch model, with dropout at hidden layers
class SonarModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(60, 60)
        self.act1 = nn.ReLU()
        self.dropout1 = nn.Dropout(0.2)
        self.layer2 = nn.Linear(60, 30)
        self.act2 = nn.ReLU()
        self.dropout2 = nn.Dropout(0.2)
        self.output = nn.Linear(30, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.act1(self.layer1(x))
        x = self.dropout1(x)
        x = self.act2(self.layer2(x))
```

```

        x = self.dropout2(x)
        x = self.sigmoid(self.output(x))
        return x

# Helper function to train the model and return the validation result
def model_train(model, X_train, y_train, X_val, y_val,
                n_epochs=300, batch_size=16):
    loss_fn = nn.BCELoss()
    optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.8)
    batch_start = torch.arange(0, len(X_train), batch_size)

    model.train()
    for epoch in range(n_epochs):
        for start in batch_start:
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

    # evaluate accuracy after training
    model.eval()
    y_pred = model(X_val)
    acc = (y_pred.round() == y_val).float().mean()
    acc = float(acc)
    return acc

# run 10-fold cross validation
kfold = StratifiedKFold(n_splits=10, shuffle=True)
accuracies = []
for train, test in kfold.split(X, y):
    # create model, train, and get accuracy
    model = SonarModel()
    acc = model_train(model, X[train], y[train], X[test], y[test])
    print("Accuracy: %.2f" % acc)
    accuracies.append(acc)

# evaluate the model
mean = np.mean(accuracies)
std = np.std(accuracies)
print("Baseline: %.2f%% (+/- %.2f%%)" % (mean*100, std*100))

```

Listing 16.3: Modified model with two dropout layers

You can see that in this case, adding dropout layer improved the accuracy a bit.

```

Accuracy: 0.86
Accuracy: 1.00
Accuracy: 0.86
Accuracy: 0.90
Accuracy: 0.90
Accuracy: 0.86

```

```
Accuracy: 0.81
Accuracy: 0.81
Accuracy: 0.70
Accuracy: 0.85
Baseline: 85.50% (+/- 7.36%)
```

Output 16.3: Result of 10-fold cross-validation

16.5 Dropout in Evaluation Mode

Dropout will randomly reset some of the input to zero. If you wonder what happens after you have finished training, the answer is nothing! The PyTorch dropout layer should run like an identity function when the model is in evaluation mode. That's why you have `model.eval()` before you evaluate the model. This is important because the goal of dropout layer is to make sure the network learn enough clues about the input for the prediction, rather than depend on a rare phenomenon in the data. But on inference, you should provide as much information as possible to the model.

16.6 Tips for Using Dropout

The original paper on Dropout provides experimental results on a suite of standard machine learning problems. As a result, they provide a number of useful heuristics to consider when using Dropout in practice.

- ▷ Generally, use a small dropout value of 20%–50% of neurons, with 20% providing a good starting point. A probability too low has minimal effect, and a value too high results in under-learning by the network.
- ▷ Use a larger network. You are likely to get better performance when Dropout is used on a larger network, giving the model more of an opportunity to learn independent representations.
- ▷ Use Dropout on incoming (visible) as well as hidden units. Application of Dropout at each layer of the network has shown good results.
- ▷ Use a large learning rate with decay or a large momentum. Increase your learning rate by a factor of 10 to 100 or use a high momentum value of 0.9 or 0.99. This is contrary to the intuition, but the authors argue that dropout introduced a lot of noise and this can help speed up learning.
- ▷ Constrain the size of network weights. A large learning rate can result in very large network weights. Imposing a constraint on the size of network weights, such as max-norm regularization, with a size of 4 or 5 has been shown to improve results.

16.7 Further Readings

Below are resources you can use to learn more about Dropout in neural networks and deep learning models.

Papers

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”. *Journal of Machine Learning Research*, 15(56), 2014, pp. 1929–1958.

<https://jmlr.org/papers/v15/srivastava14a.html>

Geoffrey E. Hinton, Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan R. Salakhutdinov. “Improving neural networks by preventing co-adaptation of feature detectors”, 2012.

<https://arxiv.org/abs/1207.0580>

Online materials

Connectionist Bench (Snoar, Mines vs. Rocks) Data Set. UCI Machine Learning Repository.

[https://archive.ics.uci.edu/ml/datasets/Connectionist+Bench+\(Sonar,+Mines+vs.+Rocks\)](https://archive.ics.uci.edu/ml/datasets/Connectionist+Bench+(Sonar,+Mines+vs.+Rocks))

How does the dropout method work in deep learning? Quora.

<https://www.quora.com/How-does-the-dropout-method-work-in-deep-learning>

Dropout. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.Dropout.html>

16.8 Summary

In this chapter, you discovered the Dropout regularization technique for deep learning models. You learned:

- ▷ What Dropout is and how it works
- ▷ How you can use Dropout on your own deep learning models.
- ▷ Tips for getting the best results from Dropout on your own models.

In the next chapter, you will learn about how learning rate schedule can speed up your training.

Using Learning Rate Schedule in PyTorch Training

17

Training a neural network or large deep learning model is a difficult optimization task.

The classical algorithm to train neural networks is called stochastic gradient descent. It has been well established that you can achieve increased performance and faster training on some problems by using a learning rate that changes during training.

In this chapter, you will discover what is learning rate schedule and how you can use different learning rate schedules for your neural network models in PyTorch.

After reading this chapter, you will know:

- ▷ The role of learning rate schedule in model training
- ▷ How to use learning rate schedule in PyTorch training loop
- ▷ How to setup your own learning rate schedule

Let's get started.

Overview

This chapter is divided into three parts; they are

- ▷ Learning Rate Schedule for Training Models
- ▷ Applying Learning Rate Schedule in PyTorch Training
- ▷ Custom Learning Rate Schedules

17.1 Learning Rate Schedule for Training Models

Gradient descent is an algorithm of numerical optimization. What it does is to update parameters using the formula:

$$w := w - \alpha \frac{dy}{dw}$$

In this formula, w is the parameter, e.g., the weight in a neural network, and y is the objective, e.g., the loss function. What it does is to move w to the direction that you can minimize y .

The direction is provided by the differentiation, $\frac{dy}{dw}$, but how much you should move w is controlled by the *learning rate* α .

An easy start is to use a constant learning rate in gradient descent algorithm. But you can do better with a *learning rate schedule*. A schedule is to make learning rate adaptive to the gradient descent optimization procedure, so you can increase performance and reduce training time.

In the neural network training process, data is feed into the network in batches, with many batches in one epoch. Each batch triggers one training step, which the gradient descent algorithm updates the parameters once. However, usually the learning rate schedule is updated once for each training epoch only.

You can update the learning rate as frequent as each step but usually it is updated once per epoch because you want to know how the network performs in order to determine how the learning rate should update. Regularly, a model is evaluated with validation dataset once per epoch.

There are multiple ways of making learning rate adaptive. At the beginning of training, you may prefer a larger learning rate so you improve the network coarsely to speed up the progress. In a very complex neural network model, you may also prefer to gradually increase the learning rate at the beginning because you need the network to explore on the different dimensions of prediction. At the end of training, however, you always want to have the learning rate smaller. Since at that time, you are about to get the best performance from the model and it is easy to overshoot if the learning rate is large.

Therefore, the simplest and perhaps most used adaptation of the learning rate during training are techniques that reduce the learning rate over time. These have the benefit of making large changes at the beginning of the training procedure when larger learning rate values are used and decreasing the learning rate so that a smaller rate and, therefore, smaller training updates are made to weights later in the training procedure.

This has the effect of quickly learning good weights early and fine-tuning them later.

Next, let's look at how you can setup learning rate schedules in PyTorch.

17.2 Applying Learning Rate Schedules in PyTorch Training

In PyTorch, a model is updated by an optimizer and learning rate is a parameter of the optimizer. Learning rate schedule is an algorithm to update the learning rate in an optimizer.

Below is an example of creating a learning rate schedule:

```
import torch
import torch.optim as optim
import torch.optim.lr_scheduler as lr_scheduler

scheduler = lr_scheduler.LinearLR(optimizer,
                                  start_factor=1.0, end_factor=0.3, total_iters=10)
```

Listing 17.1: Setting up a learning rate schedule

There are many learning rate scheduler provided by PyTorch in `torch.optim.lr_scheduler` submodule. All the scheduler needs the optimizer to update as first argument. Depends on the scheduler, you may need to provide more arguments to setup one.

Let's start with an example model. In below, a model is to solve the ionosphere binary classification problem. This is a small dataset that you can download from the UCI Machine Learning repository¹. Place the data file in your working directory with the filename `ionosphere.csv`.

The ionosphere dataset is good for practicing with neural networks because all the input values are small numerical values of the same scale.

A small neural network model is constructed with a single hidden layer with 34 neurons, using the ReLU activation function. The output layer has a single neuron and uses the sigmoid activation function in order to output probability-like values.

Plain stochastic gradient descent algorithm is used, with a fixed learning rate 0.1. The model is trained for 50 epochs. The state parameters of an optimizer can be found in `optimizer.param_groups`; which the learning rate is a floating point value at `optimizer.param_groups[0]["lr"]`. At the end of each epoch, the learning rate from the optimizer is printed.

The complete example is listed below.

```
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

# load dataset, split into input (X) and output (y) variables
dataframe = pd.read_csv("ionosphere.csv", header=None)
dataset = dataframe.values
X = dataset[:,0:34].astype(float)
y = dataset[:,34]

# encode class values as integers
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# convert into PyTorch tensors
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# train-test split for evaluation of the model
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# create model
model = nn.Sequential()
```

¹<http://archive.ics.uci.edu/ml/machine-learning-databases/ionosphere/ionosphere.data>

```

    nn.Linear(34, 34),
    nn.ReLU(),
    nn.Linear(34, 1),
    nn.Sigmoid()
)

# Train the model
n_epochs = 50
batch_size = 24
batch_start = torch.arange(0, len(X_train), batch_size)
lr = 0.1
loss_fn = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=lr)
model.train()
for epoch in range(n_epochs):
    for start in batch_start:
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    print("Epoch %d: SGD lr=%.4f" % (epoch, optimizer.param_groups[0]["lr"]))

# evaluate accuracy after training
model.eval()
y_pred = model(X_test)
acc = (y_pred.round() == y_test).float().mean()
acc = float(acc)
print("Model accuracy: %.2f%%" % (acc*100))

```

Listing 17.2: Model training with fixed learning rate

Running this model produces:

```

Epoch 0: SGD lr=0.1000
Epoch 1: SGD lr=0.1000
Epoch 2: SGD lr=0.1000
Epoch 3: SGD lr=0.1000
Epoch 4: SGD lr=0.1000
...
Epoch 45: SGD lr=0.1000
Epoch 46: SGD lr=0.1000
Epoch 47: SGD lr=0.1000
Epoch 48: SGD lr=0.1000
Epoch 49: SGD lr=0.1000
Model accuracy: 86.79%

```

Output 17.1: Training progress with learning rate reported

You can confirm that the learning rate didn't change over the entire training process. Let's make the training process start with a larger learning rate and end with a smaller rate. To

introduce a learning rate scheduler, you need to run its `step()` function in the training loop. The code above is modified into the following:

```
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
import torch.optim.lr_scheduler as lr_scheduler
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

# load dataset, split into input (X) and output (y) variables
dataframe = pd.read_csv("ionosphere.csv", header=None)
dataset = dataframe.values
X = dataset[:,0:34].astype(float)
y = dataset[:,34]

# encode class values as integers
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# convert into PyTorch tensors
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# train-test split for evaluation of the model
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# create model
model = nn.Sequential(
    nn.Linear(34, 34),
    nn.ReLU(),
    nn.Linear(34, 1),
    nn.Sigmoid()
)

# Train the model
n_epochs = 50
batch_size = 24
batch_start = torch.arange(0, len(X_train), batch_size)
lr = 0.1
loss_fn = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=lr)
scheduler = lr_scheduler.LinearLR(optimizer,
                                  start_factor=1.0, end_factor=0.5, total_iters=30)

model.train()
for epoch in range(n_epochs):
    for start in batch_start:
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
```

```

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        before_lr = optimizer.param_groups[0]["lr"]
        scheduler.step()
        after_lr = optimizer.param_groups[0]["lr"]
        print("Epoch %d: SGD lr %.4f -> %.4f" % (epoch, before_lr, after_lr))

# evaluate accuracy after training
model.eval()
y_pred = model(X_test)
acc = (y_pred.round() == y_test).float().mean()
acc = float(acc)
print("Model accuracy: %.2f%%" % (acc*100))

```

Listing 17.3: Model training with learning rate scheduler

It prints:

```

Epoch 0: SGD lr 0.1000 -> 0.0983
Epoch 1: SGD lr 0.0983 -> 0.0967
Epoch 2: SGD lr 0.0967 -> 0.0950
Epoch 3: SGD lr 0.0950 -> 0.0933
Epoch 4: SGD lr 0.0933 -> 0.0917
...
Epoch 28: SGD lr 0.0533 -> 0.0517
Epoch 29: SGD lr 0.0517 -> 0.0500
Epoch 30: SGD lr 0.0500 -> 0.0500
Epoch 31: SGD lr 0.0500 -> 0.0500
...
Epoch 48: SGD lr 0.0500 -> 0.0500
Epoch 49: SGD lr 0.0500 -> 0.0500
Model accuracy: 88.68%

```

Output 17.2: Change in learning rates

First thing to notice is the improved accuracy, which is the reason why you would be interested in a different learning rate schedule.

In the above, `LinearLR()` is used. It is a linear rate scheduler and it takes three additional parameters, the `start_factor`, `end_factor`, and `total_iters`. You set `start_factor` to 1.0, `end_factor` to 0.5, and `total_iters` to 30, therefore it will make a multiplicative factor decrease from 1.0 to 0.5, in 30 equal steps. After 30 steps, the factor will stay at 0.5. This factor is then multiplied to the original learning rate at the optimizer. Hence you will see the learning rate decreased from $0.1 \times 1.0 = 0.1$ to $0.1 \times 0.5 = 0.05$.

Besides `LinearLR()`, you can also use `ExponentialLR()`, its syntax is:

```
scheduler = lr_scheduler.ExponentialLR(optimizer, gamma=0.99)
```

Listing 17.4: Using exponential learning rate scheduler

If you replaced `LinearLR()` with this, you will see the learning rate updated as follows:

```
Epoch 0: SGD lr 0.1000 -> 0.0990
Epoch 1: SGD lr 0.0990 -> 0.0980
Epoch 2: SGD lr 0.0980 -> 0.0970
Epoch 3: SGD lr 0.0970 -> 0.0961
Epoch 4: SGD lr 0.0961 -> 0.0951
...
Epoch 45: SGD lr 0.0636 -> 0.0630
Epoch 46: SGD lr 0.0630 -> 0.0624
Epoch 47: SGD lr 0.0624 -> 0.0617
Epoch 48: SGD lr 0.0617 -> 0.0611
Epoch 49: SGD lr 0.0611 -> 0.0605
```

Output 17.3: Learning rate updated in training loop

In which the learning rate is updated by multiplying with a constant factor `gamma` in each scheduler update.

17.3 Custom Learning Rate Schedules

There is no general rule that a particular learning rate schedule works the best. Sometimes, you like to have a special learning rate schedule that PyTorch didn't provide. A custom learning rate schedule can be defined using a custom function. For example, you want to have a learning rate that:

$$lr_n = \frac{lr_0}{1 + \alpha n}$$

on epoch n , which lr_0 is the initial learning rate, at epoch 0, and α is a constant. You can implement a function that given the epoch n calculate learning rate lr_n :

```
def lr_lambda(epoch):
    # LR to be 0.1 * (1/(1+0.01*epoch))
    base_lr = 0.1
    factor = 0.01
    return base_lr/(1+factor*epoch)
```

Listing 17.5: Function to compute custom learning rate

Then, you can setup a `LambdaLR()` to update the learning rate according to this function:

```
scheduler = lr_scheduler.LambdaLR(optimizer, lr_lambda)
```

Listing 17.6: Setting up learning rate scheduler using a custom function

Modifying the previous example to use `LambdaLR()`, you have the following:

```
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
import torch.optim.lr_scheduler as lr_scheduler
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
```

```

# load dataset, split into input (X) and output (y) variables
dataframe = pd.read_csv("ionosphere.csv", header=None)
dataset = dataframe.values
X = dataset[:,0:34].astype(float)
y = dataset[:,34]

# encode class values as integers
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# convert into PyTorch tensors
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# train-test split for evaluation of the model
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# create model
model = nn.Sequential(
    nn.Linear(34, 34),
    nn.ReLU(),
    nn.Linear(34, 1),
    nn.Sigmoid()
)

def lr_lambda(epoch):
    # LR to be 0.1 * (1/(1+0.01*epoch))
    base_lr = 0.1
    factor = 0.01
    return base_lr/(1+factor*epoch)

# Train the model
n_epochs = 50
batch_size = 24
batch_start = torch.arange(0, len(X_train), batch_size)
lr = 0.1
loss_fn = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=lr)
scheduler = lr_scheduler.LambdaLR(optimizer, lr_lambda)
model.train()
for epoch in range(n_epochs):
    for start in batch_start:
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    before_lr = optimizer.param_groups[0]["lr"]
    scheduler.step()
    after_lr = optimizer.param_groups[0]["lr"]

```

```

    print("Epoch %d: SGD lr %.4f -> %.4f" % (epoch, before_lr, after_lr))

# evaluate accuracy after training
model.eval()
y_pred = model(X_test)
acc = (y_pred.round() == y_test).float().mean()
acc = float(acc)
print("Model accuracy: %.2f%%" % (acc*100))

```

Listing 17.7: Model training with custom learning rate schedule

Which produces:

```

Epoch 0: SGD lr 0.0100 -> 0.0099
Epoch 1: SGD lr 0.0099 -> 0.0098
Epoch 2: SGD lr 0.0098 -> 0.0097
Epoch 3: SGD lr 0.0097 -> 0.0096
Epoch 4: SGD lr 0.0096 -> 0.0095
...
Epoch 45: SGD lr 0.0069 -> 0.0068
Epoch 46: SGD lr 0.0068 -> 0.0068
Epoch 47: SGD lr 0.0068 -> 0.0068
Epoch 48: SGD lr 0.0068 -> 0.0067
Epoch 49: SGD lr 0.0067 -> 0.0067

```

Output 17.4: Learning rate update in training loop

Note that although the function provided to `LambdaLR()` assumes an argument `epoch`, it is not tied to the epoch in the training loop but simply counts how many times you invoked `scheduler.step()`.

17.4 Tips for Using Learning Rate Schedules

This section lists some tips and tricks to consider when using learning rate schedules with neural networks.

- ▷ *Increase the initial learning rate.* Because the learning rate will very likely decrease, start with a larger value to decrease from. A larger learning rate will result in a lot larger changes to the weights, at least in the beginning, allowing you to benefit from the fine-tuning later.
- ▷ *Use a large momentum.* Many optimizers can consider momentum. Using a larger momentum value will help the optimization algorithm continue to make updates in the right direction when your learning rate shrinks to small values.
- ▷ *Experiment with different schedules.* It will not be clear which learning rate schedule to use, so try a few with different configuration options and see what works best on your problem. Also, try schedules that change exponentially and even schedules that respond to the accuracy of your model on the training or test datasets.

17.5 Further Readings

Below is the documentation for more details on using learning rates in PyTorch:

Articles

Ionosphere Data Set. UCI Machine Learning Repository.

<https://archive.ics.uci.edu/ml/datasets/Ionosphere>

How to adjust learning rate. PyTorch documentation.

<https://pytorch.org/docs/stable/optim.html#how-to-adjust-learning-rate>

17.6 Summary

In this chapter, you discovered learning rate schedules for training neural network models. After reading this chapter, you learned:

- ▷ How learning rate affects your model training
- ▷ How to setup learning rate schedule in PyTorch
- ▷ How to create a custom learning rate schedule

In the next chapter, you will learn about the `DataLoader` class that comes with PyTorch and how it can help you make a more concise training loop.

Training a PyTorch Model with DataLoader and Dataset

18

When you build and train a PyTorch deep learning model, you can provide the training data in several different ways. Ultimately, a PyTorch model works like a function that takes a PyTorch tensor and returns you another tensor. You have a lot of freedom in how to get the input tensors. Probably the easiest is to prepare a large tensor of the entire dataset and extract a small batch from it in each training step. But you will see that using the `DataLoader` can save you a few lines of code in dealing with data.

In this chapter, you will see how you can use the `Data` and `DataLoader` in PyTorch. After finishing this chapter, you will learn:

- ▷ How to create and use `DataLoader` to train your PyTorch model
- ▷ How to use `Data` class to generate data on the fly

Let's get started.

Overview

This chapter is divided into three parts; they are:

- ▷ What is `DataLoader`?
- ▷ Using `DataLoader` in a Training Loop

18.1 What is DataLoader?

To train a deep learning model, you need data. Usually data is available as a dataset. In a dataset, there are a lot of data sample or instances. You can ask the model to take one sample at a time but usually you would let the model to process one batch of several samples. You may create a batch by extracting a slice from the dataset, using the slicing syntax on the tensor. For a better quality of training, you may also want to shuffle the entire dataset on each epoch so no two batch would be the same in the entire training loop. Sometimes, you may introduce *data augmentation* to manually introduce more variance to the data. This is common for image-related tasks, which you can randomly tilt or zoom the image a bit to generate a lot of data sample from a few images.

You can imagine there can be a lot of code to write to do all these. But it is much easier with the `DataLoader`.

The following is an example of how to create a `DataLoader` and take a batch from it. In this example, the sonar dataset¹ is used and ultimately, it is converted into PyTorch tensors and passed on to `DataLoader`:

```
import pandas as pd
import torch
from torch.utils.data import DataLoader
from sklearn.preprocessing import LabelEncoder

# Read data, convert to NumPy arrays
data = pd.read_csv("sonar.csv", header=None)
X = data.iloc[:, 0:60].values
y = data.iloc[:, 60].values

# encode class values as integers
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# convert into PyTorch tensors
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# create DataLoader, then take one batch
loader = DataLoader(list(zip(X,y)), shuffle=True, batch_size=16)
for X_batch, y_batch in loader:
    print(X_batch, y_batch)
    break
```

Listing 18.1: Loading the sonar dataset and convert to PyTorch tensors

You can see from the output of above that `X_batch` and `y_batch` are PyTorch tensors. The `loader` is an instance of `DataLoader` class which can work like an iterable. Each time you read from it, you get a batch of features and targets from the original dataset.

When you create a `DataLoader` instance, you need to provide a list of sample pairs. Each sample pair is one data sample of feature and the corresponding target. A list is required because `DataLoader` expect to use `len()` to find the total size of the dataset and using array index to retrieve a particular sample. The batch size is a parameter to `DataLoader` so it knows how to create a batch from the entire dataset. You should almost always use `shuffle=True` so every time you load the data, the samples are shuffled. It is useful for training because in each epoch, you are going to read every batch once. When you proceed from one epoch to another, as `DataLoader` knows you depleted all the batches, it will re-shuffle so you get a new combination of samples.

¹[http://archive.ics.uci.edu/ml/datasets/connectionist+bench+\(sonar,+mines+vs.+rocks\)](http://archive.ics.uci.edu/ml/datasets/connectionist+bench+(sonar,+mines+vs.+rocks))

18.2 Using DataLoader in a Training Loop

The following is an example to make use of `DataLoader` in a training loop:

```
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split

# train-test split for evaluation of the model
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# setup DataLoader for training set
loader = DataLoader(list(zip(X_train, y_train)), shuffle=True, batch_size=16)

# create model
model = nn.Sequential(
    nn.Linear(60, 60),
    nn.ReLU(),
    nn.Linear(60, 30),
    nn.ReLU(),
    nn.Linear(30, 1),
    nn.Sigmoid()
)

# Train the model
n_epochs = 200
loss_fn = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=0.1)
model.train()
for epoch in range(n_epochs):
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

# evaluate accuracy after training
model.eval()
y_pred = model(X_test)
acc = (y_pred.round() == y_test).float().mean()
acc = float(acc)
print("Model accuracy: %.2f%%" % (acc*100))
```

Listing 18.2: Training a model with `DataLoader`

You can see that once you created the `DataLoader` instance, the training loop can only be easier. In the above, only the training set is packaged with a `DataLoader` because you need to loop through it in batches. You can also create a `DataLoader` for the test set and use it for model evaluation, but since the accuracy is computed over the entire test set rather than in a batch, the benefit of `DataLoader` is not significant.

Putting everything together, below is the complete code.

```
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
from torch.utils.data import DataLoader
from sklearn.preprocessing import LabelEncoder

# Read data, convert to NumPy arrays
data = pd.read_csv("sonar.csv", header=None)
X = data.iloc[:, 0:60].values
y = data.iloc[:, 60].values

# encode class values as integers
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# convert into PyTorch tensors
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# train-test split for evaluation of the model
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

# set up DataLoader for training set
loader = DataLoader(list(zip(X_train, y_train)), shuffle=True, batch_size=16)

# create model
model = nn.Sequential(
    nn.Linear(60, 60),
    nn.ReLU(),
    nn.Linear(60, 30),
    nn.ReLU(),
    nn.Linear(30, 1),
    nn.Sigmoid()
)

# Train the model
n_epochs = 200
loss_fn = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=0.1)
model.train()
for epoch in range(n_epochs):
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

# evaluate accuracy after training
model.eval()
y_pred = model(X_test)
```

```
acc = (y_pred.round() == y_test).float().mean()
acc = float(acc)
print("Model accuracy: %.2f%%" % (acc*100))
```

Listing 18.3: Reading data from CSV and train a model with DataLoader

18.3 Create Data Iterator using Dataset Class

In PyTorch, there is a `Dataset` class that can be tightly coupled with the `DataLoader` class. Recall that `DataLoader` expects its first argument can work with `len()` and with array index. The `Dataset` class is a base class for this. The reason you may want to use `Dataset` class is there are some special handling before you can get the data sample. For example, data should be read from database or disk and you only want to keep a few samples in memory rather than prefetch everything. Another example is to perform real-time preprocessing of data, such as random augmentation that is common in image tasks.

To use `Dataset` class, you just subclass from it and implement two member functions. Below is an example:

```
from torch.utils.data import Dataset

class SonarDataset(Dataset):
    def __init__(self, X, y):
        # convert into PyTorch tensors and remember them
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.float32)

    def __len__(self):
        # this should return the size of the dataset
        return len(self.X)

    def __getitem__(self, idx):
        # this should return one sample from the dataset
        features = self.X[idx]
        target = self.y[idx]
        return features, target
```

Listing 18.4: Creating a Dataset

This is not the most powerful way to use `Dataset` but simple enough to demonstrate how it works. With this, you can create a `DataLoader` and use it for model training. Modifying from the previous example, you have the following:

```
...

# setup DataLoader for training set
dataset = SonarDataset(X_train, y_train)
loader = DataLoader(dataset, shuffle=True, batch_size=16)

# create model
model = nn.Sequential(
```

```

    nn.Linear(60, 60),
    nn.ReLU(),
    nn.Linear(60, 30),
    nn.ReLU(),
    nn.Linear(30, 1),
    nn.Sigmoid()
)

# Train the model
n_epochs = 200
loss_fn = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=0.1)
model.train()
for epoch in range(n_epochs):
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

# evaluate accuracy after training
model.eval()
y_pred = model(torch.tensor(X_test, dtype=torch.float32))
y_test = torch.tensor(y_test, dtype=torch.float32)
acc = (y_pred.round() == y_test).float().mean()
acc = float(acc)
print("Model accuracy: %.2f%%" % (acc*100))

```

Listing 18.5: Using *DataLoader* with *Dataset*

You setup `dataset` as an instance of `SonarDataset` which you implemented the `__len__()` and `__getitem__()` functions. This is used in place of the list in the previous example to setup the `DataLoader` instance. Afterward, everything is the same in the training loop. Note that you still use PyTorch tensors directly for the test set in the example.

In the `__getitem__()` function, you take an integer that works like an array index and returns a pair, the features and the target. You can implement anything in this function: Run some code to generate a synthetic data sample, read data on the fly from the internet, or add random variations to the data. You will also find it useful in the situation that you cannot keep the entire dataset in memory, so you can load only the data samples that you need it.

In fact, since you created a PyTorch dataset, you don't need to use scikit-learn to split data into training set and test set. In `torch.utils.data` submodule, you have a function `random_split()` that works with `Dataset` class for the same purpose. A full example is below:

```

import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, random_split, default_collate
from sklearn.preprocessing import LabelEncoder

```

```

# Read data, convert to NumPy arrays
data = pd.read_csv("sonar.csv", header=None)
X = data.iloc[:, 0:60].values
y = data.iloc[:, 60].values

# encode class values as integers
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y).reshape(-1, 1)

class SonarDataset(Dataset):
    def __init__(self, X, y):
        # convert into PyTorch tensors and remember them
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.float32)

    def __len__(self):
        # this should return the size of the dataset
        return len(self.X)

    def __getitem__(self, idx):
        # this should return one sample from the dataset
        features = self.X[idx]
        target = self.y[idx]
        return features, target

# set up DataLoader for data set
dataset = SonarDataset(X, y)
trainset, testset = random_split(dataset, [0.7, 0.3])
loader = DataLoader(trainset, shuffle=True, batch_size=16)

# create model
model = nn.Sequential(
    nn.Linear(60, 60),
    nn.ReLU(),
    nn.Linear(60, 30),
    nn.ReLU(),
    nn.Linear(30, 1),
    nn.Sigmoid()
)

# Train the model
n_epochs = 200
loss_fn = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=0.1)
model.train()
for epoch in range(n_epochs):
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

```



```
# create one test tensor from the testset
X_test, y_test = default_collate(testset)
model.eval()
y_pred = model(X_test)
acc = (y_pred.round() == y_test).float().mean()
acc = float(acc)
print("Model accuracy: %.2f%%" % (acc*100))
```

Listing 18.6: Complete example of training a model with *DataLoader* and *Dataset*

It is very similar to the example you have before. Beware that the PyTorch model still needs a tensor as input, not a *Dataset*. Hence in the above, you need to use the `default_collate()` function to collect samples from a dataset into tensors.

18.4 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Articles

torch.utils.data. PyTorch documentation.

<https://pytorch.org/docs/stable/data.html>

Datasets & DataLoaders. PyTorch tutorial.

https://pytorch.org/tutorials/beginner/basics/data_tutorial.html

18.5 Summary

In this chapter, you learned how to use *DataLoader* to create shuffled batches of data and how to use *Dataset* to provide data samples. Specifically you learned:

- ▷ *DataLoader* as a convenient way of providing batches of data to the training loop
- ▷ How to use *Dataset* to produce data samples
- ▷ How combine *Dataset* and *DataLoader* to generate batches of data on the fly for model training

As you have seen in many examples so far, you make use of scikit-learn for various tasks such as train-test split. In the next chapter, you will learn how to wrap your PyTorch model into a scikit-learn model so you can make use of its model selection routines.

Use PyTorch Deep Learning Models with scikit-learn

The most popular deep learning libraries in Python for research and development are TensorFlow/Keras and PyTorch, due to their simplicity. The scikit-learn library, however, is the most popular library for general machine learning in Python. In this chapter, you will discover how to use deep learning models from PyTorch with the scikit-learn library in Python. This will allow you to leverage the power of the scikit-learn library for tasks like model evaluation and model hyper-parameter optimization. After completing this lesson you will know:

- ▷ How to wrap a PyTorch model for use with the scikit-learn machine learning library
- ▷ How to easily evaluate PyTorch models using cross-validation in scikit-learn
- ▷ How to tune PyTorch model hyperparameters using grid search in scikit-learn

Let's get started.

Overview

This chapter is in four parts; they are:

- ▷ Overview of skorch
- ▷ Evaluate Deep Learning Models with Cross-Validation
- ▷ Running k -Fold Cross-validation with scikit-learn
- ▷ Grid Search Deep Learning Model Parameters

19.1 Overview of skorch

PyTorch is a popular library for deep learning in Python, but the focus of the library is deep learning, not all of machine learning. In fact, it strives for minimalism, focusing on only what you need to quickly and simply define and build deep learning models. The scikit-learn library in Python is built upon the SciPy stack for efficient numerical computation. It is a fully featured library for general purpose machine learning and provides many useful utilities in developing deep learning models. Not least of which are:

- ▷ Evaluation of models using resampling methods like k -fold cross-validation
- ▷ Efficient search and evaluation of model hyperparameters
- ▷ Connecting multiple steps of a machine learning workflow into a pipeline

PyTorch cannot work with scikit-learn directly. But thanks to the duck-typing nature of Python language, it is easy to adapt a PyTorch model for use with scikit-learn. Indeed, the `skorch` module is built for this purpose. With `skorch`, you can make your PyTorch model work just like a scikit-learn model. You may find it easier to use.

In the following sections, you will work through examples of using the `NeuralNetClassifier` wrapper for a classification neural network created in PyTorch and used in the scikit-learn library. The test problem is the Sonar dataset¹. This is a small dataset with all numerical attributes that is easy to work with.

The following examples assume you have successfully installed PyTorch, `skorch`, and scikit-learn. If you use `pip` for your Python modules, you may install them with:

```
pip install torch skorch scikit-learn
```

Listing 19.1: Installing skorch

If you're using `conda`, `skorch` is only available in the forge channel:

```
conda install -c conda-forge pytorch skorch scikit-learn
```

Listing 19.2: Installing skorch

19.2 Evaluate Deep Learning Models with Cross-Validation

The `NeuralNet` class, or more specialized `NeuralNetClassifier`, `NeuralNetBinaryClassifier`, and `NeuralNetRegressor` classes in `skorch` are factory wrappers for PyTorch models. They take an argument `model` which is a class or a function to call to get your model. In return, these wrapper classes allow you to specify loss function and optimizer, then the training loop comes for free. This is the convenience compared to using PyTorch directly.

Below is a simple example of training a binary classifier on the Sonar dataset:

```
import pandas as pd
import torch
import torch.nn as nn
from sklearn.preprocessing import LabelEncoder
from skorch import NeuralNetBinaryClassifier

# Read data
data = pd.read_csv("sonar.csv", header=None)
X = data.iloc[:, 0:60]
y = data.iloc[:, 60]
```

¹[https://archive.ics.uci.edu/ml/datasets/Connectionist+Bench+\(Sonar,+Mines+vs.+Rocks\)](https://archive.ics.uci.edu/ml/datasets/Connectionist+Bench+(Sonar,+Mines+vs.+Rocks))

```

# Binary encoding of labels
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# Convert to 2D PyTorch tensors
X = torch.tensor(X.values, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32)

# Define the model
class SonarClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(60, 60)
        self.act1 = nn.ReLU()
        self.layer2 = nn.Linear(60, 60)
        self.act2 = nn.ReLU()
        self.layer3 = nn.Linear(60, 60)
        self.act3 = nn.ReLU()
        self.output = nn.Linear(60, 1)

    def forward(self, x):
        x = self.act1(self.layer1(x))
        x = self.act2(self.layer2(x))
        x = self.act3(self.layer3(x))
        x = self.output(x)
        return x

# create the skorch wrapper
model = NeuralNetBinaryClassifier(
    SonarClassifier,
    criterion=torch.nn.BCEWithLogitsLoss,
    optimizer=torch.optim.Adam,
    lr=0.0001,
    max_epochs=150,
    batch_size=10
)

# run
model.fit(X, y)

```

Listing 19.3: Training a binary classifier on the Sonar dataset

In this model, you used `torch.nn.BCEWithLogitsLoss` as the loss function (that is indeed the default of `NeuralNetBinaryClassifier`). It is to combine the sigmoid function with binary cross-entropy loss, so that you don't need to put the sigmoid function at the output of the model. It is sometimes preferred to provide better numerical stability, as the exponential function in sigmoid and log function in cross-entropy can cancel out each other and combining them together in the same function allows more accurate floating point computation.

In addition, you specified the training parameters such as the number of epochs and batch size in the skorch wrapper. Then you just need to call `fit()` function with the input feature and target. The wrapper will help you initialize a model and train it. If you learned Keras before, you also see it brings PyTorch syntax closer to that of Keras.

Running the above will produce the following:

epoch	train_loss	valid_acc	valid_loss	dur
1	0.6952	0.5476	0.6921	0.0135
2	0.6930	0.5476	0.6920	0.0114
3	0.6925	0.5476	0.6919	0.0104
4	0.6922	0.5238	0.6918	0.0118
5	0.6919	0.5238	0.6917	0.0112
...				
146	0.2942	0.4524	0.9425	0.0115
147	0.2920	0.4524	0.9465	0.0123
148	0.2899	0.4524	0.9495	0.0112
149	0.2879	0.4524	0.9544	0.0121
150	0.2859	0.4524	0.9583	0.0118

Output 19.1: Training progress produced by `model.fit()`

Note that `skorch` is positioned as a wrapper for PyTorch models to adapt to scikit-learn interface. Therefore, you should use the model as if it is a scikit-learn model. For example, to train your binary classification model, it is expected the target to be a vector rather than an $n \times 1$ matrix. And to run the model for inference, you should use `model.predict(X)` or `model.predict_proba(X)`. It is also why you should use `NeuralNetBinaryClassifier`, such that the classification-related scikit-learn functions are provided as model methods.

19.3 Running k -Fold Cross-validation with scikit-learn

Using a wrapper over your PyTorch model already save you a lot of boilerplate code on building your own training loop. But the entire suite of machine learning functions from scikit-learn is the real productivity boost.

One example is to use the model selection functions from scikit-learn. Let's say you want to evaluate this model design with k -fold cross-validation. Normally, it means to take a dataset, split it into k portions, then run a loop to select one of these portion as test set and the rest as training set to train a model from scratch and obtain an evaluation score. It is not difficult to do but you need to write several lines of code to implement these.

Indeed, we can make use of the k -fold and cross-validation function from scikit-learn, as follows:

```
from sklearn.model_selection import StratifiedKFold
from sklearn.model_selection import cross_val_score

model = NeuralNetBinaryClassifier(
    SonarClassifier,
    criterion=torch.nn.BCEWithLogitsLoss,
    optimizer=torch.optim.Adam,
    lr=0.0001,
    max_epochs=150,
    batch_size=10,
    verbose=False
```

```
)

kfold = StratifiedKFold(n_splits=5, shuffle=True)
results = cross_val_score(model, X, y, cv=kfold)
print(results)
```

*Listing 19.4: Using k -fold cross-validation from *scikit-learn* with a PyTorch model*

The parameter `verbose=False` in `NeuralNetBinaryClassifier` is to stop the display of progress while the model is trained, since there was a lot. The above code will print the validation score, as follows:

```
[0.76190476 0.76190476 0.78571429 0.75609756 0.75609756]
```

Output 19.2: Result of k -fold cross-validation

These are the evaluation scores. Because it is a binary classification model, they are the average accuracy. There are five of them because it is obtained from a k -fold cross-validation with $k = 5$, each for a different test set. Usually you evaluate a model with the mean and standard deviation of the cross-validation scores:

```
print("mean = %.3f; std = %.3f" % (results.mean(), results.std()))
```

Listing 19.5: Printing the result of k -fold cross-validation

which is

```
mean = 0.764; std = 0.011
```

Output 19.3: Result of k -fold cross-validation

A good model should produce a high score (in this case, accuracy close to 1) and low standard deviation. A high standard deviation means the model is not very consistent with different test sets.

Putting everything together, the following is the complete code:

```
import pandas as pd
import torch
import torch.nn as nn
from sklearn.preprocessing import LabelEncoder
from skorch import NeuralNetBinaryClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score

# Read data
data = pd.read_csv("sonar.csv", header=None)
X = data.iloc[:, 0:60]
y = data.iloc[:, 60]

# Binary encoding of labels
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)
```

```

# Convert to 2D PyTorch tensors
X = torch.tensor(X.values, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32)

# Define the model
class SonarClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(60, 60)
        self.act1 = nn.ReLU()
        self.layer2 = nn.Linear(60, 60)
        self.act2 = nn.ReLU()
        self.layer3 = nn.Linear(60, 60)
        self.act3 = nn.ReLU()
        self.output = nn.Linear(60, 1)

    def forward(self, x):
        x = self.act1(self.layer1(x))
        x = self.act2(self.layer2(x))
        x = self.act3(self.layer3(x))
        x = self.output(x)
        return x

# create the skorch wrapper
model = NeuralNetBinaryClassifier(
    SonarClassifier,
    criterion=torch.nn.BCEWithLogitsLoss,
    optimizer=torch.optim.Adam,
    lr=0.0001,
    max_epochs=150,
    batch_size=10,
    verbose=False
)

# k-fold
kfold = StratifiedKFold(n_splits=5, shuffle=True)
results = cross_val_score(model, X, y, cv=kfold)
print("mean = %.3f; std = %.3f" % (results.mean(), results.std()))

```

Listing 19.6: Running k-fold cross-validation on a PyTorch model using skorch

In comparison, the following is an equivalent implementation with a neural network model in scikit-learn:

```

import pandas as pd
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.preprocessing import LabelEncoder

# load dataset
data = pd.read_csv("sonar.csv", header=None)
# split into input (X) and output (Y) variables, in numpy arrays
X = data.iloc[:, 0:60].values
y = data.iloc[:, 60].values

```

```
# binary encoding of labels
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# create model
model = MLPClassifier(hidden_layer_sizes=(60,60,60), activation='relu',
                      max_iter=150, batch_size=10, verbose=False)

# evaluate using 10-fold cross validation
kfold = StratifiedKFold(n_splits=5, shuffle=True)
results = cross_val_score(model, X, y, cv=kfold)
print("mean = %.3f; std = %.3f" % (results.mean(), results.std()))
```

Listing 19.7: Running k-fold cross-validation on a scikit-learn model

Which you should see how `skorch` is to make a drop-in replacement of scikit-learn model with a model from PyTorch.

19.4 Grid Search Deep Learning Model Parameters

The previous example showed how easy it is to wrap your deep learning model from PyTorch and use it in functions from the scikit-learn library. In this example, you will go a step further. The function that you specify to the model argument when creating the `NeuralNetBinaryClassifier` or `NeuralNetClassifier` wrapper can take many arguments. You can use these arguments to further customize the construction of the model. In addition, you know you can provide arguments to the `fit()` function.

In this example, you will use grid search to evaluate different configurations for your neural network model and report on the combination that provides the best estimated performance. To make it interesting, let's modify the PyTorch model such that it takes a parameter to decide how deep you want it to be:

```
import torch.nn as nn

class SonarClassifier(nn.Module):
    def __init__(self, n_layers=3):
        super().__init__()
        self.layers = []
        self.acts = []
        for i in range(n_layers):
            self.layers.append(nn.Linear(60, 60))
            self.acts.append(nn.ReLU())
            self.add_module(f"layer{i}", self.layers[-1])
            self.add_module(f"act{i}", self.acts[-1])
        self.output = nn.Linear(60, 1)

    def forward(self, x):
        for layer, act in zip(self.layers, self.acts):
            x = act(layer(x))
```



```
x = self.output(x)
return x
```

Listing 19.8: A PyTorch model

In this design, we hold the hidden layers and their activation functions in Python lists. Because the PyTorch components are not immediate attributes of the class, you will not see them in `model.parameters()`. That will be a problem on training. This can be mitigated by using `self.add_module()` to register the components. An alternative is to use `nn.ModuleList()` instead of a Python list, so that you provided enough clues to tell where to find the components of the model.

The skorch wrapper is still the same. With it, you can have a model compatible to scikit-learn. As you can see, there are parameters to setup the deep learning model as well as training parameters such as learning rate (`lr`) specified in the wrapper, you have many possible variations. The `GridSearchCV` function from scikit-learn is to provide grid search cross-validation. You can provide a list of values for each parameter and ask scikit-learn to try out *all combinations* and report the best set of parameters according to the metric you specified. An example is as follows:

```
from sklearn.model_selection import GridSearchCV

model = NeuralNetBinaryClassifier(
    SonarClassifier,
    criterion=torch.nn.BCEWithLogitsLoss,
    optimizer=torch.optim.Adam,
    lr=0.0001,
    max_epochs=150,
    batch_size=10,
    verbose=False
)

param_grid = {
    'module__n_layers': [1, 3, 5],
    'lr': [0.1, 0.01, 0.001, 0.0001],
    'max_epochs': [100, 150],
}

grid_search = GridSearchCV(model, param_grid, scoring='accuracy', verbose=1, cv=3)
result = grid_search.fit(X, y)
```

Listing 19.9: Running grid search using scikit-learn on a PyTorch model

You passed in `model` to `GridSearchCV()`, which is a skorch wrapper. You also passed in `param_grid`, which specified to vary:

- ▷ the parameter `n_layers` in the PyTorch model (i.e., the `SonarClassifier` class), that controls the depth of the neural network
- ▷ the parameter `lr` in the wrapper, that controls the learning rate at the optimizer
- ▷ the parameter `max_epochs` in the wrapper, that controls the number of training epochs to run

Note the use of double underscore to pass on parameters to the PyTorch model. In fact, this allows you to configure other parameters too. For example, you can setup `optimizer__weight_decay` to pass on `weight_decay` parameters to the Adam optimizer (which is for setting up L2 regularization).

Running this can take a while to compute because it tries all combinations, each evaluated with 3-fold cross-validation. You do not want to run this often but it can be useful for you to design models.

After the grid search is finished, the performance and combination of configurations for the best model are displayed, followed by the performance of all combinations of parameters, as below:

```
print("Best: %f using %s" % (result.best_score_, result.best_params_))
means = result.cv_results_['mean_test_score']
stds = result.cv_results_['std_test_score']
params = result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))
```

Listing 19.10: Showing the result of grid search

This might take about 5 minutes to complete on your workstation executed on the CPU (GPU might be faster). Running the example shows the results below.

```
Best: 0.649551 using {'lr': 0.001, 'max_epochs': 150, 'module__n_layers': 1}
0.533678 (0.003611) with: {'lr': 0.1, 'max_epochs': 100, 'module__n_layers': 1}
0.533678 (0.003611) with: {'lr': 0.1, 'max_epochs': 100, 'module__n_layers': 3}
0.533678 (0.003611) with: {'lr': 0.1, 'max_epochs': 100, 'module__n_layers': 5}
0.533678 (0.003611) with: {'lr': 0.1, 'max_epochs': 150, 'module__n_layers': 1}
0.533678 (0.003611) with: {'lr': 0.1, 'max_epochs': 150, 'module__n_layers': 3}
0.533678 (0.003611) with: {'lr': 0.1, 'max_epochs': 150, 'module__n_layers': 5}
0.644651 (0.062160) with: {'lr': 0.01, 'max_epochs': 100, 'module__n_layers': 1}
0.567495 (0.049728) with: {'lr': 0.01, 'max_epochs': 100, 'module__n_layers': 3}
0.533678 (0.003611) with: {'lr': 0.01, 'max_epochs': 100, 'module__n_layers': 5}
0.615804 (0.061966) with: {'lr': 0.01, 'max_epochs': 150, 'module__n_layers': 1}
0.620290 (0.078243) with: {'lr': 0.01, 'max_epochs': 150, 'module__n_layers': 3}
0.533678 (0.003611) with: {'lr': 0.01, 'max_epochs': 150, 'module__n_layers': 5}
0.635335 (0.108412) with: {'lr': 0.001, 'max_epochs': 100, 'module__n_layers': 1}
0.582126 (0.058072) with: {'lr': 0.001, 'max_epochs': 100, 'module__n_layers': 3}
0.563423 (0.136916) with: {'lr': 0.001, 'max_epochs': 100, 'module__n_layers': 5}
0.649551 (0.075676) with: {'lr': 0.001, 'max_epochs': 150, 'module__n_layers': 1}
0.558178 (0.071443) with: {'lr': 0.001, 'max_epochs': 150, 'module__n_layers': 3}
0.567909 (0.088623) with: {'lr': 0.001, 'max_epochs': 150, 'module__n_layers': 5}
0.557971 (0.041416) with: {'lr': 0.0001, 'max_epochs': 100, 'module__n_layers': 1}
0.587026 (0.079951) with: {'lr': 0.0001, 'max_epochs': 100, 'module__n_layers': 3}
0.606349 (0.092394) with: {'lr': 0.0001, 'max_epochs': 100, 'module__n_layers': 5}
0.563147 (0.099652) with: {'lr': 0.0001, 'max_epochs': 150, 'module__n_layers': 1}
0.534023 (0.057187) with: {'lr': 0.0001, 'max_epochs': 150, 'module__n_layers': 3}
0.634921 (0.057235) with: {'lr': 0.0001, 'max_epochs': 150, 'module__n_layers': 5}
```

Output 19.4: Result of grid search

You can see that the grid search discovered that using a learning rate of 0.001 with 150 epochs and only a single hidden layer achieved the best cross-validation score of approximately 65% on this problem.

In fact, you can see if you can improve the result by first standardizing input features. Since the wrapper allows you to use PyTorch model with scikit-learn, you can also use the scikit-learn's standardizer in realtime, and create a machine learning pipeline:

```
from sklearn.pipeline import Pipeline, FunctionTransformer
from sklearn.preprocessing import StandardScaler

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('float32', FunctionTransformer(func=lambda X: torch.tensor(X, dtype=torch.float32),
                                validate=False)),
    ('sonarmodel', model.initialize()),
])
```

Listing 19.11: Setting up a scikit-learn pipeline with skorch model

The new object `pipe` you created is another scikit-learn model that works just like the `model` object, except a standard scaler is applied before the data is passed on to the neural network. Therefore you can run a grid search on this pipeline, with a little tweak on the way parameters are specified:

```
param_grid = {
    'sonarmodel__module__n_layers': [1, 3, 5],
    'sonarmodel__lr': [0.1, 0.01, 0.001, 0.0001],
    'sonarmodel__max_epochs': [100, 150],
}

grid_search = GridSearchCV(pipe, param_grid, scoring='accuracy', verbose=1, cv=3)
result = grid_search.fit(X, y)
print("Best: %f using %s" % (result.best_score_, result.best_params_))
means = result.cv_results_['mean_test_score']
stds = result.cv_results_['std_test_score']
params = result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))
```

Listing 19.12: Running grid search on a pipeline

Two key points to note here: Since PyTorch models are running on 32-bit floats by default but NumPy arrays are usually 64-bit floats. These data types are not aligned, but scikit-learn's scaler always return you a NumPy array. Therefore you need to do type conversion in the middle of the pipeline, using a `FunctionTransformer` object.

Moreover, in a scikit-learn pipeline, each step is referred by a name, such as `scaler` and `sonarmodel`. Therefore, the parameters set for the pipeline need to carry the name as well. In the example above, we use `sonarmodel__module__n_layers` as a parameter for grid search. This refers to the `sonarmodel` part of the pipeline (which is your skorch wrapper), the `module` part therein (which is your PyTorch model), and its `n_layers` parameter. Note the use of double underscore for hierarchy separation.

Putting everything together, the following is the complete code:

```
import pandas as pd
import torch
import torch.nn as nn
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline, FunctionTransformer
from sklearn.preprocessing import StandardScaler, LabelEncoder
from skorch import NeuralNetBinaryClassifier

# Read data
data = pd.read_csv("sonar.csv", header=None)
X = data.iloc[:, 0:60]
y = data.iloc[:, 60]

# Binary encoding of labels
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)

# Convert to 2D PyTorch tensors
X = torch.tensor(X.values, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32)

class SonarClassifier(nn.Module):
    def __init__(self, n_layers=3):
        super().__init__()
        self.layers = []
        self.acts = []
        for i in range(n_layers):
            self.layers.append(nn.Linear(60, 60))
            self.acts.append(nn.ReLU())
            self.add_module(f"layer{i}", self.layers[-1])
            self.add_module(f"act{i}", self.acts[-1])
        self.output = nn.Linear(60, 1)

    def forward(self, x):
        for layer, act in zip(self.layers, self.acts):
            x = act(layer(x))
        x = self.output(x)
        return x

model = NeuralNetBinaryClassifier(
    SonarClassifier,
    criterion=torch.nn.BCEWithLogitsLoss,
    optimizer=torch.optim.Adam,
    lr=0.0001,
    max_epochs=150,
    batch_size=10,
    verbose=False
)
```

```

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('float32', FunctionTransformer(func=lambda X: torch.tensor(X, dtype=torch.float32),
                                validate=False)),
    ('sonarmodel', model.initialize()),
])

param_grid = {
    'sonarmodel__module__n_layers': [1, 3, 5],
    'sonarmodel__lr': [0.1, 0.01, 0.001, 0.0001],
    'sonarmodel__max_epochs': [100, 150],
}

grid_search = GridSearchCV(pipe, param_grid, scoring='accuracy', verbose=1, cv=3)
result = grid_search.fit(X, y)
print("Best: %f using %s" % (result.best_score_, result.best_params_))
means = result.cv_results_['mean_test_score']
stds = result.cv_results_['std_test_score']
params = result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))

```

Listing 19.13: Running grid search using scikit-learn on a pipeline of a PyTorch model

19.5 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Articles

skorch documentation.

<https://skorch.readthedocs.io/en/latest/>

Grid search cross-validator. scikit-learn documentation.

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

Stratified K-Folds cross-validator. scikit-learn documentation.

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html

Pipeline. scikit-learn documentation.

<https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html>

19.6 Summary

In this chapter, you discovered how to wrap your PyTorch deep learning models and use them in the scikit-learn general machine learning library. You learned:

- ▷ Specifically how to wrap PyTorch models so that they can be used with the scikit-learn machine learning library.

- ▷ How to use a wrapped PyTorch model as part of evaluating model performance in scikit-learn.
- ▷ How to perform hyperparameter tuning in scikit-learn using a wrapped PyTorch model.

You can see that using scikit-learn for standard machine learning operations such as model evaluation and model hyperparameter optimization can save a lot of time over implementing these schemes yourself. Wrapping your model allowed you to leverage powerful tools from scikit-learn to fit your deep learning models into your general machine learning process.

In the next chapter, you will see more examples on the hyperparameter optimization.

Optimize Hyperparameters with Grid Search

20

The “weights” of a neural network is referred as “parameters” in PyTorch code and it is fine-tuned by optimizer during training. On the contrary, hyperparameters are the parameters of a neural network that is fixed by design and not tuned by training. Examples are the number of hidden layers and the choice of activation functions. Hyperparameter optimization is a big part of deep learning. The reason is that neural networks are notoriously difficult to configure, and a lot of parameters need to be set. On top of that, individual models can be very slow to train.

In this chapter, you will discover how to use the grid search capability from the scikit-learn Python machine learning library to tune the hyperparameters of PyTorch deep learning models. After reading this chapter, you will know:

- ▷ How to wrap PyTorch models for use in scikit-learn and how to use grid search
- ▷ How to grid search common neural network parameters, such as learning rate, dropout rate, epochs, and number of neurons
- ▷ How to define your own hyperparameter tuning experiments on your own projects

Let’s get started.

Overview

In this chapter, you will see how to use the scikit-learn grid search capability with a suite of examples that you can copy-and-paste into your own project as a starting point. Below is a list of the topics we are going to cover:

- ▷ How to use PyTorch models in scikit-learn
- ▷ How to use grid search in scikit-learn
- ▷ How to tune batch size and training epochs
- ▷ How to tune optimization algorithms
- ▷ How to tune learning rate and momentum
- ▷ How to tune network weight initialization
- ▷ How to tune activation functions

- ▷ How to tune dropout regularization
- ▷ How to tune the number of neurons in the hidden layer

20.1 How to Use PyTorch Models in scikit-learn

PyTorch models can be used in scikit-learn if wrapped with `skorch`. This is to leverage the duck-typing nature of Python to make the PyTorch model provide similar API as a scikit-learn model, so everything in scikit-learn can work along. In `skorch`, there are `NeuralNetClassifier` for classification neural networks and `NeuralNetRegressor` for regression neural networks. You may need to run the follownig command to install the module.

```
pip install skorch
```

Listing 20.1: Installing skorch

To use these wrappers, you must define a your PyTorch model as a class using `nn.Module`, then pass the name of the class to the `module` argument when constructing the `NeuralNetClassifier` class. For example:

```
class MyClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        ...

    def forward(self, x):
        ...
        return x

# create the skorch wrapper
model = NeuralNetClassifier(
    module=MyClassifier
)
```

Listing 20.2: Syntax of a PyTorch model created as subclass of `nn.Module`

The constructor for the `NeuralNetClassifier` class can take default arguments that are passed on to the calls to `model.fit()` (the way to invoke a training loop in scikit-learn models), such as the number of epochs and the batch size. For example:

```
model = NeuralNetClassifier(
    module=MyClassifier,
    max_epochs=150,
    batch_size=10
)
```

Listing 20.3: Wrapping a PyTorch model in skorch

The constructor for the `NeuralNetClassifier` class can also take new arguments that can be passed to your model class' constructor, but you have to prepend it with `module__` (with two underscores). These new arguments may carry a default value in the constructor but they will be overridden when the wrapper instantiate the model. For example:


```

import torch.nn as nn
from skorch import NeuralNetClassifier

class SonarClassifier(nn.Module):
    def __init__(self, n_layers=3):
        super().__init__()
        self.layers = []
        self.acts = []
        for i in range(n_layers):
            self.layers.append(nn.Linear(60, 60))
            self.acts.append(nn.ReLU())
            self.add_module(f"layer{i}", self.layers[-1])
            self.add_module(f"act{i}", self.acts[-1])
        self.output = nn.Linear(60, 1)

    def forward(self, x):
        for layer, act in zip(self.layers, self.acts):
            x = act(layer(x))
        x = self.output(x)
        return x

model = NeuralNetClassifier(
    module=SonarClassifier,
    max_epochs=150,
    batch_size=10,
    module__n_layers=2
)

```

Listing 20.4: Creating a PyTorch model and wrap it with skorch

You can verify the result by initializing a model and print it:

```
print(model.initialize())
```

Listing 20.5: Initialize a skorch model

In this example, you should see:

```

<class 'skorch.classifier.NeuralNetClassifier'[initialized](
  module_=SonarClassifier(
    (layer0): Linear(in_features=60, out_features=60, bias=True)
    (act0): ReLU()
    (layer1): Linear(in_features=60, out_features=60, bias=True)
    (act1): ReLU()
    (output): Linear(in_features=60, out_features=1, bias=True)
  ),
)

```

Output 20.1: An initialized skorch model

20.2 How to Use Grid Search in scikit-learn

Grid search is a model hyperparameter optimization technique. It simply exhaust all combinations of the hyperparameters and find the one that gave the best score. In scikit-learn, this technique is provided in the `GridSearchCV` class. When constructing this class, you must provide a dictionary of hyperparameters to evaluate in the `param_grid` argument. This is a map of the model parameter name and an array of values to try.

By default, accuracy is the score that is optimized, but other scores can be specified in the `score` argument of the `GridSearchCV` constructor. The `GridSearchCV` process will then construct and evaluate one model for each combination of parameters. Cross-validation is used to evaluate each individual model, and the default of 3-fold cross-validation is used, although you can override this by specifying the `cv` argument to the `GridSearchCV` constructor.

Below is an example of defining a simple grid search:

```
param_grid = {
    'epochs': [10,20,30]
}
grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
grid_result = grid.fit(X, Y)
```

Listing 20.6: Grid search in scikit-learn

By setting the `n_jobs` argument in the `GridSearchCV` constructor to `-1`, the process will use all cores on your machine. Otherwise the grid search process will only run in single thread, which is slower in the multi-core CPUs.

Once completed, you can access the outcome of the grid search in the result object returned from `grid.fit()`. The `best_score_` member provides access to the best score observed during the optimization procedure, and the `best_params_` describes the combination of parameters that achieved the best results. You can learn more about the `GridSearchCV` class in the scikit-learn API documentation.

20.3 Problem Description

Now that you know how to use PyTorch models with scikit-learn and how to use grid search in scikit-learn, let's look at a bunch of examples.

All examples will be demonstrated on a small standard machine learning dataset called the Pima Indians onset of diabetes classification dataset¹. This is a small dataset with all numerical attributes that is easy to work with.

As you proceed through the examples in this chapter, you will aggregate the best parameters. This is not the best way to grid search because parameters may be correlated. You should evaluate all parameters at the same time, but it is good for demonstration purposes.

¹<https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.csv>

20.4 How to Tune Batch Size and Number of Epochs

In this first simple example, you will look at tuning the batch size and number of epochs used when fitting the network.

The batch size in iterative gradient descent is the number of patterns shown to the network before the weights are updated. It is also an optimization in the training of the network, defining how many patterns to read at a time and keep in memory.

The number of epochs is the number of times the entire training dataset is shown to the network during training. Some networks are sensitive to the batch size, such as LSTM recurrent neural networks and Convolutional Neural Networks.

Here you will evaluate a suite of different minibatch sizes from 10 to 100 in steps of 20. The full code listing is provided below:

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from skorch import NeuralNetClassifier
from sklearn.model_selection import GridSearchCV

# load the dataset, split into input (X) and output (y) variables
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
X = dataset[:,0:8]
y = dataset[:,8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# PyTorch classifier
class PimaClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer = nn.Linear(8, 12)
        self.act = nn.ReLU()
        self.output = nn.Linear(12, 1)
        self.prob = nn.Sigmoid()

    def forward(self, x):
        x = self.act(self.layer(x))
        x = self.prob(self.output(x))
        return x

# create model with skorch
model = NeuralNetClassifier(
    PimaClassifier,
    criterion=nn.BCELoss,
    optimizer=optim.Adam,
    verbose=False
)

# define the grid search parameters
```

```

param_grid = {
    'batch_size': [10, 20, 40, 60, 80, 100],
    'max_epochs': [10, 50, 100]
}
grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
grid_result = grid.fit(X, y)

# summarize results
print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))

```

Listing 20.7: Tuning minibatch size using grid search

Running this example produces the following output:

```

Best: 0.714844 using {'batch_size': 10, 'max_epochs': 100}
0.665365 (0.020505) with: {'batch_size': 10, 'max_epochs': 10}
0.588542 (0.168055) with: {'batch_size': 10, 'max_epochs': 50}
0.714844 (0.032369) with: {'batch_size': 10, 'max_epochs': 100}
0.671875 (0.022326) with: {'batch_size': 20, 'max_epochs': 10}
0.696615 (0.008027) with: {'batch_size': 20, 'max_epochs': 50}
0.714844 (0.019918) with: {'batch_size': 20, 'max_epochs': 100}
0.666667 (0.009744) with: {'batch_size': 40, 'max_epochs': 10}
0.687500 (0.033603) with: {'batch_size': 40, 'max_epochs': 50}
0.707031 (0.024910) with: {'batch_size': 40, 'max_epochs': 100}
0.667969 (0.014616) with: {'batch_size': 60, 'max_epochs': 10}
0.694010 (0.036966) with: {'batch_size': 60, 'max_epochs': 50}
0.694010 (0.042473) with: {'batch_size': 60, 'max_epochs': 100}
0.670573 (0.023939) with: {'batch_size': 80, 'max_epochs': 10}
0.674479 (0.020752) with: {'batch_size': 80, 'max_epochs': 50}
0.703125 (0.026107) with: {'batch_size': 80, 'max_epochs': 100}
0.680990 (0.014382) with: {'batch_size': 100, 'max_epochs': 10}
0.670573 (0.013279) with: {'batch_size': 100, 'max_epochs': 50}
0.687500 (0.017758) with: {'batch_size': 100, 'max_epochs': 100}

```

Output 20.2: Result of grid search on minibatch size

You can see that the batch size of 10 and 100 epochs achieved the best result of about 71% accuracy (but you should also take into account the accuracy's standard deviation).

20.5 How to Tune the Training Optimization Algorithm

All deep learning library should offer a variety of optimization algorithms. PyTorch is no exception.

In this example, you will tune the optimization algorithm used to train the network, each with default parameters.

This is an odd example because often, you will choose one approach a priori and instead focus on tuning its parameters on your problem (see the next example). Here, you will evaluate the suite of optimization algorithms available in PyTorch.

The full code listing is provided below:

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from skorch import NeuralNetClassifier
from sklearn.model_selection import GridSearchCV

# load the dataset, split into input (X) and output (y) variables
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
X = dataset[:,0:8]
y = dataset[:,8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# PyTorch classifier
class PimaClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer = nn.Linear(8, 12)
        self.act = nn.ReLU()
        self.output = nn.Linear(12, 1)
        self.prob = nn.Sigmoid()

    def forward(self, x):
        x = self.act(self.layer(x))
        x = self.prob(self.output(x))
        return x

# create model with skorch
model = NeuralNetClassifier(
    PimaClassifier,
    criterion=nn.BCELoss,
    max_epochs=100,
    batch_size=10,
    verbose=False
)

# define the grid search parameters
param_grid = {
    'optimizer': [optim.SGD, optim.RMSprop, optim.Adagrad, optim.Adadelta,
                  optim.Adam, optim.Adamax, optim.NAdam],
}
grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
grid_result = grid.fit(X, y)

# summarize results
print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
```

```
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))
```

Listing 20.8: Grid search on the choices of optimizer

Running this example produces the following output:

```
Best: 0.721354 using {'optimizer': <class 'torch.optim.adamax.Adamax'>}
0.674479 (0.036828) with: {'optimizer': <class 'torch.optim.sgd.SGD'>}
0.700521 (0.043303) with: {'optimizer': <class 'torch.optim.rmsprop.RMSprop'>}
0.682292 (0.027126) with: {'optimizer': <class 'torch.optim.adagrad.Adagrad'>}
0.572917 (0.051560) with: {'optimizer': <class 'torch.optim.adadelta.Adadelta'>}
0.714844 (0.030758) with: {'optimizer': <class 'torch.optim.adam.Adam'>}
0.721354 (0.019225) with: {'optimizer': <class 'torch.optim.adamax.Adamax'>}
0.709635 (0.024360) with: {'optimizer': <class 'torch.optim.nadam.NAdam'>}
```

Output 20.3: Result of grid search on the choices of optimizer

The results suggest that the Adamax optimization algorithm is the best with a score of about 72% accuracy.

It is worth to mention that `GridSearchCV` will recreate your model often so every trial is independent. The reason it can be done is because of the `NeuralNetClassifier` wrapper, which knows the name of the class for your PyTorch model and instantiate one for you upon request.

20.6 How to Tune Learning Rate and Momentum

It is common to pre-select an optimization algorithm to train your network and tune its parameters.

By far, the most common optimization algorithm is plain old Stochastic Gradient Descent (SGD) because it is so well understood. In this example, you will look at optimizing the SGD learning rate and momentum parameters.

The learning rate controls how much to update the weight at the end of each batch, and the momentum controls how much to let the previous update influence the current weight update.

You will try a suite of small standard learning rates and a momentum values from 0.2 to 0.8 in steps of 0.2, as well as 0.9 (because it can be a popular value in practice). In PyTorch, the way to set the learning rate and momentum is the following:

```
optimizer = optim.SGD(lr=0.001, momentum=0.9)
```

Listing 20.9: Setting up an optimizer

In the `skorch` wrapper, you will can route the parameters to the optimizer with the prefix `optimizer__`.

Generally, it is a good idea to also include the number of epochs in an optimization like this as there is a dependency between the amount of learning per batch (learning rate), the number of updates per epoch (batch size), and the number of epochs.

The full code listing is provided below:

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from skorch import NeuralNetClassifier
from sklearn.model_selection import GridSearchCV

# load the dataset, split into input (X) and output (y) variables
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
X = dataset[:,0:8]
y = dataset[:,8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# PyTorch classifier
class PimaClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer = nn.Linear(8, 12)
        self.act = nn.ReLU()
        self.output = nn.Linear(12, 1)
        self.prob = nn.Sigmoid()

    def forward(self, x):
        x = self.act(self.layer(x))
        x = self.prob(self.output(x))
        return x

# create model with skorch
model = NeuralNetClassifier(
    PimaClassifier,
    criterion=nn.BCELoss,
    optimizer=optim.SGD,
    max_epochs=100,
    batch_size=10,
    verbose=False
)

# define the grid search parameters
param_grid = {
    'optimizer_lr': [0.001, 0.01, 0.1, 0.2, 0.3],
    'optimizer_momentum': [0.0, 0.2, 0.4, 0.6, 0.8, 0.9],
}
grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
grid_result = grid.fit(X, y)

# summarize results
```

```
print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))
```

Listing 20.10: Grid search on learning rate and momentum

Running this example produces the following output.

```
Best: 0.682292 using {'optimizer_lr': 0.001, 'optimizer_momentum': 0.9}
0.648438 (0.016877) with: {'optimizer_lr': 0.001, 'optimizer_momentum': 0.0}
0.671875 (0.017758) with: {'optimizer_lr': 0.001, 'optimizer_momentum': 0.2}
0.674479 (0.022402) with: {'optimizer_lr': 0.001, 'optimizer_momentum': 0.4}
0.677083 (0.011201) with: {'optimizer_lr': 0.001, 'optimizer_momentum': 0.6}
0.679688 (0.027621) with: {'optimizer_lr': 0.001, 'optimizer_momentum': 0.8}
0.682292 (0.026557) with: {'optimizer_lr': 0.001, 'optimizer_momentum': 0.9}
0.671875 (0.019918) with: {'optimizer_lr': 0.01, 'optimizer_momentum': 0.0}
0.648438 (0.024910) with: {'optimizer_lr': 0.01, 'optimizer_momentum': 0.2}
0.546875 (0.143454) with: {'optimizer_lr': 0.01, 'optimizer_momentum': 0.4}
0.567708 (0.153668) with: {'optimizer_lr': 0.01, 'optimizer_momentum': 0.6}
0.552083 (0.141790) with: {'optimizer_lr': 0.01, 'optimizer_momentum': 0.8}
0.451823 (0.144561) with: {'optimizer_lr': 0.01, 'optimizer_momentum': 0.9}
0.348958 (0.001841) with: {'optimizer_lr': 0.1, 'optimizer_momentum': 0.0}
0.450521 (0.142719) with: {'optimizer_lr': 0.1, 'optimizer_momentum': 0.2}
0.450521 (0.142719) with: {'optimizer_lr': 0.1, 'optimizer_momentum': 0.4}
0.450521 (0.142719) with: {'optimizer_lr': 0.1, 'optimizer_momentum': 0.6}
0.348958 (0.001841) with: {'optimizer_lr': 0.1, 'optimizer_momentum': 0.8}
0.348958 (0.001841) with: {'optimizer_lr': 0.1, 'optimizer_momentum': 0.9}
0.444010 (0.136265) with: {'optimizer_lr': 0.2, 'optimizer_momentum': 0.0}
0.450521 (0.142719) with: {'optimizer_lr': 0.2, 'optimizer_momentum': 0.2}
0.348958 (0.001841) with: {'optimizer_lr': 0.2, 'optimizer_momentum': 0.4}
0.552083 (0.141790) with: {'optimizer_lr': 0.2, 'optimizer_momentum': 0.6}
0.549479 (0.142719) with: {'optimizer_lr': 0.2, 'optimizer_momentum': 0.8}
0.651042 (0.001841) with: {'optimizer_lr': 0.2, 'optimizer_momentum': 0.9}
0.552083 (0.141790) with: {'optimizer_lr': 0.3, 'optimizer_momentum': 0.0}
0.348958 (0.001841) with: {'optimizer_lr': 0.3, 'optimizer_momentum': 0.2}
0.450521 (0.142719) with: {'optimizer_lr': 0.3, 'optimizer_momentum': 0.4}
0.552083 (0.141790) with: {'optimizer_lr': 0.3, 'optimizer_momentum': 0.6}
0.450521 (0.142719) with: {'optimizer_lr': 0.3, 'optimizer_momentum': 0.8}
0.450521 (0.142719) with: {'optimizer_lr': 0.3, 'optimizer_momentum': 0.9}
```

Output 20.4: Result of the grid search on learning rate and momentum

You can see that, with SGD, the best results were achieved using a learning rate of 0.001 and a momentum of 0.9 with an accuracy of about 68%.

20.7 How to Tune Network Weight Initialization

Neural network weight initialization used to be simple: use small random values.

Now there is a suite of different techniques to choose from. You can get a laundry list from `torch.nn.init` documentation.

In this example, you will look at tuning the selection of network weight initialization by evaluating all the available techniques.

You will use the same weight initialization method on each layer. Ideally, it may be better to use different weight initialization schemes according to the activation function used on each layer. In the example below, you will use a rectifier for the hidden layer. Use sigmoid for the output layer because the predictions are binary. The weight initialization is implicit in PyTorch models. Therefore you need to write your own logic to initialize the weight, after the layer is created but before it is used. Let's modify the PyTorch as follows:

```
import torch.nn as nn

# PyTorch classifier
class PimaClassifier(nn.Module):
    def __init__(self, weight_init=nn.init.xavier_uniform_):
        super().__init__()
        self.layer = nn.Linear(8, 12)
        self.act = nn.ReLU()
        self.output = nn.Linear(12, 1)
        self.prob = nn.Sigmoid()
        # manually init weights
        weight_init(self.layer.weight)
        weight_init(self.output.weight)

    def forward(self, x):
        x = self.act(self.layer(x))
        x = self.prob(self.output(x))
        return x
```

Listing 20.11: Initializing weight in a PyTorch model

An argument `weight_init` is added to the class `PimaClassifier` and it expects one of the initializers from `torch.nn.init`. In `GridSearchCV`, you need to use the `module__` prefix to make `NeuralNetClassifier` route the parameter to the model's class constructor.

The full code listing is provided below:

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.init as init
import torch.optim as optim
from skorch import NeuralNetClassifier
from sklearn.model_selection import GridSearchCV

# load the dataset, split into input (X) and output (y) variables
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
X = dataset[:,0:8]
y = dataset[:,8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# PyTorch classifier
```

```

class PimaClassifier(nn.Module):
    def __init__(self, weight_init=init.xavier_uniform_):
        super().__init__()
        self.layer = nn.Linear(8, 12)
        self.act = nn.ReLU()
        self.output = nn.Linear(12, 1)
        self.probab = nn.Sigmoid()
        # manually init weights
        weight_init(self.layer.weight)
        weight_init(self.output.weight)

    def forward(self, x):
        x = self.act(self.layer(x))
        x = self.probab(self.output(x))
        return x

# create model with skorch
model = NeuralNetClassifier(
    PimaClassifier,
    criterion=nn.BCELoss,
    optimizer=optim.Adamax,
    max_epochs=100,
    batch_size=10,
    verbose=False
)

# define the grid search parameters
param_grid = {
    'module__weight_init': [init.uniform_, init.normal_, init.zeros_,
                           init.xavier_normal_, init.xavier_uniform_,
                           init.kaiming_normal_, init.kaiming_uniform_]
}
grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
grid_result = grid.fit(X, y)

# summarize results
print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))

```

Listing 20.12: Grid search on the weight initialization algorithm

Running this example produces the following output.

```

Best: 0.697917 using {'module__weight_init': <function kaiming_uniform_ at 0x112020c10>}
0.348958 (0.001841) with: {'module__weight_init': <function uniform_ at 0x1120204c0>}
0.602865 (0.061708) with: {'module__weight_init': <function normal_ at 0x112020550>}
0.652344 (0.003189) with: {'module__weight_init': <function zeros_ at 0x112020820>}
0.691406 (0.030758) with: {'module__weight_init': <function xavier_normal_ at 0x112020af0>}
0.592448 (0.171589) with: {'module__weight_init': <function xavier_uniform_ at 0x112020a60>}

```

```
0.563802 (0.152971) with: {'module__weight_init': <function kaiming_normal_ at 0x112020ca0>}
0.697917 (0.013279) with: {'module__weight_init': <function kaiming_uniform_ at 0x112020c10>}
```

Output 20.5: Result of the grid search on weight initialization algorithm

The best results were achieved with a Kaiming-uniform weight initialization scheme achieving a performance of about 70%.

20.8 How to Tune the Neuron Activation Function

The activation function controls the nonlinearity of individual neurons and when to fire.

Generally, the rectifier activation function is the most popular. However, it used to be the sigmoid and the tanh functions, and these functions may still be more suitable for different problems.

In this example, you will evaluate some activation functions available in PyTorch. You will only use these functions in the hidden layer, as a sigmoid activation function is required in the output for the binary classification problem. Similar to the previous example, this is an argument to the class constructor of the model, and you will use the `module__` prefix for the `GridSearchCV` parameter grid.

Generally, it is a good idea to prepare data to the range of the different activation functions (e.g., apply a scaler), which you will not do in this case.

The full code listing is provided below:

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.init as init
import torch.optim as optim
from skorch import NeuralNetClassifier
from sklearn.model_selection import GridSearchCV

# load the dataset, split into input (X) and output (y) variables
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
X = dataset[:,0:8]
y = dataset[:,8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# PyTorch classifier
class PimaClassifier(nn.Module):
    def __init__(self, activation=nn.ReLU):
        super().__init__()
        self.layer = nn.Linear(8, 12)
        self.act = activation()
        self.output = nn.Linear(12, 1)
        self.prob = nn.Sigmoid()
        # manually init weights
        init.kaiming_uniform_(self.layer.weight)
        init.kaiming_uniform_(self.output.weight)
```

```

def forward(self, x):
    x = self.act(self.layer(x))
    x = self.prob(self.output(x))
    return x

# create model with skorch
model = NeuralNetClassifier(
    PimaClassifier,
    criterion=nn.BCELoss,
    optimizer=optim.Adamax,
    max_epochs=100,
    batch_size=10,
    verbose=False
)

# define the grid search parameters
param_grid = {
    'module__activation': [nn.Identity, nn.ReLU, nn.ELU, nn.ReLU6,
                          nn.GELU, nn.Softplus, nn.Softsign, nn.Tanh,
                          nn.Sigmoid, nn.Hardsigmoid]
}
grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
grid_result = grid.fit(X, y)

# summarize results
print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))

```

Listing 20.13: Grid search on the activation function

Running this example produces the following output.

```

Best: 0.699219 using {'module__activation': <class 'torch.nn.modules.activation.ReLU'>}
0.687500 (0.025315) with: {'module__activation': <class 'torch.nn.modules.linear.Identity'>}
0.699219 (0.011049) with: {'module__activation': <class 'torch.nn.modules.activation.ReLU'>}
0.674479 (0.035849) with: {'module__activation': <class 'torch.nn.modules.activation.ELU'>}
0.621094 (0.063549) with: {'module__activation': <class 'torch.nn.modules.activation.ReLU6'>}
0.674479 (0.017566) with: {'module__activation': <class 'torch.nn.modules.activation.GELU'>}
0.558594 (0.149189) with: {'module__activation': <class 'torch.nn.modules.activation.Softplus'>}
0.675781 (0.014616) with: {'module__activation': <class 'torch.nn.modules.activation.Softsign'>}
0.619792 (0.018688) with: {'module__activation': <class 'torch.nn.modules.activation.Tanh'>}
0.643229 (0.019225) with: {'module__activation': <class 'torch.nn.modules.activation.Sigmoid'>}
0.636719 (0.022326) with: {'module__activation': <class 'torch.nn.modules.activation.Hardsigmoid'>}

```

Output 20.6: Result of the grid search on the activation function

It shows that ReLU activation function achieved the best results with an accuracy of about 70%.

20.9 How to Tune Dropout Regularization

In this example, you will look at tuning the dropout rate for regularization in an effort to limit overfitting and improve the model's ability to generalize.

For the best results, dropout is best combined with a weight constraint such as the max norm constraint, which is implemented in the forward pass function.

This involves fitting both the dropout percentage and the weight constraint. We will try dropout percentages between 0.0 and 0.9 (1.0 does not make sense) and MaxNorm weight constraint values between 0 and 5.

The full code listing is provided below.

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.init as init
import torch.optim as optim
from skorch import NeuralNetClassifier
from sklearn.model_selection import GridSearchCV

# load the dataset, split into input (X) and output (y) variables
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
X = dataset[:,0:8]
y = dataset[:,8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# PyTorch classifier
class PimaClassifier(nn.Module):
    def __init__(self, dropout_rate=0.5, weight_constraint=1.0):
        super().__init__()
        self.layer = nn.Linear(8, 12)
        self.act = nn.ReLU()
        self.dropout = nn.Dropout(dropout_rate)
        self.output = nn.Linear(12, 1)
        self.probab = nn.Sigmoid()
        self.weight_constraint = weight_constraint
        # manually init weights
        init.kaiming_uniform_(self.layer.weight)
        init.kaiming_uniform_(self.output.weight)

    def forward(self, x):
        # maxnorm weight before actual forward pass
        with torch.no_grad():
            norm = self.layer.weight.norm(2, dim=0, keepdim=True) \
                .clamp(min=self.weight_constraint / 2)
            desired = torch.clamp(norm, max=self.weight_constraint)
            self.layer.weight *= (desired / norm)
        # actual forward pass
        x = self.act(self.layer(x))
        x = self.dropout(x)
```

```

        x = self.prob(self.output(x))
        return x

# create model with skorch
model = NeuralNetClassifier(
    PimaClassifier,
    criterion=nn.BCELoss,
    optimizer=optim.Adamax,
    max_epochs=100,
    batch_size=10,
    verbose=False
)

# define the grid search parameters
param_grid = {
    'module__weight_constraint': [1.0, 2.0, 3.0, 4.0, 5.0],
    'module__dropout_rate': [0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9]
}
grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
grid_result = grid.fit(X, y)

# summarize results
print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))

```

Listing 20.14: Grid search on the dropout percentage and weight constraint

Running this example produces the following output.

```

Best: 0.701823 using {'module__dropout_rate': 0.1, 'module__weight_constraint': 2.0}
0.669271 (0.015073) with: {'module__dropout_rate': 0.0, 'module__weight_constraint': 1.0}
0.692708 (0.035132) with: {'module__dropout_rate': 0.0, 'module__weight_constraint': 2.0}
0.589844 (0.170180) with: {'module__dropout_rate': 0.0, 'module__weight_constraint': 3.0}
0.561198 (0.151131) with: {'module__dropout_rate': 0.0, 'module__weight_constraint': 4.0}
0.688802 (0.021710) with: {'module__dropout_rate': 0.0, 'module__weight_constraint': 5.0}
0.697917 (0.009744) with: {'module__dropout_rate': 0.1, 'module__weight_constraint': 1.0}
0.701823 (0.016367) with: {'module__dropout_rate': 0.1, 'module__weight_constraint': 2.0}
0.694010 (0.010253) with: {'module__dropout_rate': 0.1, 'module__weight_constraint': 3.0}
0.686198 (0.025976) with: {'module__dropout_rate': 0.1, 'module__weight_constraint': 4.0}
0.679688 (0.026107) with: {'module__dropout_rate': 0.1, 'module__weight_constraint': 5.0}
0.701823 (0.029635) with: {'module__dropout_rate': 0.2, 'module__weight_constraint': 1.0}
0.682292 (0.014731) with: {'module__dropout_rate': 0.2, 'module__weight_constraint': 2.0}
0.701823 (0.009744) with: {'module__dropout_rate': 0.2, 'module__weight_constraint': 3.0}
0.701823 (0.026557) with: {'module__dropout_rate': 0.2, 'module__weight_constraint': 4.0}
0.687500 (0.015947) with: {'module__dropout_rate': 0.2, 'module__weight_constraint': 5.0}
0.686198 (0.006639) with: {'module__dropout_rate': 0.3, 'module__weight_constraint': 1.0}
0.656250 (0.006379) with: {'module__dropout_rate': 0.3, 'module__weight_constraint': 2.0}
0.565104 (0.155608) with: {'module__dropout_rate': 0.3, 'module__weight_constraint': 3.0}
0.700521 (0.028940) with: {'module__dropout_rate': 0.3, 'module__weight_constraint': 4.0}
0.669271 (0.012890) with: {'module__dropout_rate': 0.3, 'module__weight_constraint': 5.0}
0.661458 (0.018688) with: {'module__dropout_rate': 0.4, 'module__weight_constraint': 1.0}

```

```

0.669271 (0.017566) with: {'module__dropout_rate': 0.4, 'module__weight_constraint': 2.0}
0.652344 (0.006379) with: {'module__dropout_rate': 0.4, 'module__weight_constraint': 3.0}
0.680990 (0.037783) with: {'module__dropout_rate': 0.4, 'module__weight_constraint': 4.0}
0.692708 (0.042112) with: {'module__dropout_rate': 0.4, 'module__weight_constraint': 5.0}
0.666667 (0.006639) with: {'module__dropout_rate': 0.5, 'module__weight_constraint': 1.0}
0.652344 (0.011500) with: {'module__dropout_rate': 0.5, 'module__weight_constraint': 2.0}
0.662760 (0.007366) with: {'module__dropout_rate': 0.5, 'module__weight_constraint': 3.0}
0.558594 (0.146610) with: {'module__dropout_rate': 0.5, 'module__weight_constraint': 4.0}
0.552083 (0.141826) with: {'module__dropout_rate': 0.5, 'module__weight_constraint': 5.0}
0.548177 (0.141826) with: {'module__dropout_rate': 0.6, 'module__weight_constraint': 1.0}
0.653646 (0.013279) with: {'module__dropout_rate': 0.6, 'module__weight_constraint': 2.0}
0.661458 (0.008027) with: {'module__dropout_rate': 0.6, 'module__weight_constraint': 3.0}
0.553385 (0.142719) with: {'module__dropout_rate': 0.6, 'module__weight_constraint': 4.0}
0.669271 (0.035132) with: {'module__dropout_rate': 0.6, 'module__weight_constraint': 5.0}
0.662760 (0.015733) with: {'module__dropout_rate': 0.7, 'module__weight_constraint': 1.0}
0.636719 (0.024910) with: {'module__dropout_rate': 0.7, 'module__weight_constraint': 2.0}
0.550781 (0.146818) with: {'module__dropout_rate': 0.7, 'module__weight_constraint': 3.0}
0.537760 (0.140094) with: {'module__dropout_rate': 0.7, 'module__weight_constraint': 4.0}
0.542969 (0.138144) with: {'module__dropout_rate': 0.7, 'module__weight_constraint': 5.0}
0.565104 (0.148654) with: {'module__dropout_rate': 0.8, 'module__weight_constraint': 1.0}
0.657552 (0.008027) with: {'module__dropout_rate': 0.8, 'module__weight_constraint': 2.0}
0.428385 (0.111418) with: {'module__dropout_rate': 0.8, 'module__weight_constraint': 3.0}
0.549479 (0.142719) with: {'module__dropout_rate': 0.8, 'module__weight_constraint': 4.0}
0.648438 (0.005524) with: {'module__dropout_rate': 0.8, 'module__weight_constraint': 5.0}
0.540365 (0.136861) with: {'module__dropout_rate': 0.9, 'module__weight_constraint': 1.0}
0.605469 (0.053083) with: {'module__dropout_rate': 0.9, 'module__weight_constraint': 2.0}
0.553385 (0.139948) with: {'module__dropout_rate': 0.9, 'module__weight_constraint': 3.0}
0.549479 (0.142719) with: {'module__dropout_rate': 0.9, 'module__weight_constraint': 4.0}
0.595052 (0.075566) with: {'module__dropout_rate': 0.9, 'module__weight_constraint': 5.0}

```

Output 20.7: Result of the grid search on dropout rate and weight constraint

You can see that the dropout rate of 10% and the weight constraint of 2.0 resulted in the best accuracy of about 70%.

20.10 How to Tune the Number of Neurons in the Hidden Layer

The number of neurons in a layer is an important parameter to tune. Generally the number of neurons in a layer controls the representational capacity of the network, at least at that point in the topology.

Generally, a large enough single layer network can approximate any other neural network, due to the *universal approximation theorem*.

In this example, you will look at tuning the number of neurons in a single hidden layer. You will try values from 1 to 30 in steps of 5. A larger network requires more training and at least the batch size and number of epochs should ideally be optimized with the number of neurons.

The full code listing is provided below.

```

import numpy as np
import torch
import torch.nn as nn
import torch.nn.init as init
import torch.optim as optim
from skorch import NeuralNetClassifier
from sklearn.model_selection import GridSearchCV

# load the dataset, split into input (X) and output (y) variables
dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
X = dataset[:,0:8]
y = dataset[:,8]
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

class PimaClassifier(nn.Module):
    def __init__(self, n_neurons=12):
        super().__init__()
        self.layer = nn.Linear(8, n_neurons)
        self.act = nn.ReLU()
        self.dropout = nn.Dropout(0.1)
        self.output = nn.Linear(n_neurons, 1)
        self.prob = nn.Sigmoid()
        self.weight_constraint = 2.0
        # manually init weights
        init.kaiming_uniform_(self.layer.weight)
        init.kaiming_uniform_(self.output.weight)

    def forward(self, x):
        # maxnorm weight before actual forward pass
        with torch.no_grad():
            norm = self.layer.weight.norm(2, dim=0, keepdim=True) \
                .clamp(min=self.weight_constraint / 2)
            desired = torch.clamp(norm, max=self.weight_constraint)
            self.layer.weight *= (desired / norm)
        # actual forward pass
        x = self.act(self.layer(x))
        x = self.dropout(x)
        x = self.prob(self.output(x))
        return x

# create model with skorch
model = NeuralNetClassifier(
    PimaClassifier,
    criterion=nn.BCELoss,
    optimizer=optim.Adamax,
    max_epochs=100,
    batch_size=10,
    verbose=False
)

# define the grid search parameters
param_grid = {
    'module__n_neurons': [1, 5, 10, 15, 20, 25, 30]
}

```



```

}
grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
grid_result = grid.fit(X, y)

# summarize results
print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
means = grid_result.cv_results_['mean_test_score']
stds = grid_result.cv_results_['std_test_score']
params = grid_result.cv_results_['params']
for mean, stdev, param in zip(means, stds, params):
    print("%f (%f) with: %r" % (mean, stdev, param))

```

Listing 20.15: Grid search on the number of neurons

Running this example produces the following output.

```

Best: 0.708333 using {'module__n_neurons': 30}
0.654948 (0.003683) with: {'module__n_neurons': 1}
0.666667 (0.023073) with: {'module__n_neurons': 5}
0.694010 (0.014382) with: {'module__n_neurons': 10}
0.682292 (0.014382) with: {'module__n_neurons': 15}
0.707031 (0.028705) with: {'module__n_neurons': 20}
0.703125 (0.030758) with: {'module__n_neurons': 25}
0.708333 (0.015733) with: {'module__n_neurons': 30}

```

Output 20.8: Result of the grid search on the number of neurons

You can see that the best results were achieved with a network with 30 neurons in the hidden layer with an accuracy of about 71%.

20.11 Tips for Hyperparameter Optimization

This section lists some handy tips to consider when tuning hyperparameters of your neural network.

- ▷ *k-Fold Cross-Validation.* You can see that the results from the examples in this chapter show some variance. A default cross-validation of 3 was used, but perhaps $k = 5$ or $k = 10$ would be more stable. Carefully choose your cross-validation configuration to ensure your results are stable.
- ▷ *Review the Whole Grid.* Do not just focus on the best result, review the whole grid of results and look for trends to support configuration decisions. Of course, there will be more combinations, and it takes a longer time to evaluate.
- ▷ *Parallelize.* Use all your cores if you can, neural networks are slow to train and we often want to try a lot of different parameters. Consider to run it on a cloud platform, such as AWS.
- ▷ *Use a Sample of Your Dataset.* Because networks are slow to train, try training them on a smaller sample of your training dataset, just to get an idea of general directions of parameters rather than optimal configurations.

- ▷ *Start with Coarse Grids.* Start with coarse-grained grids and zoom into finer grained grids once you can narrow the scope.
- ▷ *Do Not Transfer Results.* Results are generally problem specific. Try to avoid favorite configurations on each new problem that you see. It is unlikely that optimal results you discover on one problem will transfer to your next project. Instead, look for broader trends like number of layers or relationships between parameters.
- ▷ *Reproducibility is a Problem.* Although we set the seed for the random number generator in NumPy, the results are not 100% reproducible. There is more to reproducibility when grid searching wrapped PyTorch models than is presented in this chapter.

20.12 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Book Chapter

Michael Nielsen. Chapter 4, “A visual proof that neural nets can compute any function”. In: *Neural Networks and Deep Learning*. Determination Press, 2015.

<http://neuralnetworksanddeeplearning.com/chap4.html>

Articles

Grid search cross-validator. scikit-learn documentation.

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

skorch documentation.

<https://skorch.readthedocs.io/en/latest/>

torch.nn.init. PyTorch API.

<https://pytorch.org/docs/stable/nn.init.html>

torch.nn. PyTorch API.

<https://pytorch.org/docs/stable/nn.html>

torch.optim. PyTorch API.

<https://pytorch.org/docs/stable/optim.html>

Universal approximation theorem. Wikipedia.

https://en.wikipedia.org/wiki/Universal_approximation_theorem

20.13 Summary

In this chapter, you discovered how you can tune the hyperparameters of your deep learning networks in Python using PyTorch and scikit-learn.

Specifically, you learned:

- ▷ How to wrap PyTorch models for use in scikit-learn and how to use grid search.

- ▷ How to grid search a suite of different standard neural network parameters for PyTorch models.
- ▷ How to design your own hyperparameter optimization experiments.

In the next chapter, you will learn how to create checkpoints from a training loop.

Managing Training Process with Checkpoints and Early Stopping

21

A large deep learning model can take a long time to train. You lose a lot of work if the training process interrupted in the middle. But sometimes, you actually want to interrupt the training process in the middle because you know going any further would not give you a better model. In this chapter, you will discover how to control the training loop in PyTorch such that you can resume an interrupted process, or early stop the training loop.

After completing this chapter, you will know:

- ▷ The importance of checkpointing neural network models when training
- ▷ How to checkpoint a model during training and restore it later
- ▷ How to terminate training loop early with checkpointing

Let's get started.

Overview

This chapter is in two parts; they are:

- ▷ Checkpointing Neural Network Models
- ▷ Checkpointing with Early Stopping

21.1 Checkpointing Neural Network Models

A lot of systems have states. If you can save all its state from a system and restore it later, you can always move back in a particular point in time about how a system behaves. If you worked on Microsoft Word and saved multiple versions of a document because you don't know if you want to revert back your edit, it is the same idea here.

Same applies to long-running processes. Application checkpointing is a fault tolerance technique. In this approach, a snapshot of the state of the system is taken in case of system failure. If there is a problem, you can resume from the snapshot. The checkpoint may be used directly or as the starting point for a new run, picking up where it left off. When training deep learning models, the checkpoint captures the weights of the model. These weights can be used to make predictions as-is or as the basis for ongoing training.

PyTorch does not provide any function for checkpointing but it has functions for retrieving and restoring weights of a model. So you can implement checkpointing logic with them. Let's make a checkpoint and a resume function, which simply save weights from a model and load them back:

```
import torch

def checkpoint(model, filename):
    torch.save(model.state_dict(), filename)

def resume(model, filename):
    model.load_state_dict(torch.load(filename))
```

Listing 21.1: Functions to save and load a model

Below is how you would usually do to train a PyTorch model. The dataset used is fetched from OpenML platform. It is a binary classification dataset. PyTorch DataLoader is used in this example to make the training loop more concise.

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader, random_split, default_collate
from sklearn.datasets import fetch_openml
from sklearn.preprocessing import LabelEncoder

data = fetch_openml("electricity", version=1, parser="auto")

# Label encode the target, convert to float tensors
X = data['data'].astype('float').values
y = data['target']
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# train-test split for model evaluation
trainset, testset = random_split(TensorDataset(X, y), [0.7, 0.3])

# Define the model
model = nn.Sequential(
    nn.Linear(8, 12),
    nn.ReLU(),
    nn.Linear(12, 12),
    nn.ReLU(),
    nn.Linear(12, 1),
    nn.Sigmoid(),
)

# Train the model
n_epochs = 100
loader = DataLoader(trainset, shuffle=True, batch_size=32)
```

```

X_test, y_test = default_collate(testset)
loss_fn = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=0.1)
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    model.eval()
    y_pred = model(X_test)
    acc = (y_pred.round() == y_test).float().mean()
    print(f"End of epoch {epoch}: accuracy = {float(acc)*100:.2f}%")

```

Listing 21.2: Training a PyTorch model

If you want to add checkpoints to the training loop above, you can do it at the end of the outer for-loop, where the model validation with the test set is done. Let's say, the following:

```

...
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    model.eval()
    y_pred = model(X_test)
    acc = (y_pred.round() == y_test).float().mean()
    print(f"End of epoch {epoch}: accuracy = {float(acc)*100:.2f}%")
    checkpoint(model, f"epoch-{epoch}.pth")

```

Listing 21.3: Checkpointing a model in the training loop

You will see a number of files created in your working directory. This code is going to checkpoint the model from epoch 7, for example, into file `epoch-7.pth`. Each of these file is a ZIP file with the pickled model weight. Nothing forbid you to checkpoint inside the inner for-loop but due to the overhead it incurs, it is not a good idea to checkpoint too frequent.

As a fault tolerance technique, by adding a few lines of code before the training loop, you can resume from a particular epoch:

```

start_epoch = 0
if start_epoch > 0:
    resume_epoch = start_epoch - 1
    resume(model, f"epoch-{resume_epoch}.pth")

for epoch in range(start_epoch, n_epochs):
    model.train()

```

```

for X_batch, y_batch in loader:
    y_pred = model(X_batch)
    loss = loss_fn(y_pred, y_batch)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
model.eval()
y_pred = model(X_test)
acc = (y_pred.round() == y_test).float().mean()
print(f"End of epoch {epoch}: accuracy = {float(acc)*100:.2f}%")
checkpoint(model, f"epoch-{epoch}.pth")

```

Listing 21.4: Resume a training loop by loading from a checkpoint

That is, if the training loop was interrupted in the middle of epoch 8 so the last checkpoint is from epoch 7, setting `start_epoch = 8` above will do.

Note that if you do so, the `random_split()` function that generate the training set and test set may give you different split due to the random nature. If that's a concern for you, you should have a consistent way of creating the datasets (e.g., save the splitted data so you can reuse them).

Sometimes, there are states outside of the model and you may want to checkpoint it as well. One particular example is the optimizer, which in cases like Adam, there are dynamically adjusted momentum. If you restarted your training loop, you may want to restore the momentum at the optimizer as well. It is not difficult to do. The idea is to make your `checkpoint()` function more complicated, e.g.

```

torch.save({
    'optimizer': optimizer.state_dict(),
    'model': model.state_dict(),
}, filename)

```

Listing 21.5: Save both the optimizer and the model in the checkpoint

and correspondingly, change your `resume()` function:

```

checkpoint = torch.load(filename)
model.load_state_dict(checkpoint['model'])
optimizer.load_state_dict(checkpoint['optimizer'])

```

Listing 21.6: Reset both the optimizer and the model from a checkpoint

This works because in PyTorch, the `torch.save()` and `torch.load()` function are backed by pickle, so you can use it with a list or dict container.

To put everything together, below is the complete code:

```

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader, random_split, default_collate
from sklearn.datasets import fetch_openml
from sklearn.preprocessing import LabelEncoder

```

```

data = fetch_openml("electricity", version=1, parser="auto")

# Label encode the target, convert to float tensors
X = data['data'].astype('float').values
y = data['target']
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# train-test split for model evaluation
trainset, testset = random_split(TensorDataset(X, y), [0.7, 0.3])

def checkpoint(model, optimizer, filename):
    torch.save({
        'optimizer': optimizer.state_dict(),
        'model': model.state_dict(),
    }, filename)

def resume(model, optimizer, filename):
    checkpoint = torch.load(filename)
    model.load_state_dict(checkpoint['model'])
    optimizer.load_state_dict(checkpoint['optimizer'])

# Define the model
model = nn.Sequential(
    nn.Linear(8, 12),
    nn.ReLU(),
    nn.Linear(12, 12),
    nn.ReLU(),
    nn.Linear(12, 1),
    nn.Sigmoid(),
)

# Train the model
n_epochs = 100
start_epoch = 0
loader = DataLoader(trainset, shuffle=True, batch_size=32)
X_test, y_test = default_collate(testset)
loss_fn = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=0.1)

if start_epoch > 0:
    resume_epoch = start_epoch - 1
    resume(model, optimizer, f"epoch-{resume_epoch}.pth")

for epoch in range(start_epoch, n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()

```



```

        loss.backward()
        optimizer.step()
    model.eval()
    y_pred = model(X_test)
    acc = (y_pred.round() == y_test).float().mean()
    print(f"End of epoch {epoch}: accuracy = {float(acc)*100:.2f}%")
    checkpoint(model, optimizer, f"epoch-{epoch}.pth")

```

Listing 21.7: Example to train a model with checkpointing

21.2 Checkpointing with Early Stopping

Checkpointing is not only for fault tolerance. You can also use it to keep your best model. How to define what is the best is subjective but considering the score from the test set is a sensible method. Let's say to keep only the best model ever found, you can modify the training loop as follows:

```

...
best_accuracy = -1
for epoch in range(start_epoch, n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    model.eval()
    y_pred = model(X_test)
    acc = (y_pred.round() == y_test).float().mean()
    acc = float(acc) * 100
    print(f"End of epoch {epoch}: accuracy = {acc:.2f}%")
    if acc > best_accuracy:
        best_accuracy = acc
        checkpoint(model, "best_model.pth")

resume(model, "best_model.pth")

```

Listing 21.8: Training loop to checkpoint the best model

The variable `best_accuracy` is to keep track on the highest accuracy (`acc`) obtained so far, which is in a percentage range of 0 to 100. Whenever a higher accuracy is observed, the model is checkpointed to the file `best_model.pth`. The best model is restored after the entire training loop, via the `resume()` function you created before. Afterward, you can make predictions with the model on unseen data. Beware that, if you're using a different metric for checkpointing, e.g., the cross-entropy loss, the better model should come with a lower cross-entropy. Thus you should keep track on the lowest cross-entropy obtained.

You can also checkpoint the model per epoch unconditionally together with the best model checkpointing, as you are free to create multiple checkpoint files. Since the code above is the find the best model and make a copy of it, you may usually see a further optimization

to the training loop by stopping it early if the hope to see model improvement is slim. This is the early stopping technique that can save time in training.

The code above validates the model with test set at the end of each epoch and keeps the best model found into a checkpoint file. The simplest strategy for early stopping is to set up a threshold of k epochs. If you didn't see the model improved over the last k epochs, you terminate the training loop in the middle. This can be implemented as follows:

```
early_stop_thresh = 5
best_accuracy = -1
best_epoch = -1
for epoch in range(start_epoch, n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    model.eval()
    y_pred = model(X_test)
    acc = (y_pred.round() == y_test).float().mean()
    acc = float(acc) * 100
    print(f"End of epoch {epoch}: accuracy = {acc:.2f}%")
    if acc > best_accuracy:
        best_accuracy = acc
        best_epoch = epoch
        checkpoint(model, "best_model.pth")
    elif epoch - best_epoch > early_stop_thresh:
        print("Early stopped training at epoch %d" % epoch)
        break # terminate the training loop

resume(model, "best_model.pth")
```

Listing 21.9: Training loop to implement early stopping

The threshold `early_stop_thresh` was set to 5 above. There is a variable `best_epoch` that remembers the epoch of the best model. If the model has not been improved for long enough, the outer for-loop will be terminated.

This design is a relief on one of the design parameter, `n_epochs`. You can now make `n_epochs` the *maximum* number of epochs to train the model, hence a larger number than needed and assured that usually your training loop will stop earlier. This is also a strategy to avoid overfitting: If the model is indeed perform worse on test set as you trained it further, this early stopping logic will interrupt the training and restore the best checkpoint.

Tying everything together, the following is the complete code for checkpointing with early stopping:

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader, random_split, default_collate
```

```

from sklearn.datasets import fetch_openml
from sklearn.preprocessing import LabelEncoder

data = fetch_openml("electricity", version=1, parser="auto")

# Label encode the target, convert to float tensors
X = data['data'].astype('float').values
y = data['target']
encoder = LabelEncoder()
encoder.fit(y)
y = encoder.transform(y)
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

# train-test split for model evaluation
trainset, testset = random_split(TensorDataset(X, y), [0.7, 0.3])

def checkpoint(model, filename):
    torch.save(model.state_dict(), filename)

def resume(model, filename):
    model.load_state_dict(torch.load(filename))

# Define the model
model = nn.Sequential(
    nn.Linear(8, 12),
    nn.ReLU(),
    nn.Linear(12, 12),
    nn.ReLU(),
    nn.Linear(12, 1),
    nn.Sigmoid(),
)

# Train the model
n_epochs = 10000 # more than we needed
loader = DataLoader(trainset, shuffle=True, batch_size=32)
X_test, y_test = default_collate(testset)
loss_fn = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=0.1)

early_stop_thresh = 5
best_accuracy = -1
best_epoch = -1

for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    model.eval()
    y_pred = model(X_test)

```

```

acc = (y_pred.round() == y_test).float().mean()
acc = float(acc) * 100
print(f"End of epoch {epoch}: accuracy = {acc:.2f}%")
if acc > best_accuracy:
    best_accuracy = acc
    best_epoch = epoch
    checkpoint(model, "best_model.pth")
elif epoch - best_epoch > early_stop_thresh:
    print("Early stopped training at epoch %d" % epoch)
    break # terminate the training loop

resume(model, "best_model.pth")

```

Listing 21.10: Training a model with early stopping

You may see the above code to produce:

```

End of epoch 0: accuracy = 61.84%
End of epoch 1: accuracy = 55.90%
End of epoch 2: accuracy = 63.95%
End of epoch 3: accuracy = 66.87%
End of epoch 4: accuracy = 64.77%
End of epoch 5: accuracy = 60.03%
End of epoch 6: accuracy = 67.16%
End of epoch 7: accuracy = 66.01%
End of epoch 8: accuracy = 62.88%
End of epoch 9: accuracy = 64.28%
End of epoch 10: accuracy = 68.63%
End of epoch 11: accuracy = 70.56%
End of epoch 12: accuracy = 64.62%
End of epoch 13: accuracy = 65.63%
End of epoch 14: accuracy = 66.81%
End of epoch 15: accuracy = 65.11%
End of epoch 16: accuracy = 55.81%
End of epoch 17: accuracy = 54.59%
Early stopped training at epoch 17

```

Output 21.1: Progress of training a model with early stopping

It stopped at end of epoch 17 for the best model obtained from epoch 11. Due to the stochastic nature of algorithm, you may see the result slightly different. But for sure, even when the maximum number of epochs set to 10000 above, the training loop indeed stopped much earlier.

Of course, you can design a more sophisticated early stopping strategy, e.g., run for at least N epochs and then allow to early stop after k epochs. You have all the freedom to tweak the code above to make the best training loop to fit your need.

21.3 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Articles

torch.utils.data. PyTorch documentation.

<https://pytorch.org/docs/stable/data.html>

torch.utils.checkpoint. PyTorch documentation.

<https://pytorch.org/docs/stable/checkpoint.html>

Fetch dataset from OpenML. scikit-learn documentation.

https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_openml.html

Label encoder. scikit-learn documentation.

<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html>

21.4 Summary

In this chapter, you discovered the importance of checkpointing deep learning models for long training runs. You learned:

- ▷ What is checkpointing and why it is useful
- ▷ How to checkpoint your model and how to restore the checkpoint
- ▷ Different strategies to use checkpoints
- ▷ How to implement early stopping with checkpointing

Sometimes, it would be helpful if you can *see* the architecture of a model. In the next chapter, you will learn how to visualize a PyTorch model graphically.

Visualizing a PyTorch Model

PyTorch is a deep learning library. You can build very sophisticated deep learning models with PyTorch. However, there are times you want to have a graphical representation of your model architecture. In this chapter, you will learn:

- ▷ How to save your PyTorch model in an exchange format
- ▷ How to use Netron to create a graphical representation.

Let's get started.

Overview

This chapter is divided into two parts; they are

- ▷ Why Graphical Representation of a PyTorch Model is Hard
- ▷ How to Use Netron to Create a Model Graph

22.1 Why Graphical Representation of a PyTorch Model is Hard

PyTorch is a very flexible library for deep learning. Strictly speaking, it never mandates how you should build your model as long as it works like a function that can transform an input tensor into output tensor. This is a problem: With a model, you never know how it works unless you follow the input tensor and collect the trace until you get the output tensor. Therefore it is not trivial to convert a PyTorch model into a picture.

There are multiple libraries to solve this problem. But in general, there are only two ways to solve it: You can follow a tensor on the forward pass and see what operation (i.e., layers) are applied, or follow a tensor on the backward pass and see how the gradient propagated to the input. You can only find clues about the structure inside a model in this manner.

22.2 How to Use Netron to Create a Model Graph

When you save a PyTorch model, you are saving its states. You can get the model states using `model.state_dict()`. While the weight tensors have names that helps you to restore them to a model, you do not have the clues on how the weights are connected to each other. The only way you can connect tensors and find out their relationship is to get the tensor gradient: When you run a model and get an output, the calculations involved, including the dependencies to other tensors, are remembered by each intermediate tensors so automatic differentiation can be performed.

Indeed, if you want to know the algorithm behind a PyTorch model, this is also the way to go. There are only a few tools to create graphics from a PyTorch model. In below, you will learn about the tool Netron. It is a “deep learning model viewer”. It is a software that you can install and run on macOS, Linux, and Windows. You can visit the page below and download the software for your platform:

▷ <https://github.com/lutzroeder/netron/releases>

There is also an online version available, that you can see your model by uploading a model file.

Netron cannot visualize a PyTorch model from the saved states because there’s not enough clues to tell about the structure of the model. However, PyTorch allows you to convert the model to an exchange format, ONNX, that Netron can understand.

Let’s start with an example. In below you created a simple model to classify the iris dataset. It is a classification problem with three classes. Therefore the model should output a vector of three values. A complete code that you would create for this problem is as follows, which the dataset is obtained from scikit-learn:

```
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

data = load_iris()
X = data['data']
y = data['target']
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.long)

# split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=True)

class IrisModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(4, 8)
        self.act = nn.ReLU()
        self.output = nn.Linear(8, 3)
```

```

def forward(self, x):
    x = self.act(self.hidden(x))
    x = self.output(x)
    return x

# loss metric and optimizer
model = IrisModel()
loss_fn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# prepare model and training parameters
n_epochs = 100
batch_size = 10
batch_start = torch.arange(0, len(X_train), batch_size)

# training loop
for epoch in range(n_epochs):
    for start in batch_start:
        # take a batch
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        # forward pass
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        # backward pass
        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()

# validating model
y_pred = model(X_test)
acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
acc = float(acc)*100
print("Model accuracy: %.2f%%" % acc)

```

Listing 22.1: Training a PyTorch model using dataset from scikit-learn

Running the above produces the following, for example:

```
Model accuracy: 97.78%
```

Output 22.1: Accuracy of the model trained

So you know that `model` is a PyTorch model that can take a tensor and return a tensor. If you have ONNX module installed (e.g., running `pip install onnx`), you can *convert* this model into ONNX format using the `torch.onnx.export()` function:

```

torch.onnx.export(model, X_test, 'iris.onnx',
                  input_names=["features"], output_names=["logits"])

```

Listing 22.2: Convert a trained model into ONNX format

Running this will create a file `iris.onnx` at the local directory. You need to provide a *sample tensor* that works with the model as input (`X_test` in the example above). This is because

during the conversion, it needs to follow this sample tensor to understand what operations shall be applied so you can convert the algorithm step by step into ONNX format. Every weight in a PyTorch model is a tensor and there is a name assigned to them. But the input and output tensors are not named usually, hence you need to provide a name to them when you run `export()`. These names should be provided as a list of strings because in general, a model can take multiple tensors and return multiple tensors.

Usually you should run `export()` after the training loop. It is because the ONNX model created contains a complete model that you can run without the PyTorch library. You want to save the optimized weight to it. However, for the purpose of visualizing a model in Netron, the quality of the model is not a concern. You can run `export()` as soon as the PyTorch model is created.

After you launched Netron, you can open the saved ONNX file. In this example, you should see the following screen:

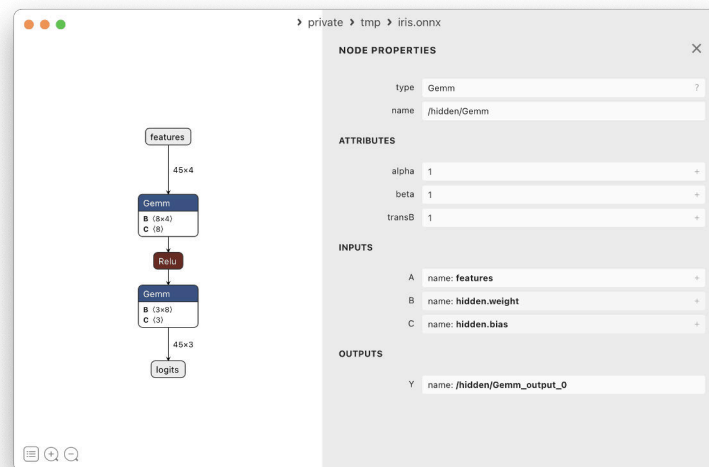


Figure 22.1: A graphical representation of the model

It shows how the input tensor connected to the output tensor via different operations in the deep learning model. The name of the input and output tensors you provided to the `export()` function is used in the visualization. Clicking on a box will give you more details about that particular tensor or operation. The names of operations as you can see in Netron may be different from what you call them in PyTorch, however. In the screen above, you see the `nn.Linear()` layer becomes “Gemm”, which stands for “general matrix multiply” operation. You can even examine the weights on a layer with Netron by a few points and clicks.

If you would like to keep a copy of this visualization, you can export it into PNG format in Netron.

22.3 Further Readings

Netron is an open source project which you can find its source code in Github:

▷ <https://github.com/lutzroeder/netron>

An online version of Netron is available below:

▷ <https://netron.app/>

Another library for visualization is torchviz, but unlike the example you saw above, it traces the model from a backward pass:

▷ <https://github.com/szagoruyko/pytorchviz>

22.4 Summary

In this chapter, you learned how to visualize a model. In particular, you learned:

- ▷ Why visualizing a PyTorch model is difficult
- ▷ How to convert a PyTorch model into ONNX format
- ▷ How to use Netron to visualize a ONNX model

In the next chapter, you will learn how to collect and plot the metrics from the training process.

Understand Model Behavior During Training by Visualizing Metrics

23

You can learn a lot about neural networks and deep learning models by observing their performance over time during training. For example, if you see the training accuracy went worse with training epochs, you know you have issue with the optimization. Probably your learning rate is too fast. In this chapter, you will discover how you can review and visualize the performance of PyTorch models over time during training. After completing this chapter, you will know:

- ▷ What metrics to collect during training
- ▷ How to plot the metrics on training and validation datasets from training
- ▷ How to interpret the plot to tell about the model and training progress

Let's get started.

Overview

This chapter is in two parts; they are:

- ▷ Collecting Metrics from a Training Loop
- ▷ Plotting the Training History

23.1 Collecting Metrics from a Training Loop

In deep learning, training a model with gradient descent algorithm means to take a forward pass to infer loss metric from the input using the model and a loss function, then a backward pass to compute the gradient from the loss metric, and an update process to apply the gradient to update the model parameters. While these are the basic steps you must take, you can do a bit more along the process to collect additional information.

A model that trained correctly should expect the loss metric to decrease, as the loss is the objective to optimize. The loss metric to use should depend on the problem.

For regression problems, the closer the model's prediction to the actual value the better. Therefore, you want to keep track on the mean square error (MSE), or sometimes root mean square error (RMSE), mean absolute error (MAE), or mean absolute percentage error (MAPE).

Although not used as a loss metric, you may also be interested in the maximum error produced by your model.

For classification problems, usually the loss metric is cross-entropy. But the value of cross-entropy is not very intuitive. Therefore, you may also want to keep track on the accuracy of prediction, true positive rate, precision, recall, F1 scores, and so on.

Collecting these metrics from a training loop is trivial. Let's start with a basic regression example of deep learning using PyTorch with the California housing dataset:

```
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Read data
data = fetch_california_housing()
X, y = data.data, data.target

# train-test split for model evaluation
X_train_raw, X_test_raw, y_train, y_test = train_test_split(X, y, train_size=0.7,
                                                            shuffle=True)

# Standardizing data
scaler = StandardScaler()
scaler.fit(X_train_raw)
X_train = scaler.transform(X_train_raw)
X_test = scaler.transform(X_test_raw)

# Convert to 2D PyTorch tensors
X_train = torch.tensor(X_train, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1, 1)

# Define the model
model = nn.Sequential(
    nn.Linear(8, 24),
    nn.ReLU(),
    nn.Linear(24, 12),
    nn.ReLU(),
    nn.Linear(12, 6),
    nn.ReLU(),
    nn.Linear(6, 1)
)

# loss function and optimizer
loss_fn = nn.MSELoss() # mean square error
optimizer = optim.Adam(model.parameters(), lr=0.001)

n_epochs = 100 # number of epochs to run
batch_size = 32 # size of each batch
```

```
batch_start = torch.arange(0, len(X_train), batch_size)

for epoch in range(n_epochs):
    for start in batch_start:
        # take a batch
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        # forward pass
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        # backward pass
        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()
```

Listing 23.1: A PyTorch example

This implementation is primitive, but you obtained `loss` as a tensor in each step in the process which provides hints to the optimizer to improve the model. To know about the progress of the training, you can, of course, print this loss metric at every step. But you can also save this value so you can visualize it later. When you do that, beware that you do not want to save a tensor but simply its value. It is because the PyTorch tensor here remembers how it comes with its value so automatic differentiation can be done. These additional data are occupying memory but you do not need them.

Hence you can modify the training loop to the following:

```
mse_history = []

for epoch in range(n_epochs):
    for start in batch_start:
        # take a batch
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        # forward pass
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        mse_history.append(float(loss))
        # backward pass
        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()
```

Listing 23.2: Training loop with the loss metric remembered in a Python list

In training a model, you should evaluate it with a test set which is segregated from the training set. Usually it is done once in an epoch, after all the training steps in that epoch. The test result can also be saved for visualization later. In fact, you can obtain multiple metrics from the test set if you want to. Hence you can add to the training loop as follows:

```

mae_fn = nn.L1Loss() # create a function to compute MAE
train_mse_history = []
test_mse_history = []
test_mae_history = []

for epoch in range(n_epochs):
    model.train()
    for start in batch_start:
        # take a batch
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        # forward pass
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        train_mse_history.append(float(loss))
        # backward pass
        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()
    # validate model on test set
    model.eval()
    with torch.no_grad():
        y_pred = model(X_test)
        mse = loss_fn(y_pred, y_test)
        mae = mae_fn(y_pred, y_test)
        test_mse_history.append(float(mse))
        test_mae_history.append(float(mae))

```

Listing 23.3: Training loop to collect multiple metrics

You can define your own function to compute the metrics or use one that already implemented from PyTorch library. It is a good practice to switch the model to evaluation mode on evaluation. It is also good practice to run the evaluation under the `no_grad()` context, in which you explicitly tell PyTorch that you have no intention to run automatic differentiation on the tensors.

However, there is a problem in the code above: The MSE from training set is computed once per training step based on one batch while the metrics from the test set are computed once per epoch and based on the entire test set. They are not directly comparable. In fact, if you look at the MSE from training steps, you will find it *very noisy*. The better way is to summarize the MSE from the same epoch to one number (e.g., their mean) so you can compare to the test set's data.

Making this change, following is the complete code:

```

import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

```

```

# Read data
data = fetch_california_housing()
X, y = data.data, data.target

# train-test split for model evaluation
X_train_raw, X_test_raw, y_train, y_test = train_test_split(X, y, train_size=0.7,
                                                            shuffle=True)

# Standardizing data
scaler = StandardScaler()
scaler.fit(X_train_raw)
X_train = scaler.transform(X_train_raw)
X_test = scaler.transform(X_test_raw)

# Convert to 2D PyTorch tensors
X_train = torch.tensor(X_train, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1, 1)

# Define the model
model = nn.Sequential(
    nn.Linear(8, 24),
    nn.ReLU(),
    nn.Linear(24, 12),
    nn.ReLU(),
    nn.Linear(12, 6),
    nn.ReLU(),
    nn.Linear(6, 1)
)

# loss function, metrics, and optimizer
loss_fn = nn.MSELoss() # mean square error
mae_fn = nn.L1Loss() # mean absolute error
optimizer = optim.Adam(model.parameters(), lr=0.001)

n_epochs = 100 # number of epochs to run
batch_size = 32 # size of each batch
batch_start = torch.arange(0, len(X_train), batch_size)

train_mse_history = []
test_mse_history = []
test_mae_history = []

for epoch in range(n_epochs):
    model.train()
    epoch_mse = []
    for start in batch_start:
        # take a batch
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        # forward pass
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        epoch_mse.append(float(loss))

```

```

    # backward pass
    optimizer.zero_grad()
    loss.backward()
    # update weights
    optimizer.step()
    mean_mse = sum(epoch_mse) / len(epoch_mse)
    train_mse_history.append(mean_mse)
    # validate model on test set
    model.eval()
    with torch.no_grad():
        y_pred = model(X_test)
        mse = loss_fn(y_pred, y_test)
        mae = mae_fn(y_pred, y_test)
        test_mse_history.append(float(mse))
        test_mae_history.append(float(mae))

```

Listing 23.4: Training a model with metrics collected during the training loop

23.2 Plotting the Training History

In the code above, you collected metrics in a Python list, one each per epoch. Therefore, it is trivial to plot them into a line graph using matplotlib. Below is an example:

```

import matplotlib.pyplot as plt
import numpy as np
plt.plot(np.sqrt(train_mse_history), label="Train RMSE")
plt.plot(np.sqrt(test_mse_history), label="Test RMSE")
plt.plot(test_mae_history, label="Test MAE")
plt.xlabel("epochs")
plt.legend()
plt.show()

```

Listing 23.5: Visualize the collected metric using matplotlib

It plots, for example, the following:

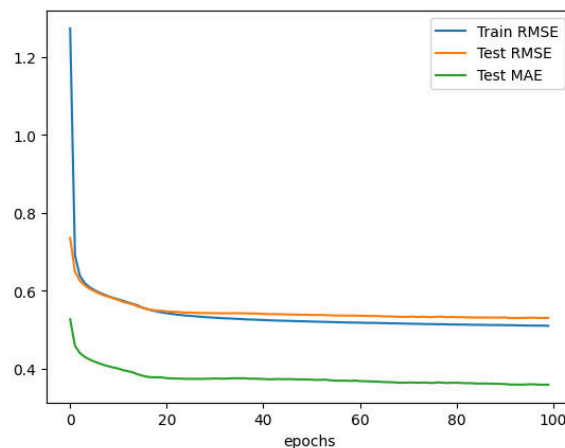


Figure 23.1: The model metric as training progressed

Plots like this can provide an indication of useful things about the training of the model, such as:

- ▷ Its speed of convergence over epochs (slope)
- ▷ Whether the model may have already converged (plateau of the line)
- ▷ Whether the model may be over-learning the training data (inflection for validation line)

In a regression example like the above, the metrics MAE and MSE should both decrease if the model gets better. In a classification example, however, accuracy metric should increase while the cross-entropy loss should decrease as more training has been done. This is what you are expected to see from the plot.

These curves should eventually flatten, meaning you cannot improve the model any further based on the current dataset, model design, and algorithms. You want this to happen as soon as possible, so your model *converge* faster as your training is efficient. You also want the metric to flatten at a high accuracy or low loss region, so your model is effective in prediction.

The other property to watch for in the plots is how different are the metrics from training and validation. In the above, you see the training set's RMSE is higher than test set's RMSE at the beginning but very soon, the curves crossed and the test set's RMSE is higher at the end. This is expected, as eventually the model will fit better to the training set but it is the test set that can predict how the model performs on future, unseen data.

You need to be careful to interpret the curves or metrics in a microscopic scale. In the plot above, you see that the training set's RMSE is extremely large compare to that of test set's in epoch 0. Their difference may not be that drastic, but since you collected the training set's RMSE by taking the MSE of each steps during the first epoch, your model probably not doing well in the first few steps but much better at the last few steps of the epoch. Taking average across all the steps may not be a fair comparison as the MSE from test set is based on the model after the last step.

Your model is *overfit* if you see the training set's metric is much better than that from test set. This can hint that you should stop your training at an earlier epoch or your model's design need some regularization, such as dropout layer.

In the plot above, while you collected mean square error (MSE) for the regression problem but you plotted root mean square error (RMSE) instead, so you can compare to the mean absolute error (MAE) in the same scale. Probably you should also collect the MAE of the training set as well. The two MAE curves should behave similarly to that of the RMSE curves.

Putting everything together, the following is the complete code:

```
import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```

# Read data
data = fetch_california_housing()
X, y = data.data, data.target

# train-test split for model evaluation
X_train_raw, X_test_raw, y_train, y_test = train_test_split(X, y, train_size=0.7,
                                                            shuffle=True)

# Standardizing data
scaler = StandardScaler()
scaler.fit(X_train_raw)
X_train = scaler.transform(X_train_raw)
X_test = scaler.transform(X_test_raw)

# Convert to 2D PyTorch tensors
X_train = torch.tensor(X_train, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1, 1)

# Define the model
model = nn.Sequential(
    nn.Linear(8, 24),
    nn.ReLU(),
    nn.Linear(24, 12),
    nn.ReLU(),
    nn.Linear(12, 6),
    nn.ReLU(),
    nn.Linear(6, 1)
)

# loss function, metrics, and optimizer
loss_fn = nn.MSELoss() # mean square error
mae_fn = nn.L1Loss() # mean absolute error
optimizer = optim.Adam(model.parameters(), lr=0.001)

n_epochs = 100 # number of epochs to run
batch_size = 32 # size of each batch
batch_start = torch.arange(0, len(X_train), batch_size)

train_mse_history = []
test_mse_history = []
test_mae_history = []

for epoch in range(n_epochs):
    model.train()
    epoch_mse = []
    for start in batch_start:
        # take a batch
        X_batch = X_train[start:start+batch_size]
        y_batch = y_train[start:start+batch_size]
        # forward pass
        y_pred = model(X_batch)

```

```

        loss = loss_fn(y_pred, y_batch)
        epoch_mse.append(float(loss))
        # backward pass
        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()
    mean_mse = sum(epoch_mse) / len(epoch_mse)
    train_mse_history.append(mean_mse)
    # validate model on test set
    model.eval()
    with torch.no_grad():
        y_pred = model(X_test)
        mse = loss_fn(y_pred, y_test)
        mae = mae_fn(y_pred, y_test)
        test_mse_history.append(float(mse))
        test_mae_history.append(float(mae))

plt.plot(np.sqrt(train_mse_history), label="Train RMSE")
plt.plot(np.sqrt(test_mse_history), label="Test RMSE")
plt.plot(test_mae_history, label="Test MAE")
plt.xlabel("epochs")
plt.legend()
plt.show()

```

Listing 23.6: Training a PyTorch model with the metric visualized

23.3 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

L1Loss. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.L1Loss.html>

MSELoss. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.MSELoss.html>

23.4 Summary

In this chapter, you discovered the importance of collecting and reviewing metrics while training your deep learning models. You learned:

- ▷ What metrics to look for during model training
- ▷ How to compute and collect metrics in a PyTorch training loop
- ▷ How to visualize the metrics from a training loop
- ▷ How to interpret the metrics to infer details about the training experience

The examples so far are all fully-connected networks and there are other type of networks. In the next chapter, you will see some examples.

Advanced Models

IV

From MLP to CNN and RNN

Traditional neural networks are the multilayer perceptron architecture. It is built with fully-connected layers. This type of layers is simple in algorithm but with a lot of parameters. In more advanced application of deep learning, you will see other kind of architecture. In this chapter, you will be introduced to two popular architectures of deep learning. After you finished this chapter, you will learn

- ▷ What is convolutional neural network (CNN)
- ▷ What is recurrent neural network (RNN)
- ▷ Why CNN and RNN can do better in some applications

Let's get started.

Overview

This chapter is in two parts; they are:

- ▷ Introduction to Convolutional Neural Network
- ▷ Introduction to Recurrent Neural Network

24.1 Introduction to Convolutional Neural Network

Convolutional neural networks (CNNs) are deep neural networks that are designed to work with image data. They are commonly used for object recognition, image classification, and image segmentation. The key building blocks of a CNN are convolutional layers and pooling layers. Usually fully connected layers are also used as output layers in CNN models.

Convolutional layers are responsible for feature extraction by applying a set of learnable filters to the input image. Unlike the fully-connected layers, convolutional layers produce output with a sliding window which the filter is applied repeatedly. Filters are multiplied with different regions of the input image to identify the presence of specific features. Because filters are applied in a sliding window, the features are *spatially invariant* meaning a particular element in the image (e.g., a cat) is not required to appear at a specific location. Also, because of the filters are applied in a window, they are usually much smaller than the image.

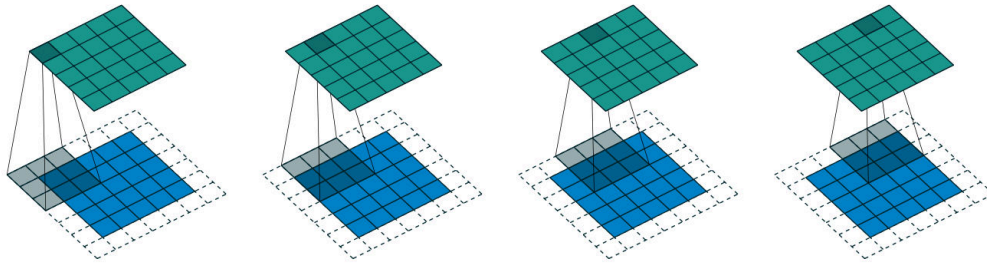


Figure 24.1: Operation of a convolutional layer: Blue is the input, gray is the filter, and green is the output. Image from “A guide to convolution arithmetic for deep learning”

Pooling layers in CNNs are to reduce the dimensionality of the feature maps generated by the convolutional layers. Similar to convolutional layers, it is applied with a sliding window. But instead of multiplying a filter, usually some aggregation function is used, such as taking the average or finding the maximum in the window. By carefully designing how the window slides, you can convert a high resolution image to lower resolution using pooling layer. This is how eventually you can reduce a million-pixel image into a binary classification (e.g., whether a cat is in the image), for example.

Fully-connected layers are used for classification by taking the output of the convolutional and pooling layers and passing it through a set of fully-connected neurons. Therefore, you usually see them near the output of the network model.

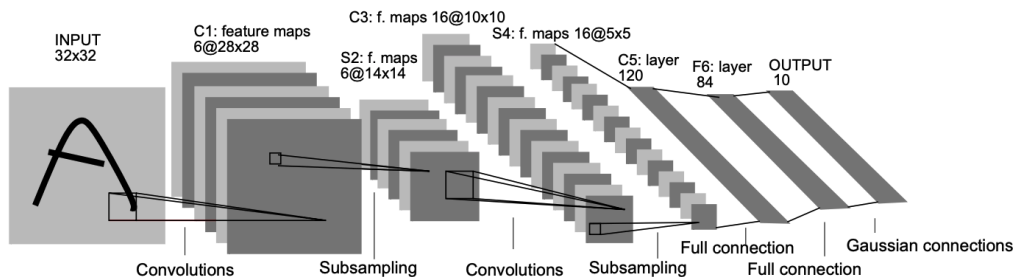


Figure 24.2: One of the earliest CNN, with convolutional layers, pooling layers (subsampling), and fully-connected layers. Image from “Gradient-Based Learning Applied to Document Recognition”

CNNs have been applied to a variety of image-related tasks such as object detection, facial recognition, and image segmentation. For example, it can be used to identify pedestrians, cars, and traffic signals on a street view to make a self-driving car.

CNNs are better suited for image-related tasks than Multilayer Perceptrons (MLPs) due to their ability to capture spatial information. MLPs process the input data as a flat vector, which means that they cannot take into account the spatial relationships between the different parts of an image. In contrast, CNNs keep the input image in two-dimensional and use convolutional layers to extract features from different regions of an image, allowing them to capture spatial information effectively.

Additionally, CNNs have fewer parameters than MLPs, which makes them more computationally efficient. MLPs require a large number of parameters to learn complex

patterns in data, whereas CNNs can learn the same patterns using fewer parameters. This makes CNNs more suitable for tasks that involve processing large amounts of data, such as image classification and object detection.

24.2 Introduction to Recurrent Neural Network

Recurrent neural networks (RNNs) are used for processing sequential data such as natural language text and speech. They are designed to capture the temporal dependencies between the input data.

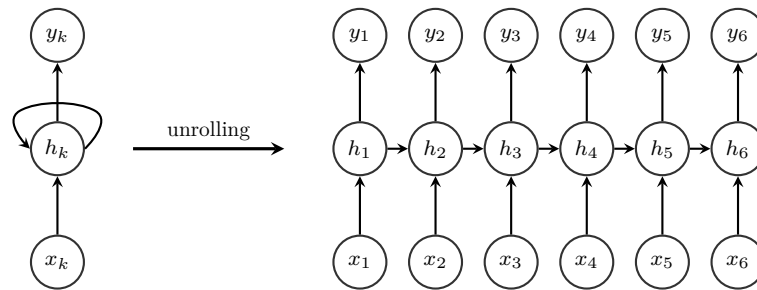


Figure 24.3: Structure of a recurrent neural network, before and after unrolling

The core building block of an RNN is the *recurrent layer*. The key idea behind a recurrent layer is to have one or more hidden state, which accumulates information about the input as the input sequence is provided. While the input is a sequence, the neural network is the same for each step, and produces output of the corresponding step. It is similar to an MLP processing one step of input at a time, except the hidden state is involved in producing the output and the input is indeed a sequence with many steps.

The hidden state of the recurrent layer is fed back into the layer at the next time step, allowing the network to maintain a memory of previous inputs. Therefore, sometimes you will see recurrent layer *unrolled* to emphasize the sequence structure of input and output. However, the repeating structure is indeed just one recurrent layer. The hidden state is not a learnable parameter, but rather, it is a state variable that is accumulated on the input sequence. It works as if it is the summary of the input sequence the RNN has seen so far. The processing of the input and the algorithm to update the hidden state is learned, but it is reused for every step of the input sequence. It is called *recurrent* because algorithmically, the recurrent layer is processing the input by applying the same algorithm in a loop on every step of the sequence.

RNNs can be more effective than Multilayer Perceptrons (MLPs) for sequential data tasks because they are designed to capture the temporal dependencies between the input data. MLPs process the input data as a flat vector, which means that they cannot take into account the order of the data. Moreover, since the input data is considered as a flat vector, inherently MLP is encouraged to process different time steps in the sequence differently. In contrast, RNNs use recurrent layers to maintain a memory of previous inputs, and consider each time step of the sequence as homogeneous, allowing them to capture the temporal dependencies in the data effectively.

The memory held by the hidden state makes RNNs well suited for tasks with variable length, such as language modeling, machine translation, and speech recognition. MLPs, on the other hand, require fixed-length inputs, which can be a limitation for tasks that involve variable-length data. In language processing, RNNs are used to generate, classify, and translate text. In speech recognition, RNN is used to convert spoken words into text, which RNN can analyze the sequence of sound waves produced by speech and identify the words being spoken.

24.3 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Vincent Dumoulin and Francesco Visin. “A guide to convolution arithmetic for deep learning”, 2016.

<https://arxiv.org/abs/1603.07285>

Yann LeCun, Leon Bottou, Yoshua Bengio, and Patrick Haffner. “Gradient-Based Learning Applied to Document Recognition”. In: *Proceedings of the IEEE*. Nov. 1998.

<http://yann.lecun.com/exdb/publis/pdf/lecun-01a.pdf>

Andrej Karpathy. *The Unreasonable Effectiveness of Recurrent Neural Networks*. May 2015.

<http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

Recurrent neural network. Wikipedia.

https://en.wikipedia.org/wiki/Recurrent_neural_network

24.4 Summary

CNN and RNN are two powerful architectures of deep learning models that are widely used in computer vision and natural language processing respectively. Understanding these architectures is essential for building effective deep learning models for a wide range of applications.

In the next chapter, you will learn to implement a CNN using PyTorch.

Building a Convolutional Neural Network in PyTorch

25

Neural networks are built with layers connected to each other. There are different kinds of layers. For image related applications, you can always find convolutional layers. It is a layer with very few parameters but applied over a large sized input. It is powerful because it can preserve the spatial structure of the image. Therefore, it is used to produce state-of-the-art results on computer vision neural networks. In this chapter, you will learn about the convolutional layer and the network it built. After completing this chapter, you will know:

- ▷ What are convolutional layers and pooling layers
- ▷ How they fit together in a neural network
- ▷ How a neural network using convolutional layers is designed

Let's get started.

Overview

This chapter is divided into four parts; they are

- ▷ The Case for Convolutional Neural Networks
- ▷ Building Blocks of Convolutional Neural Networks
- ▷ An Example of Convolutional Neural Network
- ▷ What Are in Feature Maps?

25.1 The Case for Convolutional Neural Networks

Let's consider making a neural network to process grayscale image as input, which is the simplest use case in deep learning for computer vision.

A grayscale image is an array of pixels. Each pixel is usually a value in a range of 0 to 255. An image with size 32×32 would have 1024 pixels. Taking it as input to a neural network would mean the first layer to have at least 1024 input weights.

Looking at the pixel value has little use to understanding the picture, because data is hiding in the spatial structure (e.g., whether there is a horizontal line or a vertical line on

the picture). Hence the traditional neural network will find it difficult to find out information from image input.

Convolutional neural network is to use convolutional layers to preserve spatial information of pixels. It learns how alike are the neighboring pixels and generating *feature representations*. What the convolutional layers see from the picture is invariant to distortion in some degree. For example, a convolutional neural network could predict the same result even if the input image has shift in color, rotated or rescaled. Moreover, convolutional layers have fewer weights, thus easier to train.

25.2 Building Blocks of Convolutional Neural Networks

The simplest use case of a convolutional neural network is for classification. You will find it to contain three types of layers:

1. Convolutional layers
2. Pooling layers
3. Fully-connected layers

Neurons on a convolutional layer is called the filter. Usually it is a 2D convolutional layer in image application. The filter is a 2D patch (e.g., 3×3 pixels) that is applied on the input image pixels. The size of this 2D patch is also called the receptive field, meaning how large a portion of the image it can see at a time.

The filter of a convolutional layer is to multiply with the input pixels, and then sum up the result. This result is one pixel value at the output. The filter will move around the input image to fill out all pixel values at the output. Usually multiple filters are applied to the same input, producing multiple output tensors. These output tensors are called the *feature maps* produced by this layer. They are stacked together as one tensor and pass on to the next layer as input.

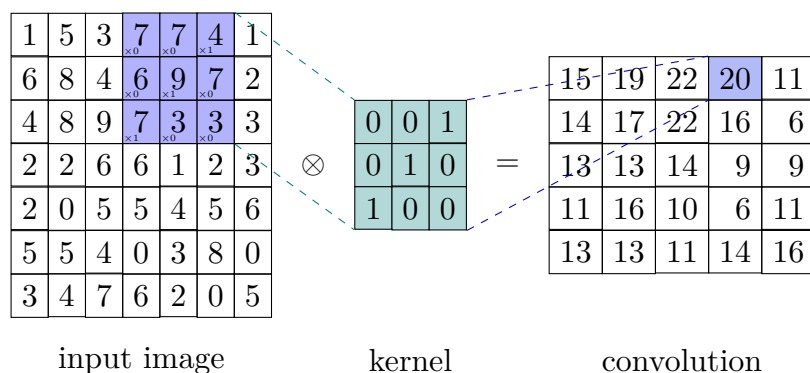


Figure 25.1: Example of a Filter Applied to a Two-Dimensional Input to Create a Feature Map

The output of a convolutional layer is called feature maps because usually it learned about the features of the input image. For example, whether there are vertical lines at the position. Learning the features from pixels is to help understanding the image at a higher

level. Multiple convolutional layers are stacked together in order to infer higher level features from lower level details.

Pooling layer is to *downsample* the previous layer's feature map. It is usually used after a convolutional layer to consolidate features learned. It can compress and generalize the feature representations. A pooling layer also has a receptive field and usually it is to take the average (average pooling) or the maximum (max pooling) over all values on the receptive field.

Fully connected layers are usually the final layers in a network. It is to take the features consolidated by previous convolutional and pooling layers as input to produce prediction. There might be multiple fully connected layers stacked together. In the case of classification, you usually see the output of the final fully connected layer applied with a softmax function to produce probability-like classification.

25.3 An Example of Convolutional Neural Network

The following is a program to do image classification on the CIFAR-10 dataset.

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision

transform = torchvision.transforms.Compose([torchvision.transforms.ToTensor()])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                       download=True, transform=transform)
testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=True, transform=transform)

batch_size = 32
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size, shuffle=True)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size, shuffle=True)

class CIFAR10Model(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=(3,3), stride=1, padding=1)
        self.act1 = nn.ReLU()
        self.drop1 = nn.Dropout(0.3)

        self.conv2 = nn.Conv2d(32, 32, kernel_size=(3,3), stride=1, padding=1)
        self.act2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(kernel_size=(2, 2))

        self.flat = nn.Flatten()

        self.fc3 = nn.Linear(8192, 512)
        self.act3 = nn.ReLU()
        self.drop3 = nn.Dropout(0.5)
```

```

        self.fc4 = nn.Linear(512, 10)

    def forward(self, x):
        # input 3x32x32, output 32x32x32
        x = self.act1(self.conv1(x))
        x = self.drop1(x)
        # input 32x32x32, output 32x32x32
        x = self.act2(self.conv2(x))
        # input 32x32x32, output 32x16x16
        x = self.pool2(x)
        # input 32x16x16, output 8192
        x = self.flat(x)
        # input 8192, output 512
        x = self.act3(self.fc3(x))
        x = self.drop3(x)
        # input 512, output 10
        x = self.fc4(x)
        return x

model = CIFAR10Model()
loss_fn = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)

n_epochs = 20
for epoch in range(n_epochs):
    for inputs, labels in trainloader:
        # forward, backward, and then weight update
        y_pred = model(inputs)
        loss = loss_fn(y_pred, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    acc = 0
    count = 0
    for inputs, labels in testloader:
        y_pred = model(inputs)
        acc += (torch.argmax(y_pred, 1) == labels).float().sum()
        count += len(labels)
    acc /= count
    print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))

torch.save(model.state_dict(), "cifar10model.pth")

```

Listing 25.1: Classification model on CIFAR-10 image dataset

The CIFAR-10 dataset provides images in 32×32 pixels in RGB color (i.e., 3 color channels). There are 10 classes, labeled in integers 0 to 9. Whenever you are working on PyTorch neural network models for images, you will find the sister library `torchvision` useful. In the above, you used it to download the CIFAR-10 dataset from the Internet and transform it into a PyTorch tensor:

```
...
transform = torchvision.transforms.Compose([torchvision.transforms.ToTensor()])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                         download=True, transform=transform)
testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                         download=True, transform=transform)
```

Listing 25.2: Load dataset and transform into PyTorch tensor

You also used a `DataLoader` in PyTorch to help creating batches for training. Training is to optimize the cross-entropy loss of the model, using stochastic gradient descent. It is a classification model, hence accuracy of classification is more intuitive than cross-entropy, which is computed at the end of each epoch, by comparing the maximum value in the output logit to the dataset's labels:

```
...
acc += (torch.argmax(y_pred, 1) == labels).float().sum()
```

Listing 25.3: Computing accuracy from logit output

It takes time to run the program above to train the network. This network should be able to achieve above 70% accuracy in classification.

It is typical in a network for image classification to be comprised of convolutional layers at an early stage, with dropout and pooling layers interleaved. Then, at a later stage, the output from convolutional layers is flattened and processed by some fully connected layers.

25.4 What Are in Feature Maps?

There are two convolutional layers in the network defined above. They are both defined with kernel size of 3×3 , hence it is looking at 9 pixels at a time to produce one output pixel. Note that the first convolutional layer is taking the RGB image as input. Hence each pixel has three channels. The second convolutional layer is taking a feature map with 32 channels as input. Each “pixel” as it sees will have 32 values. Thus the second convolutional layer has more parameters even they have the same receptive field.

Let's see what is in the feature map. Let's say we pick one input sample from the training set:

```
import matplotlib.pyplot as plt

plt.imshow(trainset.data[7])
plt.show()
```

Listing 25.4: Showing one image from the dataset

You should see that this is an image of a horse, in 32×32 pixels with RGB channels:

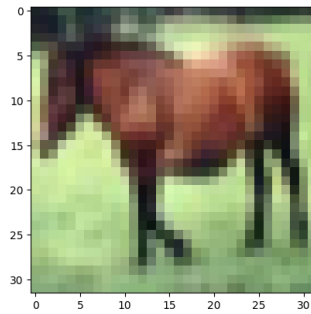


Figure 25.2: An example image

First, you need to convert this into a PyTorch tensor and make it a batch of one image. PyTorch models expect each image as a tensor in the format of (channel, height, width) but the data you read is in the format of (height, width, channel). If you use `torchvision` to transform the image into PyTorch tensors, this format conversion is done automatically. Otherwise, you need to *permute* the dimensions before use.

Afterward, pass it on through the model's first convolution layer and capture the output. You need to tell PyTorch that no gradient is needed for this calculation as you are not going to optimize the model weight:

```
X = torch.tensor([trainset.data[7]], dtype=torch.float32).permute(0,3,1,2)
model.eval()
with torch.no_grad():
    feature_maps = model.conv1(X)
```

Listing 25.5: Extracting feature map from a convolutional layer

The feature maps are in one tensor. You can visualize them using matplotlib:

```
fig, ax = plt.subplots(4, 8, sharex=True, sharey=True, figsize=(16,8))
for i in range(0, 32):
    row, col = i//8, i%8
    ax[row][col].imshow(feature_maps[0][i])
plt.show()
```

Listing 25.6: Visualizing the feature map

You may see the following:

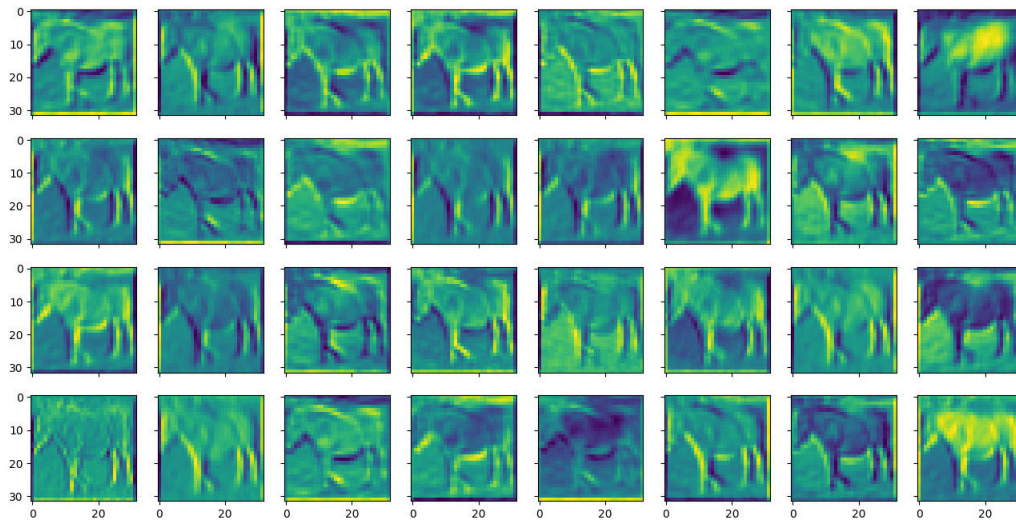


Figure 25.3: Feature map from an image

You can see that they are called feature maps because they are highlighting certain features from the input image. A feature is identified using a small window (in this case, over a 3×3 pixels filter). The input image has three color channels. Each channel has a different filter applied, and their results are combined for an output feature.

You can similarly display the feature map from the output of the second convolutional layer as follows:

```
X = torch.tensor([trainset.data[7]], dtype=torch.float32).permute(0,3,1,2)

model.eval()
with torch.no_grad():
    feature_maps = model.act1(model.conv1(X))
    feature_maps = model.drop1(feature_maps)
    feature_maps = model.conv2(feature_maps)

fig, ax = plt.subplots(4, 8, sharex=True, sharey=True, figsize=(16,8))
for i in range(0, 32):
    row, col = i//8, i%8
    ax[row][col].imshow(feature_maps[0][i])
plt.show()
```

Listing 25.7: Extracting feature map from the second convolutional layer

Which shows:

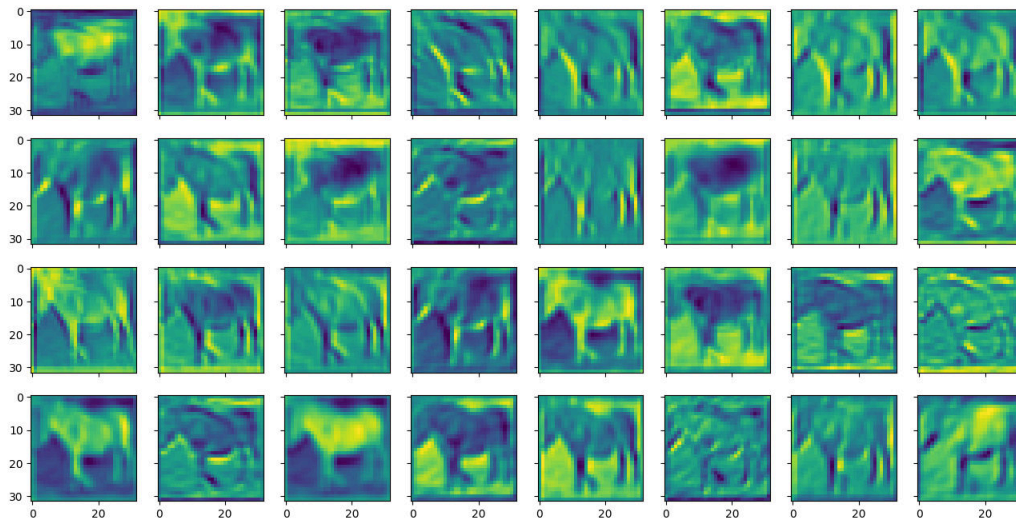


Figure 25.4: Feature map from the second convolutional layer

Compared to the output of the first convolutional layer, the feature maps from the second convolutional layer looks blurry and more abstract. But these are more useful for the model to identify the objects.

Putting everything together, the code below loads the saved model from the previous section and generate the feature maps:

```
import torch
import torch.nn as nn
import torchvision
import matplotlib.pyplot as plt

trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True)

class CIFAR10Model(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=(3,3), stride=1, padding=1)
        self.act1 = nn.ReLU()
        self.drop1 = nn.Dropout(0.3)

        self.conv2 = nn.Conv2d(32, 32, kernel_size=(3,3), stride=1, padding=1)
        self.act2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(kernel_size=(2, 2))

        self.flat = nn.Flatten()

        self.fc3 = nn.Linear(8192, 512)
        self.act3 = nn.ReLU()
        self.drop3 = nn.Dropout(0.5)

        self.fc4 = nn.Linear(512, 10)

    def forward(self, x):
```



```

        # input 3x32x32, output 32x32x32
        x = self.act1(self.conv1(x))
        x = self.drop1(x)
        # input 32x32x32, output 32x32x32
        x = self.act2(self.conv2(x))
        # input 32x32x32, output 32x16x16
        x = self.pool2(x)
        # input 32x16x16, output 8192
        x = self.flat(x)
        # input 8192, output 512
        x = self.act3(self.fc3(x))
        x = self.drop3(x)
        # input 512, output 10
        x = self.fc4(x)
        return x

model = CIFAR10Model()
model.load_state_dict(torch.load("cifar10model.pth"))

plt.imshow(trainset.data[7])
plt.show()

X = torch.tensor([trainset.data[7]], dtype=torch.float32).permute(0,3,1,2)
model.eval()
with torch.no_grad():
    feature_maps = model.conv1(X)
fig, ax = plt.subplots(4, 8, sharex=True, sharey=True, figsize=(16,8))
for i in range(0, 32):
    row, col = i//8, i%8
    ax[row][col].imshow(feature_maps[0][i])
plt.show()

with torch.no_grad():
    feature_maps = model.act1(model.conv1(X))
    feature_maps = model.drop1(feature_maps)
    feature_maps = model.conv2(feature_maps)
fig, ax = plt.subplots(4, 8, sharex=True, sharey=True, figsize=(16,8))
for i in range(0, 32):
    row, col = i//8, i%8
    ax[row][col].imshow(feature_maps[0][i])
plt.show()

```

Listing 25.8: Load a saved model and extract feature maps

25.5 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Vincent Dumoulin and Francesco Visin. “A guide to convolution arithmetic for deep learning”, 2016.

<https://arxiv.org/abs/1603.07285>

Training a Classifier. PyTorch tutorials.

https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html

Conv2d. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. Chapter 9, “Convolutional Networks”.

In: *Deep Learning*. MIT Press, 2016.

<https://www.amazon.com/dp/0262035618>

(Online version at <http://www.deeplearningbook.org>).

25.6 Summary

In this chapter, you learned how to use convolutional neural network to handle image input and how to visualize the feature maps.

Specifically, you learned:

- ▷ The structure of a typical convolutional neural networks
- ▷ What is the effect of the filter size to a convolutional layer
- ▷ What is the effect of stacking convolutional layers in a network
- ▷ How to extract and visualize feature maps from a convolutional neural network

In the next chapter, you will learn the classic example of using CNN to recognize handwritten digits.

Handwritten Digit Recognition with LeNet5 Model in PyTorch

26

A popular demonstration of the capability of deep learning techniques is object recognition in image data. The “hello world” of object recognition for machine learning and deep learning is the MNIST dataset for handwritten digit recognition. In this chapter, you will discover how to develop a deep learning model to achieve near state-of-the-art performance on the MNIST handwritten digit recognition task in PyTorch. After completing this chapter, you will know:

- ▷ How to load the MNIST dataset using torchvision
- ▷ How to develop and evaluate a baseline neural network model for the MNIST problem
- ▷ How to implement and evaluate a simple Convolutional Neural Network for MNIST
- ▷ How to implement a state-of-the-art deep learning model for MNIST

Let’s get started.

Overview

This chapter is divided into five parts; they are:

- ▷ The MNIST Handwritten Digit Recognition Problem
- ▷ Loading the MNIST Dataset in PyTorch
- ▷ Baseline Model with Multilayer Perceptrons
- ▷ Simple Convolutional Neural Network for MNIST
- ▷ LeNet5 for MNIST

26.1 The MNIST Handwritten Digit Recognition Problem

The MNIST problem is a classic problem that can demonstrate the power of convolutional neural networks. The MNIST dataset was developed by Yann LeCun, Corinna Cortes, and Christopher Burges for evaluating machine learning models on the handwritten digit classification problem. The dataset was constructed from a number of scanned document datasets available from the National Institute of Standards and Technology (NIST). This is where the name for the dataset comes from, the Modified NIST or MNIST dataset.

Images of digits were taken from a variety of scanned documents, normalized in size, and centered. This makes it an excellent dataset for evaluating models, allowing the developer to focus on machine learning with minimal data cleaning or preparation required. Each image is a 28×28 -pixel square (784 pixels total) in grayscale. A standard split of the dataset is used to evaluate and compare models, where 60,000 images are used to train a model, and a separate set of 10,000 images are used to test it.

The goal of this problem is to identify the digits on the image. There are ten digits (0 to 9) or ten classes to predict. The state-of-the-art prediction accuracy is at 99.8% level, achieved with large convolutional neural networks.

26.2 Loading the MNIST Dataset in PyTorch

The `torchvision` library is a sister project of PyTorch that provides specialized functions for computer vision tasks. There is a function in `torchvision` that can download the MNIST dataset for use with PyTorch. The dataset is downloaded the first time this function is called and stored locally, so you don't need to download again in the future. Below is a little script to download and visualize the first 16 images in the training subset of the MNIST dataset.

```
import matplotlib.pyplot as plt
import torchvision

train = torchvision.datasets.MNIST('./data', train=True, download=True)

fig, ax = plt.subplots(4, 4, sharex=True, sharey=True)
for i in range(4):
    for j in range(4):
        ax[i][j].imshow(train.data[4*i+j], cmap="gray")
plt.show()
```

Listing 26.1: Loading the MNIST dataset and display some samples

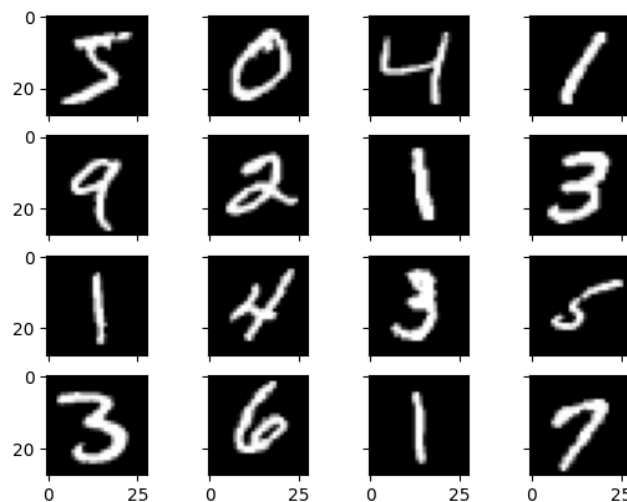


Figure 26.1: Examples from the MNIST dataset

26.3 Baseline Model with Multilayer Perceptrons

Do you really need a complex model like a convolutional neural network to get the best results with MNIST? You can get good results using a very simple neural network model with a single hidden layer. In this section, you will create a simple multilayer perceptron model that achieves accuracy of 99.81%. You will use this as a baseline for comparison to more complex convolutional neural network models. First, let's check what the data looks like:

```
import torchvision

# Load MNIST data
train = torchvision.datasets.MNIST('data', train=True, download=True)
test = torchvision.datasets.MNIST('data', train=False, download=True)
print(train.data.shape, train.targets.shape)
print(test.data.shape, test.targets.shape)
```

Listing 26.2: Showing the shape of the MNIST dataset

You should see:

```
torch.Size([60000, 28, 28]) torch.Size([60000])
torch.Size([10000, 28, 28]) torch.Size([10000])
```

Output 26.1: Shape of the MNIST dataset

The training dataset is structured as a 3-dimensional array of instance, image height, and image width. For a multilayer perceptron model, you must reduce the images down into a vector of pixels. In this case, the 28×28 -sized images will be 784 pixel input vectors. You can do this transform easily using the `reshape()` function.

The pixel values are grayscale between 0 and 255. It is almost always a good idea to perform some scaling of input values when using neural network models. Because the scale is well known and well behaved, you can very quickly normalize the pixel values to the range 0 and 1 by dividing each value by the maximum of 255.

In the following, you transform the dataset, convert to floating point, and normalize them by scaling floating point values and you can normalize them easily in the next step.

```
# each sample becomes a vector of values 0-1
X_train = train.data.reshape(-1, 784).float() / 255.0
y_train = train.targets
X_test = test.data.reshape(-1, 784).float() / 255.0
y_test = test.targets
```

Listing 26.3: Rescaling the dataset

The output targets `y_train` and `y_test` are labels in the form of integers from 0 to 9. This is a multiclass classification problem. You can convert these labels into one-hot encoding or keep them as integer labels like this case. You are going to use the cross-entropy function to evaluate the model performance and the PyTorch implementation of cross-entropy function can be applied on one-hot encoded targets or integer labeled targets.

You are now ready to create your simple neural network model. You will define your model in a PyTorch `Module` class.

```
class Baseline(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(784, 784)
        self.act1 = nn.ReLU()
        self.layer2 = nn.Linear(784, 10)

    def forward(self, x):
        x = self.act1(self.layer1(x))
        x = self.layer2(x)
        return x
```

Listing 26.4: Baseline MLP model for classification of the MNIST dataset

The model is a simple neural network with one hidden layer with the same number of neurons as there are inputs (784). A rectifier activation function is used for the neurons in the hidden layer. The output of this model are *logits*, meaning they are real numbers which can be transformed into probability-like values using a softmax function. You do not apply the softmax function explicitly because the cross-entropy function will do that for you.

You will use the stochastic gradient descent algorithm (with learning rate set to 0.01) to optimize this model. The training loop is as follows:

```
model = Baseline()

optimizer = optim.SGD(model.parameters(), lr=0.01)
loss_fn = nn.CrossEntropyLoss()
loader = torch.utils.data.DataLoader(list(zip(X_train, y_train)),
                                     shuffle=True, batch_size=100)

n_epochs = 10
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
    y_pred = model(X_test)
    acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
    print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))
```

Listing 26.5: Training the baseline model with SGD

The MNIST dataset is small. This example should complete in a minute, with the output below. This simple network can produce accuracy at 92%.

```
Epoch 0: model accuracy 84.11%
Epoch 1: model accuracy 87.53%
Epoch 2: model accuracy 89.01%
Epoch 3: model accuracy 89.76%
Epoch 4: model accuracy 90.29%
Epoch 5: model accuracy 90.69%
Epoch 6: model accuracy 91.10%
Epoch 7: model accuracy 91.48%
Epoch 8: model accuracy 91.74%
Epoch 9: model accuracy 91.96%
```

Output 26.2: Progress of the model training

Below is the complete code for the above multilayer perceptron classification on MNIST dataset.

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision

# Load MNIST data
train = torchvision.datasets.MNIST('data', train=True, download=True)
test = torchvision.datasets.MNIST('data', train=False, download=True)

# each sample becomes a vector of values 0-1
X_train = train.data.reshape(-1, 784).float() / 255.0
y_train = train.targets
X_test = test.data.reshape(-1, 784).float() / 255.0
y_test = test.targets

class Baseline(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(784, 784)
        self.act1 = nn.ReLU()
        self.layer2 = nn.Linear(784, 10)

    def forward(self, x):
        x = self.act1(self.layer1(x))
        x = self.layer2(x)
        return x

model = Baseline()

optimizer = optim.SGD(model.parameters(), lr=0.01)
loss_fn = nn.CrossEntropyLoss()
loader = torch.utils.data.DataLoader(list(zip(X_train, y_train)),
                                     shuffle=True, batch_size=100)

n_epochs = 10
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
```

```

        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
    y_pred = model(X_test)
    acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
    print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))

```

Listing 26.6: Complete code of building MNIST classification baseline model

26.4 Simple Convolutional Neural Network for MNIST

Now that you have seen how to use multilayer perceptron model to classify MNIST dataset. Let's move on to try a convolutional neural network model. In this section, you will create a simple CNN for MNIST that demonstrates how to use all the aspects of a modern CNN implementation, including convolutional layers, pooling layers, and dropout layers.

In PyTorch, convolutional layers are supposed to work on images. Tensors for images should be the pixel values with the dimensions (sample, channel, height, width) but when you load images using libraries such as PIL, the pixels are usually presented as array of dimensions (height, width, channel). The conversion to a proper tensor format can be done using a transform from the torchvision library.

```

...
transform = torchvision.transforms.Compose([
    torchvision.transforms.ToTensor(),
    torchvision.transforms.Normalize((0,), (128,)),
])
train = torchvision.datasets.MNIST('data', train=True, download=True, transform=transform)
test = torchvision.datasets.MNIST('data', train=False, download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(train, shuffle=True, batch_size=100)
testloader = torch.utils.data.DataLoader(test, shuffle=True, batch_size=100)

```

Listing 26.7: Creating a DataLoader for MNIST dataset

You need to use `DataLoader` because the transform is applied when you read the data from the `DataLoader`.

Next, define your neural network model. Convolutional neural networks are more complex than standard multilayer perceptrons, so you will start by using a simple structure that uses all the elements for state-of-the-art results. Below summarizes the network architecture.

1. The first hidden layer is a convolutional layer, `nn.Conv2d()`. The layer turns a grayscale image into 10 feature maps, with the filter size of 5×5 and a ReLU activation function. This is the input layer that expects images with the structure outlined above.
2. Next is a pooling layer that takes the max, `nn.MaxPool2d()`. It is configured with a pool size of 2×2 with stride 1. What it does is to take the maximum in a 2×2

pixel patch per channel and assign the value to the output pixel. The result is a 27×27 -pixels feature map per channel.

3. The next layer is a regularization layer using dropout, `nn.Dropout()`. It is configured to randomly exclude 20% of neurons in the layer in order to reduce overfitting.
4. Next is a layer that converts the 2D matrix data to a vector, using `nn.Flatten`. There are 10 channels from its input and each channel's feature map has size 27×27 . This layer allows the output to be processed by standard, fully connected layers.
5. Next is a fully connected layer with 128 neurons. ReLU activation function is used.
6. Finally, the output layer has ten neurons for the ten classes. You can transform the output into probability-like predictions by applying a softmax function on it.

This model is trained using cross-entropy loss and the Adam optimization algorithm. It is implemented as follows:

```
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(1, 10, kernel_size=5, stride=1, padding=2)
        self.relu1 = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=2, stride=1)
        self.dropout = nn.Dropout(0.2)
        self.flat = nn.Flatten()
        self.fc = nn.Linear(27*27*10, 128)
        self.relu2 = nn.ReLU()
        self.output = nn.Linear(128, 10)

    def forward(self, x):
        x = self.relu1(self.conv(x))
        x = self.pool(x)
        x = self.dropout(x)
        x = self.relu2(self.fc(self.flat(x)))
        x = self.output(x)
        return x

model = CNN()

optimizer = optim.Adam(model.parameters(), lr=0.01)
loss_fn = nn.CrossEntropyLoss()

n_epochs = 10
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in trainloader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
```

```

acc = 0
count = 0
for X_batch, y_batch in testloader:
    y_pred = model(X_batch)
    acc += (torch.argmax(y_pred, 1) == y_batch).float().sum()
    count += len(y_batch)
acc = acc / count
print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))

```

Listing 26.8: Training a convolutional neural network model for MNIST

Running the above takes a few minutes and produces the following:

```

Epoch 0: model accuracy 81.74%
Epoch 1: model accuracy 85.38%
Epoch 2: model accuracy 86.37%
Epoch 3: model accuracy 87.75%
Epoch 4: model accuracy 88.00%
Epoch 5: model accuracy 88.17%
Epoch 6: model accuracy 88.81%
Epoch 7: model accuracy 88.34%
Epoch 8: model accuracy 88.86%
Epoch 9: model accuracy 88.75%

```

Output 26.3: Training progress of the CNN

Not the best result but this demonstrates how convolutional layer works. Below is the complete code for using the simple convolutional network.

```

import torch
import torch.nn as nn
import torch.optim as optim
import torchvision

# Load MNIST data
transform = torchvision.transforms.Compose([
    torchvision.transforms.ToTensor(),
    torchvision.transforms.Normalize((0,), (128,)),
])
train = torchvision.datasets.MNIST('data', train=True, download=True, transform=transform)
test = torchvision.datasets.MNIST('data', train=False, download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(train, shuffle=True, batch_size=100)
testloader = torch.utils.data.DataLoader(test, shuffle=True, batch_size=100)

class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(1, 10, kernel_size=5, stride=1, padding=2)
        self.relu1 = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=2, stride=1)
        self.dropout = nn.Dropout(0.2)
        self.flat = nn.Flatten()
        self.fc = nn.Linear(27*27*10, 128)
        self.relu2 = nn.ReLU()

```

```

        self.output = nn.Linear(128, 10)

    def forward(self, x):
        x = self.relu1(self.conv(x))
        x = self.pool(x)
        x = self.dropout(x)
        x = self.relu2(self.fc(self.flat(x)))
        x = self.output(x)
        return x

model = CNN()

optimizer = optim.Adam(model.parameters())
loss_fn = nn.CrossEntropyLoss()

n_epochs = 10
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in trainloader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
    acc = 0
    count = 0
    for X_batch, y_batch in testloader:
        y_pred = model(X_batch)
        acc += (torch.argmax(y_pred, 1) == y_batch).float().sum()
        count += len(y_batch)
    acc = acc / count
    print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))

```

Listing 26.9: Complete code for classification using CNN

26.5 LeNet5 for MNIST

The previous model has only one convolutional layer. Of course, you can add more to make a deeper model. One of the earliest demonstration of the effectiveness of convolutional layers in neural networks is the “LeNet5” model. This model is developed to solve the MNIST classification problem. It has three convolutional layers and two fully connected layer to make up five trainable layers in the model, as it is named.

At the time it was developed, using hyperbolic tangent function as activation is common. Hence it is used here. This model is implemented as follows:

```

class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()

```

```

self.conv1 = nn.Conv2d(1, 6, kernel_size=5, stride=1, padding=2)
self.act1 = nn.Tanh()
self.pool1 = nn.AvgPool2d(kernel_size=2, stride=2)

self.conv2 = nn.Conv2d(6, 16, kernel_size=5, stride=1, padding=0)
self.act2 = nn.Tanh()
self.pool2 = nn.AvgPool2d(kernel_size=2, stride=2)

self.conv3 = nn.Conv2d(16, 120, kernel_size=5, stride=1, padding=0)
self.act3 = nn.Tanh()

self.flat = nn.Flatten()
self.fc1 = nn.Linear(1*1*120, 84)
self.act4 = nn.Tanh()
self.fc2 = nn.Linear(84, 10)

def forward(self, x):
    # input 1x28x28, output 6x28x28
    x = self.act1(self.conv1(x))
    # input 6x28x28, output 6x14x14
    x = self.pool1(x)
    # input 6x14x14, output 16x10x10
    x = self.act2(self.conv2(x))
    # input 16x10x10, output 16x5x5
    x = self.pool2(x)
    # input 16x5x5, output 120x1x1
    x = self.act3(self.conv3(x))
    # input 120x1x1, output 84
    x = self.act4(self.fc1(self.flat(x)))
    # input 84, output 10
    x = self.fc2(x)
    return x

```

Listing 26.10: LeNet5 model implemented in PyTorch

Compare to the previous model, LeNet5 does not have Dropout layer (because Dropout layer was invented several years after LeNet5) and use average pooling instead of max pooling (i.e., for a patch of 2×2 pixels, it is taking average of the pixel values instead of taking the maximum). But the most notable characteristic of LeNet5 model is that it uses strides and paddings to reduce the image size from 28×28 pixel down to 1×1 pixel while increasing the number of channels from a one (grayscale) into 120.

Padding means to add pixels of value 0 at the border of the image to make it a bit larger. Without padding, the output of a convolutional layer will be smaller than its input. The stride parameter controls how much the filter should move to produce the next pixel in the output. Usually it is 1 to preserve the same size. If it is larger than 1, the output is a *downsampling* of the input. Hence you see in the LeNet5 model, stride 2 was used in the pooling layers to make, for example, a 28×28 -pixel image into 14×14 .

Training this model is same as training the previous convolutional network model, as follows:

```

...
model = LeNet5()

optimizer = optim.Adam(model.parameters())
loss_fn = nn.CrossEntropyLoss()

n_epochs = 10
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in trainloader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
    acc = 0
    count = 0
    for X_batch, y_batch in testloader:
        y_pred = model(X_batch)
        acc += (torch.argmax(y_pred, 1) == y_batch).float().sum()
        count += len(y_batch)
    acc = acc / count
    print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))

```

Listing 26.11: Training the LeNet5 model

Running this you may see:

```

Epoch 0: model accuracy 89.46%
Epoch 1: model accuracy 93.14%
Epoch 2: model accuracy 94.69%
Epoch 3: model accuracy 95.84%
Epoch 4: model accuracy 96.43%
Epoch 5: model accuracy 96.99%
Epoch 6: model accuracy 97.14%
Epoch 7: model accuracy 97.66%
Epoch 8: model accuracy 98.05%
Epoch 9: model accuracy 98.22%

```

Output 26.4: Progress of training the LeNet5 model

Here, we achieved accuracy beyond 98%.

The following is the complete code.

```

import torch
import torch.nn as nn
import torch.optim as optim
import torchvision

# Load MNIST data
transform = torchvision.transforms.Compose([

```

```

    torchvision.transforms.ToTensor(),
    torchvision.transforms.Normalize((0,), (128,)),
])
train = torchvision.datasets.MNIST('data', train=True, download=True, transform=transform)
test = torchvision.datasets.MNIST('data', train=False, download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(train, shuffle=True, batch_size=100)
testloader = torch.utils.data.DataLoader(test, shuffle=True, batch_size=100)

class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, kernel_size=5, stride=1, padding=2)
        self.act1 = nn.Tanh()
        self.pool1 = nn.AvgPool2d(kernel_size=2, stride=2)

        self.conv2 = nn.Conv2d(6, 16, kernel_size=5, stride=1, padding=0)
        self.act2 = nn.Tanh()
        self.pool2 = nn.AvgPool2d(kernel_size=2, stride=2)

        self.conv3 = nn.Conv2d(16, 120, kernel_size=5, stride=1, padding=0)
        self.act3 = nn.Tanh()

        self.flat = nn.Flatten()
        self.fc1 = nn.Linear(1*1*120, 84)
        self.act4 = nn.Tanh()
        self.fc2 = nn.Linear(84, 10)

    def forward(self, x):
        # input 1x28x28, output 6x28x28
        x = self.act1(self.conv1(x))
        # input 6x28x28, output 6x14x14
        x = self.pool1(x)
        # input 6x14x14, output 16x10x10
        x = self.act2(self.conv2(x))
        # input 16x10x10, output 16x5x5
        x = self.pool2(x)
        # input 16x5x5, output 120x1x1
        x = self.act3(self.conv3(x))
        # input 120x1x1, output 84
        x = self.act4(self.fc1(self.flat(x)))
        # input 84, output 10
        x = self.fc2(x)
        return x

model = LeNet5()

optimizer = optim.Adam(model.parameters())
loss_fn = nn.CrossEntropyLoss()

n_epochs = 10
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in trainloader:
        y_pred = model(X_batch)

```

```

        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
# Validation
model.eval()
acc = 0
count = 0
for X_batch, y_batch in testloader:
    y_pred = model(X_batch)
    acc += (torch.argmax(y_pred, 1) == y_batch).float().sum()
    count += len(y_batch)
acc = acc / count
print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))

```

Listing 26.12: Complete code of classification using LeNet5 model

26.6 Resources on MNIST

The MNIST dataset is very well studied. Below are some additional resources you might want to look into.

Yann LeCun, Corinna Cortes, and Christopher J. C. Burges. *The MNIST database of handwritten digits*.

<http://yann.lecun.com/exdb/mnist/>

Rodrigo Benenson. *What is the class of this image?* Classification datasets results, 2016.

https://rodrigob.github.io/are_we_there_yet/build/classification_datasets_results.html

Digit Recognizer: Learn computer vision fundamentals with the famous MNIST data. Kaggle.

<https://www.kaggle.com/c/digit-recognizer>

Hubert Eichner. *Neural Net for Handwritten Digit Recognition in JavaScript*.

<http://myselfph.de/neuralNet.html>

Yann LeCun, Leon Bottou, Yoshua Bengio, and Patrick Haffner. “Gradient-Based Learning Applied to Document Recognition”. In: *Proceedings of the IEEE*. Nov. 1998.

<http://yann.lecun.com/exdb/publis/pdf/lecun-01a.pdf>

Yann LeCun. *LeNet-5 convolutional neural networks*.

<http://yann.lecun.com/exdb/lenet/>

26.7 Summary

In this chapter, you discovered the MNIST handwritten digit recognition problem and deep learning models developed in Python using the PyTorch library that are capable of achieving excellent results. Working through this chapter, you learned:

- ▷ How to load the MNIST dataset in PyTorch with torchvision
- ▷ How to convert the MNIST dataset into PyTorch tensors for consumption by a convolutional neural network

- ▷ How to use PyTorch to create convolutional neural network models for MNIST
- ▷ How to implement the LeNet5 model for MNIST classification

In the next chapter, you will learn about LSTM, the most common type of recurrent neural network.

LSTM for Time Series Prediction in PyTorch

27

Long Short-Term Memory (LSTM) is a structure that can be used in neural network. It is a type of recurrent neural network (RNN) that expects the input in the form of a sequence of features. It is useful for data such as time series or string of text. In this chapter, you will learn about LSTM networks. In particular,

- ▷ What is LSTM and how they are different
- ▷ How to develop LSTM network for time series prediction
- ▷ How to train a LSTM network

Let's get started.

Overview

This chapter is divided into three parts; they are

- ▷ Overview of LSTM Network
- ▷ LSTM for Time Series Prediction
- ▷ Training and Verifying Your LSTM Network

27.1 Overview of LSTM Network

LSTM cell is a building block that you can use to build a larger neural network. While the common building block such as fully-connected layer are merely matrix multiplication of the weight tensor and the input to produce an output tensor, LSTM module is much more complex.

A typical LSTM cell is illustrated as follows

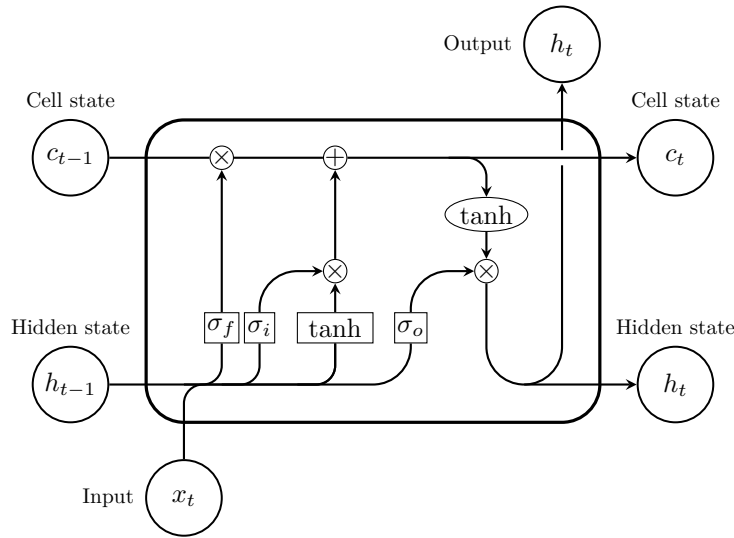


Figure 27.1: LSTM cell

It takes one time step of an input tensor x as well as a cell memory c and a hidden state h . The cell memory and hidden state can be initialized to zero at the beginning. Then within the LSTM cell, x , c , and h will be multiplied by separate weight tensors and pass through some activation functions a few times. The result is the updated cell memory and hidden state. These updated c and h will be used on the *next time step* of the input tensor. Until the end of the last time step, the output of the LSTM cell will be its cell memory and hidden state.

Specifically, the equation of one LSTM cell is as follows:

$$f_t = \sigma_f(W_f x_t + U_f h_{t-1} + b_f)$$

$$i_t = \sigma_i(W_i x_t + U_i h_{t-1} + b_i)$$

$$o_t = \sigma_o(W_o x_t + U_o h_{t-1} + b_o)$$

$$\tilde{c}_t = \sigma_c(W_c x_t + U_c h_{t-1} + b_c)$$

$$c_t = f_t \otimes c_{t-1} + i_t \otimes \tilde{c}_t$$

$$h_t = o_t \otimes \sigma_h(c_t)$$

Where W , U , b are trainable parameters of the LSTM cell. Each equation above is computed for each time step, hence with subscript t . These trainable parameters are *reused* for all the time steps. This nature of shared parameter bring the memory power to the LSTM.

Note that the above is only one design of the LSTM. There are multiple variations in the literature.

Since the LSTM cell expects the input x in the form of multiple time steps, each input sample should be a 2D tensors: One dimension for time and another dimension for features. The power of an LSTM cell depends on the size of the hidden state or cell memory, which usually has a larger dimension than the number of features in the input.

27.2 LSTM for Time Series Prediction

Let's see how LSTM can be used to build a time series prediction neural network with an example.

The problem you will look at in this chapter is the international airline passengers prediction problem. This is a problem where, given a year and a month, the task is to predict the number of international airline passengers in units of 1,000. The data ranges from January 1949 to December 1960, or 12 years, with 144 observations.

It is a regression problem. That is, given the number of passengers (in unit of 1,000) the recent months, what is the number of passengers the next month. The dataset has only one feature: The number of passengers.

Let's start by reading the data. The data can be downloaded here.

▷ <https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv>

Save this file as `airline-passengers.csv` in the local directory for the following. Below is a sample of the first few lines of the file:

```
"Month","Passengers"  
"1949-01",112  
"1949-02",118  
"1949-03",132  
"1949-04",129
```

Output 27.1: The time series dataset in the CSV file

The data has two columns, the month and the number of passengers. Since the data are arranged in chronological order, you can take only the number of passenger to make a single-feature time series. Below you will use pandas library to read the CSV file and convert it into a 2D NumPy array, then plot it using matplotlib:

```
import matplotlib.pyplot as plt  
import pandas as pd  
  
df = pd.read_csv('airline-passengers.csv')  
timeseries = df[["Passengers"]].values.astype('float32')  
  
plt.plot(timeseries)  
plt.show()
```

Listing 27.1: Extracting and plotting the time series

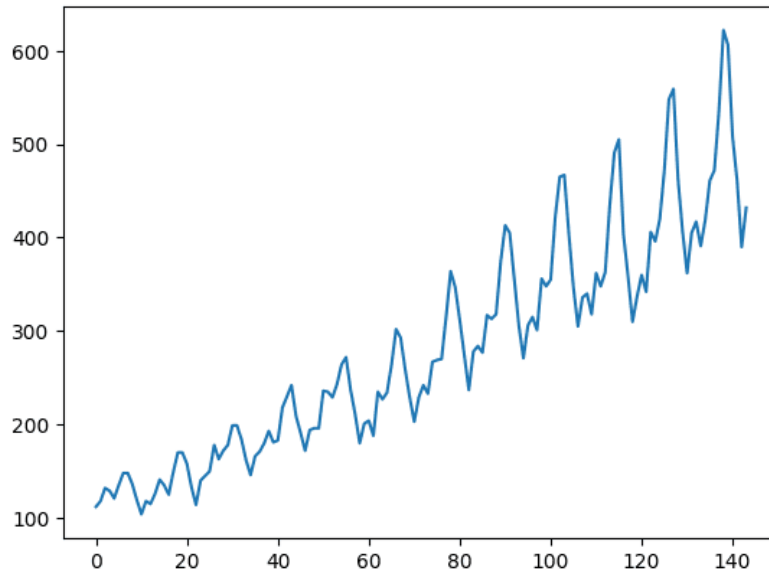


Figure 27.2: Time series as plotted

This time series has 144 time steps. You can see from the plot that there is an upward trend. There are also some periodicity in the dataset that corresponds to the summer holiday period in the northern hemisphere. Usually a time series should be “detrended” to remove the linear trend component and normalized before processing. For simplicity, these are skipped in this project.

To demonstrate the predictive power of our model, the time series is splitted into training and test sets. Unlike other dataset, usually time series data are splitted without shuffling. That is, the training set is the first half of time series and the remaining will be used as the test set. This can be easily done on a NumPy array:

```
# train-test split for time series
train_size = int(len(timeseries) * 0.67)
test_size = len(timeseries) - train_size
train, test = timeseries[:train_size], timeseries[train_size:]
```

Listing 27.2: Train-test split of the time series dataset

The more complicated problem is how do you want the network to predict the time series. Usually time series prediction is done on a window. That is, given data from time $t - w$ to time t , you are asked to predict for time $t + 1$ (or deeper into the future). The size of window w governs how much data you are allowed to look at when you make the prediction. This is also called the *look back period*.

On a long enough time series, multiple overlapping window can be created. It is convenient to create a function to generate a dataset of fixed window from a time series. Since the data is going to be used in a PyTorch model, the output dataset should be in PyTorch tensors:

```

import torch

def create_dataset(dataset, lookback):
    """Transform a time series into a prediction dataset

    Args:
        dataset: A numpy array of time series, first dimension is the time steps
        lookback: Size of window for prediction
    """
    X, y = [], []
    for i in range(len(dataset)-lookback):
        feature = dataset[i:i+lookback]
        target = dataset[i+1:i+lookback+1]
        X.append(feature)
        y.append(target)
    return torch.tensor(X), torch.tensor(y)

```

Listing 27.3: Transforming a time series dataset

This function is designed to apply windows on the time series. It is assumed to predict for one time step into the immediate future. It is designed to convert a time series into a tensor of dimensions (window sample, time steps, features). A time series of L time steps can produce roughly L windows (because a window can start from any time step as long as the window does not go beyond the boundary of the time series). Within one window, there are multiple consecutive time steps of values. In each time step, there can be multiple features. In this dataset, there is only one.

It is intentional to produce the *feature* and the *target* the same shape: For a window of three time steps, the feature is the time series from t to $t + 2$ and the target is from $t + 1$ to $t + 3$. What we are interested is $t + 3$ but the information of $t + 1$ to $t + 2$ is useful in training.

Note that the input time series is a 2D array and the output from the `create_dataset()` function will be a 3D tensor. Let's try with `lookback=1`. You can verify the shape of the output tensor as follows:

```

lookback = 1
X_train, y_train = create_dataset(train, lookback=lookback)
X_test, y_test = create_dataset(test, lookback=lookback)
print(X_train.shape, y_train.shape)
print(X_test.shape, y_test.shape)

```

Listing 27.4: Creating dataset tensor for training

which you should see:

```

torch.Size([95, 1, 1]) torch.Size([95, 1, 1])
torch.Size([47, 1, 1]) torch.Size([47, 1, 1])

```

Output 27.2: Shapes of tensors of the training and test set

Now you can build the LSTM model to predict the time series. With `lookback=1`, it is quite surely that the accuracy would not be good for too little clues to predict. But this is a good example to demonstrate the structure of the LSTM model.

The model is created as a class, in which a LSTM layer and a fully-connected layer is used.

```
...
import torch.nn as nn

class AirModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(input_size=1, hidden_size=50, num_layers=1, batch_first=True)
        self.linear = nn.Linear(50, 1)
    def forward(self, x):
        x, _ = self.lstm(x)
        x = self.linear(x)
        return x
```

Listing 27.5: LSTM model

The output of `nn.LSTM()` is a tuple. The first element is the generated hidden states, one for each time step of the input. The second element is the LSTM cell's memory and hidden states, which is not used here.

The LSTM layer is created with option `batch_first=True` because the tensors you prepared is in the dimension of (window sample, time steps, features) and where a batch is created by sampling on the first dimension.

The output of hidden states is further processed by a fully-connected layer to produce a single regression result. Since the output from LSTM is one per each input time step, you can choose to pick only the last timestep's output, which you should have:

```
x, _ = self.lstm(x)
# extract only the last time step
x = x[:, -1, :]
x = self.linear(x)
```

Listing 27.6: Modifying `forward()` to output only the next-step prediction

and the model's output will be the prediction of the next time step. But here, the fully connected layer is applied to each time step. In this design, you should extract only the last time step from the model output as your prediction. However, in this case, the window is 1, there is no difference in these two approaches.

27.3 Training and Verifying Your LSTM Network

Because it is a regression problem, MSE is chosen as the loss function, which is to be minimized by Adam optimizer. In the code below, the PyTorch tensors are combined into a dataset using `torch.utils.data.TensorDataset()` and batch for training is provided by a `DataLoader`. The model performance is evaluated once per 100 epochs, on both the training set and the test set:

```

import numpy as np
import torch.optim as optim
import torch.utils.data as data

model = AirModel()
optimizer = optim.Adam(model.parameters())
loss_fn = nn.MSELoss()
loader = data.DataLoader(data.TensorDataset(X_train, y_train), shuffle=True, batch_size=8)

n_epochs = 2000
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    if epoch % 100 != 0:
        continue
    model.eval()
    with torch.no_grad():
        y_pred = model(X_train)
        train_rmse = np.sqrt(loss_fn(y_pred, y_train))
        y_pred = model(X_test)
        test_rmse = np.sqrt(loss_fn(y_pred, y_test))
    print("Epoch %d: train RMSE %.4f, test RMSE %.4f" % (epoch, train_rmse, test_rmse))

```

Listing 27.7: Training a LSTM network

As the dataset is small, the model should be trained for long enough to learn about the pattern. Over these 2000 epochs trained, you should see the RMSE on both training set and test set decreasing:

```

Epoch 0: train RMSE 225.7571, test RMSE 422.1521
Epoch 100: train RMSE 186.7353, test RMSE 381.3285
Epoch 200: train RMSE 153.3157, test RMSE 345.3290
Epoch 300: train RMSE 124.7137, test RMSE 312.8820
Epoch 400: train RMSE 101.3789, test RMSE 283.7040
Epoch 500: train RMSE 83.0900, test RMSE 257.5325
Epoch 600: train RMSE 66.6143, test RMSE 232.3288
Epoch 700: train RMSE 53.8428, test RMSE 209.1579
Epoch 800: train RMSE 44.4156, test RMSE 188.3802
Epoch 900: train RMSE 37.1839, test RMSE 170.3186
Epoch 1000: train RMSE 32.0921, test RMSE 154.4092
Epoch 1100: train RMSE 29.0402, test RMSE 141.6920
Epoch 1200: train RMSE 26.9721, test RMSE 131.0108
Epoch 1300: train RMSE 25.7398, test RMSE 123.2518
Epoch 1400: train RMSE 24.8011, test RMSE 116.7029
Epoch 1500: train RMSE 24.7705, test RMSE 112.1551
Epoch 1600: train RMSE 24.4654, test RMSE 108.1879
Epoch 1700: train RMSE 25.1378, test RMSE 105.8224

```

```
Epoch 1800: train RMSE 24.1940, test RMSE 101.4219
Epoch 1900: train RMSE 23.4605, test RMSE 100.1780
```

Output 27.3: Training progress

It is expected to see the RMSE of test set is an order of magnitude larger. The RMSE of 100 means the prediction and the actual target would be in average off by 100 in value (i.e., 100,000 passengers in this dataset).

To better understand the prediction quality, you can indeed plot the output using matplotlib, as follows:

```
with torch.no_grad():
    # shift train predictions for plotting
    train_plot = np.ones_like(timeseries) * np.nan
    y_pred = model(X_train)
    y_pred = y_pred[:, -1, :]
    train_plot[lookback:train_size] = model(X_train)[:, -1, :]
    # shift test predictions for plotting
    test_plot = np.ones_like(timeseries) * np.nan
    test_plot[train_size+lookback:len(timeseries)] = model(X_test)[:, -1, :]
# plot
plt.plot(timeseries, c='b')
plt.plot(train_plot, c='r')
plt.plot(test_plot, c='g')
plt.show()
```

Listing 27.8: Plotting the predicted time series

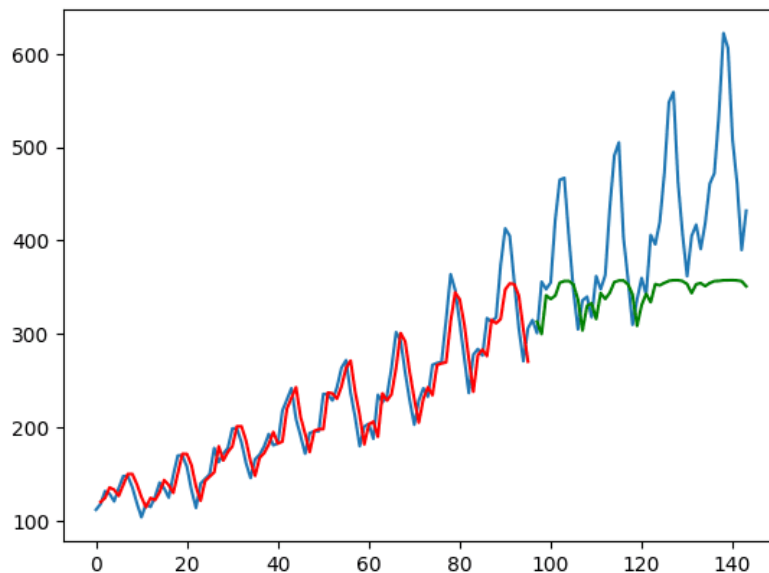


Figure 27.3: Predicted time series

From the above, you take the model's output as `y_pred` but extract only the data from the last time step as `y_pred[:, -1, :]`. This is what is plotted on the chart.

The training set is plotted in red while the test set is plotted in green. The blue curve is what the actual data looks like. You can see that the model can fit well to the training set but not very well on the test set.

Tying together, below is the complete code, except the parameter `lookback` is set to 4 this time:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
import torch.utils.data as data

df = pd.read_csv('airline-passengers.csv')
timeseries = df[["Passengers"]].values.astype('float32')

# train-test split for time series
train_size = int(len(timeseries) * 0.67)
test_size = len(timeseries) - train_size
train, test = timeseries[:train_size], timeseries[train_size:]

def create_dataset(dataset, lookback):
    """Transform a time series into a prediction dataset

    Args:
        dataset: A numpy array of time series, first dimension is the time steps
        lookback: Size of window for prediction
    """
    X, y = [], []
    for i in range(len(dataset)-lookback):
        feature = dataset[i:i+lookback]
        target = dataset[i+1:i+lookback+1]
        X.append(feature)
        y.append(target)
    return torch.tensor(X), torch.tensor(y)

lookback = 4
X_train, y_train = create_dataset(train, lookback=lookback)
X_test, y_test = create_dataset(test, lookback=lookback)

class AirModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(input_size=1, hidden_size=50, num_layers=1, batch_first=True)
        self.linear = nn.Linear(50, 1)
    def forward(self, x):
        x, _ = self.lstm(x)
        x = self.linear(x)
        return x

model = AirModel()
```

```

optimizer = optim.Adam(model.parameters())
loss_fn = nn.MSELoss()
loader = data.DataLoader(data.TensorDataset(X_train, y_train), shuffle=True, batch_size=8)

n_epochs = 2000
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    if epoch % 100 != 0:
        continue
    model.eval()
    with torch.no_grad():
        y_pred = model(X_train)
        train_rmse = np.sqrt(loss_fn(y_pred, y_train))
        y_pred = model(X_test)
        test_rmse = np.sqrt(loss_fn(y_pred, y_test))
    print("Epoch %d: train RMSE %.4f, test RMSE %.4f" % (epoch, train_rmse, test_rmse))

with torch.no_grad():
    # shift train predictions for plotting
    train_plot = np.ones_like(timeseries) * np.nan
    y_pred = model(X_train)
    y_pred = y_pred[:, -1, :]
    train_plot[lookback:train_size] = model(X_train)[:, -1, :]
    # shift test predictions for plotting
    test_plot = np.ones_like(timeseries) * np.nan
    test_plot[train_size+lookback:len(timeseries)] = model(X_test)[:, -1, :]
# plot
plt.plot(timeseries)
plt.plot(train_plot, c='r')
plt.plot(test_plot, c='g')
plt.show()

```

Listing 27.9: Training a LSTM model to predict time series with 4-step lookback

Running the above code will produce the plot below. From both the RMSE measure printed and the plot, you can notice that the model can now do better on the test set.

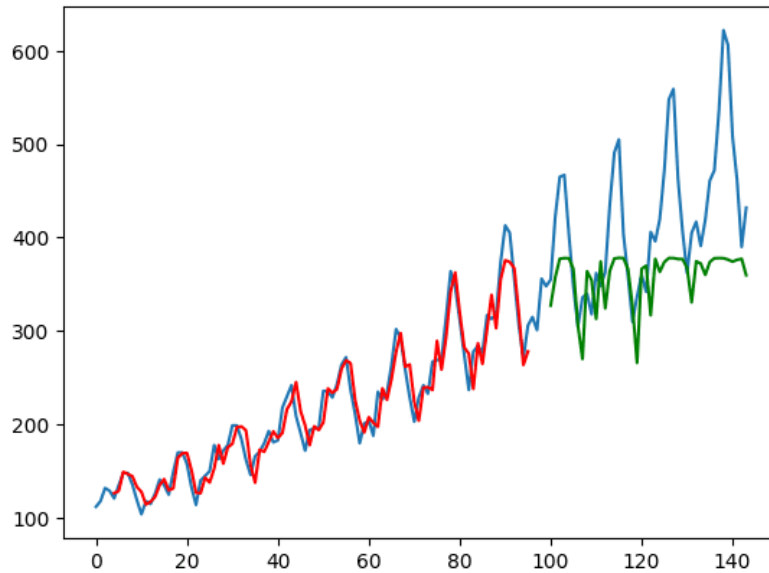


Figure 27.4: Time series prediction using LSTM network

This is also why the `create_dataset()` function is designed in such way: When the model is given a time series of time t to $t+3$ (as `lookback=4`), its output is the prediction of $t+1$ to $t+4$. However, $t+1$ to $t+3$ are also known from the input. By using these in the loss function, the model effectively was provided with more clues to train. This design is not always suitable, but you can see it is helpful in this particular example.

27.4 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

LSTM. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html>

torch.utils.data. PyTorch documentation.

<https://pytorch.org/docs/stable/data.html>

27.5 Summary

In this chapter, you discovered what is LSTM and how to use it for time series prediction in PyTorch. Specifically, you learned:

- ▷ What is the international airline passenger time series prediction dataset
- ▷ What is a LSTM cell
- ▷ How to create an LSTM network for time series prediction

In the next chapter, you will see a larger example in LSTM.

Text Generation with LSTM in PyTorch

28

Recurrent neural network can be used for time series prediction. In which, a regression neural network is created. It can also be used as generative model, which usually is a classification neural network model. A generative model is to learn certain pattern from data, such that when it is presented with some prompt, it can create a complete output that in the same style as the learned pattern.

In this chapter, you will discover how to build a generative model for text using LSTM recurrent neural networks in PyTorch. After finishing this chapter, you will know:

- ▷ Where to download a free corpus of text that you can use to train text generative models
- ▷ How to frame the problem of text sequences to a recurrent neural network generative model
- ▷ How to develop an LSTM to generate plausible text sequences for a given problem

Let's get started.

Overview

This chapter is divided into six parts; they are:

- ▷ What is a Generative Model
- ▷ Getting Text Data
- ▷ A Small LSTM Network to Predict the Next Character
- ▷ Generating Text with an LSTM Model
- ▷ Using a Larger LSTM Network
- ▷ Faster Training with GPU

28.1 What is a Generative Model

Generative model is indeed, just another machine learning model that happened to be able to create new things. Generative Adversarial Network (GAN) is a class of its own. Transformer models that uses attention mechanism is also found to be useful to generate text passages.

It is just a machine learning model because the model has been trained with existing data, so that it learned something from it. Depends on how to train it, they can work vastly different. In this chapter, a character-based generative model is created. What it means is to train a model that take a sequence of characters (alphabets and punctuations) as input and the immediate next character as the target. As long as it can predict what is the next character given what are preceding, you can run the model in a loop to generate a long piece of text.

This model is probably the simplest one. However, human language is complex. You shouldn't expect it can produce very high quality output. Even so, you need a lot of data and train the model for a long time before you can see sensible results.

28.2 Getting Text Data

Obtaining high quality data is important for a successful generative model. Fortunately, many of the classical texts are no longer protected under copyright. This means you can download all the text for these books for free and use them in experiments, like creating generative models. Perhaps the best place to get access to free books that are no longer protected by copyright is Project Gutenberg.

In this chapter, you will use a favorite book from childhood as the dataset, Alice's Adventures in Wonderland by Lewis Carroll:

▷ <https://www.gutenberg.org/ebooks/11>

Your model will learn the dependencies between characters and the conditional probabilities of characters in sequences so that you can, in turn, generate wholly new and original sequences of characters. This chapter is a lot of fun, and repeating these experiments with other books from Project Gutenberg is recommended. These experiments are not limited to text; you can also experiment with other ASCII data, such as computer source code, marked-up documents in LATEX, HTML or Markdown, and more.

You can download the complete text in ASCII format (Plaintext UTF-8) for this book for free and place it in your working directory with the filename `wonderland.txt`. Now, you need to prepare the dataset ready for modeling. Project Gutenberg adds a standard header and footer to each book, which is not part of the original text. Open the file in a text editor and delete the header and footer. The header is obvious and ends with the text:

```
*** START OF THIS PROJECT GUTENBERG EBOOK ALICE'S ADVENTURES IN WONDERLAND ***
```

Output 28.1: Header of the text file

The footer is all the text after the line of text that says:

THE END

Output 28.2: Footer of the text file

You should be left with a text file that has about 3,400 lines of text.

28.3 A Small LSTM Network to Predict the Next Character

First, you need to do some preprocessing on the data before you can build a model. Neural network models can only work with numbers, not text. Therefore you need to transform the characters into numbers. To make the problem simpler, you also want to transform all uppercase letters into lowercase.

In below, you open the text file, transform all letters into lowercase, and create a Python dict `char_to_int` to map characters into distinct integers. For example, the list of unique sorted lowercase characters in the book is as follows:

```
[ '\n', '\r', ' ', '!', '"', '"', '(', ')', '*', ',', '-', '.', ':', ';', '?', '[', ']',
'_', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p',
'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z', '\xbb', '\xbf', '\xef']
```

Listing 28.1: List of unique character tokens

Since this problem is character-based, the “vocabulary” are the distinct characters ever used in the text.

```
# load ascii text and covert to lowercase
filename = "wonderland.txt"
raw_text = open(filename, 'r', encoding='utf-8').read()
raw_text = raw_text.lower()

# create mapping of unique chars to integers
chars = sorted(list(set(raw_text)))
char_to_int = dict((c, i) for i, c in enumerate(chars))

# summarize the loaded data
n_chars = len(raw_text)
n_vocab = len(chars)
print("Total Characters: ", n_chars)
print("Total Vocab: ", n_vocab)
```

Listing 28.2: Counting the vocabulary in the text file

This should print:

```
Total Characters: 144343
Total Vocab: 44
```

Output 28.3: Number of characters and vocabulary in the text file

You can see the book has just under 150,000 characters, and when converted to lowercase, there are only 44 distinct characters in the vocabulary for the network to learn — much more than the 26 in the alphabet.

Next, you need to separate the text into inputs and targets. A window of 100 character is used here. That is, with character 1 to 100 as input, your model is going to predict for character 101. Should a window of 5 be used, the word “chapter” will become two data samples:

```
chapt -> e
hapte -> r
```

Output 28.4: Input and prediction example

In a long text such as this one, a myriad of windows can be created and this produced a dataset of a lot of samples:

```
# prepare the dataset of input to output pairs encoded as integers
seq_length = 100
dataX = []
dataY = []
for i in range(0, n_chars - seq_length, 1):
    seq_in = raw_text[i:i + seq_length]
    seq_out = raw_text[i + seq_length]
    dataX.append([char_to_int[char] for char in seq_in])
    dataY.append(char_to_int[seq_out])
n_patterns = len(dataX)
print("Total Patterns: ", n_patterns)
```

Listing 28.3: Creating a dataset from a long text using a sliding window

Running the above, you can see a total of 144,243 samples are created. Each sample is now in the form of integers, transformed using the mapping `char_to_int`. However, a PyTorch model would prefer to see the data in floating point tensors. Hence you should convert these into PyTorch tensors. LSTM layer is going to be used in the model, thus the input tensor should be of dimension (sample, time steps, features). To help training, it is also a good idea to normalize the input to 0 to 1. Hence you have the following:

```
import torch
import torch.nn as nn
import torch.optim as optim

# reshape X to be [samples, time steps, features]
X = torch.tensor(dataX, dtype=torch.float32).reshape(n_patterns, seq_length, 1)
X = X / float(n_vocab)
y = torch.tensor(dataY)
print(X.shape, y.shape)
```

Listing 28.4: Converting the dataset into PyTorch tensors

You can now define your LSTM model. Here, you define a single hidden LSTM layer with 256 hidden units. The input is single feature (i.e., one integer for one character). A dropout layer with probability 0.2 is added after the LSTM layer. The output of LSTM layer is a tuple, which the first element is the hidden states from the LSTM cell for each of the time step. It is a history of how the hidden state evolved as the LSTM cell accepts each time step of input. Presumably, the last hidden state contained the most information, hence only the

last hidden state is pass on to the output layer. The output layer is a fully-connected layer to produce logits for the 44 vocabularies. The logits can be converted into probability-like prediction using a softmax function.

```
import torch.nn as nn

class CharModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(input_size=1, hidden_size=256, num_layers=1, batch_first=True)
        self.dropout = nn.Dropout(0.2)
        self.linear = nn.Linear(256, n_vocab)
    def forward(self, x):
        x, _ = self.lstm(x)
        # take only the last output
        x = x[:, -1, :]
        # produce output
        x = self.linear(self.dropout(x))
        return x
```

Listing 28.5: LSTM model for character prediction

This is a model for single character classification of 44 classes. Therefore cross-entropy loss should be used. It is optimized using Adam optimizer. The training loop is as follows. For simplicity, no test set has created, but the model is evaluated with the training set once again at the end of each epoch to keep track on the progress.

This program can run for a long time, especially on CPU! In order to preserve the fruit of work, the best model ever found is saved for future reuse.

```
n_epochs = 40
batch_size = 128
model = CharModel()

optimizer = optim.Adam(model.parameters())
loss_fn = nn.CrossEntropyLoss(reduction="sum")
loader = data.DataLoader(data.TensorDataset(X, y), shuffle=True, batch_size=batch_size)

best_model = None
best_loss = np.inf
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
    loss = 0
    with torch.no_grad():
        for X_batch, y_batch in loader:
```



```

        y_pred = model(X_batch)
        loss += loss_fn(y_pred, y_batch)
    if loss < best_loss:
        best_loss = loss
        best_model = model.state_dict()
    print("Epoch %d: Cross-entropy: %.4f" % (epoch, loss))

torch.save([best_model, char_to_dict], "single-char.pth")

```

Listing 28.6: Training loop for the LSTM network

Running the above may produce the following:

```

...
Epoch 35: Cross-entropy: 245745.2500
Epoch 36: Cross-entropy: 243908.7031
Epoch 37: Cross-entropy: 238833.5000
Epoch 38: Cross-entropy: 239069.0000
Epoch 39: Cross-entropy: 234176.2812

```

Output 28.5: Training progress on the LSTM network

The cross-entropy almost always decreasing in each epoch. This means probably the model is not fully converged and you can train it for more epochs. Upon the training loop completed, you should have the file `single-char.pth` created to contain the best model weight ever found, as well as the character-to-integer mapping used by this model.

For completeness, below is tying everything above into one script:

```

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import torch.utils.data as data

# load ascii text and covert to lowercase
filename = "wonderland.txt"
raw_text = open(filename, 'r', encoding='utf-8').read()
raw_text = raw_text.lower()

# create mapping of unique chars to integers
chars = sorted(list(set(raw_text)))
char_to_int = dict((c, i) for i, c in enumerate(chars))

# summarize the loaded data
n_chars = len(raw_text)
n_vocab = len(chars)
print("Total Characters: ", n_chars)
print("Total Vocab: ", n_vocab)

# prepare the dataset of input to output pairs encoded as integers
seq_length = 100
dataX = []
dataY = []

```

```

for i in range(0, n_chars - seq_length, 1):
    seq_in = raw_text[i:i + seq_length]
    seq_out = raw_text[i + seq_length]
    dataX.append([char_to_int[char] for char in seq_in])
    dataY.append(char_to_int[seq_out])
n_patterns = len(dataX)
print("Total Patterns: ", n_patterns)

# reshape X to be [samples, time steps, features]
X = torch.tensor(dataX, dtype=torch.float32).reshape(n_patterns, seq_length, 1)
X = X / float(n_vocab)
y = torch.tensor(dataY)

class CharModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(input_size=1, hidden_size=256, num_layers=1, batch_first=True)
        self.dropout = nn.Dropout(0.2)
        self.linear = nn.Linear(256, n_vocab)
    def forward(self, x):
        x, _ = self.lstm(x)
        # take only the last output
        x = x[:, -1, :]
        # produce output
        x = self.linear(self.dropout(x))
        return x

n_epochs = 40
batch_size = 128
model = CharModel()

optimizer = optim.Adam(model.parameters())
loss_fn = nn.CrossEntropyLoss(reduction="sum")
loader = data.DataLoader(data.TensorDataset(X, y), shuffle=True, batch_size=batch_size)

best_model = None
best_loss = np.inf
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
    loss = 0
    with torch.no_grad():
        for X_batch, y_batch in loader:
            y_pred = model(X_batch)
            loss += loss_fn(y_pred, y_batch)
    if loss < best_loss:
        best_loss = loss

```

```

best_model = model.state_dict()
print("Epoch %d: Cross-entropy: %.4f" % (epoch, loss))

torch.save([best_model, char_to_int], "single-char.pth")

```

Listing 28.7: Complete program to train a LSTM network for character prediction

28.4 Generating Text with an LSTM Model

Given the model is well-trained, generating text using the trained LSTM network is relatively straightforward. Firstly, you need to recreate the network and load the trained model weight from the saved checkpoint. Then you need to create some prompt for the model to start on. The prompt can be anything that the model can understand. It is a seed sequence to be given to the model to obtain one generated character. Then, the generated character is added to the end of this sequence, and trim off the first character to maintain the consistent length. This process is repeated for as long as you want to predict new characters (e.g., a sequence of 1,000 characters in length). You can pick a random input pattern as your seed sequence, then print generated characters as you generate them.

A simple way to generate prompt is to pick a random sample from the original dataset, e.g., with the `raw_text` obtained in the previous section, a prompt can be created as:

```

seq_length = 100
start = np.random.randint(0, len(raw_text)-seq_length)
prompt = raw_text[start:start+seq_length]

```

Listing 28.8: Sample a random prompt from the text

But you should be reminded that you need to transform it since this prompt is a string while the model expects a vector of integers.

The entire code is merely as follows:

```

import numpy as np
import torch
import torch.nn as nn

best_model, char_to_int = torch.load("single-char.pth")
n_vocab = len(char_to_int)
int_to_char = dict((i, c) for c, i in char_to_int.items())

# reload the model
class CharModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(input_size=1, hidden_size=256, num_layers=1, batch_first=True)
        self.dropout = nn.Dropout(0.2)
        self.linear = nn.Linear(256, n_vocab)
    def forward(self, x):
        x, _ = self.lstm(x)
        # take only the last output

```

```

        x = x[:, -1, :]
        # produce output
        x = self.linear(self.dropout(x))
        return x
model = CharModel()
model.load_state_dict(best_model)

# randomly generate a prompt
filename = "wonderland.txt"
seq_length = 100
raw_text = open(filename, 'r', encoding='utf-8').read()
raw_text = raw_text.lower()
start = np.random.randint(0, len(raw_text)-seq_length)
prompt = raw_text[start:start+seq_length]
pattern = [char_to_int[c] for c in prompt]

model.eval()
print('Prompt: "%s"' % prompt)
with torch.no_grad():
    for i in range(1000):
        # format input array of int into PyTorch tensor
        x = np.reshape(pattern, (1, len(pattern), 1)) / float(n_vocab)
        x = torch.tensor(x, dtype=torch.float32)
        # generate logits as output from the model
        prediction = model(x)
        # convert logits into one character
        index = int(prediction.argmax())
        result = int_to_char[index]
        print(result, end="")
        # append the new character into the prompt for the next iteration
        pattern.append(index)
        pattern = pattern[1:]
print()
print("Done.")

```

Listing 28.9: Loading the trained model and generate text from a random prompt

Running this example first outputs the prompt used, then each character as it is generated. For example, below are the results from one run of this text generator. The prompt was:

```
Prompt: "nother rush at the stick, and tumbled head
over heels in its hurry to get hold of it; then alice, th"
```

Output 28.6: The prompt

The generated text was:

```

e was got a litule soteet of thet was sh the thiee harden an the courd, and was tuitk
↳ a little toaee th thite ththe and said to the suher, and the whrtght the pacbit
↳ sese tha woode of the soeee, and the white rabbit ses ani thr gort to the thite
↳ rabbit, and then she was aoinnene th the three baaed of the sueen and saed "ota
↳ turpe "hun mot,"

```

```

"i don't know the ter ano _enend to mere," said the maccht ar a sore of great roaee.
↳
↳ "ie you don't teink if thet soued to soeed to the boeie the mooer, io you bane
↳ thing it wo
tou het bn the crur,
"h whsh you cen not," said the manch hare.

"wes, it aadi," said the manch hare.

"weat you tail to merer ae in an a gens if gre" "he were thing," said the maccht ar a
↳ sore of geeaghen asd tothe to the thieg harden an the could.
"h dan tor toe taie thing," said the manch hare.

"wes, it aadi," said the manch hare.

"weat you tail to merer ae in an a gens if gre" "he were thing," said the maccht ar a
↳ sore of geeaghen asd tothe to the thieg harden an t

```

Output 28.7: The generated text

Let's note some observations about the generated text.

- ▷ It can emit line breaks. The original text limited the line width to 80 characters and the generative model attempted to replicate this pattern
- ▷ The characters are separated into word-like groups, and some groups are actual English words (e.g., "the," "said," and "rabbit"), but many are not (e.g., "thite," "soteet," and "tha").
- ▷ Some of the words in sequence make sense (e.g., "i don't know the"), but many do not (e.g., "he were thing").

The fact that this character-based model of the book produces output like this is very impressive. It gives you a sense of the learning capabilities of LSTM networks. However, the results are not perfect. In the next section, you will look at improving the quality of results by developing a much larger LSTM network.

28.5 Using a Larger LSTM Network

Recall that LSTM is a recurrent neural network. It takes a sequence as input, which in each step of the sequence, the input is mixed with its internal states to produce an output. Hence the output from LSTM is also a sequence. In the above, the output from the last time step is taken for further processing in the neural network but those from earlier steps are discarded. However, it is not necessarily the case. You can treat the sequence output from one LSTM layer as input to another LSTM layer. Then, you are building a larger network.

Similar to convolutional neural networks, a stacked LSTM network is supposed to have the earlier LSTM layers to learn low level features while the later LSTM layers to learn the high level features. It may not be always useful but you can try it out to see whether the model can produce a better result.

In PyTorch, making a stacked LSTM layer is easy. Let's modify the above model into the following:

```

import torch.nn as nn

class CharModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(input_size=1, hidden_size=256, num_layers=2,
                             batch_first=True, dropout=0.2)

        self.dropout = nn.Dropout(0.2)
        self.linear = nn.Linear(256, n_vocab)
    def forward(self, x):
        x, _ = self.lstm(x)
        # take only the last output
        x = x[:, -1, :]
        # produce output
        x = self.linear(self.dropout(x))
        return x

```

Listing 28.10: A LSTM network with two LSTM layers

The only change is on the parameter to `nn.LSTM()`: you set `num_layers=2` instead of 1 to add another LSTM layer. But between the two LSTM layers, you also added a dropout layer through the parameter `dropout=0.2`. Replacing this model with the previous is all the change you need to make. Rerun the training you should see the below:

```

...
Epoch 34: Cross-entropy: 203763.0312
Epoch 35: Cross-entropy: 204002.5938
Epoch 36: Cross-entropy: 210636.5625
Epoch 37: Cross-entropy: 199619.6875
Epoch 38: Cross-entropy: 199240.2969
Epoch 39: Cross-entropy: 196966.1250

```

Output 28.8: Training progress of a larger LSTM network

You should see the cross-entropy here is lower than that in the previous section. This means this model is performing better. In fact, with this model, you can see the generated text looks more sensible:

```

Prompt: "ll
say that 'i see what i eat' is the same thing as 'i eat what i see'!"

"you might just as well sa"
y it to sea," she katter said to the jury. and the thoe hardeners vhine she was
↳ seady to alic the was a long tay of the sooe of the court, and she was seady to
↳ and taid to the coor and the court.
"well you see what you see, the mookee of the soog of the season of the shase of the
↳ court!"

"i don't know the rame thing is it?" said the caterpillar.

"the cormous was it makes he it was it taie the reason of the shall bbout it, you know."

```

```

"i don't know the rame thing i can't gelp the sea," the hatter went on, "i don't know
↳ the peally was in the shall sereat it would be a teally.
the mookee of the court "

"i don't know the rame thing is it?" said the caterpillar.

"the cormous was it makes he it was it taie the reason of the shall bbout it, you know."

"i don't know the rame thing i can't gelp the sea," the hatter went on, "i don't know
↳ the peally was in the shall sereat it would be a teally.
the mookee of the court "

"i don't know the rame thing is it?" said the caterpillar.

"the
Done.

```

Output 28.9: Generated text from a larger LSTM network

Not only words are spelled correctly, the text is also more English-like. Since the cross-entropy loss is still decreasing as you trained the model, you can assume the model is not converged yet. You can expect to make the model better if you increased the training epoch.

For completeness, below is the complete code for using this new model, including training and text generation.

```

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import torch.utils.data as data

# load ascii text and covert to lowercase
filename = "wonderland.txt"
raw_text = open(filename, 'r', encoding='utf-8').read()
raw_text = raw_text.lower()

# create mapping of unique chars to integers
chars = sorted(list(set(raw_text)))
char_to_int = dict((c, i) for i, c in enumerate(chars))

# summarize the loaded data
n_chars = len(raw_text)
n_vocab = len(chars)
print("Total Characters: ", n_chars)
print("Total Vocab: ", n_vocab)

# prepare the dataset of input to output pairs encoded as integers
seq_length = 100
dataX = []
dataY = []
for i in range(0, n_chars - seq_length, 1):
    seq_in = raw_text[i:i + seq_length]
    seq_out = raw_text[i + seq_length]

```

```

    dataX.append([char_to_int[char] for char in seq_in])
    dataY.append(char_to_int[seq_out])
n_patterns = len(dataX)
print("Total Patterns: ", n_patterns)

# reshape X to be [samples, time steps, features]
X = torch.tensor(dataX, dtype=torch.float32).reshape(n_patterns, seq_length, 1)
X = X / float(n_vocab)
y = torch.tensor(dataY)

class CharModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(input_size=1, hidden_size=256, num_layers=2,
                             batch_first=True, dropout=0.2)
        self.dropout = nn.Dropout(0.2)
        self.linear = nn.Linear(256, n_vocab)
    def forward(self, x):
        x, _ = self.lstm(x)
        # take only the last output
        x = x[:, -1, :]
        # produce output
        x = self.linear(self.dropout(x))
        return x

n_epochs = 40
batch_size = 128
model = CharModel()

optimizer = optim.Adam(model.parameters())
loss_fn = nn.CrossEntropyLoss(reduction="sum")
loader = data.DataLoader(data.TensorDataset(X, y), shuffle=True, batch_size=batch_size)

best_model = None
best_loss = np.inf
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
    loss = 0
    with torch.no_grad():
        for X_batch, y_batch in loader:
            y_pred = model(X_batch)
            loss += loss_fn(y_pred, y_batch)
        if loss < best_loss:
            best_loss = loss
            best_model = model.state_dict()
    print("Epoch %d: Cross-entropy: %.4f" % (epoch, loss))

```



```

torch.save([best_model, char_to_int], "single-char.pth")

# Generation using the trained model
best_model, char_to_int = torch.load("single-char.pth")
n_vocab = len(char_to_int)
int_to_char = dict((i, c) for c, i in char_to_int.items())
model.load_state_dict(best_model)

# randomly generate a prompt
filename = "wonderland.txt"
seq_length = 100
raw_text = open(filename, 'r', encoding='utf-8').read()
raw_text = raw_text.lower()
start = np.random.randint(0, len(raw_text)-seq_length)
prompt = raw_text[start:start+seq_length]
pattern = [char_to_int[c] for c in prompt]

model.eval()
print('Prompt: "%s"' % prompt)
with torch.no_grad():
    for i in range(1000):
        # format input array of int into PyTorch tensor
        x = np.reshape(pattern, (1, len(pattern), 1)) / float(n_vocab)
        x = torch.tensor(x, dtype=torch.float32)
        # generate logits as output from the model
        prediction = model(x)
        # convert logits into one character
        index = int(prediction.argmax())
        result = int_to_char[index]
        print(result, end="")
        # append the new character into the prompt for the next iteration
        pattern.append(index)
        pattern = pattern[1:]
print()
print("Done.")

```

Listing 28.11: Complete code of training and using a larger LSTM network

28.6 Faster Training with GPU

Running programs from this chapter can be pathetically slow. Even if you have a GPU, you will not see immediate improvement. It is because the design of PyTorch, it may not use your GPU automatically. However, if you have a CUDA-capable GPU, you can improve the performance a lot by carefully moving the heavy computation away from your CPU.

A PyTorch model is a program of tensor calculation. The tensors can be stored in GPU or in CPU. Operation can be carried out as long as all the operators are in the same device. In this particular example, the model weight (i.e., those of the LSTM layers and the fully connected layer) can be moved to GPU. By doing so, the input should also be moved to the GPU before execution and the output will also be stored in the GPU unless you move it back.

In PyTorch, you can check if you have a CUDA-capable GPU using the following function:

```
torch.cuda.is_available()
```

Listing 28.12: Checking if CUDA is available on your computer

It returns a boolean to indicate if you can use GPU, which in turn, depends on the hardware model you have, whether your OS has the appropriate library installed, and whether your PyTorch is compiled with corresponding GPU support. If everything works in concert, you can create a device and assign your model to it:

```
device = torch.device("cuda:0")
model.to(device)
```

Listing 28.13: Sending a model to the GPU

If your model is running on CUDA device but your input tensor is not, you will see PyTorch complain about that and fail to proceed. To move your tensor to the CUDA device, you should run like the following:

```
y_pred = model(X_batch.to(device))
```

Listing 28.14: Sending a tensor to GPU and run on a GPU-based model

Which the `.to(device)` part will do the magic. But remember that `y_pred` produced above will also be on the CUDA device. Hence you should do the same when you run the loss function. Modifying the above program to make it capable to run on GPU will become the following:

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import torch.utils.data as data

# load ascii text and covert to lowercase
filename = "wonderland.txt"
raw_text = open(filename, 'r', encoding='utf-8').read()
raw_text = raw_text.lower()

# create mapping of unique chars to integers
chars = sorted(list(set(raw_text)))
char_to_int = dict((c, i) for i, c in enumerate(chars))

# summarize the loaded data
n_chars = len(raw_text)
n_vocab = len(chars)
print("Total Characters: ", n_chars)
print("Total Vocab: ", n_vocab)

# prepare the dataset of input to output pairs encoded as integers
seq_length = 100
dataX = []
dataY = []
```

```

for i in range(0, n_chars - seq_length, 1):
    seq_in = raw_text[i:i + seq_length]
    seq_out = raw_text[i + seq_length]
    dataX.append([char_to_int[char] for char in seq_in])
    dataY.append(char_to_int[seq_out])
n_patterns = len(dataX)
print("Total Patterns: ", n_patterns)

# reshape X to be [samples, time steps, features]
X = torch.tensor(dataX, dtype=torch.float32).reshape(n_patterns, seq_length, 1)
X = X / float(n_vocab)
y = torch.tensor(dataY)

class CharModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(input_size=1, hidden_size=256, num_layers=2,
                             batch_first=True, dropout=0.2)
        self.dropout = nn.Dropout(0.2)
        self.linear = nn.Linear(256, n_vocab)
    def forward(self, x):
        x, _ = self.lstm(x)
        # take only the last output
        x = x[:, -1, :]
        # produce output
        x = self.linear(self.dropout(x))
        return x

n_epochs = 40
batch_size = 128
model = CharModel()
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
model.to(device)

optimizer = optim.Adam(model.parameters())
loss_fn = nn.CrossEntropyLoss(reduction="sum")
loader = data.DataLoader(data.TensorDataset(X, y), shuffle=True, batch_size=batch_size)

best_model = None
best_loss = np.inf
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch.to(device))
        loss = loss_fn(y_pred, y_batch.to(device))
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
    loss = 0
    with torch.no_grad():
        for X_batch, y_batch in loader:
            y_pred = model(X_batch.to(device))

```

```

        loss += loss_fn(y_pred, y_batch.to(device))
    if loss < best_loss:
        best_loss = loss
        best_model = model.state_dict()
    print("Epoch %d: Cross-entropy: %.4f" % (epoch, loss))

torch.save([best_model, char_to_int], "single-char.pth")

# Generation using the trained model
best_model, char_to_int = torch.load("single-char.pth")
n_vocab = len(char_to_int)
int_to_char = dict((i, c) for c, i in char_to_int.items())
model.load_state_dict(best_model)

# randomly generate a prompt
filename = "wonderland.txt"
seq_length = 100
raw_text = open(filename, 'r', encoding='utf-8').read()
raw_text = raw_text.lower()
start = np.random.randint(0, len(raw_text)-seq_length)
prompt = raw_text[start:start+seq_length]
pattern = [char_to_int[c] for c in prompt]

model.eval()
print('Prompt: "%s"' % prompt)
with torch.no_grad():
    for i in range(1000):
        # format input array of int into PyTorch tensor
        x = np.reshape(pattern, (1, len(pattern), 1)) / float(n_vocab)
        x = torch.tensor(x, dtype=torch.float32)
        # generate logits as output from the model
        prediction = model(x.to(device))
        # convert logits into one character
        index = int(prediction.argmax())
        result = int_to_char[index]
        print(result, end="")
        # append the new character into the prompt for the next iteration
        pattern.append(index)
        pattern = pattern[1:]

print()
print("Done.")

```

Listing 28.15: Training and running a LSTM network on GPU if available

Compare to the code in the previous section, you should see they are essentially the same. Except the CUDA device is detected with the line:

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

Listing 28.16: Selecting the GPU if available and fallback to CPU otherwise

which will be your GPU or fall back to CPU if no CUDA device is found. Afterward, `.to(device)` is added at several strategic locations to move the computation to the GPU.

28.7 Further Readings

This character text model is a popular way of generating text using recurrent neural networks. Below are some more resources and tutorials on the topic if you are interested in going deeper.

Articles

Andrej Karpathy. *The Unreasonable Effectiveness of Recurrent Neural Networks*. May 2015.
<http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

Lars Eidnes. *Auto-Generating Clickbait With Recurrent Neural Networks*. 2015.
<https://larseidnes.com/2015/10/13/auto-generating-clickbait-with-recurrent-neural-networks/>

LSTM. PyTorch API.

<https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html>

Sequence Models and Long Short-Term Memory Networks. PyTorch tutorials.

https://pytorch.org/tutorials/beginner/nlp/sequence_models_tutorial.html

Papers

Ilya Sutskever, James Martens, and Geoffrey Hinton. “Generating Text with Recurrent Neural Networks”. In: *Proceedings of the 28th International Conference on Machine Learning*. Bellevue, WA, USA, 2011.

<https://www.cs.utoronto.ca/~ilya/pubs/2011/LANG-RNN.pdf>

28.8 Summary

In this chapter, you discovered how you can develop an LSTM recurrent neural network for text generation in PyTorch. After completing this chapter, you know:

- ▷ How to find text for classical books for free as dataset for your machine learning model
- ▷ How to train an LSTM network for text sequences
- ▷ How to use a LSTM network to generate text sequences How to optimize deep learning training in PyTorch using CUDA devices

This marks the end of this book.

Appendix

How to Setup Python on Your Workstation



It can be difficult to install a Python machine learning environment on some platforms. Python itself must be installed first and then there are many packages to install, and it can be confusing for beginners. In this tutorial, you will discover how to setup a Python machine learning development environment using Anaconda. After completing this tutorial, you will have a working Python environment to begin learning, practicing, and developing machine learning software.

A.1 Overview

In this chapter, we will cover the following steps:

1. Download Anaconda
2. Install Anaconda
3. Start and Update Anaconda
4. Install Deep Learning Libraries
5. Install Visual Studio Code environment



Note: The specific versions may differ as the software and libraries are updated frequently.

A.2 Download Anaconda

In this step, you will download the Anaconda Python package for your platform. Anaconda is a free and easy-to-use environment for scientific Python. These instructions are suitable for Windows, macOS, and Linux platforms. Windows are used below for demonstration, so you may see some Windows dialogs and file extensions.

1. Visit the Anaconda homepage <https://www.anaconda.com/>

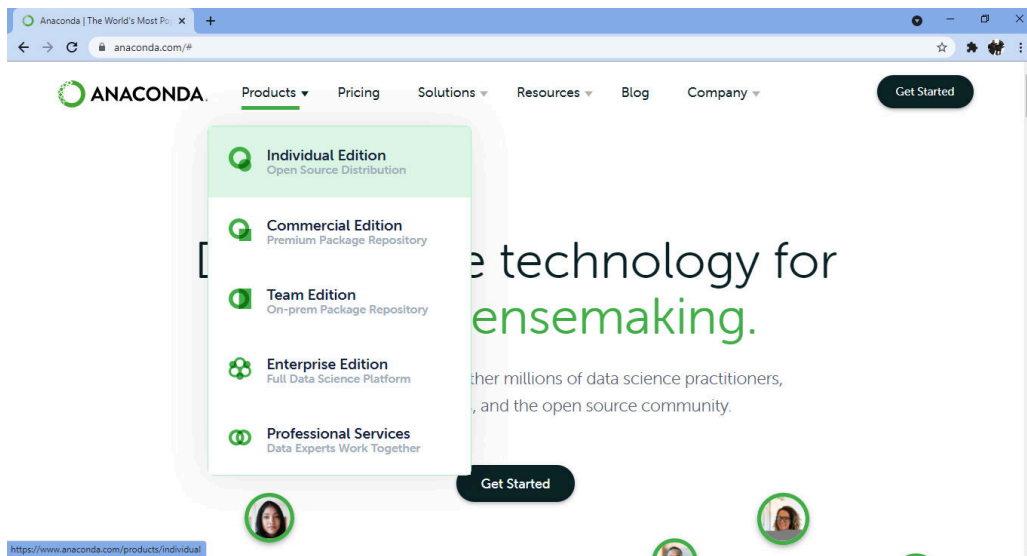


Figure A.1: Click “Products” and “Individual Edition”

2. Click “Products” from the menu and click “Individual Edition” to go to the download page <https://www.anaconda.com/products/individual-d/>.

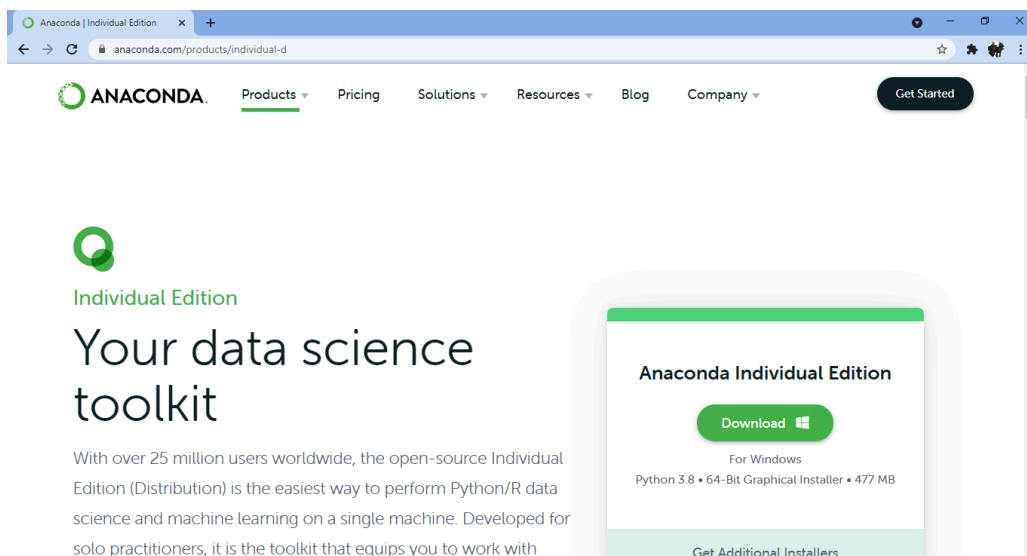


Figure A.2: Click Download

This will download the Anaconda Python package to your workstation. It will automatically give you the installer according to your OS (Windows, Linux, or MacOS). The file is about 480 MB. You should have a file with a name like:

```
Anaconda3-2021.05-Windows-x86_64.exe
```


A.3 Install Anaconda

In this step, you will install the Anaconda Python software on your system. This step assumes you have sufficient administrative privileges to install software on your system.

1. Double click the downloaded file.
2. Follow the installation wizard.

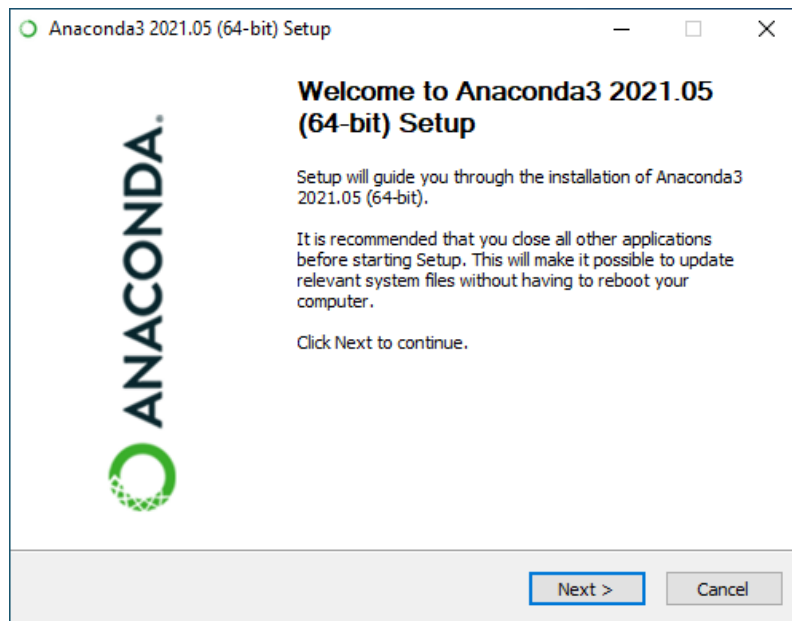


Figure A.3: Anaconda Python Installation Wizard

Installation is quick and painless. There should be no tricky questions or sticking points.

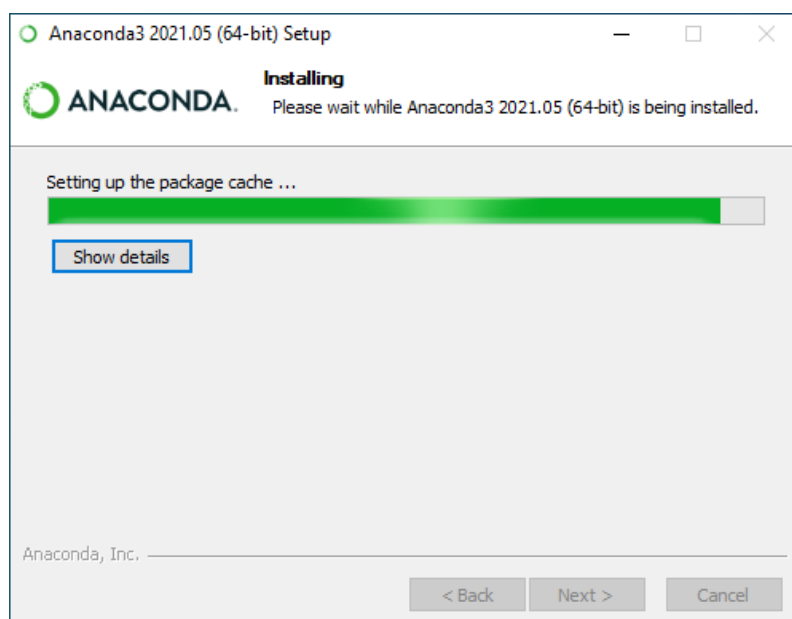


Figure A.4: Anaconda Python Installation Wizard Writing Files

The installation should take less than 10 minutes and take a bit more than 5 GB of space on your hard drive.

A.4 Start and Update Anaconda

In this step, you will confirm that your Anaconda Python environment is up to date. Anaconda comes with a suite of graphical tools called Anaconda Navigator. You can start Anaconda Navigator by opening it from your application launcher.

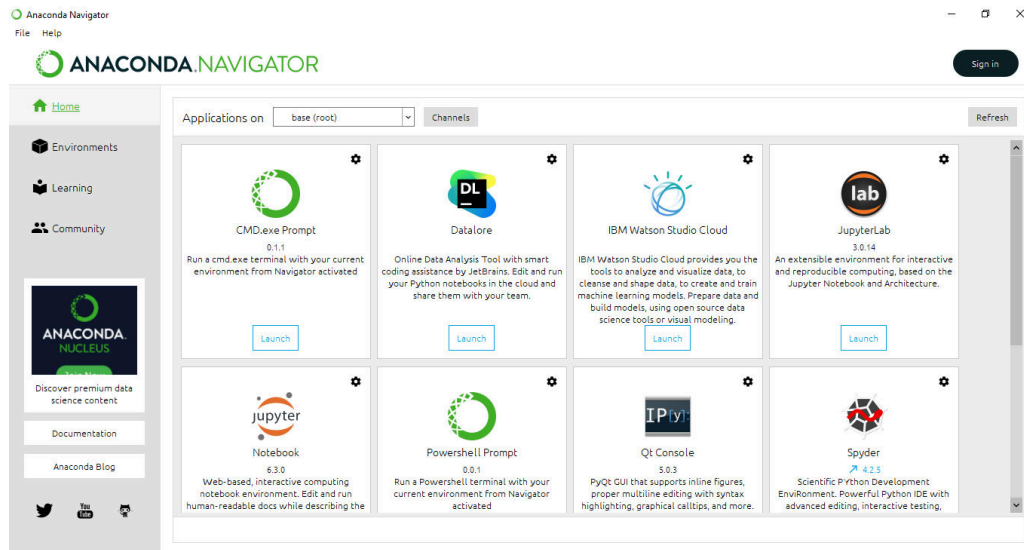


Figure A.5: Anaconda Navigator GUI

You can use the Anaconda Navigator and graphical development environments later; for now, it is recommended to start with the Anaconda command line environment called `conda`¹. Conda is fast, simple, it's hard for error messages to hide, and you can quickly confirm your environment is installed and working correctly.

1. Open a terminal or CMD.exe prompt (command line window).
2. Confirm conda is installed correctly, by typing:

```
conda -V
```

You should see the following (or something similar):

```
conda 4.10.1
```

3. Confirm Python is installed correctly by typing:

```
python -V
```

¹<https://conda.pydata.org/docs/index.html>

You should see the following (or something similar):

```
Python 3.8.8
```

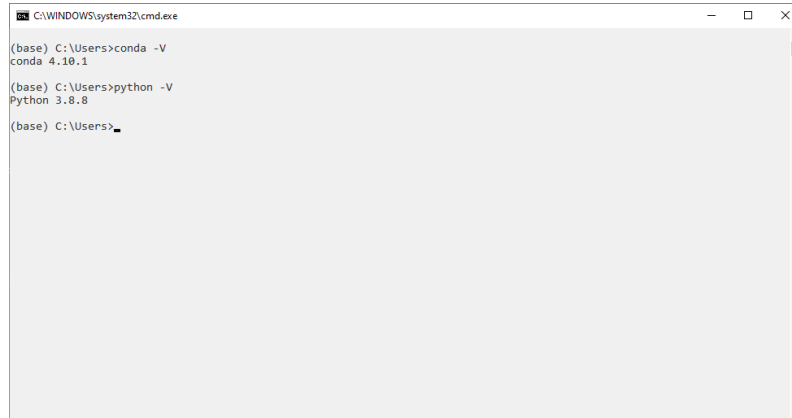
A screenshot of a Windows command prompt window titled 'C:\WINDOWS\system32\cmd.exe'. The window shows the following text: (base) C:\Users>conda -V, conda 4.10.1, (base) C:\Users>python -V, Python 3.8.8, and (base) C:\Users>_. The window has standard Windows window controls (minimize, maximize, close) in the top right corner.

Figure A.6: Confirm Conda and Python are Installed

If the commands do not work or have an error, please check the documentation for help for your platform. See some of the resources in the “Further Reading” section.

4. This step is optional. You can make community supported packages available in conda by adding a “conda-forge” channel:

```
conda config --add channels conda-forge
```

5. Confirm your conda environment is up-to-date, type:

```
conda update conda
conda update anaconda
```

You may need to install some packages and confirm the updates.

6. Confirm your SciPy environment.

The script below will print the version number of the key SciPy libraries you require for machine learning development, specifically: SciPy, NumPy, Matplotlib, Pandas, Statsmodels, and Scikit-learn. You can type “python” and type the commands in directly. Alternatively, it is recommended to open a text editor and copy-pasting the script into your editor.

```
# check library version numbers
# scipy
import scipy
print('scipy: %s' % scipy.__version__)
# numpy
import numpy
```

```
print('numpy: %s' % numpy.__version__)
# matplotlib
import matplotlib
print('matplotlib: %s' % matplotlib.__version__)
# pandas
import pandas
print('pandas: %s' % pandas.__version__)
# statsmodels
import statsmodels
print('statsmodels: %s' % statsmodels.__version__)
# scikit-learn
import sklearn
print('sklearn: %s' % sklearn.__version__)
```

Listing A.1: Code to check that key Python libraries are installed.

Save the script as a file with the name: `versions.py`. On the command line, change your directory to where you saved the script and type:

```
python versions.py
```

You should see output like the following:

```
scipy: 1.8.1
numpy: 1.23.1
matplotlib: 3.5.1
pandas: 1.4.3
statsmodels: 0.13.2
sklearn: 1.1.1
```

Output A.1: Sample output of thhe versions script

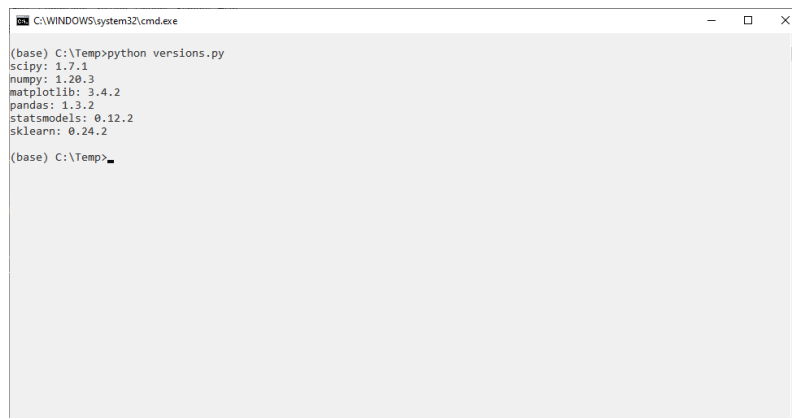


Figure A.7: Confirm Anaconda SciPy environment

A.5 Install Deep Learning Libraries

In this step, you will install Python libraries used for deep learning, specifically PyTorch:

1. Install the PyTorch and torchvision deep learning library by typing:

```
conda install pytorch torchvision
```

2. Alternatively, you may choose to install using `pip` and a specific version of PyTorch for your platform

```
pip install torch==2.0.0
```

3. Confirm your deep learning environment is installed and working correctly.

Create a script that prints the version numbers of each library, as you did before for the SciPy environment.

```
# check deep learning version numbers
# pytorch
import torch
print('pytorch: %s' % torch.__version__)
# torchvision
import torchvision
print('torchvision: %s' % torchvision.__version__)
```

Listing A.2: Code to check that key deep learning libraries are installed.

Save the script to a file `deep_versions.py`. Run the script by typing:

```
python deep_versions.py
```

Output A.2: Run script from the command line.

You should see output like:

```
pytorch: 2.0.0
torchvision: 0.15.1
```

Output A.3: Sample output of the deep learning versions script

A.6 Install Visual Studio Code for Python

If you have installed Anaconda, you will have an IDE called Spyder installed. You can develop your Python project with Spyder. The other popular way of writing Python code nowadays is to use Visual Studio Code. It is a free programming environment from Microsoft. Visual Studio Code can do a lot of things via “extensions”. Python extension is what you should get.

These instructions are suitable for Windows, macOS, and Linux platforms. Below, macOS is used to demonstrate, so you may see some Windows dialogs and file extensions.

1. Visit the Visual Studio Code homepage <https://code.visualstudio.com>

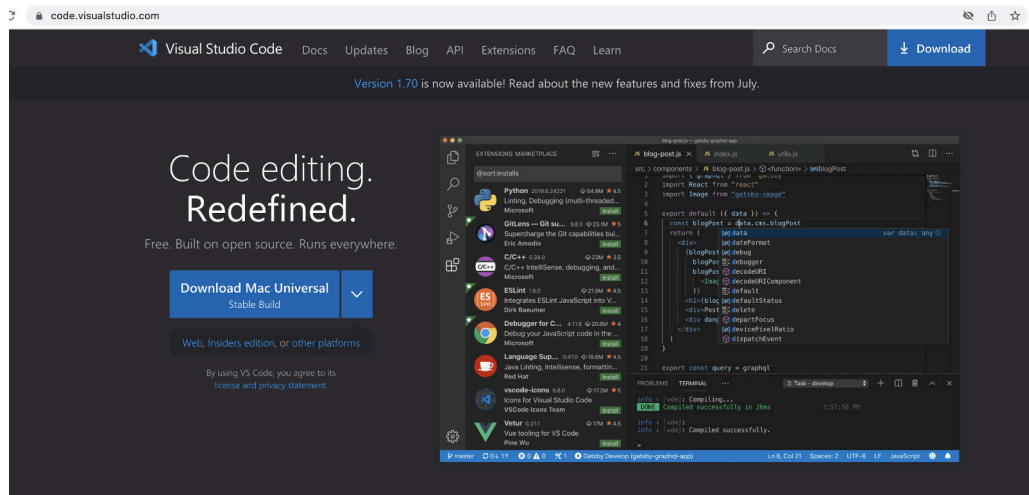


Figure A.8: Click “Products” and “Individual Edition”

2. Click the download button at the top toolbar or at the center of the screen to download a ZIP file (such as `VSCode-darwin-universal.zip`). It is around 200MB in size
3. Expand the ZIP you will find the application program. In macOS, you should move it into your `/Applications` folder

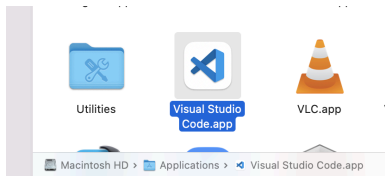


Figure A.9: Visual Studio Code in `/Applications` in macOS

4. Running Visual Studio Code will show you the main screen like the following:

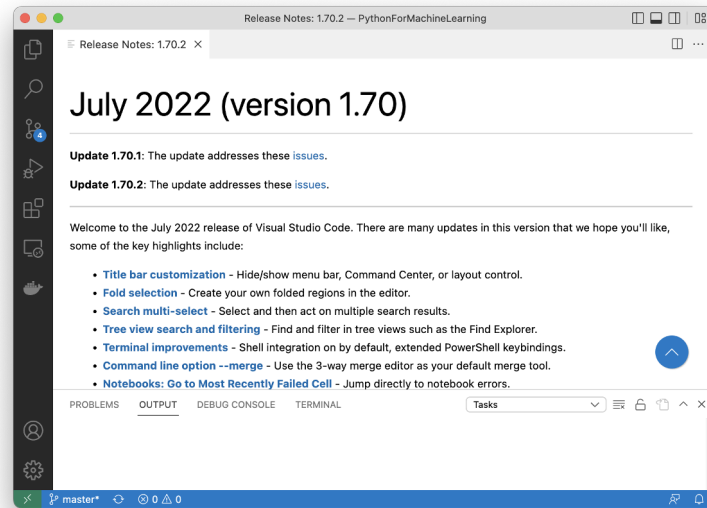


Figure A.10: Visual Studio Code main screen

You can click on the building block icon at left toolbar to open the extensions marketplace. Typing “python” on the search box will usually show the Microsoft-developed Python extension at top.

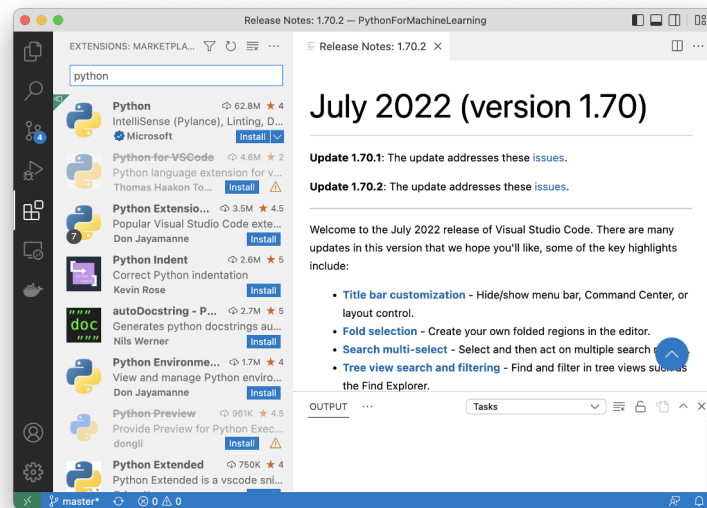


Figure A.11: Searching for Python extension in the extensions marketplace

5. Click on the “Install” button on the extension marketplace will get the extension installed. It should be very quick.

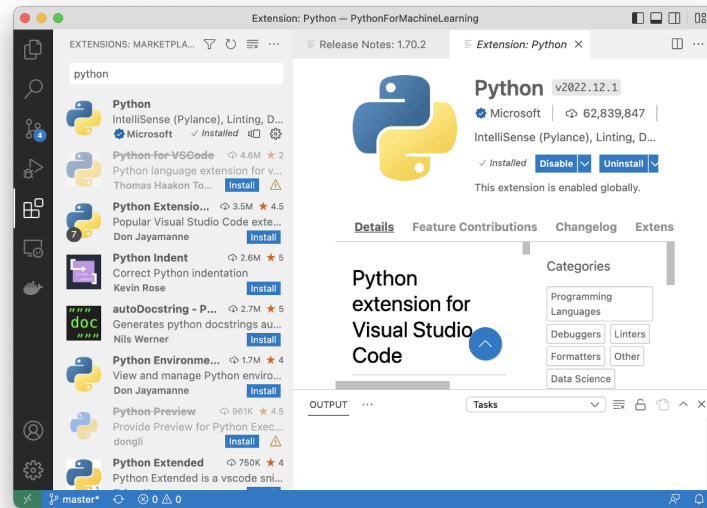


Figure A.12: Python extension installed in Visual Studio Code

Afterwards, you will find your Visual Studio Code can launch Python interpreter or Python debugger to run your program.

A.7 Further Reading

This section provides resources if you want to know more about Anaconda.

- ▷ Anaconda Documentation
<https://docs.continuum.io/>
- ▷ Anaconda Documentation: Installation
<https://docs.continuum.io/anaconda/install>
- ▷ Anaconda Navigator
<https://docs.continuum.io/anaconda/navigator.html>
- ▷ The conda command line tool
<https://conda.pydata.org/docs/index.html>
- ▷ Using conda
<https://conda.pydata.org/docs/using/>
- ▷ conda-forge
<https://conda-forge.org//docs/using/>
- ▷ Instructions for installing TensorFlow in Anaconda
<https://docs.anaconda.com/anaconda/user-guide/tasks/tensorflow/>

A.8 Summary

Congratulations, you now have a working Python development environment for machine learning. You can now learn and practice machine learning on your workstation.

How to Setup Amazon EC2 for Deep Learning on GPUs



Large deep learning models require a lot of compute time to run. You can run them on your CPU but it can take hours or days to get a result. If you have access to a GPU on your desktop, you can drastically speed up the training time of your deep learning models. In this project you will discover how you can get access to GPUs to speed up the training of your deep learning models by using the Amazon Web Service (AWS) infrastructure. For less than a dollar per hour and often a lot cheaper you can use this service from your workstation or laptop. After working through this project you will know:

- ▷ How to create an account and log-in to Amazon Web Service.
- ▷ How to launch a server instance for deep learning.
- ▷ How to configure a server instance for faster deep learning on the GPU.

Let's get started.

B.1 Overview

The process is quite simple because most of the work has already been done for us. Below is an overview of the process.

- ▷ Setup Your AWS Account.
- ▷ Launch Your Server Instance.
- ▷ Login and Run Your Code.
- ▷ Close Your Server Instance.



Note: It costs money to use a virtual server instance on Amazon. The cost is low for model development (e.g., less than one US dollar per hour), which is why this is so attractive, but it is not free. The server instance runs Linux. It is desirable although not required that you know how to navigate Linux or a Unix-like environment. We're just running our Python scripts, so no advanced skills are needed.



Note: The specific versions may differ as the software and libraries are updated frequently.

B.2 Setup Your AWS Account

You need an account on Amazon Web Services (<https://aws.amazon.com>).

1. Go to <https://aws.amazon.com>. You should click “Sign In” at the toolbar

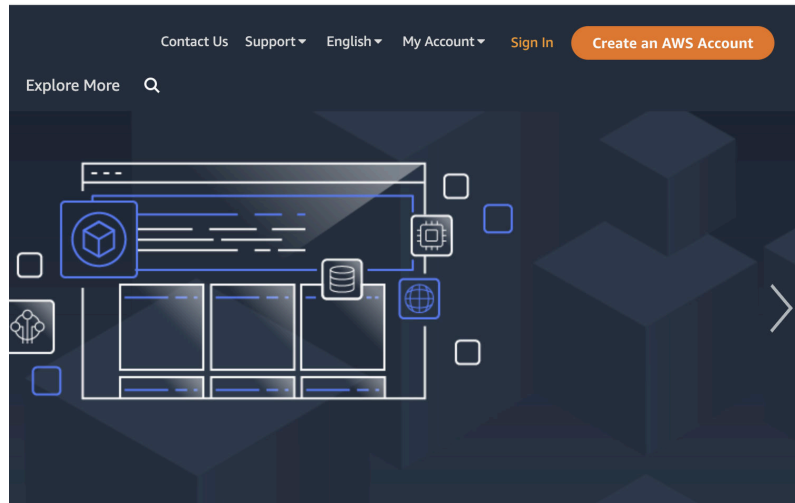


Figure B.1: AWS Sign In Button

2. This will bring you to the following screen. You can enter your login credentials, or you can sign up an account if you didn't have one yet.

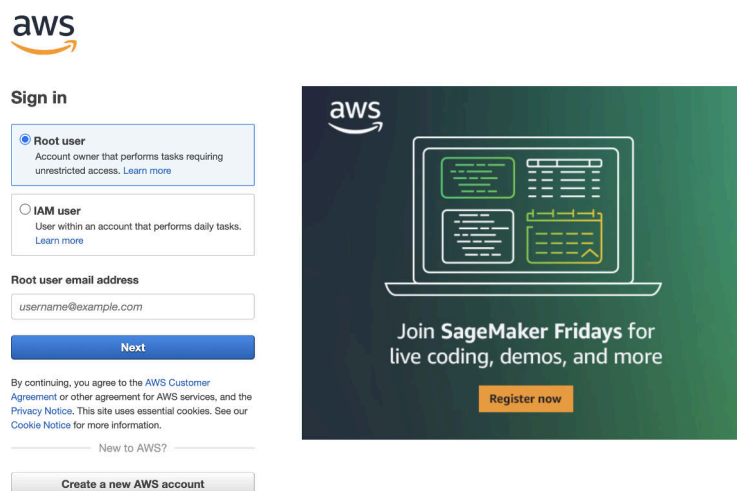


Figure B.2: Login screen for AWS console

Once you have an account you can log into the Amazon Web Services console. You will see a range of different services that you can access.

B.3 Launch Your Server Instance

Now that you have an AWS account, you want to launch an EC2 virtual server instance on which you can run Keras. Launching an instance is as easy as selecting the image to load and starting the virtual server. Thankfully there is already an image available that has almost everything we need it is called the “*Deep Learning AMI GPU PyTorch 1.13.1 (Ubuntu 20.04) 20230315*” and was created and is maintained by Amazon. Let’s launch it as an instance.

1. Login to your AWS console if you have not already.
<https://console.aws.amazon.com/console/home>
2. At the toolbar, you can select your server location. The server will cost differently in different region. Then click on EC2 for launching a new virtual server.

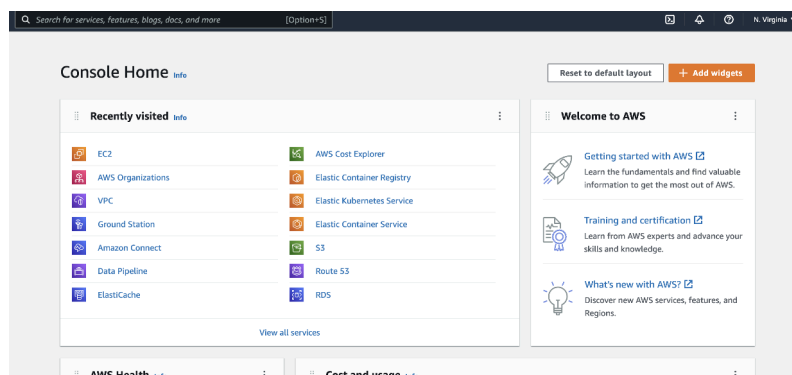


Figure B.3: AWS console

3. Click the *Launch instance* button.

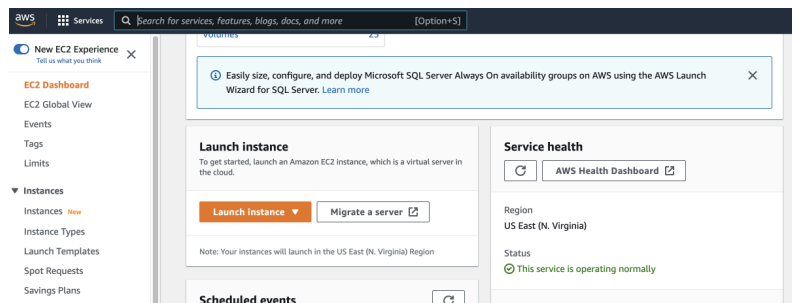


Figure B.4: Launch an EC2 instance

4. Give a name to your instance. Then click on *Browse more AMIs* and search with the keyword “deep learning”. An AMI is an Amazon Machine Image. It is a frozen instance of a server that you can select and instantiate on a new virtual server.

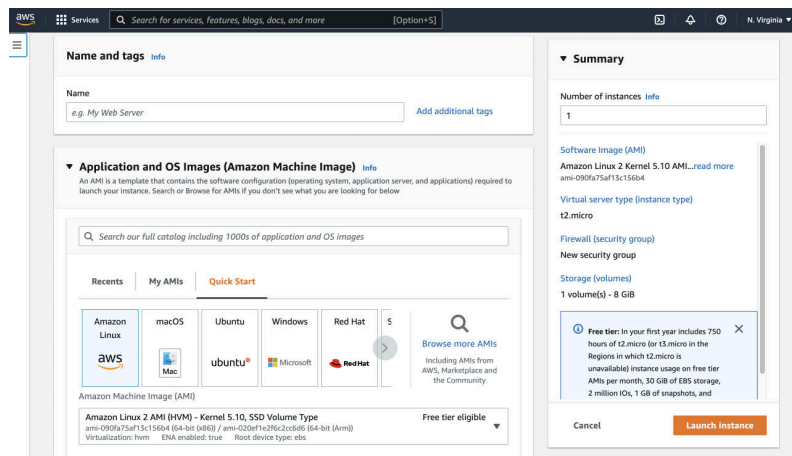


Figure B.5: Searching for an AMI

5. We will select the one with PyTorch pre-installed.

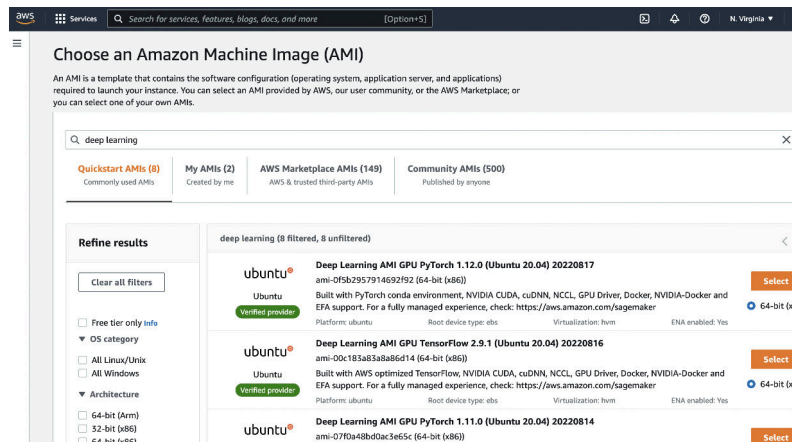
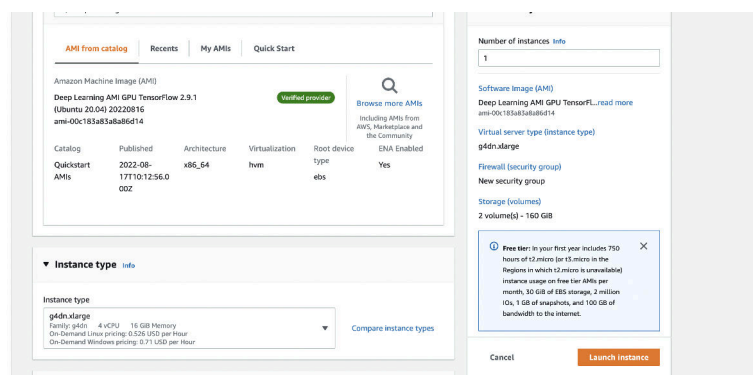


Figure B.6: Selecting “Deep Learning AMI GPU PyTorch 1.13.1 (Ubuntu 20.04)”

6. Now you need to select the hardware on which to run the image. The drop down box will show all the available hardware that are compatible to your AMI. The instance types begin with a “g” comes with GPU and those begin with a “p” includes a Tesla core GPU, which are much faster. In below, `g4dn.xlarge` is selected but you can pick one with suitable amount of memory, disk size, and cost for your project.

Figure B.7: Selecting instance type `g4ad.xlarge` for the new server

You can learn more about EC2 instance types at <https://aws.amazon.com/ec2/instance-types/>.

7. You need to select a key pair for your machine. It is used to login to the server via SSH using key-based authentication. You may select an existing one, or create a new key pair.

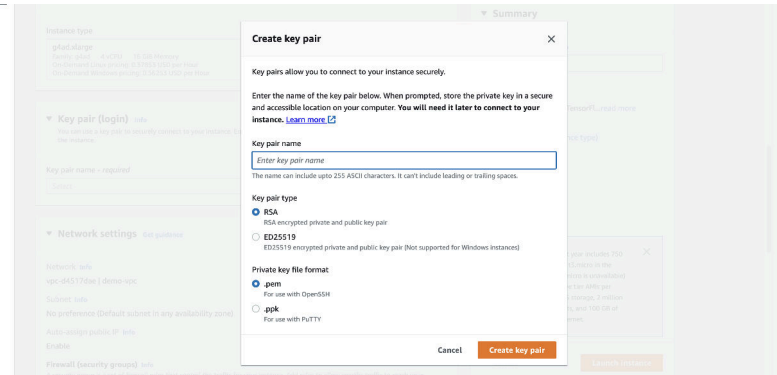


Figure B.8: Creating a new key pair

Depends on the program you use as your SSH client, you may select to download your key between PEM format or PPK format. Keep this key file safe. You will not be able to get it again after this step.

8. As a final check, you should see your instance with “Auto-assign public IP” enabled and “Allow SSH traffic from” set to “Anywhere” so we can access to the instance after launch. Then you can click *Launch* to create your instance.

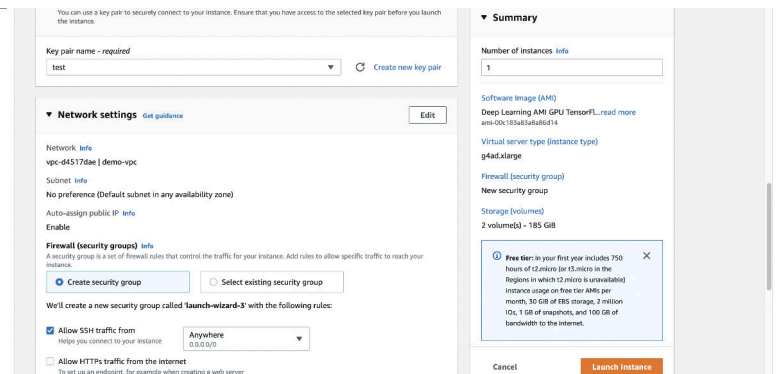


Figure B.9: Launch a new instance

Your server is now running and ready for you to log in.

B.4 Login, Configure and Run

Now that you have launched your server instance, it is time to log in and start using it.

1. Open a Terminal and change directory to where you downloaded your key pair.

- If you have not already done so, restrict the access permissions on your key pair file. This is required as part of the SSH access to your server. For example, open a terminal on your workstation and type:

```
cd Downloads
chmod 600 my-aws-key.pem
```

Listing B.1: Change permissions of your key file

- Click *Instances* in your Amazon EC2 console if you have not done so already. And find and select the instance that you just created.

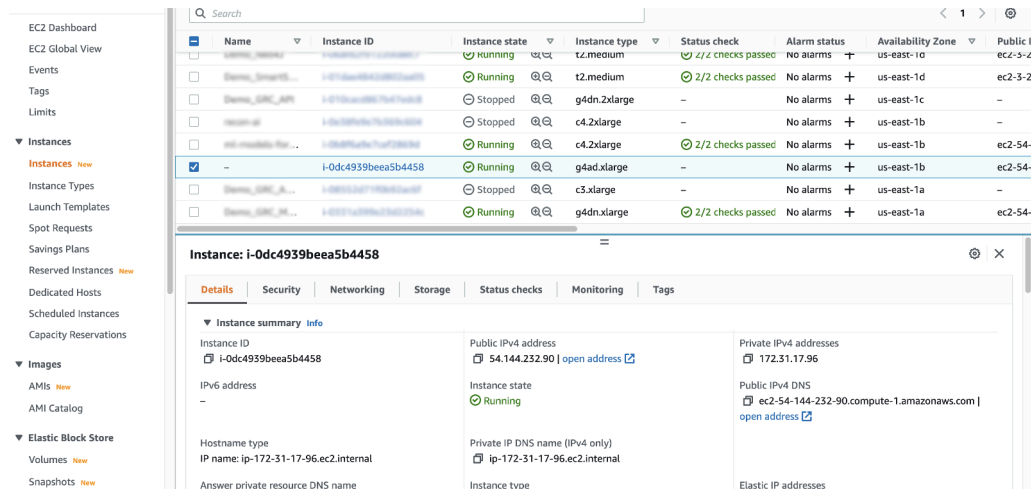


Figure B.10: Find the instance you just created

- Copy the *Public IPv4 address* (at first row of the instance detail table) to your clipboard. In this example my IP address is **54.144.232.90**. **Do not use this IP address, it will not work as your server IP address will be different.**
- Open a Terminal and change directory to where you downloaded your key pair. Login to your server using SSH, for example:

```
ssh -i my-aws-key.pem ubuntu@54.144.232.90
```

Listing B.2: Log-in To Your AWS Instance.

If prompted, type **yes** and press enter.

In the SSH command, we use the syntax `username@address` to specify the server address and the login name. The login name depends on the AMI you used. If you used Ubuntu AMI, it is `ubuntu`. But if you used Amazon Linux AMI, it would be `ec2-user`. You can see these details at <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/connection-prereqs.html>

- If your SSH session is established, you should see the login screen as follows:

```

=====
 _| _|_ )
 _| ( / Deep Learning AMI GPU PyTorch 1.13.1 (Ubuntu 20.04)
 _|\_|_|
=====

Welcome to Ubuntu 20.04.4 LTS (GNU/Linux 5.15.0-1017-aws x86_64v)

PyTorch 1.13.1 and utility libraries are installed in /usr/bin/python3.9.
To access them, use /usr/bin/python3.9.

NVIDIA driver version: 510.47.03
CUDA version: 11.2

AWS Deep Learning AMI Homepage: https://aws.amazon.com/machine-learning/amis/
Release Notes: https://docs.aws.amazon.com/dlami/latest/devguide/appendix-ami-release-notes.html
Support: https://forums.aws.amazon.com/forum.jspa?forumID=263
For a fully managed experience, check out Amazon SageMaker at https://aws.amazon.com/sagemaker
Security scan reports for python packages are located at: /opt/aws/dlami/info/
=====

* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/advantage

System information as of Mon Aug 22 20:44:29 UTC 2022

0 updates can be applied immediately.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

ubuntu@ip-172-31-17-96:~$

```

Figure B.11: Login screen for your AWS server

You are now logged into your server. You are now free to run your code.

Depends on the AMI, you may need to launch Python differently. In this particular AMI, there are Python 3.8 and 3.9 installed but all deep learning libraries are available on Python 3.9 only. You need to use `python3.9` command instead of `python` or `python3` for your projects.

B.5 Build and Run Models on AWS

This section offers some tips for running your code on AWS.

B.5.1 Copy Scripts and Data to AWS

You can get started quickly by copying your files to your running AWS instance. For example, you can copy the examples provided with this book to your AWS instance using the `scp` command as follows:

```
scp -i my-aws-key.pem -r src ubuntu@54.144.232.90:~/
```

Listing B.3: Example for copying sample code to AWS

This will copy the entire `src/` directory to your home directory on your AWS instance. You can easily adapt this example to get your larger datasets from your workstation onto your AWS instance. Note that Amazon may impose charges for moving very large amounts of data in and out of your AWS instance. Refer to Amazon documentation for relevant charges.

B.5.2 Run Models on AWS

You can run your scripts on your AWS instance as per normal:

```
python3.9 filename.py
```

Listing B.4: Example of running a Python script on AWS

In other instances, you may use another command for the Python interpreter, such as `python3`. You are using AWS to create large neural network models that may take hours or days to train. As such, it is a better idea to run your scripts as a background job. This allows you to close your terminal and your workstation while your AWS instance continues to run your script. You can easily run your script as a background process as follows:

```
nohup /path/to/script >/path/to/script.log 2>&1 < /dev/null &
```

Listing B.5: Run script as a background process.

You can then check the status and results in your `script.log` file later.

B.6 Close Your EC2 Instance

When you are finished with your work you must terminate your instance. Remember you are charged by the amount of time that you use the instance. It is cheap, but you do not want to leave an instance on if you are not using it.

1. Log out of your instance at the terminal, for example you can type:

```
exit
```

Listing B.6: Command to log-out of server instance

2. Log in to your AWS account with your web browser.
3. Click EC2.
4. Click *Instances* from the left-hand side menu, then find and select your instance.
5. At the toolbar, click *Instance state* and select *Terminate instance*. Confirm that you want to terminate your running instance.

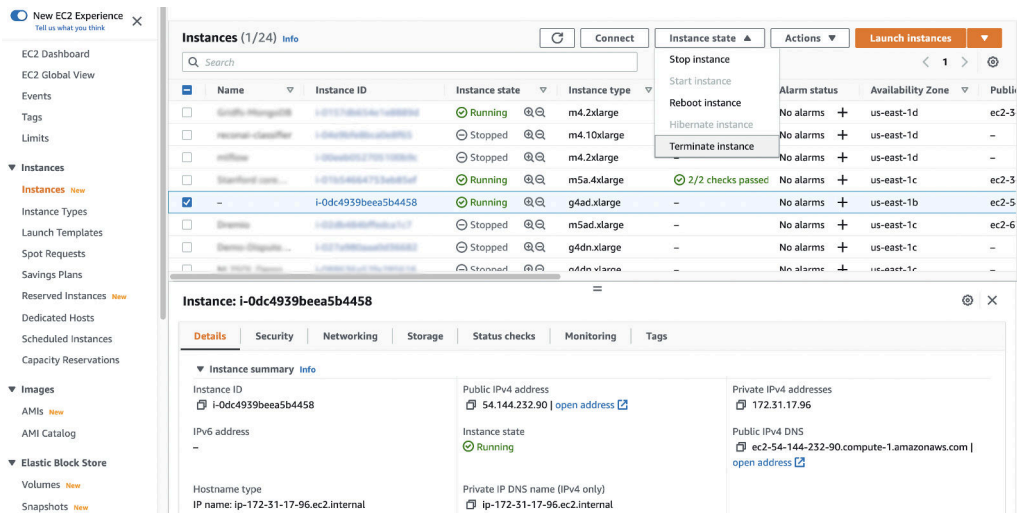


Figure B.12: Terminate an instance

There is a *Stop instance* option. That will only stop your instance from running but still occupies the resources such as storage and IP address. Hence terminate is how you stop AWS from billing you for the server.

It may take a number of seconds for the instance to close and to be removed from your list of instances.

B.7 Tips and Tricks for Using Keras on AWS

Below are some tips and tricks for getting the most out of using Keras on AWS instances.

- ▷ **Design a suite of experiments to run beforehand.** Experiments can take a long time to run and you are paying for the time you use. Make time to design a batch of experiments to run on AWS. Put each in a separate file and call them in turn from another script. This will allow you to answer multiple questions from one long run, perhaps overnight.
- ▷ **Always close your instance at the end of your experiments.** You do not want to be surprised with a very large AWS bill.
- ▷ **Try spot instances for a cheaper but less reliable option.** Amazon sell unused time on their hardware at a much cheaper price, but at the cost of potentially having your instance closed at any second. If you are learning or your experiments are not critical, this might be an ideal option for you. You can access spot instances from the *Spot Instance* option on the left hand side menu in your EC2 web console.

B.8 Further Reading

Below is a list of resources to learn more about AWS and developing deep learning models in the cloud.

- ▷ An introduction to Amazon Elastic Compute Cloud (EC2).
<http://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html>
- ▷ An introduction to Amazon Machine Images (AMI).
<http://docs.aws.amazon.com/AWSEC2/latest/UserGuide/AMIs.html>
- ▷ AWS instance types.
<https://aws.amazon.com/ec2/instance-types/>

B.9 Summary

In this lesson you discovered how you can develop and evaluate your large deep learning models in Keras using GPUs on the Amazon Web Service. You learned:

- ▷ Amazon Web Services with their Elastic Compute Cloud offers an affordable way to run large deep learning models on GPU hardware.
- ▷ How to setup and launch an EC2 server for deep learning experiments.
- ▷ How to update the Keras version on the server and confirm that the system is working correctly.
- ▷ How to run Keras experiments on AWS instances in batch as background tasks.

How Far You Have Come

You made it. Well done. Take a moment and look back at how far you have come.

- ▷ You started off with an interest in deep learning and a strong desire to be able to practice and apply deep learning using PyTorch.
- ▷ You downloaded, installed and started using PyTorch, perhaps for the first time, and started to get familiar with how to develop neural network models with the library.
- ▷ Slowly and steadily over the course of a number of lessons you learned how to use the various different features of the PyTorch library on neural network and deeper models for classical tabular, image and textual data.
- ▷ Building upon the recipes for common machine learning tasks you worked through your first machine learning problems with using deep learning models end-to-end using Python.
- ▷ Using standard templates, the recipes and experience you have gathered, you are now capable of working through new and different predictive modeling machine learning problems with deep learning on your own.

Don't make light of this. You have come a long way in a short amount of time. You have developed the important and valuable skill of being able to work through machine learning problems with deep learning end-to-end using PyTorch. This is an important platform that is used by many researchers. The sky is the limit.

I want to take a moment and sincerely thank you for letting me help you start your deep learning journey with PyTorch. I hope you keep learning and have fun as you continue to master machine learning.